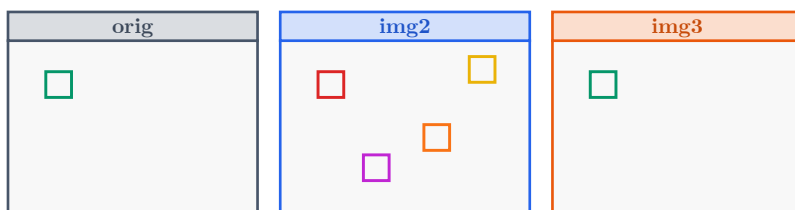


av

Advanced Pixel Lens

User's Manual

이미지 뷰어 · 비교기 · 화질 검증 도구 사용 설명서



v0.22-81-gb439fee · 2026-07-31

DDI 보상 IP 화질 검증을 위한 실무 안내서

차례

1. av 소개와 첫 실행	1
1.1. av는 어떤 도구인가	1
1.1.1. GUI 모드와 헤드리스 모드	1
1.2. 설치 위치와 실행 방법	3
1.3. 버전과 도움말 확인	3
1.3.1. 버전 — <code>--version</code>	3
1.3.2. 도움말 — <code>-h</code> / <code>--help</code>	3
1.4. 네 가지 기본 실행 형태	4
1.4.1. 인자 없이 실행	4
1.4.2. 1장 열기	4
1.4.3. 2장 비교 (A = 기준, B = 비교)	5
1.4.4. 3장 비교 (<code>--comp</code>)	5
1.5. 시작 옵션 전수	5
1.5.1. 초기 뷰 상태 — <code>--zoom</code> , <code>--sync</code> / <code>--no-sync</code>	5
1.5.2. 창과 렌더러 — <code>--fullscreen</code> , <code>--geometry</code> , <code>--windowed</code> , <code>--software</code>	5
1.5.3. 패널 외형 — <code>-nb</code> , <code>-bc</code>	6
1.5.4. 키보드 이동 폭 — <code>-p</code> / <code>--pan-step</code>	6
1.5.5. diff 시작 상태 — <code>-d</code> , <code>--diff-mode</code> , <code>--amplify</code>	7
1.5.6. 다른 창에서 다루는 시작 옵션	7
1.6. 화면 구성	8
1.6.1. 메뉴바 — File / View / Diff	8
1.6.2. 이미지 패널 — A / B / Diff	9
1.6.3. 상태바에 표시되는 정보	9
1.6.4. 파일명 토스트 (1.5초)	10
1.7. 지원 이미지 포맷	10
1.8. 종료하기	11
1.8.1. Esc 키는 종료 키가 아닙니다	11
1.9. 실무 시작 시나리오	11
2. 뷰 조작 — 줌 · 팬 · 패널	13
2.1. 뷰포트 모델	13
2.1.1. 줌 단계는 2의 거듭제곱 12단계입니다	13
2.1.2. 팬은 줌과 무관하게 이미지 픽셀 단위입니다	13
2.1.3. 화면에 나타나는 피드백	14
2.2. 줌 조작 전수	14
2.2.1. 키보드	14
2.2.2. 마우스	15
2.2.3. 시작 배율 지정과 유지	15
2.3. 팬 조작 전수	16

2.3.1. 팬 스텝 바꾸기 — <code>-p</code>	16
2.4. 패널 운용	17
2.4.1. <code>Tab</code> — 활성 패널	17
2.4.2. <code>S</code> — 동기화	17
2.4.3. 회전 · 스왑 · 테두리 · 창	17
2.5. Magnifier — <code>Ctrl+M</code>	18
2.6. Crosshair — <code>M</code>	19
2.7. Pathfinder — <code>P</code> · <code>Ctrl+P</code>	19
2.8. Overlay/Blend — <code>O</code>	19
2.9. 영속화되는 뷰 설정	20
3. 픽셀 검사와 색 판독	21
3.1. 픽셀 값 풍선말 — <code>V</code>	21
3.1.1. A/B 동시 표시 규칙	21
3.1.2. diff 모드일 때의 표시	21
3.2. 픽셀 값 우선순위와 원본값 보존	22
3.2.1. 왜 이 순서인가	22
3.3. 줌 32배 이상: 자동 픽셀 그리드와 값 오버레이	23
3.4. 픽셀 값 표시 형식 순환 — <code>Ctrl+X</code>	23
3.5. 채널 선택 — <code>Shift+R</code> / <code>Shift+G</code> / <code>Shift+B</code> / <code>Shift+Y</code> / <code>Shift+C</code>	24
3.5.1. diff 모드와의 상호작용	24
3.6. 컬러메트리 풍선말 — <code>Shift+V</code>	25
3.6.1. 헤드리스 <code>--probe</code> 와 같은 색 코어	25
3.7. 결함 픽셀 오버레이 — <code>/</code>	26
3.7.1. <code>--validate</code> 와의 관계	26
3.8. 이미지 정보 창 — <code>I</code>	27
3.9. 지원 포맷 상세	27
4. 두 영상 비교 — Diff 모드	29
4.1. Diff 패널이란	29
4.2. Diff 모드 한눈에 보기	29
4.3. 절대 · 상대 · 부호 차이 맵	30
4.3.1. Absolute — 기본 차이 맵 (<code>Ctrl+3</code>)	30
4.3.2. Relative — 어두운 영역에 민감한 상대 오차 (<code>Ctrl+4</code>)	30
4.3.3. Signed — 오버샷과 언더샷을 한 화면에 (<code>Ctrl+1</code>)	31
4.3.4. Highlight — 다른 픽셀만 빨강게 (<code>Ctrl+9</code>)	31
4.4. 컬러맵 계열	31
4.4.1. False Color — 오차 크기를 색으로 (<code>Ctrl+6</code>)	31
4.4.2. Enhance — 1 LSB 차이도 보이게 (<code>Ctrl+5</code>)	32
4.5. 지각 · 구조 메트릭 히트맵	32
4.5.1. SSIM — 구조 유사도 (<code>Ctrl+7</code>)	32

4.5.2. FLIP — 지각 오차맵 (<code>Ctrl+0</code>)	32
4.6. Alpha Blend — 겹쳐 보기 (<code>Ctrl+2</code>)	33
4.7. 파라미터 조작: 증폭 · 알파 · 허용오차	33
4.7.1. 증폭 (amplify)	33
4.7.2. 허용오차(threshold)와 Tolerance diff	34
4.7.3. <code>--comp</code> 모드에서의 대괄호 키	34
4.7.4. CLI로 초기 상태 지정	34
4.8. Diff 픽셀 리스트 창 (<code>Ctrl+D</code>)	34
4.8.1. 화면에서 직접 숫자 읽기	35
4.9. 블링크 비교 (<code>,</code>)	35
4.10. 상태바와 Info 창 판독	36
4.10.1. 상태바	36
4.10.2. Info 창 (<code>I</code>)	36
4.11. 크기가 다른 두 영상	37
4.12. 실무 시나리오: 보상 IP 링잉 확인	37
5. 세 영상 블록 비교 — <code>--comp</code>	39
5.1. 무엇을 위한 모드인가	39
5.2. 실행 방법과 CLI 옵션	40
5.2.1. 기본 실행	40
5.2.2. 세 번째 위치 인자만으로도 활성화	40
5.2.3. 옵션 전수	40
5.2.4. 블록 크기를 고르는 기준	41
5.3. worst 블록 선정 알고리즘	41
5.3.1. 1단계 — 전체 블록 스캔	41
5.3.2. 2단계 — 하이브리드 랭킹 (75% MSE + 25% peak)	41
5.3.3. 왜 하이브리드가 필요한가	42
5.3.4. 랭킹 지표 선택 — <code>--blk-metric</code>	42
5.4. 화면 요소와 오버레이	43
5.4.1. worst 블록 박스 — <code>Ctrl+B</code>	43
5.4.2. 전체 블록 그리드 — <code>G</code> / <code>--grid</code>	44
5.4.3. 승패 맵 — <code>Ctrl+W</code>	44
5.4.4. 블록 PSNR 히트맵 — <code>Ctrl+G</code>	44
5.4.5. worst 블록 리스트 창 — <code>Ctrl+D</code>	45
5.4.6. 상태바 요약과 정보 창 — <code>I</code>	45
5.5. Hover 풍선말과 echo 박스	46
5.5.1. 풍선말이 보여주는 것	46
5.5.2. 반대쪽 패널의 초록색 echo 박스	47
5.6. worst 블록 네비게이션	47
5.6.1. 점프의 스코프 규칙	48
5.6.2. 점프할 때 화면에서 일어나는 일	48

5.7. 패널 스왑 · 3-way 블링크 · 시퀀스 미러	49
5.7.1. img2 ↔ img3 스왑 — S	49
5.7.2. 3-way 블링크 — ,	49
5.7.3. 시퀀스 이동과 자동 미러 — ; / A	49
5.8. 시작 시 [comp] 요약 출력	50
5.9. 헤드리스 comp 출력 — --comp-out / --comp-batch	50
5.10. 실무 워크플로우	50
5.10.1. (a) 프레임 한 장에서 두 보상 알고리즘 중 나쁜 쪽 찾기	50
5.10.2. (b) 시퀀스 전체를 넘기며 회귀 프레임 찾기	51
5.11. comp 모드에서 달라지는 키 요약	51
6. 분석 도구 창	53
6.1. 다섯 개의 창과 공통 규칙	53
6.1.1. 창 목록	53
6.1.2. 상호 배타 규칙	53
6.1.3. 이동 · 크기 · 닫기	53
6.1.4. 어느 이미지가 A 이고 어느 것이 B 인가	54
6.2. 히스토그램 — Ctrl+H	55
6.2.1. 무엇이 보이는가	55
6.2.2. 로그 스케일	55
6.2.3. 값 읽기 — 호버 크로스헤어	55
6.2.4. Diff A-B 히스토그램	55
6.2.5. A/B 동시 표시 — Compare A/B	56
6.3. 수평 절단면 — Ctrl+L / 수직 절단면 — Ctrl+Y	56
6.3.1. 무엇이 보이는가	56
6.3.2. 어느 행 · 어느 열이 잘리는가 (중요)	56
6.3.3. 축과 정규화	57
6.3.4. 값 읽기 — 호버 크로스헤어	57
6.3.5. Diff A-B 절단면	57
6.4. 통계 창 — Ctrl+S	57
6.4.1. 무엇이 보이는가	57
6.4.2. 표시되는 통계량 전수	58
6.5. ROI 선택 모드와 ROI 통계 — Ctrl+E	58
6.5.1. ROI 지정하기	58
6.5.2. ROI Statistics 창의 구성	59
6.5.3. 컬러메트리 행 (xy / u'v' / CCT)	59
6.5.4. ROI 해제와 창 닫기	60
6.6. Scatter Plot — Ctrl+T	60
6.6.1. 무엇이 보이는가	60
6.6.2. 대각선의 의미	60
6.6.3. 채널 선택과 샘플링	60

6.7. 차트와 통계 내보내기 — <code>Shift+Ctrl+S</code>	61
6.7.1. 저장 창은 상황에 따라 달라진다	61
6.7.2. 차트 저장 창 (<code>Save Histogram</code> / <code>Save H-Line Cut</code> / <code>Save V-Line Cut</code>)	61
6.7.3. CSV 파일 형식	62
7. 이미지 시퀀스와 파일 입출력	64
7.1. 디렉토리 시퀀스 — 자동 구성과 정렬 규칙	64
7.2. 시퀀스 탐색 키	64
7.2.1. 어떤 패널이 움직이는가	64
7.2.2. 순환과 피드백	65
7.3. A 와 B 는 서로 다른 디렉토리를 독립으로 탐색합니다	65
7.4. <code>--pair</code> — 같은 파일명으로 두 디렉토리를 묶어 이동	65
7.4.1. 짝이 없는 프레임	66
7.4.2. 헤드리스 지표와 조합	66
7.5. <code>--comp</code> 의 3디렉토리 미러링	67
7.6. 슬라이드쇼 (자동 재생)	67
7.7. 파일 열기	68
7.8. 저장 — <code>Save Images</code> 창	68
7.8.1. 기본 경로	69
7.8.2. 포맷 옵션	69
7.8.3. 스크린샷과 저장의 차이	69
7.8.4. 패널 우클릭 컨텍스트 메뉴	69
7.9. 클립보드 복사 (2단계 키)	70
7.10. 실무 시나리오 — 100프레임 시퀀스에서 회귀 프레임 찾기	70
8. 헤드리스 CLI와 CI 연동	72
8.1. 헤드리스 실행 모델	72
8.1.1. 창을 열지 않는 모드 전수	72
8.1.2. <code>stdout</code> 은 데이터, <code>stderr</code> 는 요약	72
8.2. 지표 추출 — <code>--metrics</code>	73
8.2.1. 기본 사용법	73
8.2.2. 16개 열의 의미	73
8.2.3. <code>psnr_db</code> — “채널별 PSNR 평균” 관례	74
8.2.4. <code>msigned</code> — 밝기 편향 읽는 법	74
8.2.5. 시퀀스 전체 실행 — <code>--pair</code> 조합	74
8.2.6. 출력 포맷 — <code>--format csv json junit</code>	75
8.3. CI 게이트 — <code>--fail-*</code> / <code>--warn-psnr</code>	76
8.3.1. 다섯 개의 임계값	76
8.3.2. 게이트 출력	76
8.4. 픽셀 컬러메트리 — <code>--probe <X,Y></code>	76
8.4.1. 무엇을 출력하나	76
8.4.2. 검증된 기준값	77

8.4.3. B 를 함께 준 경우의 <code>delta</code> 행	77
8.5. 평판 균일도와 무라 — <code>--uniformity</code>	78
8.5.1. 무엇을 재나	78
8.5.2. 11개 열	78
8.5.3. 균일도 % 의 정의와 측정점 배치	78
8.5.4. SEMU 무라 지수 — 계산 방식과 한계	78
8.5.5. A/B/ Δ 세 행	79
8.5.6. 균일도 게이트	79
8.6. 매니페스트 배치 — <code>--batch <file -></code>	79
8.6.1. 왜 필요한가	79
8.6.2. 매니페스트 형식	80
8.6.3. 실행	80
8.7. diff 이미지 내보내기 — <code>--diff-out <png></code>	80
8.7.1. 기본 사용법	80
8.7.2. <code>--sbs</code> 합성	81
8.7.3. 주의점	81
8.8. 결함 스캔 — <code>--validate</code>	81
8.8.1. 무엇을 잡나	81
8.8.2. 종료 코드와 8비트 입력	82
8.9. 블록 비교 헤드리스 — <code>--comp-out</code> / <code>--comp-batch</code>	82
8.9.1. 블록별 테이블 — <code>--comp-out</code>	82
8.9.2. 프레임 요약 — <code>--comp-batch</code>	83
8.10. 종료 코드	84
8.11. 실전 CI 스크립트	85
8.11.1. (a) bash 회귀 게이트 루프	85
8.11.2. (b) GitHub Actions — JUnit 리포트 업로드	86
9. 색 관리와 룩(LUT · CDL)	88
9.1. 표시 파이프라인 — 어느 단계에서 무엇이 적용되는가	88
9.2. 룩은 지표와 픽셀 판독에 영향을 주지 않는다	89
9.3. ICC 프로파일과 색 관리 스위치	90
9.3.1. <code>--profile <file></code>	90
9.3.2. <code>--no-color-mgmt</code>	90
9.3.3. 현재 지원 범위와 한계	90
9.4. 3D/1D LUT — <code>--lut <file.cube></code>	91
9.4.1. 무엇을, 왜	91
9.4.2. <code>.cube</code> 파싱 규격	91
9.4.3. 보간과 적용 범위	91
9.4.4. 1D LUT 의 함정 — 메모리	92
9.4.5. 룩 ON/OFF 토글 — <code>'</code> 키	92
9.5. ASC-CDL — <code>--cdl <file.cdl></code>	92

9.5.1. 수식과 적용 순서	92
9.5.2. 파싱 범위	93
9.5.3. 33×33×33 베이킹의 정밀도 한계	94
9.6. 색 코어 — 컬러메트리 계산의 단일 소스	94
9.6.1. 변환 체인	94
9.6.2. GUI 프로브와 헤드리스가 같은 코어를 쓴다	95
9.6.3. FLIP 은 별도의 D65 경로다	96
9.7. 실무 활용 — DDI 검증 시나리오	96
9.7.1. 패널 측정 데이터와 대조하기	96
9.7.2. LUT 으로 감마 보정 후 비교하기	96
9.8. 요약과 주의점	97
10. 설정 영속화와 문제 해결	98
10.1. 설정 파일 <code>.av.ini</code>	98
10.1.1. 위치와 이름	98
10.1.2. 저장 시점과 읽는 시점	98
10.1.3. 파일 형식	98
10.1.4. 저장되는 키 전수	99
10.1.5. 저장은 되지만 다음 실행에 반영되지 않는 두 키	100
10.1.6. 영속화되지 않는 상태	100
10.1.7. 창 배치 파일 <code>.av_imgui.ini</code>	101
10.2. 설정 초기화와 부분 되돌리기	101
10.2.1. 전체 초기화	101
10.2.2. 특정 설정만 되돌리기	101
10.2.3. 설정을 고정해서 잠그는 방법	101
10.3. 모드별 키 재정의 대조표	101
10.3.1. <code>--comp</code> 가 매 프레임 강제로 끄는 상태	102
10.4. 문제 해결 Q&A	102
10.4.1. Q1. OpenGL 초기화에 실패하거나 원격 화면에서 창이 안 뜹니다	102
10.4.2. Q2. SSH 로 접속한 서버라 GUI 를 못 씁니다. 수치만 뽑고 싶습니다	103
10.4.3. Q3. 두 영상 크기가 달라 비교가 안 됩니다 (종료 코드 5)	103
10.4.4. Q4. HDR 이미지에 NaN 이나 Inf 가 섞인 것 같습니다	103
10.4.5. Q5. 16비트 PPM 의 픽셀 값이 이상하게 보입니다	104
10.4.6. Q6. <code>--comp</code> 인데 diff 키가 안 먹습니다	104
10.4.7. Q7. <code>--comp</code> 인데 블록 박스가 안 보입니다	104
10.4.8. Q8. 키를 눌렀는데 아무 반응이 없습니다	105
10.4.9. Q9. <code>Ctrl+D</code> 를 눌러도 diff 픽셀 리스팅 창이 뜨지 않습니다	105
10.4.10. Q10. 이 파일 형식은 왜 안 열립니까	105
10.4.11. Q11. 한글 파일명이 깨지고 글자가 작게 나옵니다	106
10.4.12. Q12. 설정이 저장되지 않습니다	106
10.5. 성능 팁	106
10.5.1. 큰 이미지와 시퀀스	106

10.5.2. 무거운 계산이 언제 일어나는가	106
10.5.3. 소프트웨어 렌더러 사용 시 제약	107
11. 부록 — 전체 레퍼런스	108
11.1. 키보드 단축키 전수표	108
11.1.1. Navigation — 이동	108
11.1.2. Zoom — 줌	108
11.1.3. Display — 표시	109
11.1.4. Channel — 채널	109
11.1.5. Analysis — 분석	109
11.1.6. Overlay — 오버레이	110
11.1.7. Sequence — 시퀀스	110
11.1.8. Blink — 블링크 비교기	110
11.1.9. Diff — 차이 비교	110
11.1.10. Comp — 3영상 블록 비교 (<code>--comp</code>)	111
11.1.11. File / Copy / Help	112
11.1.12. 핫키 창 목록에는 없지만 실제로 동작하는 키	112
11.2. CLI 옵션 전수표	113
11.2.1. GUI 옵션 — 창과 표시	113
11.2.2. GUI 옵션 — 비교와 색	113
11.2.3. 헤드리스 옵션	118
11.2.4. CI 게이트 옵션	119
11.2.5. 인자 검증 규칙	119
11.3. 종료 코드	120
11.4. CSV / JSON 스키마 요약	120
11.4.1. <code>--metrics</code> / <code>--batch</code> — 16열	120
11.4.2. <code>--probe</code> — 11열	121
11.4.3. <code>--uniformity</code> — 11열	121
11.4.4. <code>--comp-out</code> — 11열 (블록 단위)	122
11.4.5. <code>--comp-batch</code> — 10열 (프레임 단위)	122
11.4.6. <code>--validate</code> — 2열	123
11.5. 지원 파일 포맷	123
11.6. 용어집	123
11.6.1. 지표 용어	123
11.6.2. 색 용어	124
11.6.3. <code>--comp</code> 와 실무 용어	124

그림 차례

그림 1.1	입력 이미지 개수와 옵션에 따른 패널 레이아웃.	4
그림 1.2	av 창의 구성 요소. 활성 패널은 테두리가 굵게 강조된다.	8
그림 2.1	이미지 좌표에서 화면 좌표로의 변환. 줌은 배율을, 팬은 평행이동을 담당한다.	13
그림 3.1	픽셀 값 판독 우선순위와 표시 형식 변환. 원본 값 보존이 최우선이다.	22
그림 4.1	두 영상이 Diff 패널과 지표로 흘러가는 경로. 지표는 표시 룩의 영향을 받지 않는다.	29
그림 4.2	주요 Diff 모드의 색 해석. 같은 색이라도 모드마다 의미가 다르다.	31
그림 5.1	<code>--comp</code> 3패널 화면. 마우스를 <code>img2</code> 의 블록에 올리면 나머지 두 패널에 초록 echo 박스가 뜨고, 비교 패널(<code>img3</code>)에는 자기 통계로 계산한 같은 형식의 풍선말이 함께 나타난다.	39
그림 5.2	<code>worst</code> 블록 선정은 지표 MSE 랭킹 75%와 피크 픽셀 오차 랭킹 25%를 섞는다. 오른쪽 두 블록은 MSE는 비슷하지만 성격이 전혀 다른 결함이다.	42
그림 5.3	<code>Tab</code> 의 순회 대상은 마우스가 올라가 있는 패널로 결정된다. 대괄호 키는 마우스 위치와 무관하게 대상이 고정된다.	48
그림 6.1	이미지 패널을 중심으로 열리는 분석 창들과 단축키.	53
그림 7.1	디렉토리 시퀀스 동기 이동. <code>--pair</code> 는 두 디렉토리를, <code>--comp</code> 는 세 디렉토리를 같은 파일명으로 묶는다.	66
그림 8.1	헤드리스 모드의 CI 연동 흐름. 창을 띄우지 않고 종료 코드로 합격/불합격을 알린다.	73
그림 9.1	표시 파이프라인과 측정 경로의 분리. 룩(LUT/CDL)은 화면에만 적용되며 지표·픽셀 값 판독에는 영향을 주지 않는다.	88

이 설명서를 읽는 법

- 처음 쓰는 사람은 1장 → 2장 → 3장 순서로 읽으면 기본 조작이 끝납니다.
- 두 영상을 비교하려면 4장, 세 영상을 블록 단위로 비교하려면 5장을 봅니다.
- 스크립트·CI 자동화가 목적이면 8장만 읽어도 됩니다.
- 키를 빨리 찾으려면 11장 부록의 전체 단축키 표를 보거나, 실행 중에 `Ctrl+Shift+H`로 찾기 창을 여십시오.
- 이 문서의 모든 예시는 실제로 동작하는 명령 문법입니다.

1. av 소개와 첫 실행

이 장에서는 av가 어떤 도구이고 어디에 설치되어 있으며, 명령 한 줄로 어떻게 띄우는지를 다룹니다. 실행 형태 네 가지(인자 없음 / 1장 / 2장 / 3장), 시작 옵션 전체, 그리고 처음 화면에 보이는 것들(메뉴바·패널·상태바·파일명 토스트)과 종료 방법까지 익히면 나머지 장을 읽을 준비가 끝납니다.

1.1. av는 어떤 도구인가

av는 “Advanced Pixel Lens”의 줄임말로, 이미지를 **픽셀 단위로 파고들어 보는 뷰어**이자 두(또는 세) 장의 이미지를 나란히 놓고 차이를 계량하는 **비교기**입니다. 창 제목과 앱 아이콘에도 `Advanced Pixel Lens` 라는 이름이 그대로 쓰입니다.

항목	내용
언어	C++20
윈도우/입력	SDL3 (<code>SDL_CreateWindow</code> , 스캔코드 기반 키 처리)
UI	Dear ImGui (메뉴바·상태바·분석 창·풍선말 전부 ImGui)
렌더링	OpenGL 3.3 Core (GLSL <code>#version 150</code>). 실패하거나 <code>--software</code> 지정 시 SDL 소프트웨어 렌더러로 자동 폴백
설정 파일	<code>~/.av.ini</code> (종료 시 자동 저장)

주 용도는 두 가지입니다.

1. **DDI(디스플레이 드라이버 IC) 보상 IP 검증** — 보상 전 원본과 보상 IP가 뺀 결과를 A/B로 띄워 놓고, PSNR·SSIM·FLIP 같은 지표와 블록 단위 최악 구간을 찾아 화질 열화가 어디서 발생했는지 짚어냅니다. 원본 대비 FW 결과와 HW 결과를 한 화면에서 셋으로 비교하는 `--comp` 모드(5장 참조)가 이 목적에 특화된 기능입니다.
2. **일반 영상 알고리즘 A/B 비교** — 스케일러, 노이즈 리덕션, 톤매핑 등 임의의 두 결과물을 비교하는 범용 도구로도 그대로 씁니다.

1.1.1. GUI 모드와 헤드리스 모드

av는 하나의 실행 파일이지만 실행 방식은 두 갈래입니다. 이 장부터 7장까지는 GUI 모드를, 8장은 헤드리스 모드를 다룹니다.

모드	트리거	동작
GUI	헤드리스 플래그가 없을 때	창을 띄우고 이벤트를 처리한다.

										람 이 눈 으 로 보 고 키 로 조 작
헤드리스	--metrics	--batch	--comp-out	--comp-batch	--diff-out	--validate	--probe	--uniformity	중 하나	SDL-OpenGL. 창 을 아 예 만 들 지 않 고 계 산 결 과 (CSV/ JSON/ JUnit 또 는 PNG) 만 내 보 낸 뒤 즉 시 종 료. SSH-CI 파 이 프 라 인 용

참고 헤드리스 분기는 `main()` 의 맨 앞, `parse_cli()` 직후에 일어납니다. 즉 헤드리스 플래그가 하나라도 있으면 그래픽 드라이버가 없는 서버에서도 안전하게 돛니다. 자세한 컬럼 규격과 CI 게이트(exit 10 등)는 8장을 보십시오.

1.2. 설치 위치와 실행 방법

세 플랫폼 모두 `PATH` 에 잡히는 위치에 설치되어 있으므로, 어느 디렉토리에서든 `av` 만 치면 실행됩니다.

플랫폼	설치 위치	실행 예
macOS	<code>~/local/bin/av</code>	<code>av a.png b.png</code>
Linux (alexws)	번들 디렉토리 설치 (<code>av</code> 래퍼 + <code>av.bin</code> + <code>lib/</code>)	<code>av a.png b.png</code>
Windows	<code>C:\Windows\av.exe</code>	<code>av a.png b.png</code>

참고 Linux 번들은 실행 파일과 의존 라이브러리를 한 디렉토리에 묶어 배포하는 형태입니다. 실제로 실행되는 것은 래퍼 스크립트 `av` 이고, 그 안에서 번들된 `ld-linux` 와 `lib/` 를 지정해 `av.bin` 을 띄웁니다. 오프라인 서버에 tar로 풀어 놓기만 해도 동작합니다.

팁 SSH로 접속한 Linux에서 GUI를 띄우면 `av`가 `SSH_CONNECTION` 환경변수를 보고 자동으로 소프트웨어 렌더러로 전환하며 아래 한 줄을 출력합니다. `--software` 를 손으로 붙일 필요가 없습니다.

```
[av] X11 forwarding detected (SSH) - using software renderer
```

1.3. 버전과 도움말 확인

1.3.1. 버전 — `--version`

```
av --version
```

출력은 다음 형식입니다. `AV_VERSION_FULL` 은 `git describe` 기반이고 날짜는 최신 커밋 날짜입니다.

```
av v0.22-80-g096f41f (updated 2026-07-31)
```

버그를 보고하거나 CI 로그를 남길 때는 이 한 줄을 그대로 첨부하십시오. 태그 뒤의 `-80-g096f41f` 는 “태그 이후 80개 커밋, 해시 096f41f”라는 뜻이라 빌드를 정확히 특정할 수 있습니다.

1.3.2. 도움말 — `-h` / `--help`

```
av --help
```

앞부분은 다음과 같습니다(전체는 40행 남짓이며, 뒤쪽은 5장·8장·9장에서 다루는 옵션들입니다).

```
Usage: av [options] [imageA] [imageB]

Options:
  --diff-mode <mode>  none|alphablend|abs|rel|highlight|falsecolor|ssim|flip|signed|enhance
  (default: none)
  --zoom <factor>      fit|0|1|2 etc. (0/fit=window, 1=1:1 actual; saved)
  --sync               Enable viewport sync (default: on)
  --no-sync            Disable viewport sync
  ...
  --amplify <val>      Diff amplification 0.1-100 (default: 1.0)
  --fullscreen         Start in fullscreen
  --geometry <WxH>     Initial window size (default: 1280x720)
  -p, --pan-step <N>   Shift+hjkl jump size in pixels (default: 32)
  -bc <A> <B> <D>     Border colours for A/B/Diff panels as 6-digit hex
  e.g. -bc ff00ff ffff00 00ffff (default: magenta/yellow/cyan)
```

```

-d, --diff          Show pixel-absolute diff (shortcut)
--software          Force SDL software renderer (no OpenGL)
--windowed          Start in windowed mode (title bar + resizable)
-nb                Start with panel borders hidden
--version           Print version and exit
-h, --help          Print this help

```

`-h` 와 `--help`, `--version` 은 모두 **출력 후 즉시 종료(exit 0)**합니다. 창이 뜨지 않으므로 스크립트에서 안전하게 호출할 수 있습니다.

주의 알 수 없는 옵션(`--` 로 시작하는 미지의 토큰)을 주면 av는 `Unknown option: ...` 을 stderr로 찍고 **exit 1**로 죽습니다. 오타가 조용히 무시되지 않으므로, CI 스크립트에서 옵션 철자를 바꿀 때 안심해도 됩니다.

1.4. 네 가지 기본 실행 형태

위치 인자(파일 경로)는 최대 3개까지 받으며, 순서대로 imageA, imageB, imageC로 해석됩니다. 몇 개를 주느냐에 따라 레이아웃이 자동으로 결정됩니다.

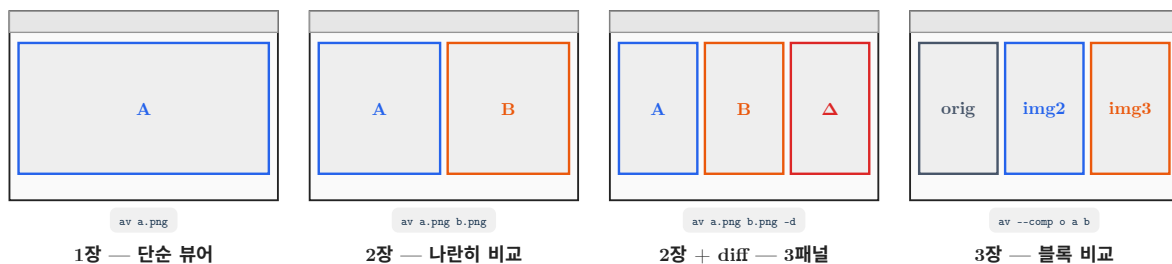


그림 1.1: 입력 이미지 개수와 옵션에 따른 패널 레이아웃.

형태	명령	화면
인자 없음	<code>av</code>	빈 뷰어. 메뉴 File → Open Image A... 또는 <code>Shift+Ctrl+O</code> 로 여는다. 창에 파일을 드래그 앤 드롭해도 로드됨
1장	<code>av result.png</code>	단일 패널이 창 전체를 차지
2장 (A,B 비교)	<code>av orig.png comp.png</code>	좌우 반반 A B. diff 모드를 켜면 A B Diff 3분할로 바뀜
3장 (<code>--comp</code>)	<code>av --comp orig.png fw.png hw.png</code>	3분할 orig img2 img3. diff는 강제로 꺼짐

1.4.1. 인자 없이 실행

```
av
```

빈 화면에서 시작합니다. 파일을 여는 방법은 세 가지입니다.

- `Shift+Ctrl+O` (macOS는 `Shift+Cmd+O`) — 현재 활성 패널에 열기
- 메뉴바 File → Open Image A... / Open Image B... — 패널을 지정해서 열기
- 탐색기에서 창으로 **드래그 앤 드롭** — A가 비어 있으면 A에, 차 있으면 B에 로드

1.4.2. 1장 열기

```
av /data/ddi/compensated/frame_0001.png
```

한 장만 주면 패널 하나가 창 전체를 씹니다. 이 상태에서도 `;` / `A`로 같은 디렉토리의 다음/이전 이미지를 순회할 수 있습니다(7장 참조).

1.4.3. 2장 비교 (A = 기준, B = 비교)

```
av /data/ddi/orig/frame_0001.png /data/ddi/comp/frame_0001.png
```

왼쪽(A)이 기준, 오른쪽(B)이 비교 대상입니다. PSNR·SSIM·FLIP을 비롯한 모든 지표는 “A 대비 B”로 계산되므로, DDI 검증에서는 반드시 원본을 A에, 보상 IP 출력을 B에 놓으십시오. 순서를 잘못 넣었다면 `Shift+Space`로 A와 B를 맞바꿀 수 있습니다.

1.4.4. 3장 비교 (--comp)

```
av --comp /data/ddi/orig/f001.png /data/ddi/fw/f001.png /data/ddi/hw/f001.png
```

원본 하나와 비교 대상 둘(예: FW 모델 결과와 HW RTL 결과)을 한 화면에 놓고, 각 블록별로 어느 쪽이 원본에 더 가까운지 판정하는 전용 모드입니다. 이 모드에서는 diff가 강제로 비활성화되며, 시작할 때 `[comp]`로 시작하는 요약이 stdout에 출력됩니다. 자세한 사용법(블록 크기, worst 블록 순회, 승패 맵 등)은 5장을 보십시오.

참고 세 번째 위치 인자를 주면 `--comp`를 생략해도 자동으로 3영상 모드가 켜집니다. 반대로 `--comp`를 켜는데 이미지가 3개가 아니면 `--comp needs three images: av --comp <orig> <img2> <img3>`를 찍고 `exit 1`로 종료합니다.

1.5. 시작 옵션 전수

1.5.1. 초기 뷰 상태 — --zoom, --sync / --no-sync

옵션	기본값	설명
<code>--zoom <factor></code>	<code>fit</code> (= 0)	초기 배율. <code>fit</code> 또는 0이면 창에 맞춤, 1이면 1:1 실제 크기, 2면 2배 값은 <code>~/av.ini</code> 의 <code>zoom=</code> 키에 영속화 되며, 다음 실행에서 <code>--zoom</code> 을 생략하면 저장값이 그대로 재현됨. 배율은 0.125 ~ 256으로 클램프됨
<code>--sync</code>	<code>on</code>	A/B 패널의 줌·팬을 잠금 동기화. GUI에서는 <code>[S]</code> 로 토글
<code>--no-sync</code>	—	동기화를 끄고 시작. 해상도가 다른 두 영상을 각각 따로 보고 싶을 때

```
av --zoom 4 orig.png comp.png      # 4배로 시작 (이후 실행에도 4배가 유지됨)
av --zoom fit orig.png comp.png    # 창 맞춤으로 되돌리기
av --no-sync orig.png comp.png     # A/B를 따로 움직이기
```

주의 `--zoom`은 **영속**되지만 `--sync / --no-sync`는 CLI 값이 항상 이깁니다. `~/av.ini`에 `sync_viewports=0`이 저장돼 있어도, `--no-sync` 없이 실행하면 기본값 `on`이 덮어씹힙니다. 동기화를 끈 상태로 시작하고 싶다면 매번 `--no-sync`를 붙이십시오.

팁 DDI 보상 결과를 볼 때는 `--zoom 8` 정도로 시작해 두면 매 프레임 확대 조작을 반복하지 않아도 됩니다. `--zoom`으로 지정한 배율은 `;` / `A`로 다음 영상을 넘길 때도 유지되므로 시퀀스를 훑을 때 화면이 튀지 않습니다.

1.5.2. 창과 렌더러 — --fullscreen, --geometry, --windowed, --software

옵션	기본값	설명
<code>--fullscreen</code>	<code>off</code>	<code>SDL_WINDOW_FULLSCREEN</code> 으로 시작. 프로젝터·검증용 모니터에 꽉 채워 띄울 때
<code>--geometry <WxH></code>	<code>1280x720</code>	초기 창 크기(픽셀). <code>x</code> 또는 <code>X</code> 구분자 모두 허용

<code>--windowed</code>	off	타이틀바가 있는 일반 창 모드로 시작. 지정하지 않으면 테두리 없는 최대화 창 이 기본
<code>--software</code>	off	OpenGL을 쓰지 않고 SDL 소프트웨어 렌더러 강제. 원격 데스크톱·가상머신·GL 드라이버 문제 시

```
av --geometry 1920x1080 --windowed orig.png comp.png
av --fullscreen --comp orig.png fw.png hw.png
av --software orig.png comp.png      # GL이 뻥뻥일 때
```

주의 기본(= `--windowed` 미지정)은 `SDL_WINDOW_BORDERLESS` 에 `SDL_WINDOW_MAXIMIZED` 가 함께 붙습니다. 즉 창이 곧바로 최대화되므로 `--geometry` 로 준 크기가 눈에 보이지 않을 수 있습니다. `--geometry` 를 확실히 적용하려면 `--windowed` 와 함께 쓰십시오.

창 모드는 GUI에서 **W**로 언제든지 토글할 수 있고, 그 상태는 `~/.av.ini` 의 `windowed_mode=` 에 저장됩니다. 타이틀바에 파일명이 표시되는 것은 `--windowed` 모드일 때뿐이며, 형식은 다음과 같습니다 (대괄호 안은 현재 diff 모드 태그).

```
av - f001.png | f001_comp.png [Abs]
```

테두리 없는 기본 모드에서는 제목이 `Advanced Pixel Lens` 로 고정됩니다.

1.5.3. 패널 외형 — `-nb`, `-bc`

옵션	기본값	설명
<code>-nb</code>	테두리 표시	패널 테두리를 숨긴 상태로 시작. GUI에서 B 로 토글되며 이 토글 값은 <code>~/.av.ini</code> 의 <code>show_borders=</code> 에 영속화 됨
<code>-bc <A> <D></code>	magenta / yellow / cyan	A·B·Diff 패널 테두리 색을 6자리 hex로 지정. 값 3개를 반드시 모두 줘야 함. 영속화되지 않으므로 매번 지정

```
av -bc ff00ff ffff00 00ffff orig.png comp.png # 기본값과 동일
av -bc 00ff00 ff8000 ffffffff orig.png comp.png # 초록 / 주황 / 흰색
av -nb orig.png comp.png                       # 테두리 없이 시작
```

테두리는 알파 230으로 그려지며 A(왼쪽)=마젠타 `ff00ff`, B(오른쪽)=노랑 `ffff00`, Diff=시안 `00ffff` 가 기본입니다. `--comp` 3분할에서도 같은 세 색이 orig / img2 / img3 순으로 재사용됩니다. hex 파싱에 실패하면 `Invalid hex colour: ...` 를 찍고 exit 1로 종료합니다.

팁 캡처해서 보고서에 넣을 때는 `-nb` 로 테두리를 지우면 이미지 경계가 깔끔하게 잘립니다. 반대로 화면 캡처를 여러 장 이어 붙일 때는 색 테두리가 어느 패널인지 구분해 주므로 켜 두는 편이 낫습니다.

1.5.4. 키보드 이동 폭 — `-p` / `--pan-step`

옵션	기본값	설명
<code>-p <N></code> , <code>--pan-step <N></code>	32	<code>[Shift] + [h] [j] [k] [l]</code> (또는 <code>[Shift] + 방향키</code>) 로 이동할 때의 점프 폭(이미지 픽셀). <code>[Shift]</code> 없이 누르면 항상 1픽셀. 영속화되지 않음

```
av -p 64 orig.png comp.png      # 한 번에 64픽셀씩 이동
av --pan-step 8 orig.png comp.png # 미세 이동 (8x8 블록 단위 겹쳐놓)
```

팁 DDI 보상 IP가 8x8 또는 16x16 블록 단위로 동작한다면 `--pan-step` 을 그 블록 크기와 맞춰 두십시오. `Shift+l` 을 누를 때마다 정확히 블록 하나씩 넘어가므로, 블록 경계 아티팩트를 한 칸씩 옮기 좋습니다.

1.5.5. diff 시작 상태 — `-d`, `--diff-mode`, `--amplify`

옵션	기본값	설명
<code>-d</code> , <code>--diff</code>	off	<code>--diff-mode abs</code> 와 동일한 단축 옵션. 픽셀 절대차 diff로 바로 시작
<code>--diff-mode <mode></code>	none	시작 diff 모드 지정. 아래 10개 토큰 중 하나 (<code>none</code> = 끄, 실제 모드는 9종)
<code>--amplify <val></code>	1.0	diff 증폭 배율. 도움말상 범위는 0.1 ~ 100 . GUI에서 <code>[</code> / <code>]</code> 로 조절할 때는 0.5 ~ 100 으로 클램프되고 <code>\</code> 로 1.0 리셋

`--diff-mode` 가 받는 토큰은 다음과 같습니다.

토큰	단축키	의미
none	Ctrl+D	diff 끄 (기본)
alphablend	Ctrl+2	A와 B를 알파 블렌딩
abs	Ctrl+3	픽셀 절대차 $ A-B $
rel	Ctrl+4	상대차
enhance	Ctrl+5	$[min,max] \rightarrow [128,255]$ 리맵
falsecolor	Ctrl+6	의사색 오차맵
ssim	Ctrl+7	SSIM 구조 유사도 히트맵
flip	Ctrl+0	FLIP 지각 오차맵 (magma)
signed	Ctrl+1	부호 있는 차 (파랑 = $A < B$, 빨강 = $A > B$)
highlight	Ctrl+9	임계 초과 픽셀 강조

```
av -d orig.png comp.png                # abs diff로 바로 시작
av --diff-mode signed --amplify 8 orig.png comp.png  # 부호차를 8배 증폭
av --diff-mode flip orig.png comp.png    # 지각 오차맵으로 시작
```

예시 — 보상 IP의 미세 편향을 첫 화면에서 잡아내기

DDI 보상 결과는 원본 대비 차이가 1~2 LSB에 그치는 경우가 많아 절대차를 그대로 보면 새까맣게 보입니다. 이때 `signed` 모드에 증폭을 걸어 시작하면 화면을 열자마자 편향의 **방향**이 드러납니다.

```
av --diff-mode signed --amplify 20 orig/f001.png comp/f001.png
```

패널 오른쪽 Diff 창이 전반적으로 푸르면 보상 결과가 원본보다 밝고($A < B$), 붉으면 어둡다($A > B$)는 뜻입니다. 회색이면 차이가 0입니다. 자세한 모드별 해석은 4장을 보십시오.

1.5.6. 다른 장에서 다루는 시작 옵션

아래 옵션들도 시작할 때 지정하지만, 기능 설명이 긴 만큼 해당 장에서 다룹니다.

옵션	참조
<code>--blk</code> <code>--num_blk</code> <code>--blk-metric</code> <code>--grid</code>	5장 (세 영상 블록 비교)

<code>--pair</code>	7장 (이미지 시퀀스와 파일 입출력)
<code>--metrics</code> <code>--format</code> <code>--fail-psnr</code> <code>--warn-psnr</code> <code>--fail-ssim</code> <code>--fail-flip</code> <code>--fail-maxerr</code> <code>--batch</code> <code>--comp-out</code> <code>--comp-batch</code> <code>--diff-out</code> <code>--sbs</code> <code>--validate</code> <code>--probe</code> <code>--uniformity</code> <code>--fail-uniformity</code> <code>--fail-semu</code>	8장 (헤드리스 CLI와 CI 연동)
<code>--profile</code> <code>--lut</code> <code>--cdl</code> <code>--no-color-mgmt</code>	9장 (색 관리와 룩)

1.6. 화면 구성

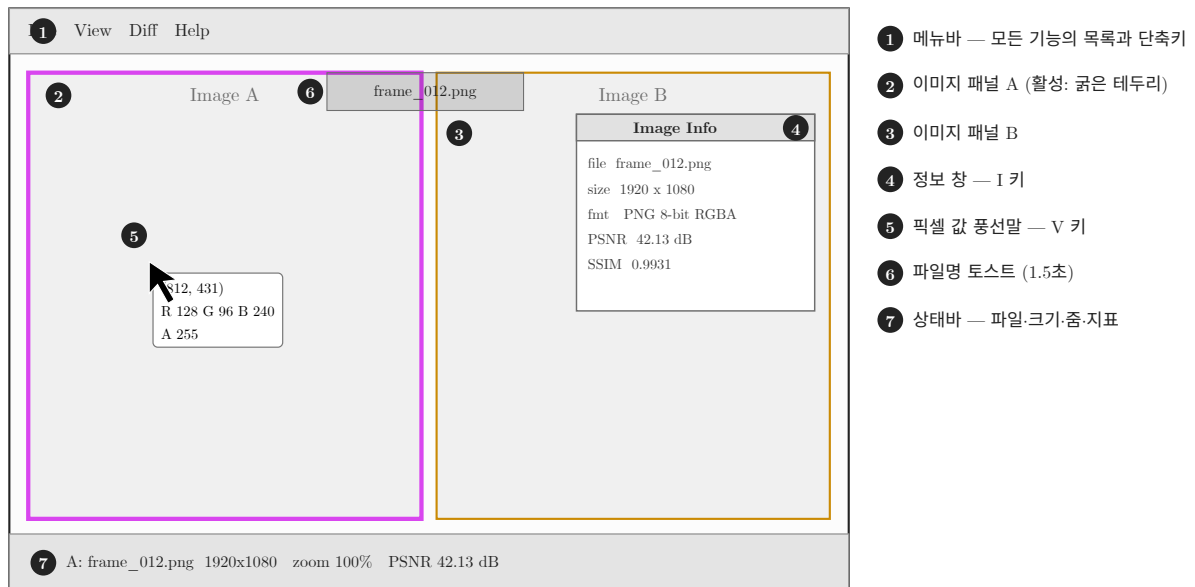


그림 1.2: av 창의 구성 요소. 활성 패널은 테두리가 굵게 강조된다.

기본 상태에서는 이미지가 화면 전체를 차지하고 **메뉴바와 상태바는 숨겨져 있습니다**. `U`를 누르면 위쪽에 메뉴바, 아래쪽에 24픽셀 높이의 상태바가 나타납니다. 이 표시 여부는 `~/.av.ini`의 `show_ui=`에 저장되므로, 한 번 켜 두면 다음 실행에도 유지됩니다. 메뉴바 오른쪽 끝에는 항상 `NN fps | U: hide UI`가 표시되어 다시 숨기는 방법을 알려 줍니다.

1.6.1. 메뉴바 — File / View / Diff

File 메뉴:

항목	단축키	동작
Open Image A...	—	A 패널에 파일 열기
Open Image B...	—	B 패널에 파일 열기
Open Image...	<code>Shift+Cmd+O</code>	현재 활성 패널에 열기
Save Images...	<code>Shift+Cmd+S</code>	A/B/Diff 저장 다이얼로그 토글 (7장)
Quit	<code>Q</code>	종료

View 메뉴 (표시·분석 창 토글이 모여 있습니다):

항목	단축키	참조
Fit to Window	—	로드된 패널을 창 맞춤으로. 2장
1:1 Pixel	Space	1:1 배율 + 중앙 정렬. 2장
Sync Viewports	S	A/B 줌.팬 동기화 토글. 2장
Pathfinder: Image	P	미니맵 표시. 2장
Pathfinder: Schematic	Ctrl+P	도식형 미니맵. 2장
Show Image Info	I	영상 정보 + PSNR 창. 3장
Show Pixel Info	V	커서 픽셀값 풍선말. 3장
Colorimetry Probe (xy/CCT)	Shift+V	CIE xy·u'v'·CCT 병기. 3장
Show Histogram	Ctrl+H	히스토그램. 6장
Show H-Line Cut	Ctrl+L	수평 절단면. 6장
Show V-Line Cut	Ctrl+Y	수직 절단면. 6장
Show Statistics	Ctrl+S	통계 창. 6장
ROI Stats	Ctrl+E	ROI 선택 + 통계. 6장
Scatter Plot	Ctrl+T	A vs B 산점도. 6장
Overlay/Blend	O	겹쳐 보기. 하위에 Blend/Curtain 모드와 알파 슬라이더. 4장
Show Borders	B	패널 테두리 토글
Pixel Format	Ctrl+X	하위 메뉴 <code>Decimal (128)</code> / <code>Hex (0x80)</code> / <code>Hex (80h)</code> . 3장
Hotkey Reference	Ctrl+Shift+H	전체 단축키 목록 창

Diff 메뉴는 앞서 표로 정리한 10개 모드(`Off` , `Alpha Blend` , `Absolute` , `Relative` , `Enhance` , `FalseColor` , `SSIM` , `FLIP` , `Signed` , `Highlight`)를 각자의 단축키와 함께 나열하고, 맨 아래에 `Amplify` 슬라이더(0.5~50)를 둡니다. 이 메뉴는 소스의 `kDiffModes` 테이블 하나에서 생성되므로 메뉴·상태바·창 제목·CLI 토큰이 어긋날 일이 없습니다.

참고 `Show Histogram` / `Show H-Line Cut` / `Show V-Line Cut` / `Show Statistics` 네 창은 서로 배타적입니다. 하나를 켜면 나머지 셋이 자동으로 닫힙니다.

1.6.2. 이미지 패널 — A / B / Diff

레이아웃은 로드된 영상 수와 모드에 따라 자동으로 결정됩니다.

상황	패널 배치
1장	단일 패널 (창 전체)
2장, diff 꺼짐	A B (좌우 반반)
2장, diff 켜짐	A B Diff (3등분)
<code>--comp</code>	orig img2 img3 (3등분), diff 강제 해제
Overlay 모드 (<code>O</code>)	단일 패널에 A와 B를 겹쳐 표시
Blink 모드 (<code>,</code>)	단일 패널에서 A와 B를 자동 교대

패널 테두리 색이 곧 그 패널의 정체입니다. 기본값 기준으로 마젠타 = A(원본), 노랑 = B(비교), 시안 = Diff입니다.

1.6.3. 상태바에 표시되는 정보

상태바는 왼쪽부터 순서대로, **해당 항목이 활성화일 때만** 나타납니다.

항목	내용
Zoom: 400%	활성 패널의 현재 배율 (항상 표시)
A: 1920x1080	A 영상 해상도. 미로드 시 A: 뒤에 회색 대시만 표시
B: 1920x1080	B 영상 해상도
Diff: Abs (A-B) x8.0	diff 모드 태그, 방향(스왑 시 B-A), 증폭 배율
SSIM: 0.9987	SSIM 모드일 때 점수. 계산 중이면 computing... 0.99 초과 = 초록, 0.90 초과 = 노랑, 그 이하 = 빨강
FLIP: 0.0123	FLIP 모드일 때 평균 오차(낮을수록 좋음). 0.05 미만 = 초록, 0.15 미만 = 노랑
P2 42.1dB P3 39.8dB W2 120 W3 86 T 34	--comp 전용. img2/img3의 원본 대비 PSNR과 블록 승/패/무 집계 (5장)
PSNR: 41.2 dB	diff 모드일 때 자동 계산된 PSNR. 40dB 초과 = 초록, 30dB 초과 = 노랑, 그 이하 = 빨강. 동일 영상이면 PSNR: inf
[Diff: 812 / 2073600 px]	Highlight 모드에서 임계 초과 픽셀 수 / 전체
Thr: 4 (0.4% exceed)	임계값과 초과 비율
A [12/240]	시퀀스 위치 (현재 인덱스 / 전체 개수). A·B 각각 표시
ROI mode [x,y WxH]	ROI 선택 모드와 현재 영역
Overlay:Blend 50%	Overlay 모드와 블렌드 비율
▶ 와 1.4s / 3.0s	슬라이드쇼 잔여 시간 / 간격
● 와 BLINK A 0.50s	블링크 현재 위상과 간격
Crosshair	크로스헤어 오버레이 켜짐
Sync: on	줌.팬 동기화 상태 (항상 표시)
60 fps	오른쪽 끝에 정렬된 프레임 레이트

팁 검증 결과를 캡처할 때는 U로 상태바를 켜 채 찍으십시오. 해상도·diff 모드·PSNR·시퀀스 인덱스가 한 줄에 다 들어가므로, 캡처 이미지 자체가 “어떤 조건에서 얻은 화면인지”를 증명하는 메타데이터가 됩니다.

1.6.4. 파일명 토스트 (1.5초)

이미지를 로드하거나 시퀀스를 넘길 때마다, 화면 **상단 중앙**에 어두운 둥근 배지가 떠서 A: frame_0001.png처럼 패널 라벨과 파일명(basename)을 1.5초간 보여 준 뒤 사라집니다. 로드 실패하면 배경이 붉게 바뀌고 A (failed): frame_0001.png로 표시됩니다.

이 토스트가 필요한 이유는 기본 실행이 테두리 없는 창이라 타이틀바가 없기 때문입니다. ; / A로 240장짜리 시퀀스를 빠르게 넘길 때, 지금 몇 번 파일을 보고 있는지 눈으로 확인하는 유일한 수단입니다(정확한 인덱스는 상태바의 A [12/240]로 확인).

참고 A/B 라벨은 데이터 슬롯이 아니라 **화면상의 위치** 기준입니다. Shift+Space로 A와 B를 스왑한 상태라면 슬롯 0의 영상이 오른쪽에 있으므로 토스트도 B로 표시됩니다.

1.7. 지원 이미지 포맷

디렉토리 시퀀스 스캔과 파일 열기가 인식하는 확장자는 다음 8종입니다.

.png .jpg .jpeg .bmp .tga .pgm .ppm .hdr

- **PNG** — 8/16비트, 알파 지원
- **JPEG** — EXIF 방향 자동 적용
- **BMP, TGA** — 기본 지원 (TGA는 알파 채널 포함)
- **HDR** — Radiance RGBE. 내부적으로 float(RGBA32F)로 유지되어 NaN/Inf 검사(1) 대상이 됨
- **PGM/PPM** — P2/P3(ASCII)와 P5/P6(바이너리)를 자체 파서로 읽으며, 10비트·12비트 등 `maxval` 이 255가 아닌 파일도 **원본값 그대로** 보존합니다. DDI 검증에서 10비트 패널 데이터를 다룰 때 특히 중요합니다.

포맷별 비트 심도, 픽셀값 표시 우선순위(PPM 원본값 → HDR float → LDR uint8), 저장 시 선택 가능한 형식은 3장과 7장에서 자세히 다룹니다.

1.8. 종료하기

방법	동작
<code>Q</code>	즉시 종료. 확인 창 없음
메뉴 File → <code>Quit</code>	<code>Q</code> 와 동일
창 닫기 버튼 / <code>Cmd+Q</code> 등 OS 종료	<code>SDL_EVENT_QUIT</code> 또는 <code>SDL_EVENT_WINDOW_CLOSE_REQUESTED</code> 수신 후 종료

종료 직전에 `save_app_ini()` 가 호출되어 현재 설정이 `~/.av.ini` 에 기록됩니다. 저장되는 항목은 테두리 표시, 픽셀값 형식, 돋보기, 크로스헤어, 동기화, UI 표시, 각 분석 창의 열림 상태, 채널 모드, 창 모드, 오버레이, 초기 zoom 등입니다 (10장 참조).

1.8.1. Esc 키는 종료 키가 아닙니다

`Esc` 는 **열려 있는 것을 하나씩 닫는** 키입니다. 아래 순서대로 검사해서 켜져 있는 첫 항목 하나만 끄고 멈춥니다. 따라서 창을 여러 개 띄웠다면 `Esc` 를 여러 번 눌러야 모두 닫힙니다.

1. 블링크 모드 (끄면서 이전 sync 상태로 복원)
2. Diff 픽셀 리스트 창
3. 핫키 도움말 창
4. 히스토그램 → H-Line Cut → V-Line Cut → 통계 창
5. ROI 통계 창 → 산점도 창
6. Image Info 창 → 픽셀값 풍선말
7. 저장 다이얼로그 → 크로스헤어
8. ROI 선택 모드 → Overlay 모드

아무것도 열려 있지 않으면 `Esc` 는 아무 일도 하지 않습니다.

주의 `Ctrl+C` → `1/2/3` 클립보드 복사 모드가 활성화일 때는 `Esc` 가 그 모드를 취소하는 용도로 먼저 소비됩니다 (5초 후 자동 취소). 3장 참조.

1.9. 실무 시작 시나리오

예시 — DDI 보상 IP 회귀 검증 첫 5분

새 RTL 빌드가 나왔고, 원본 240프레임과 보상 결과 240프레임이 각각 `/data/ddi/orig` , `/data/ddi/hw` 에 있다고 합시다.

1단계 — **눈으로 먼저 본다.** 8배 확대에 부호차 diff, 20배 증폭으로 시작합니다.

```
av --zoom 8 --diff-mode signed --amplify 20 -p 16 \
/data/ddi/orig/f001.png /data/ddi/hw/f001.png
```

U로 상태바를 켜면 PSNR: 41.2 dB 가 바로 보입니다. **Shift+l**을 누를 때마다 16픽셀(= 블록 하나)씩 이동하며 블록 경계를 훑고, **;**로 다음 프레임으로 넘기면 상단 토스트가 A: f002.png 를 알려 줍니다.

2단계 — FW 모델과 HW 결과를 동시에 본다. 어느 쪽이 원본에 가까운지 블록 단위로 판정합니다(5장).

```
av --comp --blk 16x16 --num_blk 24 \
/data/ddi/orig/f001.png /data/ddi/fw/f001.png /data/ddi/hw/f001.png
```

3단계 — 수치로 못을 박는다. 240프레임 전체를 헤드리스로 돌려 CSV로 남기고, PSNR 게이트를 걸어 CI에서 자동 판정합니다(8장).

```
av --pair --metrics --fail-psnr 38 \
/data/ddi/orig/f001.png /data/ddi/hw > report.csv
```

팁 자주 쓰는 조합은 셸 alias로 박아 두십시오. 예를 들어 alias avd='av --zoom 8 --diff-mode signed --amplify 20 -p 16' 처럼 만들어 두면 avd orig.png comp.png 한 줄로 검증 환경이 재현됩니다. --zoom 은 어차피 ~/.av.ini 에 저장되지만, alias로 명시해 두면 다른 사람의 머신에서도 같은 화면이 나온다는 장점이 있습니다.

2. 뷰 조작 — 줌 · 팬 · 패널

이 장에서는 av의 뷰포트 모델(줌 · 팬 · fit)과 그것을 움직이는 모든 키보드 · 마우스 조작을 다룹니다. 이어서 패널 운용(활성 패널 전환, 동기화, 스왑, 회전, 테두리), 정밀 검사 보조 도구인 Magnifier · Crosshair · Pathfinder, 그리고 두 영상을 한 패널에 겹쳐 보는 Overlay/Blend · Curtain 모드를 설명합니다. DDI 보상 IP 출력을 원본과 비교할 때 “어디를 얼마나 확대해서 볼 것인가”를 결정하는 것이 이 장의 전부입니다.

2.1. 뷰포트 모델

패널마다 독립된 뷰포트 상태가 하나씩 있습니다. 뷰포트는 다음 네 값으로 완전히 기술됩니다.

필드	단위 · 범위	의미와 기본값
zoom	배율, 0.125 ~ 256	표시 배율. 키보드·휠 줌은 12단계 목록 위에서만 움직입니다
pan_x	이미지 픽셀	화면 중심에서의 가로 오프셋. 값이 커지면 뷰가 왼쪽을 봅니다. 기본 0
pan_y	이미지 픽셀	화면 중심에서의 세로 오프셋. 값이 커지면 뷰가 위쪽을 봅니다. 기본 0
fit	on off	창 맞춤 모드. 기본 on — 매 프레임 패널 크기에 맞춰 배율을 다시 계산합니다

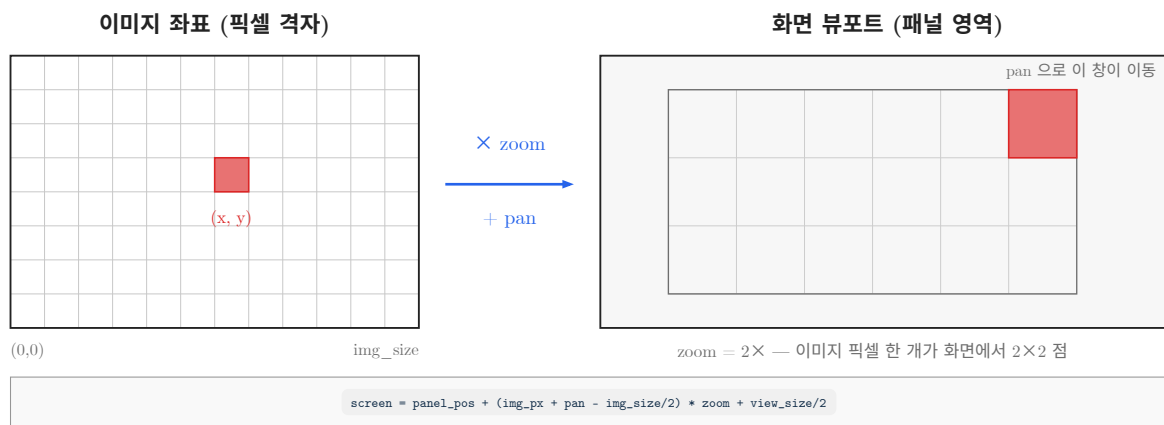


그림 2.1: 이미지 좌표에서 화면 좌표로의 변환. 줌은 배율을, 팬은 평행이동을 담당한다.

2.1.1. 줌 단계는 2의 거듭제곱 12단계입니다

0.125 0.25 0.5 1 2 4 8 16 32 64 128 256

+/-, Z/X, 마우스 휠은 모두 이 목록에서 한 칸씩 이동합니다. 따라서 픽셀 격자가 항상 정수배로 정렬되어, 8x8 · 16x16 블록 단위로 동작하는 보상 IP의 블록 경계를 어긋남 없이 볼 수 있습니다. 배율은 항상 0.125 ~ 256 범위로 클램프됩니다.

fit 모드는 예외입니다. fit은 “패널 폭/이미지 폭”과 “패널 높이/이미지 높이” 중 작은 값을 그대로 쓰므로 0.42 처럼 12단계에 없는 연속값이 됩니다. 이 상태에서 줌 인/아웃을 하면 가장 가까운 단계로 스냅된 뒤 한 칸 이동합니다.

2.1.2. 팬은 줌과 무관하게 이미지 픽셀 단위입니다

키보드 팬 1스텝은 화면 픽셀이 아니라 **이미지 픽셀** 1개입니다. 즉 4x 줌에서 H를 한 번 누르면 화면상으로는 4픽셀이 움직입니다. 반대로 fit 상태처럼 배율이 1보다 작으면 한 번 눌러도 화면에서는 1픽셀도 움직이지 않은 것처럼 보일 수 있습니다.

팬 값은 매 프레임 클램프되어, 이미지가 패널 안에 최소 50화면픽셀은 남도록 제한됩니다. 실수로 이미지를 화면 밖으로 완전히 날려 버릴 수 없습니다.

2.1.3. 화면에 나타나는 피드백

- **Zoom HUD:** 배율이 바뀌면 화면 상단 중앙에 **8x** 같은 알약 모양 배지가 뜨고 5초 뒤 서서히 사라집니다 (마지막 1초 동안 페이드). 배율이 1 미만이면 **0.50x** 처럼 소수 둘째 자리까지 표시합니다.
- **상태바:** **Zoom: 400%** 와 **Sync: on|off** 가 항상 보입니다. 단 상태바와 메뉴바는 기본이 **숨김**이므로 **U**로 먼저 켜야 합니다.
- 상태바의 배율은 **활성 패널**의 배율입니다. 활성 패널은 **Tab**으로 바꿉니다.

주의 키보드 줌.팬은 활성 패널이 아니라 **언제나 왼쪽 패널 A의 뷰포트**에 적용된 뒤, 동기화(**S**, 기본 on)가 켜져 있을 때만 오른쪽 패널 B로 복사됩니다. **--no-sync** 상태에서 **Tab**으로 B를 활성화해도 키보드로는 B를 움직일 수 없습니다 — 이때는 B 패널 위에 마우스를 올리고 휠.드래그를 쓰십시오. 마우스 조작은 항상 **커서 아래 패널**에 적용됩니다.

2.2. 줌 조작 전수

2.2.1. 키보드

키	동작
+ · = · Numpad+	한 단계 확대 (×2). = 와 + 는 같은 물리 키라 Shift 유무와 무관합니다
- · Numpad-	한 단계 축소 (÷2)
Z	한 단계 확대 (+ 와 동일)
Shift+Z	한 단계 축소
X	한 단계 축소 (Z/X 쌍으로 왼손 조작)
0	1:1 (100%) + 중앙 정렬. $2^{-0} = 1$ 배율입니다
1~8	2^{-n} 고정 배율 + 중앙 정렬. 1 =2×, 2 =4×, 3 =8×, ... 8 =256×
9	줌 동작 없음 (2^{-9} 은 범위 밖이라 무시)
F	fit 토글. 커먼 창에 맞추고, 끄면 직전 배율로 돌아갑니다
Space	1:1 (100%) + 중앙 정렬. 0 과 같은 동작

주의 **0**은 **fit**이 아니라 **1:1(100%)** 입니다. 앱 안의 Hotkey Reference (**Ctrl+Shift+H**) 표와 View 메뉴의 단축키 힌트에는 **0**이 “Fit to Window”, **1**이 “1:1 Pixel”로 적혀 있지만, 실제 키 처리는 2^{-n} 규칙을 따릅니다(**1**은 2×). fit이 필요하다면 **F** 또는 메뉴의 **View → Fit to Window** 항목을 쓰십시오 — 메뉴 항목 자체는 이름대로 정확히 동작합니다.

참고 숫자 키가 줌으로 동작하려면 두 가지 조건이 필요합니다.

첫째, **Ctrl**을 같이 누르지 않아야 합니다 — **Ctrl**+숫자는 diff 모드 전환입니다 (4장 참조).

둘째, 클립보드 복사 모드가 아니어야 합니다 — **Ctrl/Cmd+C** 직후 5초 동안은 **1/2/3**이 각각 Image A/B/Diff 복사로 가로채집니다. **4~8**을 누르면 복사 모드가 취소되면서 원래의 줌 동작이 그대로 수행됩니다. **Esc**로도 취소됩니다.

또한 줌에 쓰이는 숫자는 **상단 숫자열**뿐입니다. 넘패드 숫자는 줌으로 동작하지 않습니다 (넘패드는 Numpad+ / Numpad- 만 줌에 연결되어 있습니다).

주의 `X`는 줌 아웃이지만 `Ctrl+X`는 픽셀값 표기 형식 순환(Dec → 0x80 → 80h)입니다 (3장 참조). `Shift+X`나 `Cmd+X`는 줌 아웃으로 동작합니다.

2.2.2. 마우스

조작	동작
스크롤 휠	커서 아래 패널을 한 단계 확대/축소. 동기화가 켜져 있으면 반대편 패널의 배율도 따라갑니다
<code>Alt</code> + 휠	줌이 아니라 시퀀스 탐색입니다 (7장 참조)
우클릭 드래그	영역 줌 . 노란 테두리의 선택 사각형이 그려지고, 버튼을 놓으면 그 영역이 들어가는 가장 큰 2^{-n} 배율로 스냅하며 선택 중심이 화면 중심에 옵니다
좌 더블클릭	커서 지점을 중심으로 $\times 2$ 확대
우 더블클릭	커서 지점을 중심으로 $\div 2$ 축소
<code>Ctrl</code> + 좌 더블클릭	커서 지점을 중심으로 $\times 16$ 확대 (한 번에 픽셀 레벨까지)
<code>Ctrl</code> + 우 더블클릭	커서 지점을 중심으로 $\div 16$ 축소
우클릭 짧게 누르기	드래그 폭이 5픽셀 미만이고 0.5초 이상 누르면 컨텍스트 메뉴(Save As... / Copy to Clipboard)가 열립니다

영역 줌은 “이 무라 덩어리만 보고 싶다”처럼 관심 영역이 화면에 이미 보일 때 가장 빠릅니다. 배율을 계산할 필요 없이 드래그 한 번으로 정수배 확대가 끝납니다.

2.2.3. 시작 배율 지정과 유지

옵션	설명
<code>--zoom fit</code>	창 맞춤으로 시작 (기본값)
<code>--zoom 1</code>	1:1(100%)로 시작
<code>--zoom 4</code>	4× 고정 배율로 시작

`--zoom`에 0보다 큰 값을 주면 그 배율이 **영상을 새로 로드할 때마다** 다시 적용됩니다. 즉 `;` / `A`로 시퀀스를 넘겨도 배율이 fit으로 되돌아가지 않고 고정 배율이 유지됩니다 (7장 참조). 이 값은 `av.ini`에 `zoom=`으로 저장되므로, CLI에 `--zoom`을 주지 않으면 지난번 값이 그대로 재현됩니다.

예시 — 저계조 밴딩을 픽셀 단위로 확인하기

DDI 보상 전/후 그레이 램프에서 밴딩 계단이 몇 픽셀 폭인지 세어 보는 경우입니다.

```
av orig.png comp_out.png --zoom 4
```

- 4× 상태로 뜹니다. 밴딩 의심 구간이 화면에 보이면 그 위를 **우클릭 드래그**로 감쌉니다 — 예컨대 32× 정도로 스냅됩니다.
- 화면 상단에 `32x` HUD가 5초간 뜨고, 상태바에는 `Zoom: 3200%`가 표시됩니다.
- 32× 이상에서는 픽셀 격자와 픽셀값이 자동으로 그려집니다 (3장 참조).
- 원래 그림 전체를 다시 보려면 `F`, 픽셀 등배로 돌아가려면 `0`입니다.

2.3. 팬 조작 전수

키 · 조작	동작
<code>H</code> · <code>←</code>	왼쪽으로 1픽셀 이동
<code>L</code> · <code>→</code>	오른쪽으로 1픽셀 이동
<code>K</code> · <code>↑</code>	위쪽으로 1픽셀 이동
<code>J</code> · <code>↓</code>	아래쪽으로 1픽셀 이동
<code>Shift</code> + 위 8키	<code>pan_step</code> 픽셀만큼 빠르게 이동 (기본 32)
<code>Cmd+Shift+H</code>	왼쪽 끝단으로 점프
<code>Cmd+Shift+L</code>	오른쪽 끝단으로 점프
<code>Cmd+Shift+K</code>	위쪽 끝단으로 점프
<code>Cmd+Shift+J</code>	아래쪽 끝단으로 점프
<code>G</code>	이미지 중심으로 복귀 (<code>pan_x = pan_y = 0</code>)
좌클릭 드래그	커서 아래 패널을 자유롭게 이동. 드래그 양은 배율로 나뉘어 이미지 픽셀로 환산됩니다

2.3.1. 팬 스텝 바꾸기 — `-p`

```
av orig.png comp_out.png -p 8
```

`-p` (= `--pan-step`)는 `Shift` + 이동의 한 걸음 크기를 이미지 픽셀 단위로 지정합니다. 기본값은 32입니다.

팁 보상 IP의 블록 크기와 `-p` 값을 맞추면 `Shift+L`을 누를 때마다 정확히 블록 하나씩 이동합니다. 8x8 블록 알고리즘을 검증할 때 `-p 8`, 16x16이면 `-p 16`으로 두고 `Shift+H`/`Shift+L`로 블록 열을 훑으면 블록 경계 아티팩트를 놓치지 않습니다. 블록 단위 비교 자체는 5장(`--comp`)을 보십시오.

참고 끝단 점프는 `Cmd` (macOS) 또는 `Win` (Windows · Linux) 키와 `Shift`를 함께 누른 `H`/`J`/`K`/`L`에만 연결되어 있습니다. 방향키에는 끝단 점프가 없습니다 — `Cmd+Shift+←`는 그냥 `pan_step` 이동입니다. 또 `Ctrl+Shift+H`는 팬이 아니라 Hotkey Reference 창 토글이므로 주의하십시오.

주의 `Shift+↑`/`Shift+↓`는 슬라이드쇼가 재생 중일 때만 재생 간격 조절로 바뀝니다 (7장 참조). 슬라이드쇼가 꺼져 있으면 평소대로 `pan_step` 세로 이동입니다. 좌우는 언제나 팬입니다.

주의 `--comp` (3영상 블록 비교) 모드에서 `G`는 **이미지 중심 복귀가 아니라 블록 그리드 경계선 토글**입니다. 이 모드에서 중앙으로 돌아가려면 `0`이나 `Space` (1:1 + 센터)를 쓰십시오. 자세한 내용은 5장을 참조하십시오.

예시 — 보상 결과의 좌측 끝 열을 검사하기

패널 경계 근처의 보상 오차는 화면 끝단에 몰려 있는 경우가 많습니다.

```
av orig.png comp_out.png --zoom 8 -p 8
```

1. `Cmd+Shift+H`로 이미지의 가장 왼쪽 열까지 즉시 이동합니다.
2. `Cmd+Shift+K`로 위쪽 끝단까지 올라갑니다 — 이제 좌상단 모서리입니다.

3. **Shift+J**를 반복해 8픽셀(=블록 한 줄)씩 아래로 내려가며 훑습니다.
4. 동기화가 켜져 있으므로 원본 패널과 보상 결과 패널이 같은 좌표를 봅니다.
5. 전체 그림으로 복귀할 때는 **G** 다음 **F**.

2.4. 패널 운용

두 영상을 열면 화면이 좌(A) · 우(B)로 나뉘고, diff 모드에서는 A | B | Diff 3분할, `--comp`에서는 orig | img2 | img3 3분할이 됩니다. 레이아웃 자체는 1장 · 4장 · 5장에서 다루고, 여기서는 레이아웃과 무관한 공통 조작만 정리합니다.

키	동작
Tab	활성 패널 전환 (A ↔ B). 활성 패널의 이미지 테두리가 2px → 4px로 굵어집니다
S	줌/팬 동기화 토글. 상태바에 <code>Sync: on off</code> 로 표시됩니다
Shift+Space	A/B 이미지 스왑 (두 장이 모두 로드된 경우에만)
R	로드된 모든 이미지를 시계 방향 90도 회전
Ctrl+R	반시계 방향 90도 회전
B	패널 테두리 표시 토글
W	윈도우 모드 토글 (타이틀바 표시 ↔ 테두리 없는 최대화)
U	UI 오버레이(메뉴바 + 상태바) 토글

2.4.1. **Tab** — 활성 패널

활성 패널은 다음 네 가지를 결정합니다.

- 상태바의 `Zoom`: 값과 Zoom HUD가 어느 패널 기준인지
- `;`/**A** 시퀀스 탐색이 어느 패널을 넘길지 (7장 참조)
- **Shift+Ctrl+0** 파일 열기가 어느 패널로 로드할지 (7장 참조)
- 이미지 테두리 굵기(활성 4px / 비활성 2px)

앞서 경고했듯이 **키보드 줌·팬 대상은 바뀌지 않습니다**.

2.4.2. **S** — 동기화

기본값은 **on**이며 CLI로도 지정할 수 있습니다. `--sync`는 명시적으로 켜고, `--no-sync`는 끕니다.

```
av orig.png comp_out.png --sync
av orig.png comp_out.png --no-sync
```

동기화가 켜져 있으면 한쪽 패널의 배율·팬이 그대로 반대쪽에 복사됩니다. 원본과 보상 결과를 같은 좌표에서 비교해야 하는 대부분의 DDI 검증 작업에서는 켜 채로 두는 것이 맞습니다. 두 영상의 해상도가 다르거나 서로 다른 영역을 동시에 보고 싶을 때만 `--no-sync`를 쓰십시오. 이 값은 `av.ini`에 저장됩니다.

주의 `--comp` 모드에서 **S**는 동기화 토글이 아니라 **img2 ↔ img3 패널 위치 스왑**이고, **Tab**은 활성 패널 전환이 아니라 **worst 블록 순회**입니다. 5장을 참조하십시오.

2.4.3. 회전 · 스왑 · 테두리 · 창

- **회전**: **R**/**Ctrl+R**은 활성 패널만이 아니라 **로드된 A · B 두 장 모두**를 회전시키고, 회전 후 두 뷰포트를 fit + 중앙으로 초기화합니다. 세로로 촬영된 패널 캡처를 눕혀 볼 때 유용합니다.
- **스왑**: **Shift+Space**는 A/B의 표시 위치를 바꿉니다. diff의 부호 방향도 **A-B**에서 **B-A**로 뒤집히므로 상태바 표기를 확인하십시오 (4장 참조).

- **테두리:** **B**는 패널 구분 테두리와 이미지 외곽선을 함께 끕니다. 색은 기본적으로 A=마젠타, B=노랑, Diff=시안이며 **-bc**로 바꿉니다. 처음부터 숨기려면 **-nb**입니다.

```
av -bc ff00ff ffff00 00ffff orig.png comp_out.png
av -nb orig.png comp_out.png
```

- **창:** **W**를 누르면 타이틀바가 있는 창 모드가 되고 크기가 **--geometry** (기본 1280x720)로 복원되며 화면 중앙에 배치됩니다. 지정 크기가 화면보다 크면 사용 가능 영역의 80%로 줄여 잡습니다. 다시 누르면 타이틀바 없는 최대화 상태로 돌아갑니다. **--windowed**로 처음부터 창 모드로 띄울 수 있습니다.

```
av --windowed --geometry 2560x1440 orig.png comp_out.png
```

- **UI:** **U**는 메뉴바와 상태바를 켜고 끕니다. **기본값은 숨김**이므로, 처음 실행했을 때 메뉴가 보이지 않는 것은 정상입니다. 스크린샷을 찍을 때는 꺼 두고, 배율·PSNR·시퀀스 위치를 확인할 때는 켜 두면 됩니다.

2.5. Magnifier — **Ctrl+M**

Magnifier는 커서 주변을 확대해 보여 주는 돋보기입니다. **기본값이 켜짐**이며 **Ctrl+M**으로 토글하고, 상태는 **av.ini**에 **magnifier_active**로 저장됩니다.

가장 중요한 성질은 **한 패널이 아니라 화면에 보이는 모든 패널에 동시에 뜬다**는 점입니다. 마우스가 올라간 패널은 “어느 픽셀을 볼지”만 정하고, 나머지 패널은 그 **같은 이미지 좌표**를 자기 픽셀로 확대해 나란히 보여 줍니다. 두 영상 모드면 A·B 두 개, diff를 켜면 Diff까지 세 개, **--comp** 모드면 orig·img2·img3 세 개가 한꺼번에 뜨므로 같은 자리를 눈으로 바로 대조할 수 있습니다.

항목	값 · 동작
확대 영역	커서 픽셀을 중심으로 16x16 이미지 픽셀
표시 대상	보이는 모든 패널 — 각 확대경 좌상단에 A B Diff 또는 orig img2 img3 이름표
확대 배율	패널 폭·높이에 맞춰 자동 (최대 512x512 = 픽셀당 32배, 최소 112px)
표시 위치	각 패널의 가로 중앙. 커서가 패널 위쪽이면 아래 절반, 아래쪽이면 위 절반에 배치되어 커서를 가리지 않음
중심 표시	정중앙 픽셀에 노란 테두리
하단 라벨	(x, y) 좌표 + 그 픽셀의 RGB: r, g, b. Diff 패널에서는 diff: r, g, b
자동 숨김	뷰 배율이 32x 이상이면 표시하지 않습니다 (이미 픽셀이 충분히 크므로)
경계 제한	Ctrl 을 누르고 있는 동안 마우스 커서가 이미지 영역 밖으로 못 나갑니다

라벨의 픽셀값 표기는 **Ctrl+X**로 고른 형식(10진 / 0x80 / 80h)을 따르고, PPM/PGM 원본값이 있으면 8비트로 뭉개지 않은 원본값을 보여 줍니다 (3장 참조). 채널 단독 보기(**Shift+R** / **Shift+G** / **Shift+B**) 중에는 해당 채널 값만 **R: 128**처럼 표시됩니다.

팁 **Ctrl** 홀드의 경계 제한은 이미지 **맨 가장자리 픽셀**을 읽을 때를 위한 기능입니다. 보상 IP는 경계 픽셀 처리에서 오차가 커지기 쉬운데, 제한 없이 마우스를 움직이면 커서가 이미지 밖으로 미끄러져 값이 사라집니다. **Ctrl**을 누른 채 가장자리로 밀면 커서가 이미지 안에 붙잡히고 마지막 유효 픽셀로 클램프되어 0번 열/행의 값을 안정적으로 읽을 수 있습니다.

예시 — 보상 오차가 가장 큰 픽셀 주변을 찾기

```
av orig.png comp_out.png --diff-mode abs --amplify 8
```

1. Diff 패널에서 밝게 튀는 지점을 찾습니다 (4장 참조).
2. Magnifier가 켜져 있으므로 커서를 올리기만 하면 16x16 이웃이 크게 확대되어 A·B(그리고 Diff) 패널에 동시에 뜹니다 — 같은 좌표를 바로 대조할 수 있습니다.
3. 툴팁 아래 `diff: 3, 0, 1` 처럼 중심 픽셀의 채널별 차이가 그대로 보입니다.
4. **Ctrl**을 누른 채 가장자리까지 밀어 경계 픽셀도 확인합니다.
5. 더 넓게 보려면 **Ctrl+M**으로 툴팁을 끄고 **5** (32×)로 직접 확대합니다 — 32× 이상에서는 Magnifier가 어차피 자동으로 숨겨집니다.

2.6. Crosshair — **M**

M은 십자선 오버레이를 켭니다. 커서가 있는 패널에 패널 전체를 가로지르는 반투명 노란 십자선이 그려지고, 커서 옆에 `(x, y)` 좌표와 그 픽셀의 값이 함께 표시됩니다. Diff 패널에서는 차이값이 표시됩니다. 상태바에 **Crosshair** 표시가 뜨며, 상태는 `av.ini`에 `show_crosshair`로 저장됩니다 (기본 꺼짐).

십자선은 “두 패널에서 같은 스캔라인을 보고 있는가”를 눈으로 확인할 때 편리합니다. 가로 라인 프로파일 일을 실제로 그려 보려면 **Ctrl+L** (H-Line Cut)을 쓰십시오 (6장 참조).

2.7. Pathfinder — **P · **Ctrl+P****

Pathfinder는 확대 상태에서 “지금 전체 이미지의 어디를 보고 있는지”를 알려 주는 미니맵입니다.

키	모드
P	Image 모드 — 이미지 썸네일 위에 현재 뷰 영역을 노란 테두리로 표시 (기본값)
Ctrl+P	Schematic 모드 — 썸네일 없이 회색 박스 위에 뷰 영역을 반투명 노랑으로 채워 표시

같은 키를 다시 누르면 꺼집니다(모드 0). 미니맵은 왼쪽 패널의 좌하단에 8픽셀 여백을 두고 그려지며, 폭은 패널 폭의 20% 또는 150픽셀 중 작은 쪽, 높이는 패널 높이의 30%를 넘지 않습니다.

참고 Pathfinder는 **fit** 상태에서는 **표시되지 않습니다**. 전체가 다 보이는 상황에서는 미니맵이 의미가 없기 때문입니다. **F**로 fit을 끄거나 줌을 하면 나타납니다. 또한 왼쪽 패널에만 그려집니다.

2.8. Overlay/Blend — **O**

O는 두 영상을 좌우로 나란히 두는 대신 **한 패널에 겹쳐** 보여 주는 모드입니다. 두 장이 모두 로드되어 있어야 하며, 켜면 화면이 1패널로 합쳐집니다. 상태는 `av.ini`에 `overlay_active`로 저장됩니다.

모드는 두 가지이고 메뉴 **View → Overlay/Blend** 아래에서 고릅니다 (메뉴바가 보이려면 **U**로 UI를 켜야 합니다).

모드	동작
Blend	A와 B를 alpha 비율로 섞습니다. <code>0.00</code> = A만, <code>1.00</code> = B만, 기본 <code>0.50</code> . 메뉴의 <code>Blend: 0.50</code> 슬라이더로 조절합니다
Curtain	분할선을 기준으로 왼쪽은 A, 오른쪽은 B를 보여 줍니다. 분할선 위에 <code>A B</code> 라벨이 표시되며 기본 위치는 화면 폭의 50%

Curtain 모드에서는 **좌클릭 드래그로 분할선을 좌우로 움직입니다**. 분할선 위치는 0~1 범위로 클램프됩니다.

상태바에는 `Overlay:Blend 50%` 또는 `Overlay:Curtain 50%` 처럼 현재 모드와 alpha가 표시됩니다.

주의 Curtain 모드에서는 좌클릭 드래그가 팬이 아니라 분할선 이동으로 쓰입니다. 이 모드에서 화면을 이동하려면 키보드 팬(`H`/`J`/`K`/`L`)을 쓰거나, 잠시 Blend 모드로 돌아가십시오. 우클릭 영역 줌도 Curtain 모드에서는 동작하지 않습니다 (휠 줌은 됩니다).

참고 `--comp` 3영상 모드에서는 Overlay가 매 프레임 강제로 꺼집니다. diff · 블링크 · A/B 스왑도 마찬가지입니다 (5장 참조).

예시 — Curtain으로 보상 경계 확인하기

```
av orig.png comp_out.png
```

1. `O`로 Overlay를 켭니다 — 한 화면에 A와 50% 혼합으로 겹쳐 보입니다.
2. `U`로 메뉴바를 켜고 **View → Overlay/Blend → Curtain mode**를 고릅니다.
3. 화면 중앙에 흰 세로선과 `A | B` 라벨이 나타납니다.
4. 분할선을 좌우로 천천히 끌면 같은 좌표에서 원본과 보상 결과가 교대로 드러나 계조 이동이나 색 틀어짐이 즉시 눈에 띕니다.
5. 전역 밝기 차이를 보고 싶다면 Blend 모드로 돌아가 슬라이더를 `0.00` 과 `1.00` 사이에서 왕복시키거나, 자동 교대 비교인 블링크(`,`)를 쓰십시오.

2.9. 영속화되는 뷰 설정

아래 항목은 종료할 때 홈 디렉토리의 `av.ini` (실제 경로는 `~/.av.ini`)에 저장되어 다음 실행에 그대로 복원됩니다.

설정 키	토글 키	기본값 · 의미
<code>sync_viewports</code>	<code>S</code>	<code>1</code> — 패널 줌/팬 동기화
<code>show_borders</code>	<code>B</code>	<code>1</code> — 패널 테두리 표시
<code>show_ui</code>	<code>U</code>	<code>0</code> — 메뉴바 + 상태바 표시
<code>windowed_mode</code>	<code>W</code>	<code>0</code> — 타이틀바 있는 창 모드
<code>magnifier_active</code>	<code>Ctrl+M</code>	<code>1</code> — Magnifier
<code>show_crosshair</code>	<code>M</code>	<code>0</code> — 십자선 오버레이
<code>overlay_active</code>	<code>O</code>	<code>0</code> — Overlay/Blend 모드
<code>zoom</code>	<code>--zoom</code>	<code>0</code> — <code>0</code> =fit, <code>>0</code> =고정 배율

반대로 **저장되지 않는** 값도 알아 두면 좋습니다. 현재 팬 좌표, 현재 배율(고정 배율 설정 `zoom` 과는 별개), 활성 패널, 회전 상태, Pathfinder 모드, Overlay의 alpha와 분할선 위치는 세션이 끝나면 사라집니다. `--zoom`, `-p`, `-bc`, `--geometry`, `--no-sync` 같은 CLI 옵션과 `av.ini`의 우선순위, 파일 형식, 초기화 방법은 10장을 참조하십시오.

3. 픽셀 검사와 색 판독

이 장은 av 의 이름(Advanced Pixel **L**ens)이 가리키는 핵심 기능, 즉 “화면에 보이는 그림”이 아니라 “그 아래 깔린 숫자”를 읽는 방법을 다룹니다. 커서 아래 픽셀의 좌표와 값을 읽는 풍선말, 좀 32배 이상에서 자동으로 커지는 픽셀 그리드와 값 오버레이, 10/12-bit 원본값 보존 규칙, 채널 분리 보기, CIE 색도 판독, 그리고 float/HDR 영상의 결함 픽셀 검출까지를 순서대로 설명합니다. DDI 보상 IP 의 출력이 원본과 LSB 몇 단계나 어긋났는지, 그 어긋남이 색도상 몇 $\Delta u'v'$ 인지를 확인하는 것이 이 장의 실무 목표입니다.

3.1. 픽셀 값 풍선말 — **V**

V 를 누르면 마우스 커서 아래 픽셀의 좌표와 값을 보여 주는 풍선말이 커집니다. 메뉴 `View > Show Pixel Info` 로도 같은 토글을 조작할 수 있습니다. 이 상태는 `$HOME/.av.ini` 의 `show_pixel_info=0|1` 로 영속화되므로, 한 번 켜 두면 다음 실행에서도 그대로 켜진 채 시작합니다.

풍선말은 두 줄로 구성됩니다.

줄	내용
1행	이미지 좌표 <code>(x, y)</code> — 회색 글자. 좌상단이 <code>(0, 0)</code>
2행	<code>R: G: B: 값</code> — 각각 빨강/초록/파랑 글자색

풍선말은 커서 기준 오른쪽 위(`+16, -40` px)에 뜨고, 화면 오른쪽이나 위쪽 경계에 닿으면 자동으로 반대편으로 접힙니다. 글꼴은 전용 대형 폰트(26 px)라서 멀리서도 값을 읽을 수 있습니다.

3.1.1. A/B 동시 표시 규칙

값 자체는 **커서가 올라가 있는 패널 하나만** 표시합니다. 두 영상을 나란히 띄운 상태에서 왼쪽 패널 위에 커서를 두면 A 값이, 오른쪽으로 옮기면 같은 좌표의 B 값이 나옵니다. 뷰포트 동기화(**S**)가 켜져 있으면 두 패널의 같은 화면 위치가 같은 이미지 좌표를 가리키므로, 커서를 좌우로 왕복시키는 것만으로 A/B 값을 즉시 대조할 수 있습니다.

커서 위치	풍선말이 보여 주는 값
A 패널(왼쪽)	A 영상의 해당 픽셀 값
B 패널(오른쪽)	B 영상의 해당 픽셀 값
Diff 패널(3분할 오른쪽)	<code> A - B </code> 채널별 절댓값
<code>--comp</code> 세 번째 패널	<code>img3(comp.img_c)</code> 의 값
<code>--comp</code> 3-way 블링크 중	현재 블링크 단계의 영상 값(<code>orig/img2/img3</code>)

참고 `Shift+Space` 로 A/B 를 스왑한 상태에서도 풍선말은 “지금 그 패널에 실제로 그려지고 있는 영상”의 값을 읽습니다. 화면과 숫자가 어긋나지 않습니다.

3.1.2. diff 모드일 때의 표시

diff 모드(4장 참조)에서는 화면이 `A | B | Diff` 3분할이 되고, Diff 패널 위의 풍선말은 두 영상의 **채널별 절댓값 차이** `|A - B|` 를 표시합니다. 이때 값 계산은 아래 “픽셀 값 우선순위”를 그대로 따르므로, 16-bit PNM 쌍이라면 8-bit 로 눌린 값이 아니라 원본 스케일의 차이가 나옵니다. 예를 들어 10-bit(maxval 1023) 원본 쌍에서 `R:3 G:0 B:1` 이면 R 채널이 3 LSB 어긋났다는 뜻입니다.

주의 Diff 패널 위에서는 `Shift+V` 컬러메트리 줄이 나오지 않습니다. 색도 판독은 A 또는 B 패널 위에서만 동작합니다(뒤의 “컬러메트리 풍선말” 절 참조).

예시 — 보상 IP 출력의 LSB 오차 확인

```
av orig_10bit.ppm comp_10bit.ppm
```

1. `V` 로 풍선말을 겁니다.
2. `5` 로 32배 줌(2^5)까지 확대하고 관심 영역으로 팬합니다.
3. 커서를 왼쪽 패널의 의심 픽셀에 올리면 `(412, 77) / R:640 G:640 B:641` .
4. 커서를 오른쪽 패널의 같은 화면 위치로 옮기면 `R:637 G:640 B:641` .
5. `Ctrl+3` 으로 diff 모드에 들어가 Diff 패널에 커서를 두면 `R:3 G:0 B:0` — R 채널만 3 LSB 낮게 보상되었음이 확정됩니다.

3.2. 픽셀 값 우선순위와 원본값 보존

av 는 한 픽셀에 대해 최대 세 종류의 버퍼를 가질 수 있고, 값을 표시할 때는 항상 같은 우선순위로 고릅니다. 이 규칙은 풍선말, 줌 오버레이, diff 값 표시가 모두 공유합니다.

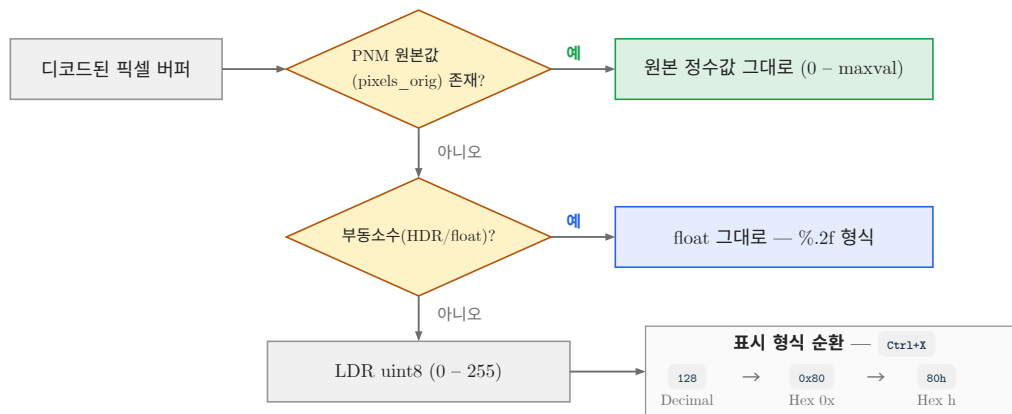


그림 3.1: 픽셀 값 판독 우선순위와 표시 형식 변환. 원본 값 보존이 최우선이다.

순위	버퍼	표시 형식과 조건
1	<code>pixels_orig</code>	PNM 원본값(uint16, RGB 3채널). <code>ppm_maxval > 0</code> 이고 버퍼가 비어 있지 않을 때. 범위 <code>0 ~ maxval</code> (8-bit 255, 10-bit 1023, 12-bit 4095, 16-bit 65535). <code>Ctrl+X</code> 형식 순환 대상
2	<code>pixels_f32</code>	HDR float. 풍선말은 <code>%.3f</code> , 줌 오버레이는 <code>%.2f</code> . 음수나 1 초과 값도 그대로 표시하며, 16진 형식 순환 대상이 아님
3	<code>pixels</code>	LDR uint8 RGBA8. 범위 <code>0 ~ 255</code> . <code>Ctrl+X</code> 형식 순환 대상

3.2.1. 왜 이 순서인가

로더는 PNM(P2/P3/P5/P6)을 **자체 파서**로 읽어 원본 정수값을 `pixels_orig` 에 그대로 보존한 뒤, 화면 표시용 8-bit 텍스처를 `value * 255 / maxval` 로 따로 만듭니다. 즉 10-bit 값 641 은 화면에는 159 로 그려지지만 메모리에는 641 이 남아 있습니다. 만약 우선순위가 표시용 버퍼부터였다면 10-bit 원본에서 1 LSB 차이가 8-bit 로 놀리면서 사라져 버립니다. 그래서 원본값이 있으면 무조건 원본값을 먼저 씁니다.

HDR 을 두 번째에 두는 이유는 float 값에 정수 표기(16진 포함)를 강제하면 정보가 손실되기 때문이고, uint8 을 마지막에 두는 이유는 그것이 항상 존재하는 최후의 fallback 이기 때문입니다.

팁 DDI 보상 IP 검증에서는 톤 곡선/감마 보상이 10~12-bit 도메인에서 일어납니다. 중간 산출물을 8-bit PNG 로 덤프하면 검증하려는 오차 자체가 반올림에 묻히므로, 파이프라인 중간 덤프는 P6 16-bit PNM(또는 maxval 을 실제 비트폭에 맞춘 PNM) 으로 남기는 것을 권장합니다. av 는 그 값을 그대로 읽어 줍니다.

주의 `pixels_f32` 는 HDR(Radiance) 영상과 `maxval > 255` 인 PNM 에만 만들어집니다. PNM 의 float 버퍼는 $\text{원본값} / \text{maxval}$ 로 정규화되어 항상 $0 \sim 1$ 안에 들어가므로, 뒤에 나오는 결함 픽셀 오버레이 (/) 에서는 아무것도 검출되지 않습니다. 이는 정상 동작입니다.

3.3. 줌 32배 이상: 자동 픽셀 그리드와 값 오버레이

확대 배율에 따라 두 단계의 보조 표시가 자동으로 켜집니다. 별도 토클 키는 없습니다.

줌 임계값	자동으로 켜지는 것
<code>zoom >= 16</code>	셰이더가 그리는 은은한 회색 픽셀 경계선. 픽셀 경계를 40 % 세기의 중간 회색으로 섞어 그립니다 (소프트웨어 렌더러도 동일)
<code>zoom >= 32</code>	흰색 픽셀 격자선 + 셀 중앙에 R/G/B 값 3줄 . 매그니파이어는 이 배율부터 자동으로 숨습니다(픽셀이 이미 충분히 큼)

값 글자 크기는 $\min(\text{zoom} * 0.25, 20)$ px 이고, 계산 결과가 6 px 미만이면 값은 그리지 않습니다. 즉 32배에서는 8 px, 80배 이상에서는 20 px 로 고정됩니다. 글자색은 R 이 빨강, G 가 초록, B 가 파랑이며 1 px 검은 그림자가 깔려 밝은 배경에서도 읽힙니다.

채널 모드(다음 절)가 RGB 가 아니면 3줄 대신 해당 채널 한 줄만 그립니다. Diff 패널에서도 같은 격자와 값이 나오며, 값은 $|A - B|$ 입니다.

예시 — 32배 확대에서 블록 경계 아티팩트 읽기

```
av --zoom 32 orig.ppm comp.ppm
```

창이 열리자마자 32배 상태로 시작하므로 격자와 숫자가 바로 보입니다. `--zoom` 은 `fit` (또는 `0`), `1` (`1:1`), `2`, `4`, `8`, `16`, `32` ... 를 받고 내부 줌 단계 `0.125 ~ 256` 범위로 맞춰집니다. 이 값은 `.av.ini` 의 `zoom=` 으로 영속화되므로, 다음 실행 때도 같은 배율로 시작합니다(2장 참조).

3.4. 픽셀 값 표시 형식 순환 — **Ctrl+X**

Ctrl+X 를 누를 때마다 정수 픽셀값의 표기 방식이 3단계로 순환합니다. 풍선말, 줌 오버레이, diff 값 표시가 모두 동시에 바뀝니다.

단계	표기	설명
0	Decimal	128
1	Hex 0x	0x80
2	Hex h	80h

메뉴에서는 `View > Pixel Format` 아래의 `Decimal (128)` / `Hex (0x80)` / `Hex (80h)` 항목으로 직접 고를 수 있습니다(현재 선택 항목에 체크 표시). 선택은 `$HOME/.av.ini` 에 `pixel_format=0|1|2` 로 영속화됩니다.

참고 0 패딩은 “최소” 2자리입니다. 10-bit 원본값 641 은 0x281 처럼 3자리로 그대로 나옵니다. 값이 잘리지 않습니다.

주의 HDR float 값(%.2f / %.3f)은 16진 순환 대상이 아닙니다. HDR 영상에서 **Ctrl+X** 를 눌러도 표시가 변하지 않는 것은 정상입니다.

팁 레지스터 덤프나 RTL 시뮬레이션 로그와 대조할 때는 Hex 0x 또는 Hex h 로 바꿔 두면 눈으로 값을 변환할 필요가 없습니다. DDI 보상 LUT 의 엔트리를 16진으로 관리하는 팀이라면 첫 실행에서 한 번만 바꿔 두면 이후 계속 유지됩니다.

3.5. 채널 선택 — **Shift+R** / **Shift+G** / **Shift+B** / **Shift+Y** / **Shift+C**

한 채널만 분리해서 보면 특정 서브픽셀에만 걸린 보상 오차나 컬러 필터 계열 아티팩트가 훨씬 잘 드러납니다.

키	동작
Shift+R	Red 채널만 회색조로 표시
Shift+G	Green 채널만 회색조로 표시
Shift+B	Blue 채널만 회색조로 표시
Shift+Y	Rec.709 루마 Y' 그레이뷰 토글(다시 누르면 RGB 복귀)
Shift+C	RGB 전체로 복귀(리셋)

Y' 그레이뷰는 감마 인코딩된 표시값에 Rec.709 계수 $Y' = 0.2126 R' + 0.7152 G' + 0.0722 B'$ 를 적용합니다. GPU 셰이더와 소프트웨어 렌더러가 동일한 계수를 씁니다.

주의 여기서의 Y' 는 **비디오 루마**(감마 인코딩된 값에 적용)이고, **Shift+V** 컬러메트릭과 `--probe` 가 쓰는 CIE Y 는 **선형광 휘도**입니다. 계수 숫자가 거의 같아 보이지만 적용 도메인이 다르므로 두 값을 같은 것으로 취급하면 안 됩니다. 이 구분은 `src/color.h` 주석에 명시되어 있습니다.

3.5.1. diff 모드와의 상호작용

채널 모드는 표시 필터가 아니라 **diff 연산에도 함께 적용**됩니다.

diff 모드	Shift+R/G/B 가 켜졌을 때의 동작
Absolute / Relative	해당 채널의 차이만 회색조로 표시
Highlight	해당 채널의 차이만 임계 판정에 사용
FalseColor	3채널 평균이 아니라 해당 채널의 차이를 컬러맵에 매핑
Enhance	해당 채널만 [min,max] → [128,255] 로 재매핑
AlphaBlend	혼합 결과의 해당 채널만 회색조로 표시
Signed	항상 Rec.709 루마 기준 — 채널 모드 영향 없음

톨러런스 임계값(4장 참조)도 채널 모드를 따릅니다. RGB 모드에서는 3채널 최대 차이와 비교하지만, 단일 채널 모드에서는 그 채널의 차이만 임계와 비교합니다. “Identical” 오버레이와 Diff Listing 창 의 동일 판정 역시 채널별로 따로 평가되므로, “RGB 로는 다르지만 G 채널만 보면 완전히 같다”는 상태를 바로 확인할 수 있습니다.

주의 `Shift+Y` 의 Y' 그레이뷰는 A/B 패널에만 적용됩니다. diff 세이더는 R/G/B 단일 채널만 처리하므로, 루마 모드에서 Diff 패널은 RGB diff 그대로 그려집니다. 다만 픽셀 값 풍선말은 루마 모드에서 R/G/B 세 값을 모두 보여 줍니다.

채널 모드는 `.av.ini` 에 `channel_mode=0|1|2|3|4` (RGB/R/G/B/Luma)로 영속화됩니다.

예시 — 서브픽셀 렌더링 보상의 채널 편중 확인

```
av --diff-mode falsecolor --amplify 8 orig.png comp.png
```

`Shift+R` → `Shift+G` → `Shift+B` 로 채널을 돌려 보면서 FalseColor 지도의 밝기 분포를 비교합니다. R 에서만 광범위하게 색이 올라온다면 보상 IP 의 R 채널 계수 또는 R 서브픽셀 경로를 의심합니다. `Shift+C` 로 언제든지 RGB 전체 보기로 돌아옵니다.

3.6. 컬러메트리 풍선말 — `Shift+V`

`Shift+V` 는 픽셀 풍선말 아래에 CIE 색도 정보 상자를 덧붙입니다. 커는 순간 `V` 픽셀 풍선말도 자동으로 함께 커집니다(색도 상자는 풍선말에 얹혀 그려지기 때문). 메뉴로는 `View > Colorimetry Probe (xy/CCT)` 입니다.

색도 상자는 최대 3줄입니다.

줄	형식과 내용
A	A xy 0.3096,0.3249 6714K — A 영상의 CIE 1931 x,y (소수 4자리)와 CCT(K, 정수)
B	B xy 0.3064,0.3218 6930K — B 영상의 같은 값(B 가 로드된 경우만)
Δ	$\Delta u'v'$ 0.0024 ΔCCT 216K — A/B 모두 유효할 때만. $u'v'$ 평면상의 유클리드 거리와 CCT 차(B - A)

A/B 두 줄은 **커서가 어느 패널 위에 있는** 항상 `images[0]` (A)과 `images[1]` (B)을 같은 이미지 좌표에서 샘플링합니다. 즉 커서 한 번으로 두 영상의 색도를 동시에 읽습니다.

입력 종류	XYZ 변환 방식
PNM 원본값	값 / maxval 을 sRGB 표시값으로 보고 감마 해제 후 매트릭스 적용
HDR float	이미 선형광으로 보고 매트릭스만 적용
LDR uint8	값 / 255 를 sRGB 표시값으로 보고 감마 해제 후 매트릭스 적용

기준 백색점은 CIE 2도 표준광 D65($x=0.31270$, $y=0.32900$)이며, sRGB→XYZ Bradford 적응 매트릭스가 순백 (1,1,1) 을 정확히 이 점으로 보냅니다. CCT 는 McCamy 1992 3차 근사식을 씁니다.

3.6.1. 헤드리스 `--probe` 와 같은 색 코어

GUI 풍선말과 헤드리스 `--probe` 는 `src/color.cpp` 라는 **하나의 색 코어**를 공유합니다(전부 double 정밀도, GL/SDL 의존 없음). 따라서 화면에서 읽은 값과 CI 에서 뽑은 CSV 값이 반올림 자릿수만 다를 뿐 동일합니다. FLIP 엔진이 내부적으로 쓰는 자체 D65 와는 의도적으로 분리되어 있어, 색도 수치는 공개 레퍼런스와 맞춰집니다.

GUI 는 화면 공간을 아끼기 위해 x,y / CCT / $\Delta u'v'$ / ΔCCT 만 보여 줍니다. XYZ, $u'v'$ 원값, Duv, L^* , $\Delta E76$ 까지 필요하면 헤드리스 `--probe` 를 쓰십시오(8장 참조).

예시 — 한 픽셀의 색도 이동량 정량화

화면에서 좌표를 찍고(예: 풍선말이 (4, 4) 라고 표시), 같은 좌표를 CLI 로 넘깁니다.

```
av --probe 4,4 orig.ppm comp.ppm
```

```
which,X,Y,Z,x,y,uprime,vprime,CCT,Duv,Lstar
```

```
A,0.799808,0.839443,0.944302,0.309577,0.324918,0.197187,0.465657,6713.57,0.00270091,93.4263
```

```
B,0.797736,0.837855,0.967665,0.306438,0.321849,0.196142,0.463513,6929.78,0.00275288,93.3572
```

```
delta,,,,,,,,,216.203,0.00238477,1.6842
```

`delta` 행은 XYZ/xy/u'v' 열을 비우고 CCT 열에 ΔCCT , Duv 열에 $\Delta u'v'$, Lstar 열에 ΔE_{76} 을 담습니다. 이 예에서 보상 결과는 CCT 가 216 K 올라갔고 $\Delta u'v'$ 는 0.0024 — 일반적인 화이트포인트 관리 기준(0.004 이내)에는 들어가지만 ΔE_{76} 1.68 은 육안 식별 한계(약 1.0) 를 넘어섭니다.

참고 `show_colorimetry` 는 `.av.ini` 에 저장되지 않습니다. 창을 닫으면 꺼진 상태로 돌아갑니다(반면 `show_pixel_info` 는 저장됩니다).

3.7. 결함 픽셀 오버레이 — /

/ 는 float/HDR 영상에서 물리적으로 있을 수 없는 값을 색으로 칠해 주는 오버레이를 토글합니다. 렌더러/보상 IP 시뮬레이터가 뱉은 float 출력에 NaN 이 섞였는지, 클리핑이 빠져 음수나 초과백색이 남았는지를 한눈에 잡아냅니다.

분류	색	조건 (R/G/B 중 하나라도)
NaN	마젠타	<code>isnan(v)</code> — 최우선. 즉시 확정
Inf	빨강	<code>isinf(v)</code> — 두 번째 우선
negative	파랑	<code>v < 0</code> — 초과백색보다 우선
superwhite	주황	<code>v > 1</code>

패널 좌상단에 `bad-px: NaN magenta / Inf red / neg blue / >1 orange` 범례가 같이 뜹니다. 마커 한 번의 크기는 `max(zoom, 3)` px 이라 축소 상태에서도 3 px 이상으로 보이며, 한 프레임/패널당 최대 30,000 개까지만 그립니다(더 많으면 나머지는 생략되므로 정확한 개수는 `--validate` 로 세십시오).

주의 대상은 `pixels_f32` 를 가진 영상뿐입니다. 8-bit LDR 영상은 구조적으로 결함 값이 나올 수 없으므로 아무것도 표시되지 않습니다. `maxval > 255` 인 PNM 도 float 버퍼가 0 ~ 1 로 정규화되어 있어 검출 대상이 없습니다. 이 토글은 `.av.ini` 에 저장되지 않습니다.

3.7.1. --validate 와의 관계

GUI 오버레이가 “어디에” 있는지를 보여 준다면, 헤드리스 `--validate` 는 “몇 개” 인지를 세고 CI 게이트로 쓸 수 있는 종료 코드를 돌려줍니다. 두 경로는 완전히 같은 4분류(`nan` / `inf` / `negative` / `superwhite`) 를 씁니다.

```
av --validate frame_hdr.hdr
```

```
class,count
nan,0
inf,0
```

```
negative,0
superwhite,0
```

0 이 아닌 항목이 있으면 stdout 끝에 최초 위반 픽셀 좌표가 최대 20개까지 `# class,x,y,channel` 형태의 주석 줄로 덧붙이고, 요약 한 줄은 stderr 로 나갑니다. 종료 코드는 NaN 또는 Inf 가 하나라도 있으면 8, 그 외에는 0 입니다. 음수/초과백색만 있는 경우는 실패로 보지 않습니다.

팁 CI 에서는 `av --validate` 를 게이트로 걸어 NaN/Inf 유입을 막고, 실패한 프레임만 GUI 로 열어 `/` 로 위치를 확인하는 2단 구성이 효율적입니다. 8-bit 영상에 `--validate` 를 걸면 “nothing to validate” 를 stderr 로 알리고 모든 카운트를 0 으로 출력한 뒤 0 으로 끝냅니다.

3.8. 이미지 정보 창 — I

I 는 폭 600 px 의 반투명(85 %) `Image Info` 창을 토글합니다. 메뉴로는 `View > Show Image Info` 이며, 상태는 `.av.ini` 의 `show_info` 로 영속화됩니다. A 또는 B 중 하나라도 로드되어 있어야 창이 뜹니다.

창에 실제로 표시되는 항목은 다음이 전부입니다.

필드	내용
A(Left): / B(Right):	파일 경로 전체(경로가 비면 (loaded)). A 는 마젠타, B 는 노랑 라벨
W x H ch=N	픽셀 해상도와 채널 수. [HDR] 태그는 Radiance 계열일 때만 붙음
C(Right):	--comp 모드에서만. img3 의 경로와 크기 (시안 라벨)
A Zoom: ... Pan: (...)	패널별 줌 배율(%)과 팬 오프셋. fit 상태면 Zoom: Fit-to-Window
PSNR: xx.xx dB	A(Left)=기준, B(Right)=비교. 창이 열리면 자동 계산

PSNR 값은 R/G/B 채널별 PSNR 의 산술 평균이며, 색으로 등급이 보입니다: 40 dB 초과는 초록, 30~40 dB 는 노랑, 30 dB 이하는 빨강입니다. 두 영상이 완전히 같으면 `PSNR: inf (identical)` (초록), 크기나 버퍼 종류가 맞지 않으면 `PSNR: N/A (size/format mismatch)` (빨강)가 나옵니다. 영상을 다시 로드하거나 시퀀스를 넘기면 자동으로 재계산됩니다.

--comp 모드에서는 PSNR 이 두 줄로 바뀝니다.

```
PSNR img2(mid) vs orig: 42.13 dB
PSNR img3(right) vs orig: 38.90 dB
```

주의 `ch=` 는 항상 4 입니다. 로더가 모든 입력을 RGBA 4채널로 강제 변환하기 때문이며, 원본 파일의 채널 수(그레이스케일 1채널 등)를 뜻하지 않습니다. 또한 이 창에는 **비트 깊이 필드가 없습니다**. 원본 비트 깊이는 `v` 풍선말의 값 범위로 판단하십시오 — 값이 255 를 넘어가면 10-bit 이상 PNM 원본값이 살아 있다는 뜻입니다. 픽셀 통계/히스토그램은 6장을 참조하십시오.

3.9. 지원 포맷 상세

av 는 PNM 계열만 자체 파서로 읽고, 나머지는 `stb_image` 로 디코딩합니다. 파일 열기 대화상자의 필터는 `png;jpg;jpeg;bmp;tga;hdr;ppm;pgm` 입니다.

포맷	지원 내용과 주의점
----	------------

PNG	8/16-bit, 알파 포함. 16-bit PNG 는 디코딩 시 8-bit 로 놀립니다 — 원본 비트 깊이를 보존하려면 16-bit PNM 을 쓰십시오
JPEG	표준 손실 압축. 블록 아티팩트가 diff 에 그대로 잡히므로 보상 검증 입력으로는 부적합
BMP	Windows 비트맵. 무손실이지만 8-bit 고정
TGA	Targa. 알파 채널 지원, 8-bit 고정
HDR	Radiance RGBE. float32 RGBA 로 로드되어 <code>pixels_f32</code> 를 채움 — <code>/</code> 결함 오버레이와 <code>--validate</code> 의 유일한 실질 대상
PNM P5/P6	PGM/PPM Binary. 자체 파서. <code>maxval</code> 1 ~ 65535, 16-bit 원본값 보존. <code>maxval > 255</code> 이면 정밀 diff 용 float 버퍼도 함께 생성
PNM P2/P3	PGM/PPM ASCII. 자체 파서. 원본값 보존. <code>#</code> 주석 줄 허용. 파일이 크면 로딩이 느립니다

P5/P2(그레이스케일)는 한 값을 R=G=B 로 복제해 3채널로 확장한 뒤 `pixels_orig` 에 담습니다. 따라서 그레이스케일 PNM 에서도 풍선말은 세 값을 동일하게 보여 줍니다.

주의 헤드리스 경로는 PNM ASCII(P2/P3)를 읽지 못합니다. `--metrics`, `--probe`, `--validate`, `--uniformity`, `--diff-out`, `--comp-out` 등 헤드리스 모드는 `stb_image` 기반의 CPU 디코더만 쓰므로 P5/P6 바이너리는 되지만 P2/P3 는 `failed to load` 로 종료 코드 4 를 냅니다. 같은 파일을 GUI 로 열면 잘 보이므로 혼동하기 쉽습니다. CI 파이프라인에서는 PNM 을 **바이너리 P5/P6** 로 덤프하십시오. 덧붙여 헤드리스 디코더는 16-bit PNM 도 8-bit 로 변환해 읽으므로, 원본 비트 깊이가 필요한 정밀 비교는 GUI 에서 수행해야 합니다.

주의 치트시트에는 JPEG 의 EXIF 방향이 자동 적용된다고 적혀 있으나, 실제 디코더 (`stb_image`) 는 EXIF 방향 태그를 해석하지 않습니다. 스마트폰으로 찍은 세로 사진이 눕는다면 `R` 시계 방향 90도 / `Ctrl+R` 반시계 방향 90도 로 직접 회전시키십시오. 회전은 A/B 양쪽에 동시에 적용되고 뷰는 `fit` 으로 초기화됩니다.

팁 DDI 보상 검증용 참조/비교 쌍은 다음 조합을 권장합니다.

- 8-bit 파이프라인: PNG(무손실, 알파 불필요 시 3채널)
- 10/12/16-bit 파이프라인: P6 바이너리 PNM(`maxval` 을 실제 비트폭에 맞춤)
- 선형 HDR 파이프라인: Radiance `.hdr`

세 경우 모두 A/B 의 해상도와 버퍼 종류가 같아야 PSNR/SSIM/FLIP 이 계산됩니다. 파일 저장/시퀀스 취급은 7장, 색 관리와 LUT 적용은 9장을 참조하십시오.

4. 두 영상 비교 — Diff 모드

이 장은 av의 핵심 기능인 두 영상(A/B) 비교를 다룹니다. 10가지 Diff 모드가 각각 무엇을 계산하고 어떤 색으로 보여 주는지, 증폭·허용오차 같은 파라미터를 어떻게 조작하는지, 그리고 차이 픽셀을 숫자로 확인하는 Diff 픽셀 리스트 창과 블링크 비교를 순서대로 설명합니다. DDI 보상 IP 검증에서는 대개 A = 원본(reference), B = 보상 결과(compensated output)로 놓고 씁니다.

4.1. Diff 패널이란

두 영상이 모두 로드된 상태에서 Diff 모드를 켜면 화면이 세 패널로 나뉩니다 — 왼쪽 A, 가운데 B, 오른쪽 Diff 입니다. 각 패널 폭은 창 폭의 1/3 이며, 기본 테두리 색은 A = 마젠타, B = 노랑, Diff = 시안입니다(`-bc` 로 변경, 10장 참조).

Diff 패널은 A 패널과 같은 뷰포트를 공유합니다. 즉 A 패널을 확대·이동하면 Diff 패널이 그대로 따라오고, 뷰포트 동기화(`s`)가 켜져 있으면 B 패널까지 세 장이 함께 움직입니다. 차이 계산은 화면에 실제로 그려지는 픽셀에 대해 매 프레임 수행되므로, 확대하면 확대한 만큼 정밀하게 관찰할 수 있습니다.

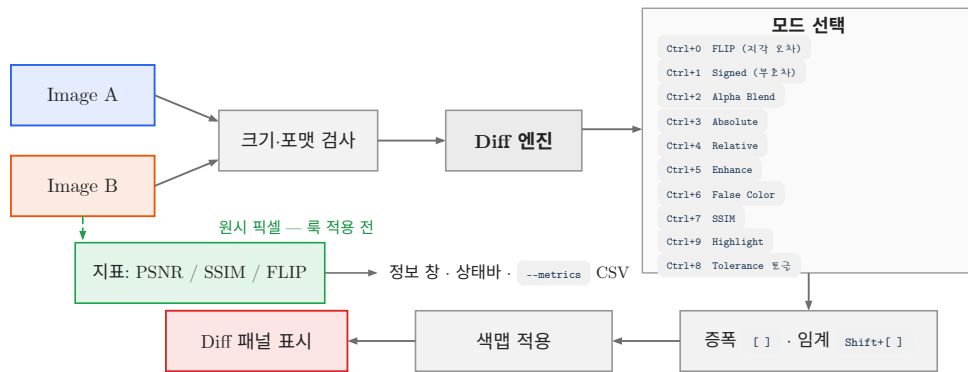


그림 4.1: 두 영상이 Diff 패널과 지표로 흘러가는 경로. 지표는 표시 룩의 영향을 받지 않는다.

Diff 모드가 켜져 있는데 두 영상이 완전히 동일하면 Diff 패널 한가운데에 초록색 `Identical` 글자가 크게 표시됩니다(Alpha Blend 모드는 제외). 채널 보기 모드 (`Shift+R` / `Shift+G` / `Shift+B`)에서는 해당 채널만 같아도 `Identical` 이 뜹니다.

참고 Diff 패널의 렌더링은 GPU(OpenGL 셰이더)와 소프트웨어 렌더러(`--software`)가 동일한 수식을 사용하도록 구현되어 있습니다. 따라서 GPU 가 없는 CI 머신이나 원격 X11 세션에서도 화면 결과가 같습니다.

4.2. Diff 모드 한눈에 보기

모드 전환은 모두 `Ctrl` + 숫자 조합이며 **토글** 방식입니다. 같은 키를 다시 누르면 그 모드가 꺼지고 Diff 가 Off 로 돌아갑니다. 메뉴 `Diff` 에서도 같은 목록을 고를 수 있습니다.

키	메뉴 항목	태그 / CLI 토큰	한 줄 요약
<code>Ctrl+0</code>	FLIP	<code>FLIP</code> / <code>flip</code>	NVIDIA FLIP-LDR 지각 오차맵 (magma)
<code>Ctrl+1</code>	Signed	<code>Signed</code> / <code>signed</code>	부호 있는 차이: 파랑 = $A < B$, 흰 = 동일, 빨강 = $A > B$
<code>Ctrl+2</code>	Alpha Blend	<code>AlphaBlend</code> / <code>alphablend</code>	A와 B를 알파 비율로 혼합해 겹쳐 보기

Ctrl+3	Absolute	Abs / abs	채널별 절대차 $ A-B $
Ctrl+4	Relative	Rel / rel	A 기준 상대 오차 $ A-B / A$
Ctrl+5	Enhance	Enhance / enhance	0이 아닌 차이를 128~255로 remap
Ctrl+6	FalseColor	FalseColor / falsecolor	오차 크기를 남색→빨강 컬러맵으로
Ctrl+7	SSIM	SSIM / ssim	구조 유사도 히트맵 + 점수
Ctrl+9	Highlight	Highlight / highlight	다른 픽셀만 빨강계, 나머지는 검정
(메뉴) Off	Off	None / none	Diff 끄기 — 2패널 A B 로 복귀

주의 Diff 메뉴의 Off 항목에는 단축키 라벨이 Ctrl+D 로 표시되지만, 실제 키보드에서 Ctrl+D 는 Diff 픽셀 리스트 창을 여닫는 키입니다(아래 절 참조). 키보드만으로 Diff 를 끄려면 **현재 켜져 있는 모드의 키를 한 번 더** 누르거나 메뉴 Diff → Off 를 고르십시오. 예: Signed 상태에서 Ctrl+I .

참고 --comp (3영상 블록 비교) 모드에서는 Diff 가 매 프레임 강제로 Off 로 유지되므로 Ctrl+0 ~ Ctrl+9 가 아무 효과가 없습니다. 3영상 비교는 5장을 참조하십시오.

Diff 모드:증폭.허용오차.블링크 간격은 ~/.av.ini 에 저장되지 않습니다. 즉 av 를 다시 실행하면 항상 Diff Off, 증폭 1.0, 허용오차 0 에서 시작합니다. 매번 같은 조건으로 열고 싶다면 CLI 옵션(아래 “CLI로 초기 상태 지정”)을 쓰십시오.

4.3. 절대 · 상대 · 부호 차이 맵

4.3.1. Absolute — 기본 차이 맵 (Ctrl+3)

채널별로 $|A - B|$ 를 구해 증폭을 곱한 뒤 0~1로 잘라 그대로 RGB 로 그립니다. 차이가 없으면 검정입니다. R 성분만 다르면 붉게, G 만 다르면 초록으로 보이므로 “어느 채널이 틀어졌는가”를 색으로 바로 읽을 수 있습니다.

CLI 단축 옵션 -d (= --diff)가 정확히 이 모드입니다.

```
av -d orig.png comp.png
```

예시 — 보상 결과의 채널 편중 확인

원본과 보상 결과를 av -d orig.png comp.png 로 열고 J 를 여덟 번 눌러 증폭을 5.0 으로 올립니다. Diff 패널 전체가 푸르스름하게 뜬다면 B 채널만 계통적으로 틀어졌다는 뜻이므로, 보상 LUT 의 B 축을 먼저 의심하면 됩니다.

4.3.2. Relative — 어두운 영역에 민감한 상대 오차 (Ctrl+4)

$|A - B|_{\max(A+0.001, 0.001)}$ 를 채널별로 계산합니다(분모의 0.001 은 0 나눗셈 방지용 eps). 같은 1 LSB 차이라도 밝은 영역에서는 작게, 어두운 영역에서는 크게 보입니다. 저계조(low gray) 보상 품질, 블랙 레벨 근처의 밴딩·색틀어짐을 볼 때 Absolute 보다 유리합니다.

팁 저계조 검증에서는 Relative 로 위치를 먼저 찾고, 그 자리를 32배 이상 확대해 Absolute 나 Enhance 로 실제 LSB 값을 확인하는 순서가 빠릅니다.

4.3.3. Signed — 오버샷과 언더샷을 한 화면에 (Ctrl+1)

Absolute 계열이 절대값이라 “어느 쪽이 밝은지”를 버리는 반면, Signed 는 부호를 살립니다. Rec.709 휘도로 $0.2126 \cdot \Delta R + 0.7152 \cdot \Delta G + 0.0722 \cdot \Delta B$ 를 구해 증폭을 곱하고 -1~+1 로 자른 뒤, 파랑—흰색—빨강 발산형(diverging) 컬러맵에 태웁니다.

색	의미
빨강	$A > B$ — A 가 더 밝다 (보상 결과가 어두워짐 = 언더샷)
흰색	$A = B$ — 차이 없음
파랑	$A < B$ — A 가 더 어둡다 (보상 결과가 밝아짐 = 오버샷)

엣지 주변에서 빨강 띠와 파랑 띠가 번갈아 나타나면 전형적인 링잉(ringing)입니다. 샤프닝·디링잉 IP, ODC(over-drive) 계수 검증에 가장 먼저 켜는 모드입니다.

```
av --diff-mode signed --amplify 8 orig.png comp.png
```

4.3.4. Highlight — 다른 픽셀만 빨강게 (Ctrl+9)

세 채널 차이 중 최대값에 증폭을 곱해 0보다 크면 빨강(밝기 128~255), 0이면 검정으로 칠합니다. 차이의 크기보다 위치가 궁금할 때 씁니다.

이 모드에서는 상태바에 [Diff: 1234 / 2073600 px] 형식으로 현재 화면에 그려진 픽셀 중 차이가 있는 픽셀 수 / 전체 픽셀 수가 함께 표시됩니다. 카운트는 화면 뷰포트 기준이므로, 영상 전체 통계를 원하면 먼저 F (fit)로 전체를 띄우고 읽으십시오.

4.4. 컬러맵 계열

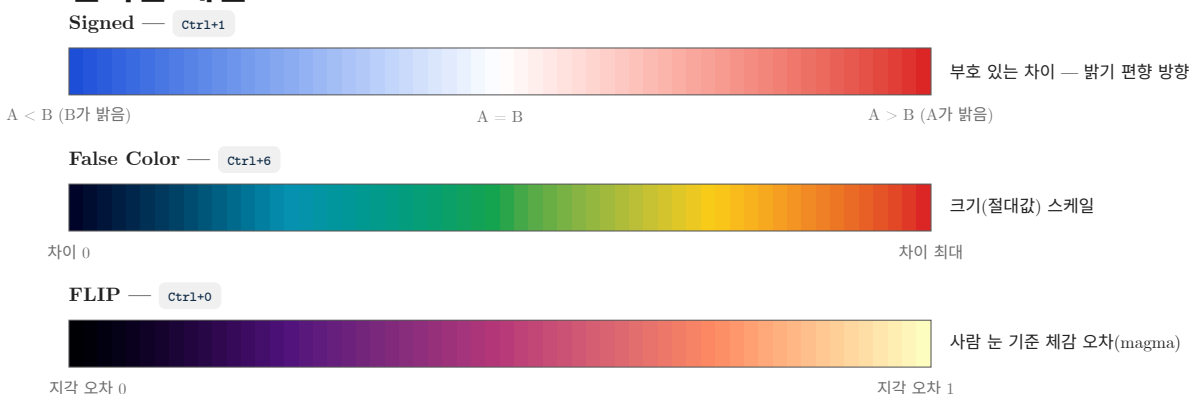


그림 4.2: 주요 Diff 모드의 색 해석. 같은 색이라도 모드마다 의미가 다르다.

4.4.1. False Color — 오차 크기를 색으로 (Ctrl+6)

세 채널 차이의 평균(단일 채널 모드에서는 해당 채널 값)에 증폭을 곱한 값을 0~1 로 정규화해 컬러맵에 태웁니다. 색 순서는 짙은 남색 → 파랑 → 초록 → 노랑 → 빨강이며, 구간 경계는 0.25 / 0.50 / 0.75 입니다.

색	정규화 오차 (증폭 반영 후)
짙은 남색 → 파랑	0.00 ~ 0.25
파랑 → 초록	0.25 ~ 0.50
초록 → 노랑	0.50 ~ 0.75
노랑 → 빨강	0.75 ~ 1.00

리포트용 그림에 쓰기 좋습니다. 증폭을 바꾸면 색 스케일도 함께 바뀌므로, 여러 장을 비교할 때는 반드시 같은 증폭값으로 캡처하십시오.

4.4.2. Enhance — 1 LSB 차이도 보이게 (Ctrl+5)

이 모드만은 화면 전체가 아니라 **두 영상 전체**를 먼저 훑어 0이 아닌 차이값의 전역 최소·최대(0~255 스케일)를 구합니다. 그리고 그 `[min, max]` 구간을 `[128, 255]` 로 선형 remap 합니다. 차이가 0인 픽셀만 검정으로 남습니다.

결과적으로 “차이가 있는 곳은 무조건 중간 회색 이상”이 되므로, 1 LSB 밖에 차이가 없는 영역도 놓치지 않습니다. Demura 잔차, 디더링 패턴 차이처럼 진폭이 아주 작은 결함을 찾을 때 가장 강력합니다.

주의 Enhance 는 스케일이 데이터에 따라 자동으로 바뀌므로 **두 캡처 간 밝기를 서로 비교할 수 없습니다**. “얼마나 다른가”는 Absolute/Signed 나 상태바 PSNR 로 확인하고, Enhance 는 “어디가 다른가”에만 쓰십시오.

4.5. 지각 · 구조 메트릭 히트맵

SSIM 과 FLIP 은 픽셀 단위 뺄셈이 아니라 사람이 느끼는 차이를 모델링한 메트릭입니다. 두 모드 모두 배경 스레드에서 전체 영상을 한 번 계산해 히트맵 텍스처를 만들고, 그 텍스처를 Diff 패널에 표시합니다. 계산 중에는 상태바에 `SSIM: computing...` / `FLIP: computing...` 이 뜹니다. 영상을 다시 로드하거나 시퀀스를 넘기면 자동으로 재계산됩니다.

4.5.1. SSIM — 구조 유사도 (Ctrl+7)

11×11 가우시안 윈도우($\sigma = 1.5$), $C_1 = (0.01 \cdot 255)^2$, $C_2 = (0.03 \cdot 255)^2$ 의 표준 SSIM 을 픽셀마다 구한 뒤, 비유사도 $\frac{1-SSIM}{2}$ 를 False Color 와 같은 컬러맵으로 그립니다. 파랑에 가까울수록 구조가 같고, 빨강에 가까울수록 구조가 깨진 곳입니다.

상태바에는 전체 평균 점수가 `SSIM: 0.9987` 형식(소수 4자리)으로 표시되며 색이 품질 신호를 겸합니다.

상태바 색	SSIM 점수
초록	0.99 초과
노랑	0.90 초과 ≤ 0.99
빨강	0.90 이하

4.5.2. FLIP — 지각 오차맵 (Ctrl+0)

NVIDIA 의 FLIP-LDR(Andersson et al., 2020)을 충실히 이식한 구현입니다. sRGB → YCxCz 변환, 대비 감도 함수(CSF) 공간 필터링, Hunt 보정 CIELab 공간의 HyAB 색차, 엣지·포인트 특징 검출을 결합해 픽셀마다 0~1 의 지각 오차를 내고 magma 컬러맵(검정 → 보라 → 주황 → 연노랑)으로 그립니다. 검정이 “차이 없음”, 밝을수록 “사람 눈에 잘 보이는 차이”입니다.

기본 ppd(pixels-per-degree)는 67.0206 으로, 폭 0.7 m 인 4K 모니터를 0.7 m 거리에서 보는 조건에 해당합니다. 화면 평균이 점수이며 **낮을수록 좋습니다**(0 = 동일).

상태바 색	FLIP 점수 (낮을수록 좋음)
초록	0.05 미만
노랑	0.05 이상 0.15 미만
빨강	0.15 이상

참고 av 의 FLIP 구현은 NVIDIA 공식 `flip` 레퍼런스 툴과 **소수 5자리까지 일치**하는 것이 검증되어 있습니다. 따라서 사내 리포트의 FLIP 수치를 레퍼런스 값과 그대로 비교해도 됩니다. 헤드리스 `--metrics` 의 FLIP 열도 같은 엔진을 씁니다(8장 참조).

팁 PSNR 이 40 dB 를 넘어 “수치상 문제 없음”인데도 눈으로 보면 거슬리는 경우가 있습니다. 이럴 때 FLIP 을 켜면 사람이 실제로 인지하는 영역만 밝게 뜨므로, “수치는 좋은데 왜 이상한가”를 설명하는 근거 그림으로 쓰기 좋습니다.

4.6. Alpha Blend — 겹쳐 보기 (Ctrl+2)

유일하게 차이를 계산하지 않는 모드입니다. Diff 패널에 $(1 - \alpha) \cdot A + \alpha \cdot B$ 를 그림니다. `alpha` 기본값은 $0.5(A\ 50\% + B\ 50\%)$ 이고 범위는 0.0~1.0 입니다 (0 = A 만, 1 = B 만). 패널 좌상단에 `A: 50% - B: 50%` HUD 가 표시됩니다.

이 모드에서만 `[` / `]` 와 `\` 의 동작이 달라집니다.

키	Alpha Blend 에서의 동작
<code>[</code> / <code>]</code>	alpha -1% / +1%
<code>Shift+[</code> / <code>Shift+]</code>	alpha -10% / +10%
<code>\</code>	alpha = 50% 로 리셋

허용오차(threshold)는 Alpha Blend 에는 적용되지 않으며, `Identical` 오버레이도 뜨지 않습니다. B 영상이 로드되지 않은 상태에서 `Ctrl+2` 를 누르면 `B image required for Alpha Blend` 토스트가 약 1.8초 동안 표시됩니다.

예시 — 정렬(alignment) 확인

패턴 영상 두 장의 위치가 1픽셀 어긋났는지 보려면 `Ctrl+2` 로 Alpha Blend 를 켜고 `[` 와 `]` 를 번갈아 눌러 alpha 를 0%와 100% 사이에서 흔들어 보십시오. 옛지가 좌우로 떨어지면 정렬이 어긋난 것이고, 제자리에서 밝기만 변하면 정렬은 맞고 계조만 다른 것입니다.

4.7. 파라미터 조작: 증폭 · 알파 · 허용오차

4.7.1. 증폭 (amplify)

차이가 너무 작아 검게만 보일 때 증폭을 올립니다. 계산된 차이에 곱해지는 배율이며, Signed / Absolute / Relative / Highlight / False Color / Enhance 에 영향을 줍니다 (SSIM · FLIP 히트맵과 Alpha Blend 는 영향 없음).

조작	변화량	비고
<code>[</code> / <code>]</code>	-0.5 / +0.5	범위 0.5 ~ 100.0
<code>\</code>	1.0 으로 리셋	Alpha Blend 에서는 alpha = 50%
메뉴 <code>Diff</code> → Amplify 슬라이더	연속 조절	슬라이더 범위 0.5 ~ 50.0
<code>--amplify <val></code>	시작값 지정	기본 1.0, 도움말 표기 범위 0.1 ~ 100

현재 값은 상태바에 `Diff: Signed (A-B) x8.0` 처럼 표시됩니다. 괄호 안의 `(A-B)` 는 비교 방향이며, `Shift+Space` 로 A/B 를 맞바꾸면 `(B-A)` 로 바뀝니다.

주의 `--amplify` 로 준 값은 클램프되지 않습니다. `--amplify 0.2` 처럼 0.5 미만을 주면 그 값으로 시작하지만, 이후 `[` 를 누르는 순간 하한 0.5 로 올라붙습니다. 재현 가능한 캡처가 필요하다면 0.5 이상의 값을 쓰십시오.

4.7.2. 허용오차(threshold)와 Tolerance diff

허용오차는 “이 값 이하의 차이는 무시”라는 기준입니다. 기본값 0 은 비활성이고, 1~255 범위(0~255 스케일 기준)로 설정합니다. 커먼 채널 최대 차이가 threshold 이하인 픽셀이 어두운 회색(40, 40, 40)으로 놀리고, 초과한 픽셀만 원래 Diff 색으로 남습니다.

키	동작
<code>Ctrl+8</code>	Tolerance diff 토글 — 0이면 1로, 그 외에는 0으로
<code>Shift+[</code> / <code>Shift+]</code>	threshold -1 / +1 (Alpha Blend 에서는 alpha $\pm 10\%$)
<code>Ctrl+\</code>	threshold = 0 (리셋)

threshold 가 1 이상이면 상태바에 `Thr: 3 (0.8% exceed)` 가 표시됩니다. 괄호 안은 현재 화면에 그려진 픽셀 중 임계를 초과한 비율입니다.

예시 — 규격 1 LSB 이내 판정

“보상 결과는 원본 대비 2 LSB 이내”라는 규격을 확인한다고 합시다. `Ctrl+3` 으로 Absolute 를 켜고 `Ctrl+8` 로 Tolerance 를 켜 뒤 `Shift+]` 를 한 번 더 눌러 `Thr: 2` 로 맞춥니다. 화면이 전부 어두운 회색이면 규격 통과이고, 남아 있는 색점이 규격 위반 픽셀입니다. 상태바의 `(x.x% exceed)` 를 그대로 리포트 수치로 쓸 수 있습니다.

4.7.3. --comp 모드에서의 대괄호 키

주의 `--comp` (3영상 블록 비교) 모드에서는 `[` / `]` 가 증폭 조절이 아니라 **worst 블록 순회**로 동작합니다(`Shift+[` / `Shift+]` 도 마찬가지). 자세한 내용은 5장을 참조하십시오.

4.7.4. CLI로 초기 상태 지정

```
av --diff-mode <mode> [--amplify <val>] A B
```

`<mode>` 에 넣을 수 있는 토큰은 `none`, `alphablend`, `abs`, `rel`, `highlight`, `falsecolor`, `ssim`, `flip`, `signed`, `enhance` 이며 기본값은 `none` 입니다. 잘못된 토큰을 주면 `Unknown diff-mode: ...` 를 출력하고 종료합니다.

```
# Signed + 증폭 8배로 바꾼 시작 (영인 검사 프리셋)
av --diff-mode signed --amplify 8 orig.png comp.png

# 절대차 탐색 옵션 (-d 는 --diff-mode abs 와 동일)
av -d orig.png comp.png

# FLIP 지각 오차맵을 전체 맞춤 배율로 열기
av --diff-mode flip --zoom fit ref.png test.png
```

차이 맵을 파일로 뽑는 헤드리스 `--diff-out` / `--sbs` 는 8장에서 다룹니다.

4.8. Diff 픽셀 리스트 창 (`Ctrl+D`)

차이가 있는 픽셀을 **숫자 표**로 확인하는 창입니다. Diff 모드가 켜져 있는 상태에서 `Ctrl+D` 를 누르면 화면 상단 가운데에 `Diff Pixels` 창이 열립니다.

열	내용
#	차이 픽셀 일련번호 (1부터)
Pos	이미지 좌표 (x,y)

A(RGB)	A 영상의 픽셀값	r,g,b
B(RGB)	B 영상의 픽셀값	r,g,b
D(RGB)	절대차	A-B (붉은 글씨)

- **정렬 순서:** 별도의 정렬 기능은 없고, 영상을 위에서 아래로 → 왼쪽에서 오른쪽으로 훑은 래스터 순서 그대로입니다.
- **페이지:** 한 번에 50개씩 표시하며, 창 폭에 따라 같은 5열 묶음이 최대 4열까지 가로로 반복 배치됩니다. 헤더에 `Showing #1 ~ #50 / 8123 different (2073600 total pixels)` 형식으로 현재 구간과 전체 차이 픽셀 수, 비교 대상 총 픽셀 수가 표시됩니다.
- **이동:** 아래쪽 `< Prev / Next >` 버튼으로 50개 단위 이동, `List from #` 에 번호를 넣고 Enter 를 치면 그 번호부터 목록을 다시 그립니다.
- **클릭 동작:** `Go to pixel #` 에 번호를 넣고 `Go` 버튼(또는 Enter)을 누르면 A/B 두 뷰포트가 모두 **32배 줌**으로 바뀌면서 해당 픽셀이 화면 정중앙에 오도록 이동합니다. 목록도 그 픽셀부터 다시 시작합니다.
- **닫기:** 창 오른쪽 아래 `Close` , 창 제목줄의 X, `Esc` , 또는 `Ctrl+D` 를 다시 누르면 닫힙니다.

값의 단위는 원본 비트심도를 따릅니다. PPM(P2/P3/P5/P6) 10비트 영상이면 0~1023 으로 표시되고, 일반 8비트 영상은 0~255 입니다. HDR/float 영상은 0~255 스케일로 환산된 정수로 나열됩니다.

채널 보기 모드에서는 선택한 채널의 차이가 0이 아닌 픽셀만 남기고, `A / B / D` 열도 `A(R)` , `B(R)` , `D(R)` 처럼 단일 값으로 바뀝니다.

주의 이 창은 Diff 패널이 한 번이라도 그려져 차이 목록이 계산된 뒤에만 내용을 표시합니다. Diff 모드가 Off 인 상태에서 `Ctrl+D` 를 눌러도 아무것도 뜨지 않습니다. 또 두 영상이 완전히 동일하면(선택 채널 기준으로 동일해도) 창이 스스로 닫힙니다 — `Identical` 오버레이가 그 답입니다.

참고 `--comp` 모드에서 `Ctrl+D` 는 diff 픽셀 리스트가 아니라 **worst 블록 리스트 창**을 토글합니다 (행을 클릭하면 세 패널이 동시에 그 블록으로 점프). 5장을 참조하십시오.

4.8.1. 화면에서 직접 숫자 읽기

Diff 패널을 32배 이상 확대하면 픽셀마다 격자선과 함께 차이값이 직접 그려집니다. RGB 모드에서는 빨강·초록·파랑 세 줄로 세로로 쌓이고, 단일 채널 모드에서는 한 줄만 나옵니다. 표시 형식은 `Ctrl+X` 로 10진수 / `0x` 16진수 / `h` 점미 16진수를 순환할 수 있습니다(3장 참조).

32배 미만에서는 마우스를 올린 자리에 돋보기가 뜨고, 그 아래에 `diff: 3, 0, 12` 형식으로 중심 픽셀의 채널별 절대차가 표시됩니다. 돋보기는 `Ctrl+M` 으로 끄고 켤 수 있으며 32배 이상에서는 자동으로 숨습니다.

4.9. 블링크 비교 (,))

두 영상을 번갈아 깜빡여 눈으로 차이를 잡는 고전적인 기법입니다. 두 영상이 모두 로드된 상태에서 ,) 를 누르면 화면이 **한 패널 전체**로 바뀌고 A 와 B 가 자동으로 교대합니다.

키	동작
,)	블링크 시작 / 종료
< (= Shift+,)	교대 간격 -0.1초

> (= Shift+.) 교대 간격 +0.1초

Esc 블링크 종료

간격의 기본값은 0.5초, 범위는 0.1~2.0초, 조절 단위는 0.1초입니다. 상태바에는 작은 원형 표시와 함께 BLINK A 0.50s 형식으로 현재 위상(A 또는 B)과 간격이 표시되고, 패널 테두리 색도 위상에 따라 A 색(마젠타) ↔ B 색(노랑)으로 바뀌므로 지금 어느 쪽을 보고 있는지 헷갈리지 않습니다.

블링크를 켜면 활성 패널의 뷰포트가 양쪽에 복사되고 뷰포트 동기화가 강제로 켜집니다(두 영상이 정확히 같은 위치에서 깜빡여야 하기 때문). 블링크를 끄면 이전 동기화 설정이 그대로 복원됩니다.

팁 확대율이 낮으면 블링크의 효과가 약합니다. 의심 영역을 8~16배로 확대한 뒤 간격을 < 로 0.2초까지 줄이면 1 LSB 밴딩 경계도 깜빡임으로 드러납니다.

--comp 모드에서 , 는 원본 → img2 → img3 를 한 패널에서 순환하는 3-way 블링크가 됩니다(5장 참조).

4.10. 상태바와 Info 창 판독

4.10.1. 상태바

U 로 UI 를 켜면 창 아래쪽 상태바에 Diff 관련 정보가 왼쪽부터 차례로 나타납니다. 표시 조건이 안 맞는 항목은 아예 나오지 않습니다.

표시	의미와 조건
Diff: Signed (A-B) x8.0	활성 모드 태그, 비교 방향, 증폭. Diff 가 켜져 있을 때
SSIM: 0.9987	SSIM 모드일 때 평균 점수 (계산 중이면 computing...)
FLIP: 0.0123	FLIP 모드일 때 평균 점수 (낮을수록 좋음)
PSNR: 42.3 dB	Diff 가 켜져 있으면 자동 계산. 완전 동일하면 PSNR: inf
[Diff: 1234 / 2073600 px]	Highlight 모드의 차이 픽셀 수 / 화면 픽셀 수
Thr: 2 (0.8% exceed)	허용오차가 1 이상일 때 임계값과 초과 비율
BLINK A 0.50s	블링크 중일 때 현재 위상과 간격

상태바 PSNR 은 R/G/B 채널별 PSNR 의 평균이며, 색이 곧 신호등입니다 — 40 dB 초과는 초록, 30 dB 초과는 노랑, 30 dB 이하는 빨강입니다. Diff 를 끄면 PSNR 표시도 사라집니다.

4.10.2. Info 창 (I)

I 로 여는 Image Info 창에는 A/B 의 경로·해상도·채널 수·HDR 여부, 각 패널의 줌과 팬 값, 그리고 PSNR 한 줄이 표시됩니다. Diff 모드와 무관하게 열리는 즉시 계산됩니다.

표시	의미
PSNR: 42.31 dB [A(Left)=ref, B(Right)=cmp]	정상 계산. A 가 기준, B 가 비교 대상
PSNR: inf (identical)	두 영상이 완전히 동일
PSNR: N/A (size/format mismatch)	해상도 또는 픽셀 포맷이 달라 계산 불가

색 규칙은 상태바와 같습니다(40 dB 초과 초록 / 30 dB 초과 노랑 / 그 이하 빨강). --comp 모드에서는 이 자리에 img2, img3 각각의 원본 대비 PSNR 두 줄이 표시됩니다 (5장 참조).

4.11. 크기가 다른 두 영상

해상도가 다른 두 장을 열어도 av 는 거부하지 않고 **겹치는 좌상단 영역** $\min(W_A, W_B) \times \min(H_A, H_B)$ 만 비교합니다. 다만 표시 지점마다 처리가 다르므로 아래를 기억해 두십시오.

기능	크기가 다를 때의 동작
Diff 패널	A 의 크기를 기준으로 뷰포트를 잡고, 겹치는 영역만 차이를 그림
<code>Identical</code> 오버레이	겹치는 영역이 같으면 표시됨 (전체가 같다는 뜻이 아님)
Diff 픽셀 리스트	겹치는 영역만 나열. 각 영상의 실제 행 쪽으로 인덱싱하므로 좌표는 정확
Info 창 PSNR	<code>N/A (size/format mismatch)</code> — 계산하지 않음
<code>--diff-out</code>	<code>[diff-out] size mismatch: ...</code> 출력 후 <code>exit 5</code>
<code>--metrics</code>	해당 행의 상태가 <code>mismatch</code> 로 기록됨

주의 상태바의 PSNR 은 크기가 다른 두 영상에서 신뢰할 수 없습니다. Info 창은 크기 불일치를 명시적으로 `N/A` 로 처리하지만, 상태바 PSNR 은 두 버퍼를 선형 인덱스로 $\min(W_A \cdot H_A, W_B \cdot H_B)$ 개까지 훑는 방식이라 폭이 다르면 행이 어긋난 채 계산됩니다. 해상도가 다르면 반드시 Info 창(`I`) 값을 기준으로 판단하고, 정량 수치가 필요하면 두 영상을 같은 해상도로 맞추는 뒤 헤드리스 `--metrics` 로 뽑으십시오(8장 참조).

4.12. 실무 시나리오: 보상 IP 링잉 확인

DDI 보상 IP 의 출력에서 엷지 링잉을 찾아 리포트까지 만드는 전형적인 흐름입니다.

예시 — 보상 전/후 링잉 검사 — Signed + 증폭 8

1. **열기** — 원본과 보상 결과를 Signed 모드로 바로 엷니다.

```
av --diff-mode signed --amplify 8 orig.png comp.png
```

2. **전체 조망** — `U` 로 상태바를 켜고 `F` 로 전체를 맞추는 뒤 상태바의 `PSNR` 값을 먼저 기록합니다.
예: `PSNR: 38.6 dB` (노랑 = 주의).
3. **링잉 위치 찾기** — 화면에서 엷지를 따라 빨강 띠와 파랑 띠가 교대로 나타나는 곳을 찾습니다.
빨강은 $A > B$ (보상 결과가 어두워짐 = 언더샷), 파랑은 $A < B$ (보상 결과가 밝아짐 = 오버샷)입니다.
4. **증폭 미세 조정** — 띠가 포화되어 뭉개져 보이면 `I` 로 증폭을 낮추고, 너무 흐리면 `J` 로 올립니다. 원위치는 `\` 입니다.
5. **정량 확인** — `Ctrl+D` 로 Diff 픽셀 리스트를 열고 `Go to pixel #` 에 번호를 넣어 `Go` 를 누르면 32배 줌으로 해당 픽셀에 붙습니다. 화면에 직접 찍히는 채널별 차이값으로 링잉의 진폭(LSB)을 읽습니다.
6. **규격 판정** — `Ctrl+8` 로 Tolerance 를 켜고 `Shift+J` 로 규격 LSB 까지 올린 뒤, 상태바 `Thr: N (x.x% exceed)` 의 초과 비율을 기록합니다.
7. **지각 관점 교차 확인** — `Ctrl+0` 으로 FLIP 을 켜 실제로 사람 눈에 보이는 수준인지 확인합니다. `FLIP: 0.04` (초록)라면 수치상 링잉이 있어도 시각 영향은 작다고 보고할 수 있습니다.
8. **증거 캡처** — `Ctrl+C` 를 누른 뒤 `3` 을 누르면 Diff 이미지가 PNG 로 클립보드에 복사됩니다. 파일로 남기려면 `Shift+Ctrl+S` 저장 대화상자를 쓰거나(7장 참조), CI 에서는 헤드리스로 뽑습니다(8장 참조).

```
av --diff-out ringing_sbs.png --diff-mode signed --amplify 8 --sbs orig.png comp.png
```

팁 같은 검사를 프레임 시퀀스 전체에 반복해야 한다면 GUI 로 한 프레임을 확인해 조건을 정한 다음, 그 조건 그대로 `--pair + --metrics + --fail-psnr` 게이트로 자동화하는 것이 정석입니다. 시퀀스는 7장, CI 연동은 8장을 참조하십시오.

5. 세 영상 블록 비교 — `--comp`

이 장은 av의 핵심 기능인 3-영상 블록 비교 모드를 다룹니다. 원본 1장과 알고리즘 결과 2장을 한 화면에 나란히 띄우고, 영상 전체를 블록 단위로 스캔해 “원본에서 가장 많이 틀어진 블록”을 자동으로 찾아 표시합니다. 어느 알고리즘이 어디에서 더 나쁜지를 숫자와 색으로 동시에 보여주므로, DDI(디스플레이 드라이버 IC) 보상 IP의 화질 검증에서 두 구현을 맞대어 놓고 판정하는 데 그대로 쓸 수 있습니다.

5.1. 무엇을 위한 모드인가

일반적인 2-영상 diff(4장 참조)는 “A와 B가 다르다”까지만 알려줍니다. 그러나 실무에서 반복되는 질문은 대개 세 항입니다. 즉 **원본(ground truth)**이 있고, 그것을 목표로 삼는 **보상 결과 2종**이 있으며, 알고 싶은 것은 “둘 중 어느 쪽이, 어느 영역에서 더 나쁜가”입니다. `--comp`는 정확히 이 형태를 위해 만들어졌습니다.

전형적인 조합은 다음과 같습니다.

원본 (왼쪽)	img2 (가운데)	img3 (오른쪽)
보상 전 입력 프레임	펌웨어(C 모델) 출력	하드웨어(RTL/게이트) 출력
골든 레퍼런스	구버전 보상 알고리즘	신버전 보상 알고리즘
시뮬레이션 기준 영상	튜닝 파라미터 셋 A	튜닝 파라미터 셋 B
원본 패널 영상	8-bit 파이프 결과	10-bit 파이프 결과

세 패널의 배치는 항상 **왼쪽 = 원본 / 가운데 = img2 / 오른쪽 = img3**로 고정되며, 패널 테두리 색은 기본값 기준으로 원본 = 마젠타, img2 = 노랑, img3 = 시안입니다 (`-bc <A> <B> <D>`로 바꿀 수 있습니다). 세 패널의 줌과 팬은 하나의 뷰포트를 공유하므로, 한 패널을 확대하면 나머지 두 패널이 같은 좌표를 같은 배율로 따라옵니다.

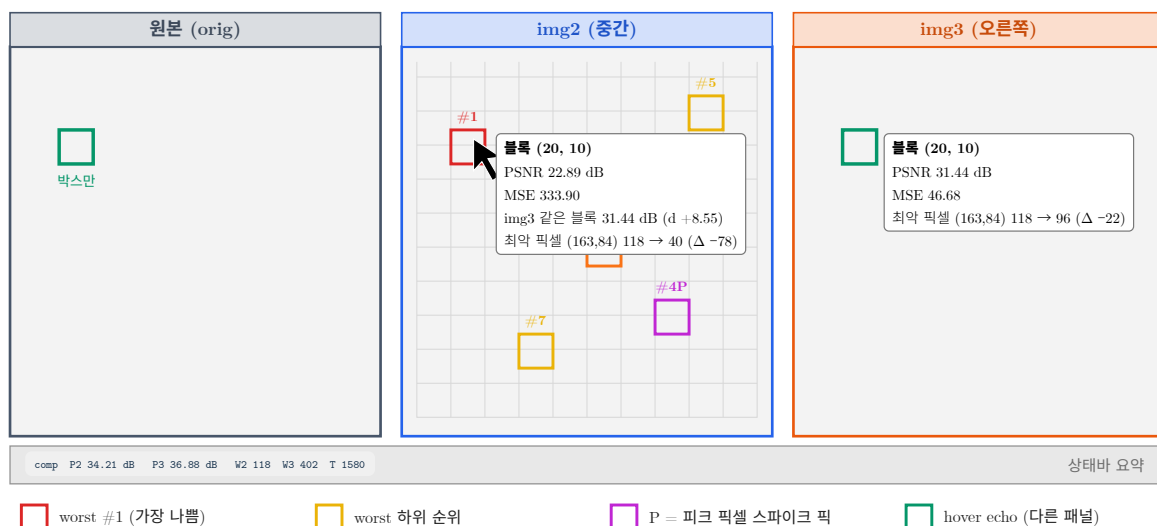


그림 5.1: `--comp` 3패널 화면. 마우스를 img2의 블록에 올리면 나머지 두 패널에 초록 echo 박스가 뜨고, 비교 패널(img3)에는 자기 통계로 계산한 같은 형식의 풍선말이 함께 나타난다.

주의 3-영상 모드에서는 diff 파이프라인이 **자동으로 비활성화** 됩니다. `--comp`를 주면 CLI 파싱 단계에서 diff 모드가 `none`으로 강제되고, 실행 중에도 매 프레임 diff 모드 · 블링크(A/B) · 오버레이

· A/B 스왑이 꺼진 상태로 유지됩니다. 따라서 `Ctrl+0` ~ `Ctrl+9` 같은 diff 키, `Shift+Space` (A/B 스왑), `0` (오버레이)는 comp 모드에서 눌러도 반응하지 않습니다. 대신 이 장에서 설명하는 comp 전용 키 체계를 사용합니다.

5.2. 실행 방법과 CLI 옵션

5.2.1. 기본 실행

가장 단순한 형태는 플래그 하나에 위치 인자 세 개입니다.

```
av --comp orig.png fw.png hw.png
```

세 영상은 **같은 크기** 여야 하고, 픽셀 타입도 짝이 맞아야 합니다(둘 다 8-bit 계열이거나 둘 다 HDR float). 크기나 포맷이 어긋나면 해당 쌍의 통계가 `size/format mismatch` 로 표시되고 그 패널의 블록 오버레이는 그려지지 않습니다.

5.2.2. 세 번째 위치 인자만으로도 활성화

`--comp` 플래그를 깜빡했더라도 위치 인자를 세 개 주면 comp 모드가 자동으로 켜집니다. CLI 파서는 위치 인자를 imageA → imageB → imageC 순으로 채우고, imageC 가 비어 있지 않은데 `--comp` 가 없으면 스스로 `--comp` 를 겁니다. 즉 아래 두 줄은 완전히 같은 동작입니다.

```
av --comp orig.png fw.png hw.png
av orig.png fw.png hw.png
```

참고 위치 인자는 최대 3개입니다. 네 번째를 주면 `Too many image arguments (max 3)` 를 출력하고 종료합니다. 반대로 `--comp` 를 주었는데 영상이 3개가 아니면 `--comp needs three images: av --comp <orig> <img2> <img3>` 를 출력하고 종료합니다.

5.2.3. 옵션 전수

옵션	기본값	설명
<code>--comp</code>	off	3-영상 블록 비교 활성화. 위치 인자 3개면 생략 가능
<code>--blk <WxH></code>	8x8	블록 크기(픽셀). 각 변 2 ~ 1024. <code>--blk 16</code> 처럼 숫자 하나만 주면 정사각형
<code>--num_blk <N></code>	16	표시할 worst 블록 개수. <code>--num_blk</code> 도 같음. 1 미만은 1 로 보정
<code>--blk-metric <m></code>	rgb	worst 랭킹 기준: <code>rgb</code> <code>y</code> <code>chroma</code>
<code>--grid [WxH N]</code>	off / 16x16	시작 시 블록 그리드 표시. 셀 크기는 <code>--blk</code> 와 독립(각 변 2 ~ 1024)
<code>--comp-out <file -></code>	—	헤드리스: 블록별 CSV/JSON 출력 후 종료 (8장 참조)
<code>--comp-batch</code>	off	헤드리스: 세 디렉토리의 같은 이름 프레임 전체를 순회해 프레임당 1행 요약 (8장 참조)

예시 — 옵션을 조합한 실행 예

16x16 블록으로 스캔하고 worst 블록을 24개까지 표시하며, 루마 기준으로 순위를 매기고, 32x32 그리드를 켜 채로 시작합니다.

```
av --comp orig.png fw.png hw.png --blk 16x16 --num_blk 24 --blk-metric y --grid 32
```

주의 `--blk` 값이 2 ~ 1024 범위를 벗어나면 `Invalid --blk: ... (expected WxH, e.g. 8x8)` 를 출력하고 종료합니다. `--blk-metric` 에 `rgb / y / chroma` 이외의 값을 주면 `Unknown --blk-metric: ... (rgb|y|chroma)` 를 출력하고 종료합니다.

5.2.4. 블록 크기를 고르는 기준

블록 크기는 “어느 정도 크기의 결함을 한 덩어리로 볼 것인가”를 정합니다.

크기	쓰임새
<code>--blk 4x4</code>	단일 픽셀~수 픽셀 규모의 점 결함, 라인 버퍼 경계 오류 추적
<code>--blk 8x8</code> (기본)	일반적인 보상 IP 검증. 대부분의 로컬 아티팩트를 잘 분리
<code>--blk 16x16</code>	블록 노이즈, 디더 패턴, 타일 경계 아티팩트
<code>--blk 32x32</code> 이상	무라·얼룩처럼 넓게 퍼지는 저주파 오차. 개수가 줄어 개괄 파악에 유리

블록 개수는 `ceil(W / blk_w) x ceil(H / blk_h)` 이며, 영상 오른쪽.아래 가장자리 블록은 남는 만큼만 잘려서 (예: `8x3`) 계산됩니다. 잘린 블록도 자기 실제 픽셀 수로 정규화되므로 통계가 왜곡되지 않습니다.

5.3. worst 블록 선정 알고리즘

5.3.1. 1단계 — 전체 블록 스캔

원본과 비교 영상을 한 번 훑으면서, 블록마다 다음을 모두 구합니다.

```
MSE_c    = sum((orig_c - cmp_c)^2) / (blk_w * blk_h)    # c = R, G, B
MSE      = (MSE_R + MSE_G + MSE_B) / 3
PSNR(dB) = 20*log10(peak) - 10*log10(MSE)              # MSE = 0 이면 999 (= inf 표기)
peak     = 255.0 (8-bit 계역 쌍) | 1.0 (HDR float 쌍)
```

동시에 블록 안에서 다음 두 값을 함께 기록합니다.

- `nerr` — R/G/B 중 하나라도 값이 다른 픽셀의 개수
- `ld|max` (peak 오차) — 블록 안 모든 픽셀에 대해 `max(|dR|, |dG|, |dB|)` 를 구한 뒤 그 최댓값, 그리고 그 픽셀의 좌표

전역 PSNR(상태바 `P2 / P3`, 정보 창 값)은 블록 PSNR 의 평균이 아니라, 영상 전체의 채널별 MSE 로 부터 구한 **채널별 PSNR 의 평균** 입니다. 이는 2-영상 모드 정보 창 의 PSNR 산출 관례와 동일하므로 4 장의 값과 직접 비교할 수 있습니다.

5.3.2. 2단계 — 하이브리드 랭킹 (75% MSE + 25% peak)

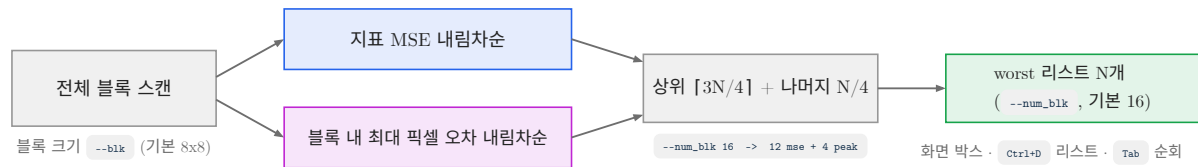
오차가 전혀 없는 블록(RGB MSE = 0)은 후보에서 제외합니다. 남은 후보를 랭킹 지표 MSE 내림차순 으로 정렬한 뒤, 표시할 개수 `want` 를 다음과 같이 둘로 나눕니다.

```
want    = min(--num_blk, 오차가 있는 블록 수)
n_mse   = min(want, (want * 3 + 3) / 4)    # 정수 나눗셈 = 75% 용건
n_peak  = want - n_mse                    # 나머지 25%
```

앞의 `n_mse` 개는 지표 MSE 상위 블록을 그대로 가져오고, 남은 `n_peak` 개는 **그 뒤쪽 후보들을 블록 내 최대 픽셀 오차(`ld|max`) 기준으로 다시 정렬해** 뽑습니다. 이렇게 뽑힌 블록에는 `spike` 표시가 붙어, 화면에서 마젠타 박스와 `P` 라벨로 구분됩니다.

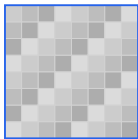
<code>--num_blk</code>	MSE 픽	peak(스파이크) 픽
4	3	1

8	6	2
16 (기본)	12	4
24	18	6
32	24	8



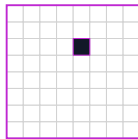
같은 MSE, 다른 결함 — 왜 두 랭킹을 섞는가

(a) 전체가 조금씩 어긋남



MSE 높음 → MSE 랭킹이 잡음

(b) 한 픽셀만 크게 틀



MSE 낮음 → 피크 랭킹만이 잡음

(b)는 8×8 안에서 63픽셀이 정확하고 1픽셀만 크게 어긋난 경우다. MSE는 64로 나누므로 값이 묻히지만, 화면에서는 반짝이는 점 결함으로 바로 보인다. 그래서 av는 상위 3/4을 지표 MSE로, 나머지 1/4을 블록 내 최대 오차로 뽑고 후자를 마젠타 P 로 표시한다. 지표 기준은 `--blk-metric rgb|yl|chroma` 로 바꾼다 — 예를 들어 크로마 결함만 추적하려면 `chroma` 를 쓴다.

그림 5.2: worst 블록 선정은 지표 MSE 랭킹 75%와 피크 픽셀 오차 랭킹 25%를 섞는다. 오른쪽 두 블록은 MSE 는 비슷하지만 성격이 전혀 다른 결함이다.

5.3.3. 왜 하이브리드가 필요한가

8x8 블록은 픽셀이 64개입니다. 이 안에서 단 한 픽셀만 크게 튀는 아티팩트 — 예를 들어 록업 테이블 경계에서 한 점만 잘못 매핑되거나, 감마 보정 반올림이 한 코드값에서만 어긋나는 경우 — 를 생각해 봅시다. 오차 120 짜리 픽셀이 하나뿐이면 블록 MSE 는 대략 $120^2 / 64 / 3 = 75$ 수준에 그칩니다. 반면 전체 픽셀이 오차 10씩 고르게 깔린 밋밋한 블록의 MSE 는 100 입니다. MSE 만으로 순위를 매기면 **눈에 확 띄는 점 결함이 눈에 띄지 않는 저역 오차보다 뒤로 밀립니다.**

DDI 보상 IP 검증에서 실제로 문제가 되는 것은 대개 전자입니다. 평균적으로 조금 어두운 결과는 육안으로 거의 구분되지 않지만, 한 점이 튀는 결함은 패널에서 반짝이는 점으로 그대로 보입니다. 그래서 av 는 상위 75%만 MSE 랭킹으로 채우고, 나머지 25%를 `ld|max` 랭킹으로 채워 두 종류의 실패를 한 화면에 함께 올립니다.

팁 마젠타 P 박스가 여럿 뜨는데 그 블록의 PSNR 은 나쁘지 않다면, “평균은 괜찮은데 국소적으로 크게 튀는” 전형적인 스파이크 아티팩트입니다. 그 블록 위에 마우스를 올려 풍선말의 `worst px` 줄에서 좌표와 `orig / this / delta` 값을 읽고, 해당 코드값이 보상 LUT 의 어느 구간에 걸리는지 역추적하십시오.

5.3.4. 랭킹 지표 선택 — `--blk-metric`

`--blk-metric` 은 위 1단계에서 계산한 세 가지 오차 에너지 중 어느 것으로 순위를 매길지 고릅니다. 화면에 표시되는 블록 PSNR/MSE 자체는 항상 RGB 기준으로 같고, **순서만** 바뀝니다.

값	정의와 용도
rgb (기본)	채널별 MSE 의 산술 평균 $(MSE_R + MSE_G + MSE_B) / 3$. 범용
y	Rec.709 루마 오차의 MSE. 밝기·콘트라스트 계열 결함, 감마·디밍 보상 검증
chroma	Rec.709 CbCr 오차의 MSE. 색틀어짐, 화이트포인트·컬러 보상 검증

계산식은 픽셀별 채널 차 `dR` , `dG` , `dB` 로부터 다음과 같이 유도합니다.

```

dY = 0.2126*dR + 0.7152*dG + 0.0722*dB
dCb = (dB - dY) / 1.8556
dCr = (dR - dY) / 1.5748

y      기준 행킹 MSE = sum(dY^2) / npx
chroma 기준 행킹 MSE = sum(0.5 * (dCb^2 + dCr^2)) / npx

```

예시 — 색 보상만 골라 보기

화이트포인트 보상 IP 를 검증하는데, 밝기 오차는 이미 허용 범위임을 알고 있고 색만 보고 싶습니다.

```
av --comp orig.png fw.png hw.png --blk-metric chroma --num_blk 24
```

이렇게 하면 루마만 어긋난 블록은 순위에서 밀리고, dCb / dCr 이 큰 블록이 위로 올라옵니다. worst 블록에 마우스를 올리면 풍선말에 `rank metric (chroma) MSE: ...` 줄이 추가로 표시되어, 어떤 기준으로 뽑혔는지 바로 확인할 수 있습니다.

5.4. 화면 요소와 오버레이

comp 모드의 모든 오버레이는 세션 한정입니다. `~/.av.ini` 에 저장되지 않으므로, 다음 실행에는 항상 아래의 기본 상태로 시작합니다.

키	기본 상태	내용
<code>Ctrl+B</code>	ON	worst 블록 박스 + hover 풍선말
<code>G</code>	OFF (<code>--grid</code> 지정 시 ON)	전체 블록 그리드 바운더리
<code>Ctrl+W</code>	OFF	블록 승패 맵 (img2 vs img3)
<code>Ctrl+G</code>	OFF	블록 PSNR 히트맵
<code>Ctrl+D</code>	OFF	worst 블록 리스트 창
<code>I</code>	—	정보 창 (img2/img3 각각의 원본 대비 PSNR)

5.4.1. worst 블록 박스 — `Ctrl+B`

img2 패널과 img3 패널에는 각자의 worst 블록이 순위 색 테두리로 그려집니다. 색은 순위에 따른 그라데이션입니다.

- #1(가장 나쁨) = 순수 빨강 `RGB(255, 0, 0)`
- 중간 순위 = 주황
- #N(마지막) = 노랑 `RGB(255, 220, 0)`
- 스파이크 픽(peak 오차로 선정) = 마젠타 `RGB(255, 70, 255)`

박스 왼쪽 위에는 `#1`, `#2` ... 순위 라벨이 붙고, 스파이크 픽은 `#7P` 처럼 `P` 가 따라붙습니다. 박스가 화면에서 너무 작으면(가로 22픽셀 또는 세로 14픽셀 미만) 라벨은 생략되고 테두리만 그려집니다. `Ctrl+B` 로 박스 전체를 잠시 끄면 보상 결과 자체를 가리지 않고 원본 화질을 눈으로 확인할 수 있습니다.

참고 worst 박스와 풍선말은 **비교 패널(img2, img3) 전용** 입니다. 원본 패널에는 worst 박스가 그려지지 않습니다 — 원본에는 “원본 대비 오차”라는 개념이 없기 때문입니다. 원본 패널에 나타나는 것은 그리드, 승패 맵, 선택(SEL) 박스, 그리고 echo 박스뿐입니다.

5.4.2. 전체 블록 그리드 — **G** / `--grid`

G 를 누르면 세 패널 모두에 옅은 하늘색 격자가 그려집니다. worst 블록만이 아니라 **모든** 블록 경계를 보여주므로, 결함이 블록 경계에 정렬돼 있는지(코덱/타일 처리 흔적) 아니면 경계를 가로지르는지(광학.패널 특성)를 판단할 때 유용합니다.

그리드 셀 크기는 `--blk` 와 독립적인 `--grid WxH` 값이며 기본은 `16x16` 입니다. `--grid` 를 값 없이 주면 그리드가 켜진 채로 시작하고 셀은 기본 `16x16` 입니다.

```
av --comp orig.png fw.png hw.png          # 그리드 OFF 로 시작
av --comp orig.png fw.png hw.png --grid    # 16x16 그리드 ON 으로 시작
av --comp orig.png fw.png hw.png --grid 32  # 32x32 그리드 ON
av --comp orig.png fw.png hw.png --blk 8x8 --grid 64x64  # 스캔은 8x8, 그리드는 64x64
```

주의 화면상 셀 한 번이 4픽셀 미만이면(즉 $\text{세크기} \times \text{줄} < 4$) 그리드는 그려지지 않습니다. Fit 줌으로 4K 영상을 보면서 `--grid 8` 을 쳐도 아무것도 보이지 않는 것은 정상이며, 줌을 올리면 나타납니다.

또한 comp 모드에서는 **G** 가 그리드 토글로 재할당되므로, 2장에서 설명한 “이미지 중심으로 이동” 기능은 comp 모드에서 **G** 로 호출할 수 없습니다.

5.4.3. 승패 맵 — **Ctrl+W**

블록마다 img2 와 img3 의 성적을 직접 비교해 우세한 쪽의 색으로 테두리를 칠합니다. 판정 기준은 두 블록 MSE 의 dB 차이입니다.

```
d = 10 * log10(mse_img3 / mse_img2)    # d > 0 이면 img2 가 우세
|d| <= 1dB -> 표시하지 않음 (동급)
d > +1dB   -> 파랑 RGB(70, 160, 255) = img2 우세
d < -1dB   -> 주황 RGB(255, 150, 40)  = img3 우세
```

테두리의 알파(진하기)는 $\min(|d| / 6, 1)$ 에 비례하므로, 차이가 6dB 이상 벌어진 블록은 가장 진하게, 1~2dB 차이는 흐리게 표시됩니다. 즉 **색은 승자, 진하기는 격차** 입니다. 어느 한쪽만 완전 일치(MSE = 0)인 블록은 최대 격차로 취급합니다.

원본 패널 왼쪽 위에는 범례가 표시됩니다.

```
win/loss: blue=img2 better, orange=img3 better (|d|>1dB)
```

승패 맵도 화면상 블록 한 번이 3픽셀 미만이면 그려지지 않습니다. 전체 집계는 상태바의 `W2` / `W3` / `T` 로 항상 확인할 수 있습니다.

5.4.4. 블록 PSNR 히트맵 — **Ctrl+G**

승패 맵이 “둘 중 누가 나은가”라면, 히트맵은 “이 패널이 절대적으로 얼마나 나쁜가”를 보여줍니다. 비교 패널(img2, img3)에만 그려지며, 원본 패널에는 나타나지 않습니다.

```
t = (50 - block_psnr) / 30          # 0 이하이면 그리지 않음, 1 로 클램프
색 = RGB(255, 230*(1-t), 0)         # t=0 노랑 -> t=1 빨강
알파 = 35 + 130*t
```

즉 블록 PSNR 이 **50dB 이상이면 투명** 하고, 50dB 부근은 옅은 노랑, 20dB 이하는 진한 빨강으로 채워 집니다. worst 블록 박스가 “가장 나쁜 16개”만 보여주는 반면, 히트맵은 영상 전체의 열화 분포를 한눈에 보여주므로 “결함이 화면 하단에만 몰려 있다”, “에지 부근 전반이 균일하게 나쁘다” 같은 패턴을 잡아냅니다.

참고 Fit 줌처럼 블록이 화면에서 아주 작아지는 경우, 히트맵은 여러 블록을 3픽셀 이상 크기의 셀로 묶고 **그 안의 최약값**으로 색을 칠합니다. 축소해도 나쁜 블록이 평균에 묻혀 사라지지 않습니다.

5.4.5. worst 블록 리스트 창 — **Ctrl+D**

Ctrl+D 는 **Comp Worst Blocks** 창을 엽니다(comp 모드에서는 4장의 diff 리스팅 대신 이 창이 열립니다). **img2** 와 **img3** 의 worst 목록을 하나로 합쳐 심각도 내림차순으로 정렬한 표이며, 최대 `--num_blk` x 2 행이 표시됩니다. 열은 7개입니다.

열	내용
#	병합 순위. 스파이크 픽이면 <code>12 P</code> 처럼 <code>P</code> 가 붙음
img	<code>img2</code> (하늘색) 또는 <code>img3</code> (주황) — 어느 패널의 블록인지
grid	블록 그리드 좌표 <code>(bx,by)</code>
PSNR(dB)	그 패널의 블록 PSNR (완전 일치면 <code>inf</code>)
MSE	그 패널의 블록 RGB MSE
other PSNR	상대 알고리즘 의 같은 위치 블록 PSNR
d max	블록 내 최대 픽셀 오차와 그 좌표 <code>37.0 (491,707)</code>

창 상단에는 `severity order (metric rgb); click a row to jump all panes. P = peak-pixel pick` 안내가 표시되며, 괄호 안 지표는 `--blk-metric` 설정을 그대로 반영합니다.

행을 클릭하면 세 패널이 동시에 그 블록 중심으로 이동하고 자동 확대되며, 흰색 선택(SEL) 박스가 붙습니다. 현재 선택된 행은 하이라이트로 표시됩니다.

#	img	grid	PSNR(dB)	MSE	other PSNR	d max
1	img3	(88,110)	-0.00	65025.00	18.21	255.0 (712,884)
2	img2	(61,88)	7.57	11371.26	3.02	255.0 (491,707)
3 P	img2	(13,201)	31.44	46.71	29.80	212.0 (109,1612)

팁 `PSNR(dB)` 와 `other PSNR` 을 나란히 읽는 것이 이 창의 핵심입니다. 두 값이 비슷하면 **두 알고리즘이 같이 실패한 영역**이므로 원인은 입력 영상 쪽(원본 자체의 난이도)일 가능성이 높고, 한쪽만 크게 낮으면 **그 구현의 고유 버그**입니다.

5.4.6. 상태바 요약과 정보 창 — **I**

comp 모드에서는 상태바에 요약이 항상 표시됩니다.

P2 23.7dB P3 3.5dB W2 31177 W3 66 T 79

항목	의미
P2	img2 의 원본 대비 전역 PSNR (완전 일치면 <code>P2 inf</code>)
P3	img3 의 원본 대비 전역 PSNR
W2	img2 가 우세한 블록 수 (<code>d > +1dB</code>)
W3	img3 가 우세한 블록 수 (<code>d < -1dB</code>)
T	무승부 블록 수 (<code> d <= 1dB</code>)

`P2` / `P3` 의 글자색은 40dB 초과 = 초록, 30~40dB = 노랑, 30dB 이하 = 빨강입니다. `W2` / `W3` 는 각각 하늘색/주황으로, 승패 맵의 색과 같은 규칙을 씁니다. 승패 집계에는 두 영상 모두 원본과 완전히 같은 블록은 포함되지 않으므로, `W2 + W3 + T` 는 전체 블록 수가 아니라 **오차가 있는 블록 수** 입니다.

`I` 로 정보 창을 열면 세 영상의 경로·해상도와 함께 PSNR 두 줄이 표시됩니다.

```
PSNR img2(mid)   vs orig: 23.71 dB
PSNR img3(right) vs orig: 3.51 dB
```

크기·포맷이 어긋난 쪽은 `N/A (size/format mismatch)` , 완전히 같으면 `inf (identical)` 로 표시됩니다.

5.5. Hover 풍선말과 echo 박스

5.5.1. 풍선말이 보여주는 것

worst 블록 박스 위에 마우스를 올리면 그 블록의 상세 통계가 풍선말로 뜹니다. 이 풍선말 한 장이 comp 모드에서 가장 정보 밀도가 높은 화면 요소입니다.

```
Block #3  img2(mid) vs orig
grid (61,88)  px (488,704)-(495,711)   8x8
PSNR: 21.47 dB   MSE: 463.221
img3(right) same block: PSNR 12.05 dB (d -9.42)
MSE R/G/B: 512.000 / 401.664 / 476.000
error pixels: 61 / 64 (95.3%)
worst px (491,707)  |d|max = 37
  orig  R:128  G:130  B:126
  this  R:165  G:151  B:140
  delta R:+37  G:+21  B:+14
```

각 줄의 의미는 다음과 같습니다.

줄	의미
Block #k ... vs orig	이 패널 내 순위와 패널 이름 (<code>img2(mid)</code> / <code>img3(right)</code>)
grid ... px ...	블록 그리드 좌표, 픽셀 범위(양끝 포함), 실제 블록 크기
pick: peak-pixel spike	스파이크 픽인 경우에만 추가되는 마젠타 줄
rank metric (y chroma) MSE	<code>--blk-metric</code> 이 <code>rgb</code> 가 아닐 때만 추가
PSNR / MSE	이 블록의 RGB 기준 PSNR 과 MSE
... same block: PSNR ... (d ...)	상대 알고리즘 의 같은 블록 성적과 차이
MSE R/G/B	채널별 MSE — 어느 채널이 주범인지 즉시 판별
error pixels	오차 픽셀 수 / 전체 픽셀 수 (백분율)
worst px	최악 픽셀 좌표와 <code> d max</code>
orig / this / delta	최악 픽셀의 원본값, 이 패널 값, 부호 있는 차이

상대 알고리즘 병기 줄의 `d` 는 “상대 블록 PSNR - 이 블록 PSNR” 입니다. 값이 음수이면 상대가 더 나쁘다는 뜻이며 글자색이 초록(이 패널 우세), 양수이면 주황(상대 우세)입니다. 상대 블록이 원본과 완전히 같으면 `PSNR inf (identical)` 로 표시됩니다.

팁 MSE R/G/B 줄에서 한 채널만 값이 튀면 채널별 보상 계수나 크로στο크 계수를, 세 채널이 비슷하면 게인·감마 경로를 먼저 의심하십시오. `delta` 줄의 부호가 전 채널 동일 부호면 밝기 편향, 서로 반대면 색 틀어짐입니다.

5.5.2. 반대쪽 패널의 초록색 echo 박스

풍선말이 뜨는 동안, 같은 위치가 **나머지 두 패널** 에도 형광 초록 테두리로 표시됩니다. 이 echo 박스는 채우기 없이 테두리만 그리므로 아래 픽셀을 가리지 않습니다.

여기서 중요한 점은, 반대쪽 **비교 패널** 에는 박스만이 아니라 같은 형식의 상세 풍선말이 함께 뜬다는 것입니다. 그리고 그 내용은 마우스가 올라간 패널의 값을 복사한 것이 아니라 **그 패널 자신의 통계** 입니다. 그 블록이 그 패널의 worst 목록에 없더라도 해당 블록 하나만 즉석에서 다시 스캔해 계산합니다.

```
img3(right) vs orig (worst #7)
grid (61,88) px (488,704)-(495,711) 8x8
PSNR 12.05 dB MSE 4051.339
MSE R/G/B: 4210.000 / 3901.125 / 4042.891
error px: 64 / 64 (100.0%)
worst px (490,706) |d|max = 121
  orig R:128 G:130 B:126
  this R:249 G:210 B:190
  delta R:+121 G:+80 B:+64
```

첫 줄 끝의 `(worst #7)` 은 그 블록이 자기 패널 worst 목록에도 들어 있을 때만 붙습니다. 없으면 `img3(right) vs orig` 로만 표시됩니다. echo 풍선말의 테두리는 초록이라 마우스가 올라간 쪽의 풍선말과 시각적으로 구분됩니다.

원본 패널에는 초록 echo 박스만 그려지고 풍선말은 뜨지 않습니다. 원본에는 비교 대상이 없기 때문입니다.

예시 — 한 번의 hover 로 두 알고리즘을 동시에 읽기

img2 패널의 #3 빨간 박스에 마우스를 올립니다. 그러면

1. img2 패널: 흰 배경 풍선말에 `PSNR 21.47 dB`, 그리고 `img3(right) same block: PSNR 12.05 dB (d -9.42)` — img3 이 9.42dB 더 나쁘다는 사실이 한 줄에.
2. img3 패널: 같은 좌표에 초록 박스 + 초록 테두리 풍선말에 img3 자신의 전체 통계 (`error px: 64 / 64`, `|d|max = 121`).
3. 원본 패널: 같은 좌표에 초록 박스 — 원본에서 그 자리가 어떤 그림인지 확인.

마우스를 한 번도 옮기지 않고 “원본은 이런 그림인데, img2 는 이만큼, img3 는 이만큼 틀어졌다”를 읽어낼 수 있습니다.

5.6. worst 블록 네비게이션

worst 박스는 화면 곳곳에 흩어져 있고, Fit 줌에서는 8x8 블록이 몇 픽셀에 불과합니다. 그래서 av 는 “다음 나쁜 블록으로 점프” 를 키 하나로 제공합니다.

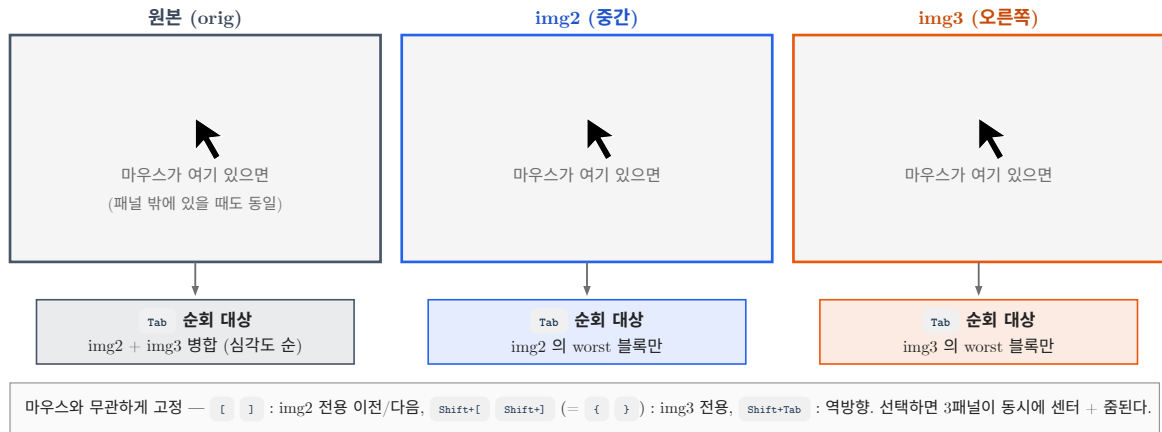


그림 5.3: Tab의 순회 대상은 마우스가 올라가 있는 패널로 결정된다. 대괄호 키는 마우스 위치와 무관하게 대상이 고정된다.

5.6.1. 점프의 스코프 규칙

키	순회 대상
Tab / Shift+Tab	마우스 커서가 올라가 있는 패널의 worst 블록만. 커서가 가운데 패널이면 img2, 오른쪽 패널이면 img3. 원본 패널 위이거나 패널 밖이면 img2 + img3 병합 심각도순
[/]	항상 img2(가운데)의 worst 블록만 이전 / 다음
Shift+[/ Shift+]	항상 img3(오른쪽)의 worst 블록만 이전 / 다음. 키보드에 따라 { / } 로 표기

Tab은 마우스 위치를 매 프레임 읽어 스코프를 정합니다. 따라서 **마우스를 가운데 패널에 놓고 Tab을 연타하면 img2의 결합만 훑고**, 오른쪽 패널로 옮겨 연타하면 img3의 결합만 훑습니다. 마우스를 원본 패널에 두면 두 알고리즘의 결합을 심각도 순서로 섞어서 훑습니다. 순회 범위가 바뀌면 위치는 처음부터 다시 시작합니다.

참고 마우스로 조준하기 번거로울 때는 [/] 계열을 쓰십시오. 마우스가 어디 있든 대상 패널이 고정되므로, 키보드만으로 “img2만 16개 훑고 → img3만 16개 훑기” 같은 작업 순서를 만들 수 있습니다.

comp 모드에서는 Tab이 패널 포커스 전환 대신 블록 순회로, [/]가 diff 증폭 조절 대신 블록 순회로 재할당됩니다.

5.6.2. 점프할 때 화면에서 일어나는 일

블록이 선택되면 다음이 동시에 일어납니다.

1. 세 패널의 뷰포트가 그 블록 중심으로 이동 합니다(Fit 해제).
2. 현재 줌이 4배 미만이면 8배로 자동 확대 합니다. 이미 4배 이상이면 배율을 유지합니다 — 사용자가 골라 둔 확대 배율을 임의로 바꾸지 않기 위해서입니다.
3. 세 패널 모두에 **흰색 선택(SEL) 박스**와 그림자 테두리가 그려집니다.
4. 박스 아래에 SEL #3 img2 처럼 **순위와 소속 패널** 라벨이 표시됩니다.

라벨의 순위는 병합 순서가 아니라 **해당 패널 안에서의 순위**이므로, 리스트 창의 # 열(병합 순위)과 다를 수 있습니다.

예시 — 결합 순회 루틴

```
av --comp orig.png fw.png hw.png --num_blk 16
```

1. 마우스를 가운데(img2) 패널 위에 둡니다.
2. `Tab` — img2 의 #1 블록으로 점프, 8배 확대, `SEL #1 img2` 표시.
3. 풍선말을 읽고(원한다면 `Ctrl+D` 리스트 창과 대조), 다시 `Tab`.
4. 16번 누르면 처음(#1)으로 순환합니다. `Shift+Tab` 으로 되돌아갈 수 있습니다.
5. 마우스를 오른쪽(img3) 패널로 옮기고 `Tab` — 이제 img3 의 #1 로 갑니다.

5.7. 패널 스왑 · 3-way 블링크 · 시퀀스 미러

5.7.1. img2 ↔ img3 스왑 — `S`

`S` 는 가운데 패널과 오른쪽 패널의 영상을 맞바꿉니다. 단순한 화면 교환이 아니라 슬롯 자체가 교환되므로 블록 통계가 다시 계산되고, `P2 / P3`, `W2 / W3`, worst 목록, 승패 맵 색이 모두 새 배치를 따릅니다.

두 결과를 오래 들여다보면 “왼쪽에 있는 쪽이 더 나빠 보이는” 위치 편향이 생기기 쉽습니다. `S` 로 자리를 바꿔 같은 판단이 유지되는지 확인하면 이 편향을 제거할 수 있습니다.

주의 comp 모드에서 `S` 는 줌/팬 동기화 토글이 아니라 패널 스왑입니다. comp 모드는 세 패널이 하나의 뷰포트를 공유하므로 동기화 토글 자체가 의미가 없습니다. 통계 창은 여전히 `Ctrl+S` 입니다.

5.7.2. 3-way 블링크 — `,`

`,` 를 누르면 화면이 단일 패널로 바뀌고 `orig` → `img2` → `img3` → `orig` 순으로 자동 순환합니다. 같은 자리에서 세 영상이 겹쳐 재생되므로, 나란히 놓고 비교할 때는 놓치는 미세한 이동·깜빡임·색 이동이 눈에 잘 들어옵니다.

- 화면 왼쪽 위에 현재 위상 이름이 크게 표시됩니다: `orig / img2 / img3`
- 패널 테두리 색도 위상에 따라 마젠타 → 노랑 → 시안으로 바뀝니다
- 간격은 기본 0.5초이며 `<` / `>` 로 0.1초 단위로 조절합니다(0.1~2.0초)
- 세 영상이 모두 로드돼 있어야 시작합니다. 다시 `,` 를 누르면 3패널로 복귀합니다

블링크 중에도 worst 박스·그리드·히트맵 등 오버레이는 그 위상의 패널 규칙대로 그려집니다(원본 위상에서는 worst 박스가 없습니다).

5.7.3. 시퀀스 이동과 자동 미러 — `;` / `A`

comp 모드에서 `;` (다음) 과 `A` (이전) 는 **언제나 원본 패널의 디렉토리** 를 기준으로 동작합니다. 어느 패널이 활성 상태이든 원본이 시퀀스를 구동하며, 새 원본 파일을 읽은 뒤 **같은 파일명** 을 img2 의 디렉토리나 img3 의 디렉토리에서 찾아 자동으로 미러 로드합니다. 이어서 블록 통계가 새 프레임 기준으로 다시 계산되고, 상태바·worst 박스·승패 맵이 즉시 갱신됩니다.

```
seq_orig/frame_0001.png  frame_0002.png  ...
seq_fw/  frame_0001.png  frame_0002.png  ...
seq_hw/  frame_0001.png  frame_0002.png  ...
```

```
av --comp seq_orig/frame_0001.png seq_fw/frame_0001.png seq_hw/frame_0001.png
```

이 상태에서 `;` 를 누르면 세 패널이 동시에 `frame_0002.png` 로 넘어갑니다. 파일명이 상단 중앙에 1.5초 토스트로 표시되고, 마지막 프레임 다음에는 처음으로 순환합니다. `Alt` + `휠`, 그리고 슬라이드쇼 (`Shift+A`) 도 같은 미러 경로를 사용합니다. 시퀀스 일반 규칙은 7장을 참조하십시오.

주의 img2 또는 img3 디렉토리에 같은 이름의 파일이 없으면 해당 패널은 비워지고 파일명이 “없음” 메시지로 표시됩니다. 세 디렉토리의 파일명 규칙을 반드시 일치시키십시오.

5.8. 시작 시 [comp] 요약 출력

comp 모드로 실행하면 창이 뜨기 **전에** stdout 으로 요약 두 줄이 출력되고 즉시 플러시됩니다. 파이프나 파일로 리다이렉트해도 바로 보이므로, GUI 를 띄운 채로도 스크립트에서 수치를 회수할 수 있습니다.

```
av --comp test/diff-org.png test/diff-2.png test/SPR3.bmp
```

```
[comp] img2 vs orig: PSNR 23.7052 dB, worst blocks shown: 16 (12 mse + 4 peak) (#1 grid(61,88)
psnr 7.57271 mse 11371.26)
[comp] img3 vs orig: PSNR 3.50965 dB, worst blocks shown: 16 (12 mse + 4 peak) (#1 grid(88,110)
psnr -0.0000 mse 65025.00)
```

한 줄의 구성은 다음과 같습니다.

필드	의미
img2 vs orig / img3 vs orig	어느 패널의 요약인지
PSNR ... dB	원본 대비 전역 PSNR. 완전 일치면 <code>inf</code>
worst blocks shown: 16 (12 mse + 4 peak)	표시된 worst 블록 총 개수와 MSE 픽 / 스파이크 픽 내역
(#1 grid(bx,by) psnr ... mse ...)	#1 블록의 그리드 좌표와 성적. worst 블록이 없으면 생략

크기.포맷이 어긋난 쪽은 `[comp] img2 vs orig: size/format mismatch` 한 줄로 대체됩니다.

예시 — GUI 실행에서 수치만 뽑아 게이트 걸기

```
av --comp orig.png fw.png hw.png | tee comp.log &
grep -o 'img2 vs orig: PSNR [0-9.]*' comp.log
```

또는 스파이크 픽 개수만 확인:

```
grep -o 'peak)' comp.log | wc -l
```

다만 정식 CI 게이트라면 GUI 없이 도는 `--comp-out` / `--comp-batch` 를 쓰는 편이 낫습니다(아래).

5.9. 헤드리스 comp 출력 — `--comp-out` / `--comp-batch`

`--comp-out <file|->` 는 창·GL 없이 블록별 전수 테이블(CSV 또는 `--format json`)을 내보내고 종료하며, `--comp-batch` 는 세 영상의 디렉토리 전체를 같은 파일명끼리 묶어 프레임당 한 줄의 요약 CSV 를 내보내고 종료합니다. 둘 다 `--comp` 와 함께 3개 영상이 있어야 하고 `--blk` / `--num_blk` 설정을 그대로 따릅니다. 컬럼 정의, 종료 코드, CI 파이프라인 연동 방법은 8장을 참조하십시오.

5.10. 실무 워크플로우

5.10.1. (a) 프레임 한 장에서 두 보상 알고리즘 중 나쁜 쪽 찾기

퍼웨어 C 모델(`fw.png`)과 RTL 결과(`hw.png`)가 같은 입력에서 서로 다른 그림을 냈고, 어느 쪽이 문제인지 30초 안에 판정해야 하는 상황입니다.

1. 실행하고 콘솔의 `[comp]` 두 줄로 전역 PSNR 을 먼저 읽습니다.

```
av --comp orig.png fw.png hw.png --blk 8x8 --num_blk 16
```

- 상태바의 **W2** / **W3** / **T** 를 봅니다. 예를 들어 **W2 31177 W3 66 T 79** 라면 **img2(fw)**가 압도적으로 우세하므로 의심 대상은 **img3(hw)** 입니다.
- Ctrl+W** 로 승패 맵을 커서 주황(= **img3** 우세) 영역이 어디에 남아 있는지 확인합니다. 특정 영역에만 주황이 몰려 있다면 그 영역이 **hw** 의 강점/약점 경계입니다.
- Ctrl+G** 로 히트맵을 커서 **img3** 패널의 빨간 영역 분포를 봅니다. 전면이 붉으면 전역 계수 오류, 한 쪽에만 몰리면 좌표 의존 로직(라인 버퍼, 타일 경계) 문제일 가능성이 큼니다.
- 마우스를 오른쪽(**img3**) 패널에 두고 **Tab** 을 눌러 **#1** 블록으로 점프합니다. 자동으로 8배 확대되며 세 패널이 같은 자리를 보여줍니다.
- 풍선말의 **MSE R/G/B** 로 주범 채널을, **worst px** 의 **orig** / **this** / **delta** 로 실제 코드값 차이를 읽습니다. 반대쪽(**img2**) 패널의 초록 풍선말과 비교하면 같은 자리에서 **fw** 는 얼마나 잘 맞췄는지 즉시 알 수 있습니다.
- 마젠타 **P** 박스가 있으면 **Ctrl+D** 리스트 창에서 **ld|max** 열이 큰 행을 눌러 스파이크 위치를 하나씩 확인합니다.
- 마지막으로 **S** 로 두 결과의 자리를 바꿔 같은 결론이 나오는지 검증하고, **,** 로 3-way 블링크를 돌려 육안 인상까지 확인합니다.

5.10.2. (b) 시퀀스 전체를 넘기며 회귀 프레임 찾기

신버전 보상 알고리즘이 특정 장면에서만 퇴행한다는 제보가 들어왔고, 100프레임짜리 시퀀스에서 그 프레임을 찾아야 하는 상황입니다.

- 세 디렉토리의 첫 프레임으로 시작합니다.

```
av --comp seq_orig/f_0001.png seq_old/f_0001.png seq_new/f_0001.png --num_blk 16
```

- Ctrl+W** 로 승패 맵을 켜 둡니다. 이제 상태바의 **W2** (구버전 우세) 와 **W3** (신버전 우세) 가 프레임 품질의 요약 지표가 됩니다.
- ;** 를 눌러 프레임을 넘깁니다. 세 패널이 같은 파일명으로 동시에 갱신되고 통계가 자동 재계산됩니다. 되돌아가려면 **A** 입니다.
- W2** 가 갑자기 커지고 **P3** 가 떨어지는 프레임이 회귀 지점입니다. 그 프레임에서 멈추고 (a) 절차로 들어갑니다.
- 프레임 수가 많아 손으로 넘기기 부담스러우면, 같은 시퀀스를 헤드리스로 한 번 돌려 회귀 프레임 목록을 먼저 뽑고 GUI 로는 해당 프레임만 확인하는 것이 빠릅니다 (**--comp-batch** , 8장 참조).

팁 회귀 프레임을 찾은 뒤에는 **--blk** 를 바꿔 가며 다시 열어 보십시오. **--blk 4x4** 에서 **worst** 블록이 흩어지면 점 결함, **--blk 32x32** 에서도 같은 자리에 뭉쳐 있으면 넓은 영역의 저주파 오차입니다. 결함의 공간 스케일을 이렇게 좁혀 두면 원인 블록(LUT, 필터 탭, 라인 메모리)을 훨씬 빨리 특정할 수 있습니다.

5.11. comp 모드에서 달라지는 키 요약

아래 키들은 일반 모드와 **comp** 모드에서 동작이 다릅니다. 전체 핫키 목록은 **Ctrl+Shift+H** 도움말 창과 11장 부록을 참조하십시오.

키	일반 모드	comp 모드
Tab	A/B 패널 포커스 전환	마우스 아래 패널의 worst 블록 순회
[/]	diff 증폭 조절	img2 worst 블록 이전 / 다음

Shift+[/ Shift+]	diff threshold 조절	img3 worst 블록 이전 / 다음
G	이미지 중심으로 이동	블록 그리드 토글
S	줌/팬 동기화 토글	img2 ↔ img3 패널 스왑
Ctrl+B	—	worst 블록 박스 토글
Ctrl+G	—	블록 PSNR 히트맵 토글
Ctrl+W	윈도우 모드 토글	승패 맵 토글
Ctrl+D	diff 리스팅 창	worst 블록 리스트 창
,	A/B 블링크	orig → img2 → img3 3-way 블링크
; / A	활성 패널의 시퀀스 이동	원본 시퀀스 이동 + img2·img3 자동 미러
I	A 대비 B PSNR 1줄	img2·img3 각각의 원본 대비 PSNR 2줄

6. 분석 도구 창

3장에서 배운 픽셀 단위 검사와 4장의 Diff 모드가 “한 점” 또는 “한 화면”을 보는 도구였다면, 이 장의 분석 창들은 이미지 전체 또는 지정한 영역을 숫자와 그래프로 환산해 줍니다. 히스토그램, 수평·수직 절단면, 전체 통계, ROI 통계, 산점도의 다섯 가지 창과 이들을 PNG·CSV로 내보내는 방법을 다룹니다. DDI 보상 IP 출력을 원본과 대조할 때, “눈으로는 비슷한데 수치로는 어떤가”를 답하는 곳이 바로 이 장입니다.

6.1. 다섯 개의 창과 공통 규칙

6.1.1. 창 목록

키	창 제목	대상
Ctrl+H	Histogram	A / B / Diff A-B
Ctrl+L	H-Line Cut	A / B / Diff A-B
Ctrl+Y	V-Line Cut	A / B / Diff A-B
Ctrl+S	Image Statistics	A / B / Diff A-B
Ctrl+E	ROI Statistics	ROI 안의 A / B / Diff A-B
Ctrl+T	Scatter Plot (A vs B)	A vs B 픽셀쌍

다섯 창 모두 상단 메뉴의 **View** 에서도 같은 항목으로 열 수 있으며, 메뉴에는 위 단축키가 그대로 표시됩니다.

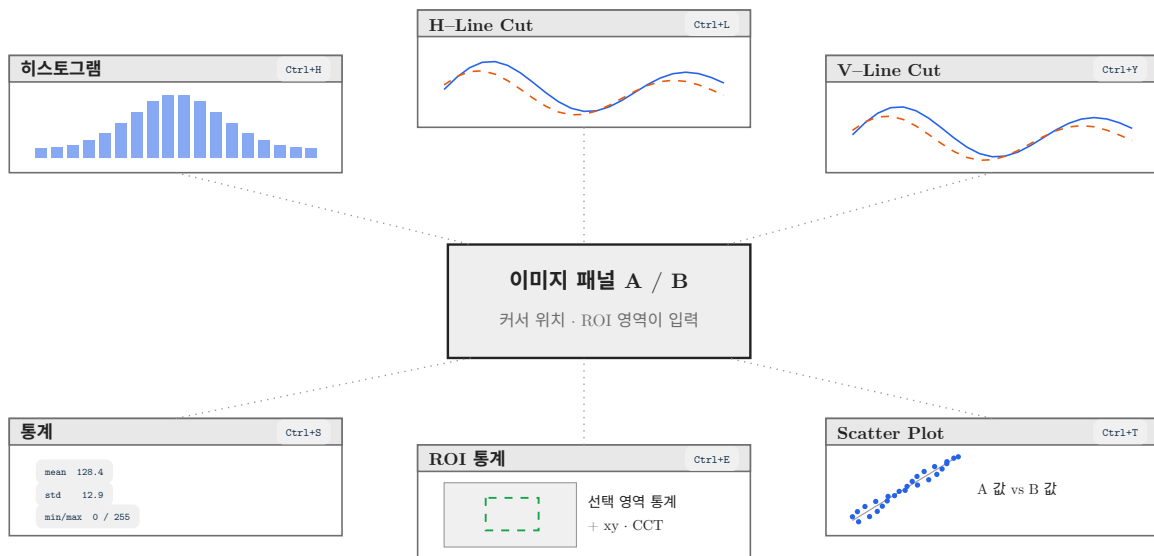


그림 6.1: 이미지 패넬을 중심으로 열리는 분석 창들과 단축키.

6.1.2. 상호 배타 규칙

Ctrl+H / Ctrl+L / Ctrl+Y / Ctrl+S 네 창은 서로 배타적입니다. 하나를 켜면 나머지 셋이 자동으로 닫힙니다. 화면 대부분을 차지하는 큰 창들이라 겹쳐 놓아도 읽을 수 없기 때문입니다.

반면 ROI Statistics (Ctrl+E)와 Scatter Plot (Ctrl+T)은 독립적이어서 위 네 창 중 하나와 동시에 띄워둘 수 있습니다.

6.1.3. 이동 · 크기 · 닫기

조작	방법
----	----

이동	창 제목 표시줄을 좌드래그
크기	창 가장자리 또는 오른쪽 아래 모서리를 좌드래그
닫기	제목 표시줄 오른쪽 x 버튼, 같은 단축키 다시 누르기, 또는 Esc

처음 열릴 때의 기본 크기는 다음과 같습니다.

창	첫 표시 크기
Histogram / H-Line Cut / V-Line Cut / Image Statistics	작업 영역의 가로 90% × 세로 85%
Scatter Plot	작업 영역의 가로 50% × 세로 70%
ROI Statistics	가로 600 px, 세로는 내용에 맞춰 자동

참고 창을 옮기거나 크기를 바꾸면 그 위치·크기는 프로그램을 실행한 디렉터리의 `.av_imgui.ini` 에 자동 저장되어 다음 실행 때 복원됩니다. 한편 “어떤 창이 열려 있었는가”는 홈 디렉터리의 `~/av.ini` 에 `show_histogram / show_hline_cut / show_vline_cut / show_stats / show_scatter_plot / histogram_compare` 키로 저장됩니다. ROI 통계 창의 열림 상태(`show_roi_stats`)만은 저장되지 않아, 다시 실행하면 항상 닫힌 상태로 시작합니다.

주의 **Esc** 는 “열려 있는 것 중 하나”를 정해진 우선순위로 닫습니다. 분석 창끼리의 순서는 `Histogram` → `H-Line Cut` → `V-Line Cut` → `Image Statistics` → `ROI Statistics` → `Scatter Plot` 이고, 그 뒤로 `Image Info` · 픽셀 풍선말 · 저장 창 · Crosshair · ROI 모드가 이어집니다. 여러 개를 띄워 놓았다면 **Esc** 를 여러 번 눌러야 모두 닫힙니다.

6.1.4. 어느 이미지가 A 이고 어느 것이 B 인가

분석 창의 A / B 는 언제나 **로드 슬롯**을 가리킵니다.

라벨	가리키는 것
A	명령줄의 첫 번째 이미지 인자 (또는 <code>Open</code> 대화상자로 왼쪽 패널에 연 이미지)
B	명령줄의 두 번째 이미지 인자 (오른쪽 패널)

```
av orig.png comp_ip_out.png
# → A = orig.png, B = comp_ip_out.png
```

다음 세 가지 규칙을 기억하면 헷갈릴 일이 없습니다.

- Shift+Space** 의 A/B 화면 스왑은 분석 창에 영향을 주지 않습니다. 화면상 좌우가 바뀌어도 분석 창의 A 는 여전히 첫 번째 인자입니다.
- comp** 모드에서 **S** 로 `img2 ↔ img3` 을 교환하면 B 통계가 바뀝니다. 이 키는 두 번째 슬롯의 내용 자체를 바꾸므로 분석 창도 새 이미지를 읽습니다 (5장 참조).
- Diff |A-B|** 섹션은 항상 A 기준입니다. 절단면 창의 Diff 섹션은 A 패널의 pan 값으로 행·열을 정합니다.

참고 `Histogram / H-Line Cut / V-Line Cut / Image Statistics` 창의 `Diff |A-B|` 섹션은 두 이미지가 모두 로드되어 있고 Diff 모드가 켜져 있을 때만 나타납니다 (`D` 등, 4장 참조). Diff 모드가 `none` 이면 A 섹션과 B 섹션만 보입니다. 예외적으로 `ROI Statistics` 의 `ROI Diff |A-B|` 표는 Diff 모드와 무관하게 두 이미지만 로드되어 있으면 언제나 계산됩니다.

6.2. 히스토그램 — Ctrl+H

6.2.1. 무엇이 보이는가

Ctrl+H 를 누르면 Histogram 창이 열리고, 로드된 이미지마다 한 섹션씩 쌓입니다. 각 섹션은 파일 이름을 제목으로 하고 그 아래 R, G, B 세 개의 차트로 분리되어 그려집니다. 세 채널을 한 그래프에 겹쳐 그리지 않기 때문에 채널별 클리핑이나 계조 결손을 서로 가리지 않고 볼 수 있습니다.

요소	의미
섹션 제목	A: orig.png / B: comp_ip_out.png / Diff A-B
빈(bin) 수	항상 256 개 (X축 0 ~ 255)
X축 눈금	0, 32, 64, 96, 128, 160, 192, 224, 255 고정
Y축 눈금	실제 픽셀 개수. 천 단위 콤마 표기 (예: 1,234,567)
막대 색	R = 빨강, G = 초록, B = 파랑

주의 8비트 이미지는 픽셀값이 그대로 빈 번호가 되지만, float/HDR 이미지는 그 이미지 안의 최댓값으로 나눈 뒤 0 ~ 255 로 환산해 담습니다. 즉 float 이미지의 X축은 “절대 코드값”이 아니라 “그 이미지 최댓값 대비 비율”입니다. 절대값이 필요하면 통계 창(Ctrl+S)의 Min / Max 행을 함께 보세요.

6.2.2. 로그 스케일

세로축은 log1p(count) 로 정규화됩니다. 자연 이미지의 히스토그램은 특정 계조에 수십만 개가 몰리고 나머지는 수십 개뿐이라, 선형 축으로는 꼬리가 완전히 보이지 않기 때문입니다. 정규화는 **채널마다 독립적으로** 그 채널의 최댓값을 기준으로 수행되므로, R 차트의 꼭대기와 G 차트의 꼭대기는 서로 다른 개수를 뜻할 수 있습니다. 실제 개수는 Y축 눈금 라벨에 그대로 적혀 있으니 그 숫자를 읽으면 됩니다.

팁 DDI 보상 결과에서 “계조 결손(빠진 코드)”을 찾을 때 로그 스케일이 결정적입니다. 선형 축이라면 개수 0 인 빈과 개수 5 인 빈이 똑같이 바닥에 붙어 보이지만, 로그 축에서는 개수 0 인 빈만 완전히 비어 있는 세로 틈으로 드러납니다.

6.2.3. 값 읽기 — 호버 크로스헤어

차트 위에 마우스를 올리면 세 채널 차트를 세로로 관통하는 흰 크로스헤어가 그려지고, 풍선말에 그 빈의 정확한 값이 표시됩니다.

Bin: 128
 R: 12,048
 G: 11,730
 B: 9,415

한 번의 호버로 R/G/B 세 값을 동시에 읽을 수 있으므로, 특정 코드값에서 채널 간 분포 편차를 비교하기에 좋습니다.

6.2.4. Diff |A-B| 히스토그램

두 이미지가 로드되어 있고 Diff 모드가 켜져 있으면 세 번째 섹션으로 Diff |A-B| 히스토그램이 추가됩니다. 8비트일 때 X축은 채널별 절대 오차 0 ~ 255 를 그대로 나타냅니다. 즉 **빈 0의 막대 높이 = 완전히 일치한 픽셀 수** 이고, 오른쪽으로 갈수록 오차가 큰 픽셀입니다.

예시 — 보상 오차 분포 확인

```
av orig.png comp_ip_out.png --diff-mode abs
```

Ctrl+H 로 히스토그램을 연 뒤 **Diff |A-B|** 섹션을 봅니다. 막대가 빈 0~2 에만 몰려 있으면 LSB 수준의 반올림 차이뿐이고, 빈 8 근처에 두 번째 봉우리가 생겼다면 특정 계조 구간에서 보상 LUT가 한 단계 어긋났다는 신호입니다. 그 봉우리에 마우스를 올려 정확한 픽셀 개수를 읽고, 4장의 **highlight** 모드로 그 픽셀들이 화면 어디에 모여 있는지 확인하십시오.

6.2.5. A/B 동시 표시 — **Compare A/B**

두 이미지가 모두 로드되면 창 맨 아래에 **Compare A/B** 체크박스가 나타납니다. 체크하면 **Compare A (magenta) / B (yellow)** 라는 제목의 추가 차트 3개(R/G/B)가 그려지고, 한 차트 위에 A는 마젠타, B는 노랑 반투명 막대로 **겹쳐서** 그려집니다.

항목	동작
A 막대	마젠타 (반투명)
B 막대	노랑 (반투명)
정규화	A와 B의 공통 최대값을 기준으로 로그 정규화 — 두 곡선의 높이를 직접 비교 가능
입력	8비트 픽셀 버퍼 기준
영속화	<code>~/av.ini</code> 의 <code>histogram_compare</code> 로 저장 (기본 꺼짐)

팁 언제 여는가: 보상 IP가 전체 밝기나 감마를 통째로 밀어 버렸는지 한눈에 판정할 때. **Compare A/B** 를 켜를 때 노랑 분포가 마젠타 분포에 대해 통째로 오른쪽으로 평행 이동했다면 오프셋 편차, 오른쪽으로 늘어났다면 게인(감마) 편차입니다. 모양은 같은데 특정 구간만 빗살처럼 비었다면 LUT 양자화 문제입니다.

6.3. 수평 절단면 — **Ctrl+L** / 수직 절단면 — **Ctrl+Y**

6.3.1. 무엇이 보이는가

Ctrl+L 은 **H-Line Cut** (가로 한 줄), **Ctrl+Y** 는 **V-Line Cut** (세로 한 줄) 창을 엽니다. 두 창의 구조는 완전히 같고, 가로/세로만 다릅니다. 히스토그램과 마찬가지로 이미지마다 한 섹션, 섹션마다 R/G/B 세 개의 꺾은선 차트가 그려집니다.

창	샘플하는 선	가로축
H-Line Cut	한 개의 행 (row)	열 좌표 0 ~ <code>width-1</code>
V-Line Cut	한 개의 열 (column)	행 좌표 0 ~ <code>height-1</code>

섹션 제목에는 파일 이름과 함께 지금 잘라 낸 위치가 표시됩니다.

```
A: orig.png row=540
B: comp_ip_out.png row=540
Diff |A-B| row=540
```

6.3.2. 어느 행 · 어느 열이 잘리는가 (중요)

절단 위치는 마우스 커서가 아니라 **그 패널 뷰포트의 정중앙**입니다. 내부적으로 다음과 같이 계산됩니다.

```
row = round(-pan_y + height / 2) # H-Line Cut
col = round(-pan_x + width / 2)  # V-Line Cut
```

따라서 **보고 싶은 줄을 패널 한가운데로 pan 해서 가져오는 것**이 사용법입니다. `j` / `k` 로 위아래, `h` / `l` 로 좌우 1픽셀씩 이동하고, `Shift` 를 함께 누르면 `--pan-step` 만큼(기본 32픽셀) 점프합니다 (2장 참조). 절단면 창을 띄워 둔 채 pan 하면 곡선이 실시간으로 갱신됩니다.

팁 `s` 로 뷰포트 동기화(`sync_viewports` , 기본 켜짐)를 유지하면 A 패널과 B 패널의 pan 이 같아져 A · B · Diff 세 섹션이 **정확히 같은 행/열**을 봅니다. 동기화를 꺼 두면 A는 540행, B는 300행을 보는 상태가 될 수 있으니 주의하십시오. Diff 섹션은 어떤 경우든 A 패널의 pan 을 기준으로 합니다.

6.3.3. 축과 정규화

요소	의미
X축 눈금	<code>1 / 2 / 5</code> × 10의 거듭제곱 규칙으로 고른 “예쁜” 픽셀 좌표 눈금 (최대 7구간)
Y축 눈금 (8비트)	실제 코드값. 최댓값 255 기준 4등분 (<code>64 / 128 / 191 / 255</code>)
Y축 눈금 (float)	그 선 안의 최댓값 기준 4등분, 소수점 2자리로 표시
곡선 점 수	차트 폭(px)과 데이터 길이 중 작은 쪽으로 균등 다운샘플

즉 4K 이미지의 3840픽셀짜리 행을 900픽셀 폭 차트에 그릴 때는 900점으로 균등 추출해 그립니다. 세밀한 스파이크를 놓치지 않으려면 창을 넓히거나 최대화하십시오.

6.3.4. 값 읽기 — 호버 크로스헤어

차트 위에 마우스를 올리면 세 채널을 관통하는 크로스헤어와 함께 풍선말이 뜹니다.

```
X: 1920          # V-Line Cut 에서는 "Y: 540"
_____
R: 128
G: 130
B: 127
```

8비트 이미지는 정수 코드값으로, float 이미지는 소수점 4자리로 표시됩니다.

6.3.5. Diff |A-B| 절단면

Diff 모드가 켜져 있으면 세 번째 섹션에 채널별 절대 오차 곡선이 그려집니다. 평평하게 바닥에 붙어 있어야 정상이고, 특정 구간에서 솟아오르면 그 좌표 구간의 보상이 틀어진 것입니다.

예시 — 가로 밴딩 / 무라 라인 프로파일 확인

```
av flat_gray128.png comp_flat_gray128.png --diff-mode abs
```

플랫 필드 패턴에서 세로 방향 밴딩이 의심되면 `Ctrl+Y` 로 `V-Line Cut` 을 엽니다. 정상이라면 세 채널 모두 128 근처의 평탄한 직선이어야 합니다. B 섹션 곡선에 주기 8픽셀의 톱니가 보이면 게이트 드라이버 블록 경계의 무라이고, 완만한 사인파 모양이면 전원 리플이나 감마 탭 보간 오차입니다. `Shift+j` 로 몇 줄씩 옮겨 가며 그 주기가 열 방향으로 유지되는지 확인하면 “라인 결함”인지 “면 결함”인지 가려낼 수 있습니다. 마찬가지로 가로 줄무늬는 `Ctrl+L` 로 확인합니다.

6.4. 통계 창 — `Ctrl+S`

6.4.1. 무엇이 보이는가

`Ctrl+S` 는 `Image Statistics` 창을 엽니다. 이미지마다 한 개의 표가 쌓이며, 표 제목에는 파일 이름과 해상도가 붙습니다 (예: `A: orig.png (3840 x 2160)`). A 섹션 제목은 마젠타, B는 노랑, `Diff |A-B|` 는 시안색으로 구분됩니다.

표는 5열 구조입니다: `Metric` | `Red (R)` | `Green (G)` | `Blue (B)` | `Description` .

6.4.2. 표시되는 통계량 전수

Metric	의미
<code>Pixels</code>	이미지의 전체 픽셀 수 (width × height). 천 단위 콤마 표기
<code>Min</code>	채널 최솟값
<code>Max</code>	채널 최댓값
<code>Dynamic Range</code>	<code>Max - Min</code>
<code>Mean</code>	산술 평균
<code>Std Dev</code>	표준편차 — 평균 주위의 퍼짐
<code>Variance</code>	분산 (표준편차의 제곱, 모분산)
<code>Median</code>	중앙값 — 이상치에 평균보다 강건
<code>Skewness</code>	왜도. 0 = 대칭, 양수 = 오른쪽 꼬리, 음수 = 왼쪽 꼬리
<code>Kurtosis (excess)</code>	초과 첨도. 0 = 정규분포, 양수 = 두꺼운 꼬리
<code>Energy</code>	픽셀값 제곱의 평균 (원점 기준 2차 모멘트)
<code>RMS</code>	<code>Energy</code> 의 제곱근 — 신호 크기
<code>Entropy (bits)</code>	Shannon 엔트로피 (bit). 높을수록 복잡·무작위

`Diff |A-B|` 섹션에는 위 13개 행에 더해 다음 4개 행이 추가됩니다.

Metric	의미
<code>MSE</code>	평균 제곱 오차. 작을수록 유사
<code>PSNR</code>	피크 신호 대 잡음비 (dB). 클수록 우수
<code>MAE</code>	평균 절대 오차
<code>Max Error</code>	단일 픽셀 최대 절대 오차

참고 `PSNR` 의 피크값은 데이터 형식을 따릅니다. 8비트 이미지는 `peak = 255`, float/HDR 이미지는 `peak = 1.0` 로 계산합니다. 두 이미지가 완전히 동일하면 `inf` 로 표시됩니다. 크기가 다른 두 이미지는 픽셀 수가 작은 쪽까지만 비교합니다. 여기의 채널별 PSNR 은 헤드리스 `--metrics` 가 출력하는 값과 같은 정의이며, CI 게이트와 수치를 맞춰 보고 싶다면 8장을 참조하십시오.

팁 언제 여는가: 보상 전후의 “전역 성질”을 한 장으로 남길 때. `Mean` 차이는 밝기 바이어스, `Std Dev` 차이는 대비 변화, `Entropy` 감소는 계조 뭉개짐(밴딩)을 뜻합니다. 특히 보상 IP가 계조를 압축했는지 판단하려면 A와 B의 `Dynamic Range` 와 `Entropy (bits)` 를 나란히 읽는 것이 가장 빠릅니다.

6.5. ROI 선택 모드와 ROI 통계 — `Ctrl+E`

6.5.1. ROI 지정하기

`Ctrl+E` 를 누르면 ROI(Region of Interest) 선택 모드가 켜집니다. 이 상태에서 아무 패널 위나 **좌클릭 드래그** 하면 하늘색 사각형이 따라 그려지고, 버튼을 놓는 순간 다음이 일어납니다.

- 화면 좌표가 현재 zoom / pan 을 역산해 이미지 픽셀 좌표로 변환됩니다.
- 사각형이 이미지 경계 안으로 잘립니다.
- `ROI Statistics` 창이 **자동으로 열립니다**.
- 선택 영역에 하늘색 테두리와 `1024 x 768` 형태의 크기 라벨이 남습니다.

동작	설명
드래그 크기 하한	가로·세로 어느 쪽이든 3 px 미만이면 무시 (오클릭 방지)
대상 패널	드래그를 시작한 패널. Diff 패널이나 Overlay 화면 위에서도 가능
휠 줌	ROI 모드에서도 마우스 휠 줌은 그대로 동작
재선택	다시 드래그하면 이전 ROI를 대체

참고 ROI는 **이미지 픽셀 좌표**로 저장되므로, 어느 패널에서 그렸든 A와 B의 같은 좌표 영역에 동일하게 적용됩니다. 그래서 왼쪽 패널에서 결합 부위를 지정하기만 하면 오른쪽 보상 결과의 같은 부위 통계가 자동으로 따라옵니다.

6.5.2. ROI Statistics 창의 구성

창 제목 자체가 현재 ROI를 알려 줍니다: `ROI Statistics [1200,860 256x192]`. 본문은 위에서부터 다음 순서입니다.

- ROI 요약 한 줄 — `ROI: (1200, 860) size 256 x 192 = 49,152 px`
- `Image A (ROI)` 표 — 통계 창(`Ctrl+S`)과 동일한 13개 지표를 ROI 안의 픽셀만으로 다시 계산 (여기서 `Pixels` 행은 ROI 픽셀 수)
- A의 ROI 평균 컬러메트리 한 줄
- `Image B (ROI)` 표 + B의 ROI 평균 컬러메트리 한 줄
- `ROI mean Δ (A→B)` 한 줄 — A와 B의 색 차이
- `ROI Diff |A-B|` 표 — 13개 지표 + `MSE` / `PSNR` / `MAE` / `Max Error`

6.5.3. 컬러메트리 행 (`xy` / `u'v'` / `CCT`)

각 이미지 표 아래에 붙는 컬러메트리 줄은 **ROI 평균색**을 CIE 좌표로 환산한 값입니다. 8비트 이미지는 평균 sRGB 값을, float 이미지는 선형 RGB 평균을 D65 기준으로 XYZ 변환합니다.

```
mean colorimetry: xy 0.3128,0.3290 u'v' 0.1978,0.4683 CCT 6503K Duv +0.0001
```

항목	의미
<code>xy</code>	CIE 1931 색도 좌표 (소수점 4자리)
<code>u'v'</code>	CIE 1976 UCS 색도 좌표 — 지각 균등성이 좋아 색차 판정에 사용
<code>CCT</code>	McCamy 근사 상관 색온도 (K)
<code>Duv</code>	흑체 궤적으로부터의 부호 있는 거리. 양수 = 녹색 쪽, 음수 = 자홍 쪽

A와 B가 모두 로드되어 있으면 그 아래에 Δ 줄이 하나 더 표시됩니다.

```
ROI mean Δ (A→B): Δu'v' 0.0021 ΔCCT -87K ΔE76 1.34
```

항목	의미
<code>Δu'v'</code>	A와 B 평균색의 <code>u'v'</code> 평면 유클리드 거리. 보통 0.002 이하를 “육안 무차” 기준으로 씁니다
<code>ΔCCT</code>	B 색온도 - A 색온도 (K). 부호 포함
<code>ΔE76</code>	CIE 1976 <code>L*a*b*</code> 색차 (CIE76). 1.0 근방이 JND 목표

팁 언제 여는가: 보상 IP가 특정 영역의 색을 틀었는지 판정할 때. 화이트 패치 위에 ROI를 씌우고 `ΔCCT` 와 `Δu'v'` 를 읽으면 “밝기는 맞는데 색온도가 87K 낮아졌다” 같은 결론을 한 줄로 얻습니다. 같은 ROI의 `ROI Diff |A-B|` 표에서 `Max Error` 를 함께 보면 “평균은 맞는데 국소 최대 오차가 크다”는 패

턴(= 노이즈 유입)도 걸러집니다. 픽셀 한 점의 컬러메트리가 필요하면 **Shift+V** 프로브를 쓰십시오 (3장 참조).

6.5.4. ROI 해제와 창 닫기

하고 싶은 것	방법
ROI 모드 끄기 + ROI 지우기	Ctrl+E 를 다시 누름
ROI 통계 창 닫기	창의 x 버튼 또는 Esc
다른 영역으로 바꾸기	ROI 모드를 켜 채 새로 드래그

주의 **Ctrl+E** 로 ROI 모드를 끄면 사각형과 통계는 사라지지만 창 자체는 남아 **Ctrl+E** 한 ROI 모드 ON → 패널 위에서 좌측적 드래그 라는 안내만 표시합니다. 창까지 치우려면 **x** 버튼이나 **Esc** 를 쓰십시오. 반대로 **View** 메뉴의 **ROI Stats** 항목은 ROI 모드와 창을 한 번에 켜고 끕니다.

6.6. Scatter Plot — **Ctrl+T**

6.6.1. 무엇이 보이는가

Ctrl+T 는 **Scatter Plot (A vs B)** 창을 엽니다. **두 이미지가 모두 로드되어 있을 때만** 그려집니다. 같은 좌표의 픽셀 한 쌍을 점 하나로 찍되, 가로축(X)은 A의 값, 세로축(Y)은 B의 값입니다. 두 축 모두 **0.00** ~ **1.00** 로 정규화되어 있고 **0.25** 간격의 눈금이 붙습니다 (8비트는 값/255).

6.6.2. 대각선의 의미

차트에는 왼쪽 아래에서 오른쪽 위로 향하는 회색 대각선이 그려져 있습니다. 이것이 **B = A**, 즉 **완전 일치선**입니다.

점 구름의 모습	해석
대각선 위에 가늘게 겹침	보상이 항등에 가까움 — 사실상 무변경
대각선보다 기울기가 큼/작음	게인(gain) 편차. 밝은 쪽으로 갈수록 오차가 벌어짐
대각선과 평행하게 위/아래로 밀림	오프셋(offset) 편차 — DC 바이어스
대각선 주위로 넓게 퍼짐	노이즈 또는 디더링. 퍼짐 폭이 곧 잡음 크기
S자로 휨	감마-톤커브 차이 — LUT 곡선이 원본과 다름
특정 구간에서 수평으로 눕는 계단	클리핑 또는 코드 뭉개짐 — 여러 입력이 한 출력으로
세로 방향 여러 갈래로 갈라짐	입력이 같은데 출력이 다름 = 공간 의존 보상(무라 보상 등)

6.6.3. 채널 선택과 샘플링

창 상단에 **R** / **G** / **B** 라디오 버튼이 있어 한 번에 한 채널만 봅니다 (기본값 **R**). 오른쪽에 **I X=A, Y=B** 라는 안내가 붙습니다. 점 색은 선택 채널을 따라 빨강/초록/파랑입니다.

성능을 위해 전체 픽셀을 다 찍지 않고 균등 간격으로 최대 **20,000 점**까지 추출합니다. 실제 개수는 창에 그대로 표시됩니다.

Total: 8,294,400 px / Sampled: 20,000

항목	값
최대 샘플 수	20,000 (고정)
샘플링 방식	선형 인덱스 균등 간격 추출 (stride)
점 반지름	샘플이 5,000개 초과면 1.0 px, 이하면 1.5 px
캐시 갱신	이미지 크기 또는 내용이 바뀔 때 자동 재추출 (시퀀스 이동·회전 포함)

주의 샘플링은 무작위가 아니라 균등 간격이므로, 이미지 가로폭의 약수와 같은 주기를 갖는 패턴(예: 폭 3840에 stride 415)에서는 특정 위상만 뽑힐 수 있습니다. 주기성이 강한 테스트 패턴을 볼 때는 ROI를 좁혀 잡고 ROI Diff |A-B| 통계로 교차 확인하십시오.

예시 — 게인 · 오프셋 편차 분리

av ramp_orig.png ramp_comp_ip.png

0~255 램프 패턴을 열고 **Ctrl+T** 를 누릅니다. **R** 채널의 점 구름이 대각선보다 살짝 왼쪽 위로 평행 이동해 있다면 R 출력에 일정한 플러스 오프셋이 실린 것이고, 원점 근처는 붙어 있는데 오른쪽 끝에서 벌어졌다면 게인이 1보다 큰 것입니다. **G** / **B** 로 전환해 세 채널의 벌어짐이 다르다면 화이트 밸런스가 계조에 따라 흔들린다는 뜻이므로, 그 지점에 ROI를 씌워 **Ctrl+E** 의 **ACCT** 로 색온도 이동량까지 정량화하십시오.

팁 언제 여는가: “보상 IP의 전달 함수(transfer curve)가 설계값과 같은가”를 한 장으로 증명해야 할 때. 산점도 한 장이면 게인·오프셋·감마·클리핑을 동시에 보여 줄 수 있어 리뷰 자료로 효율이 좋습니다.

6.7. 차트와 통계 내보내기 — **Shift+Ctrl+S**

6.7.1. 저장 창은 상황에 따라 달라진다

Shift+Ctrl+S 는 “지금 열려 있는 창”에 맞는 저장 대화상자를 엽니다.

지금 열려 있는 창	열리는 저장 창
Histogram	Save Histogram
H-Line Cut	Save H-Line Cut
V-Line Cut	Save V-Line Cut
Image Statistics	Save Statistics
(분석 창 없음)	이미지 저장 창 (7장 참조)

주의 Scatter Plot 과 ROI Statistics 에는 전용 내보내기가 없습니다. 이 둘만 열어 둔 채 **Shift+Ctrl+S** 를 누르면 차트 저장 창이 아니라 일반 이미지 저장 창(Save Images)이 열립니다 (7장 참조). 산점도와 ROI 수치는 화면에서 읽어 옮기거나, 8장의 헤드리스 **--metrics** / **--probe** 로 대체하십시오.

6.7.2. 차트 저장 창 (Save Histogram / Save H-Line Cut / Save V-Line Cut)

창 위쪽에 두 개의 전역 옵션이 있습니다.

옵션	기본값	설명
Resolution	1920 x 1080	출력 PNG 크기(px). 최소 320 x 240 으로 자동 보정
Channels	Combined	Combined = R/G/B를 한 장에, Separate R/G/B = 채널별 3장

그 아래에는 Image A / Image B / Diff 세 항목이 있고, 각 항목마다 PNG 경로와 CSV 경로를 따로 편집할 수 있는 입력칸과 개별 Save 버튼이 있습니다. 기본 경로는 원본 이미지가 있던 디렉터리에 다음 규칙으로 자동 생성됩니다.

항목	기본 파일명 규칙
----	-----------

Histogram	<code><stem>_histogram.png</code> / <code>.csv</code>
H-Line Cut	<code><stem>_hcut.png</code> / <code>.csv</code>
V-Line Cut	<code><stem>_vcut.png</code> / <code>.csv</code>
Diff	<code><stemA>_vs_<stemB>_<suffix>_diff.png</code> / <code>.csv</code>

맨 아래 `Save All Checked` 버튼은 체크된 항목의 PNG와 CSV를 한꺼번에 씁니다. 성공하면 초록색으로 `Saved: <파일명>`, 실패하면 빨간색으로 `Error: Failed to write ...` 가 표시됩니다.

참고 `Separate R/G/B` 를 고르면 지정한 경로의 확장자 앞에 `_R` / `_G` / `_B` 가 붙어 3개 파일이 생성됩니다. 예를 들어 `orig_histogram.png` 를 지정하면 `orig_histogram_R.png`, `orig_histogram_G.png`, `orig_histogram_B.png` 가 저장됩니다. CSV는 채널 분리와 무관하게 항상 한 파일에 R/G/B 열이 모두 들어갑니다.

주의 절단면 저장은 **저장 버튼을 누른 시점의 pan 위치**로 행.열을 다시 계산합니다. A와 B 항목은 각자 자기 패널의 pan 을, `Diff` 항목은 A 패널의 pan 을 씁니다. 또 화면의 `Diff` 섹션과 달리 저장 창의 `Diff` 항목은 Diff 모드가 꺼져 있어도 두 이미지만 로드되어 있으면 활성화됩니다.

6.7.3. CSV 파일 형식

종류	헤더	내용
히스토그램	<code>bin,R,G,B</code>	256행. 각 bin의 픽셀 개수
절단면	<code>position,R,G,B</code>	선 길이만큼의 행. 정규화를 되돌린 실제 값 (8비트는 정수, float는 실수)
통계	<code>Metric,R,G,B,Description</code>	지표당 한 행 + 설명 열

통계 CSV의 행 이름은 화면 표기와 조금 다릅니다 (공백 없는 식별자): `Count`, `Min`, `Max`, `DynamicRange`, `Mean`, `Variance`, `StdDev`, `Median`, `Skewness`, `Kurtosis`, `Energy`, `RMS`, `Entropy`. `Diff` 항목에는 여기에 `MSE`, `PSNR`, `MAE`, `MaxError` 4행이 더 붙습니다. 숫자는 `%.6g` 형식(유효숫자 6자리)으로 기록됩니다.

```
Metric,R,G,B,Description
Count,8294400,8294400,8294400,Pixel count
Min,0,0,0,Minimum value
Max,255,255,255,Maximum value
DynamicRange,255,255,255,Max - Min
Mean,127.412,128.006,126.883,Arithmetic mean
...
PSNR,48.2214,47.9031,48.5507,Peak signal-to-noise ratio (dB)
MaxError,7,6,8,Maximum absolute error
```

예시 — 리뷰 자료용 차트 일괄 저장

```
av orig.png comp_ip_out.png --diff-mode abs
```

1. `Ctrl+H` 로 히스토그램을 엽니다.
2. `Shift+Ctrl+S` → `Save Histogram` 창이 뜹니다.
3. `Resolution` 을 `2560 X 1440` 으로 올리고 `Separate R/G/B` 를 선택합니다.
4. `Image A` / `Image B` / `Diff` 세 항목을 모두 체크하고 `Save All Checked` .

5. 원본 디렉터리에 채널별 PNG 9장과 CSV 3개가 생성됩니다.

CSV는 그대로 스프레드시트나 스크립트로 넘겨 회차별 추세를 그릴 수 있고, PNG는 보고서에 바로 붙일 수 있습니다. 같은 데이터를 GUI 없이 뽑아 CI에 물리려면 8장의 `--metrics` / `--format` 을 사용하십시오.

팁 라인컷 CSV의 `position,R,G,B` 는 밴딩 주기를 FFT로 분석하기에 딱 좋은 형태입니다. 무라 보상 검증에서 “주기 8픽셀 성분이 보상 후 몇 dB 줄었는가”를 정량화하려면, 보상 전후 두 CSV를 같은 행 (row)에서 뽑아 비교하십시오.

7. 이미지 시퀀스와 파일 입출력

DDI 보상 IP 검증은 정지 영상 한 장으로 끝나지 않습니다. 보통 수십~수백 프레임의 테스트 패턴 시퀀스를 원본 디렉토리와 보상 결과 디렉토리로 나누어 뽑아 놓고, “몇 번째 프레임에서 화질이 무너지는가”를 찾아내야 합니다. 이 장에서는 av 가 디렉토리를 시퀀스로 다루는 방식, 두 패널을 같은 파일명으로 묶는 `--pair`, 자동 재생, 그리고 결과를 파일·클립보드로 내보내는 방법을 다룹니다.

7.1. 디렉토리 시퀀스 — 자동 구성과 정렬 규칙

av 는 별도의 “시퀀스 열기” 기능이 없습니다. **이미지 한 장을 여는 순간** 그 파일이 들어 있는 디렉토리를 스캔해서 시퀀스 목록을 자동으로 만듭니다. CLI 인자로 연 경우, 파일 다이얼로그로 연 경우, 창에 드래그앤드롭한 경우, `;` / `a` 로 이동한 경우 모두 동일한 로더를 거치므로 예외가 없습니다.

시퀀스에 포함되는 파일은 확장자로 걸러집니다 (대소문자 무시).

항목	값
스캔 대상	연 파일의 부모 디렉토리 (경로에 디렉토리가 없으면 <code>.</code>)
포함 확장자	<code>.png</code> <code>.jpg</code> <code>.jpeg</code> <code>.bmp</code> <code>.tga</code> <code>.pgm</code> <code>.ppm</code> <code>.hdr</code>
하위 디렉토리	스캔하지 않음 (1단계, 정규 파일만)
정렬 기준	파일명(basename)의 바이트 코드포인트 순 — <code>LC_ALL=C</code> 와 동일
보관 단위	패널 A / 패널 B 각각 독립된 목록과 현재 인덱스

정렬은 **natural sort 가 아닙니다**. 대문자가 소문자보다 앞서고, 숫자를 자릿수로 묶지 않습니다. 즉 `f10.png` 가 `f2.png` 보다 **앞**에 옵니다. 이 규칙은 av 가 참조하는 `input_img_file.toml` 의 순서 규칙과 일치 시키기 위한 의도적인 선택입니다.

주의 프레임 번호는 반드시 제로패딩하십시오. `frame_1.png` 부터 `frame_100.png` 까지 뽑으면 탐색 순서가 `frame_1, frame_10, frame_100, frame_11, ...` 이 됩니다. `frame_0001.png` 형태로 뽑으면 파일명 순서 = 프레임 순서가 보장됩니다.

현재 위치는 상태바에 항상 표시됩니다. 패널 A 가 100장 중 3번째라면 상태바에 `A [3/100]` 이 초록색으로 나오고, B 패널에도 시퀀스가 있으면 `B [3/100]` 이 이어서 표시됩니다. 인덱스는 1부터 셉니다.

7.2. 시퀀스 탐색 키

키	동작
<code>;</code>	활성 패널의 시퀀스에서 다음 이미지
<code>a</code>	활성 패널의 시퀀스에서 이전 이미지
<code>N</code> / <code>Shift+N</code>	다음 / 이전 (레거시 핫키, <code>;</code> / <code>a</code> 와 완전히 동일)
<code>Tab</code>	활성 패널 전환 (A ↔ B)
<code>Alt</code> + 마우스휠	커서가 올라가 있는 패널의 시퀀스를 위/아래로 이동

7.2.1. 어떤 패널이 움직이는가

`;` 와 `a` 는 **활성 패널(active panel)** 의 시퀀스만 움직입니다. 활성 패널은 `Tab` 으로 바꾸며, 화면에서는 해당 패널의 테두리가 굵어지는 것으로 구분합니다 (활성 4px, 비활성 2px). `B` 로 테두리를 꺼 둔 상태라면 활성 패널이 보이지 않으므로, 시퀀스 작업 중에는 테두리를 켜 두는 편이 안전합니다.

`Alt` + `휠`은 활성 패널과 무관하게 **마우스 커서 아래 패널**을 움직입니다. 왼손으로 키를 누르지 않고 오른손 `휠`만으로 A 와 B 를 번갈아 넘길 때 편리합니다. `Alt` 없이 `휠`을 돌리면 평소대로 줌입니다 (2장 참조).

7.2.2. 순환과 피드백

목록의 마지막에서 `;` 를 누르면 처음으로, 첫 번째에서 `a` 를 누르면 마지막으로 순환합니다. 경계에서 멈추지 않으므로 100프레임 시퀀스를 계속 넘겨도 막히지 않습니다.

로딩이 끝나면 화면 **상단 중앙**에 파일명 토스트가 1.5초 동안 뜹니다. 형식은 `A: frame_0042.png` 처럼 패널 라벨과 basename 입니다. 로드에 실패하면 배경이 붉게 바뀌고 `A (failed): frame_0042.png` 로 표시되며, **이전 이미지는 그대로 유지**됩니다(비파괴 로드). 전체화면이나 테두리 없는 모드처럼 타이틀바가 없는 상태에서도 지금 몇 번 프레임을 보고 있는지 알 수 있게 하기 위한 장치입니다.

참고 시퀀스로 다음 영상을 로드해도 줌 설정은 깨지지 않습니다. `--zoom` 이나 `.av.ini` 로 고정 배율이 지정돼 있으면 그 배율을 유지한 채 가운데 정렬하고, 지정이 없으면 (기본값 = fit) 새 영상 크기에 맞춰 다시 맞춥니다. 400% 로 확대한 채 프레임을 넘기며 같은 영역을 보려면 `--zoom 4` 로 띄우십시오.

주의 히스토그램·통계 같은 ImGui 창 의 입력란에 키보드 포커스가 있으면 `;` `a` `N` 같은 평문 키가 그 창으로 먹힙니다. 탐색이 안 되면 패널을 한 번 클릭하거나 `Esc` 로 창을 닫은 뒤 다시 누르십시오.

7.3. A 와 B 는 서로 다른 디렉토리를 독립으로 탐색합니다

패널 A 와 B 는 각자의 파일 목록과 인덱스를 따로 들고 있습니다. 왼쪽에 원본 디렉토리를, 오른쪽에 보상 결과 디렉토리를 띄워 놓고 한쪽만 넘길 수 있다는 뜻입니다.

```
av /work/golden/frame_0001.png /work/hw_out/frame_0001.png
```

이 상태에서 `Tab` 으로 B 를 활성화한 뒤 `;` 를 한 번 누르면 오른쪽만 다음 프레임으로 갑니다. 실무에서 이 독립성이 필요한 대표적인 경우는 다음과 같습니다.

- **파이프라인 지연 보정:** 보상 IP 가 1프레임 지연되어 출력을 뺀 경우, B 만 한 칸 앞으로 밀어 프레임 정렬을 맞춘 뒤 비교합니다.
- **파일명 규칙이 다른 덤프:** 원본은 `frame_0001.png`, HW 덤프는 `cap_0001.png` 처럼 이름 체계가 다르면 같은 이름 짝짓기(`--pair`)를 쓸 수 없으므로 독립 탐색으로 수동 정렬합니다.
- **기준 프레임 고정:** A 에 플랫폼 필드 기준 영상을 고정해 두고 B 만 넘기며 무라를 비교합니다.

반대로 프레임 정렬이 이미 맞고 파일명도 같다면, 한쪽만 넘어가는 것은 오히려 사고의 원인입니다. 그럴 때 쓰는 것이 다음 절의 `--pair` 입니다.

7.4. `--pair` — 같은 파일명으로 두 디렉토리를 묶어 이동

`--pair` 는 A 의 디렉토리를 기준 시퀀스로 삼고, **같은 파일명**을 B 의 디렉토리에서 찾아 항상 함께 로드합니다. `;` / `a` 를 아무리 눌러도 두 패널의 프레임이 어긋나지 않습니다.



그림 7.1: 디렉토리 시퀀스 동기 이동. `--pair` 는 두 디렉토리를, `--comp` 는 세 디렉토리를 같은 파일명으로 묶는다.

```
av --pair /work/golden/frame_0001.png /work/hw_out
```

두 번째 인자는 파일이어도 되고 디렉토리여도 됩니다. 파일을 주면 그 파일의 부모 디렉토리가 비교 디렉토리로 쓰입니다.

```
av --pair /work/golden/frame_0001.png /work/hw_out/frame_0001.png # 워낙 동일
```

규칙	동작
기준	항상 A(왼쪽) 디렉토리가 시퀀스를 구동. <code>Tab</code> 으로 B 를 활성화해도 <code>;</code> / <code>a</code> 는 A 기준
짜짓기	A 의 <code>basename</code> 을 B 디렉토리에서 그대로 찾음 (확장자 포함 완전 일치)
디렉토리 검증	A 와 B 의 디렉토리가 같으면 창을 띄우기 전에 오류 출력 후 종료
인자 개수	경로 2개 필수. 하나만 주면 오류 후 종료
짜이 없을 때	B 패널을 비우고 안내 문구 표시 (아래 참조)

7.4.1. 짜이 없는 프레임

B 디렉토리에 같은 이름의 파일이 없으면 `av` 는 조용히 넘어가지 않고 B 패널을 비운 뒤 다음과 같이 표시합니다.

```
No matching image:
frame_0057.png
in /work/hw_out
```

이 화면 자체가 “HW 덤프에서 57번 프레임이 빠졌다”는 리포트입니다. 프레임 누락은 보상 IP 검증에서 흔한 사고이므로, 시퀀스를 한 번 훑어보는 것만으로 결손을 잡아낼 수 있습니다.

주의 디렉토리 검증은 창을 열기 전에(fail-fast) 수행됩니다. A 와 B 가 같은 디렉토리면 `--pair: both directories are the same (...). Nothing to compare.` 를 `stderr` 로 출력하고 즉시 종료하므로, 스크립트에서 경로를 잘못 넘긴 것을 바로 알 수 있습니다.

7.4.2. 헤드리스 지표와 조합

`--pair` 는 GUI 전용 옵션이 아닙니다. `--metrics` 와 함께 쓰면 A 디렉토리의 모든 프레임을 순회하며 프레임당 한 줄씩 CSV 를 뱉습니다.

```
av --pair --metrics /work/golden/frame_0001.png /work/hw_out
```

짝이 없는 프레임은 `missing`, 디코드 실패는 `decode_error`, 크기/포맷 불일치는 `mismatch` 로 상태가 찍히므로 회귀 프레임과 결손 프레임을 한 번에 걸러낼 수 있습니다. 컬럼 구성과 `--fail-psnr` 같은 CI 게이트는 8장을 참조하십시오.

7.5. `--comp` 의 3디렉토리 미러링

세 영상 블록 비교 모드(`--comp`, 5장 참조)에서도 시퀀스 탐색이 동작합니다. 동작 원리는 `--pair` 와 같지만 대상이 셋입니다.

```
av --comp /work/golden/f_0001.png /work/fw_out/f_0001.png /work/hw_out/f_0001.png
```

대상	시퀀스 동작
orig (원본)	<code>;</code> / <code>a</code> 가 이 디렉토리의 목록을 기준으로 이동
img2	현재 img2 경로의 디렉토리에서 같은 파일명 을 자동 로드
img3	현재 img3 경로의 디렉토리에서 같은 파일명 을 자동 로드

`--pair` 와 마찬가지로, `Tab` 으로 다른 패널을 만져도 `;` / `a` 는 언제나 원본 디렉토리를 기준으로 움직입니다. 새 프레임이 로드되면 블록 통계가 무효화되어 다음 프레임에 대해 `worst` 블록 순위와 PSNR 이 자동으로 다시 계산됩니다.

주의 `--comp` 에서 img2 나 img3 디렉토리에 같은 이름의 파일이 없으면 해당 패널의 영상은 **해제되어 빈 패널**이 됩니다. 이때는 `--pair` 처럼 “No matching image” 안내가 아니라 `(no image)` 로만 보이며, 세 영상이 모두 갖춰지지 않았으므로 블록 비교 통계도 계산되지 않습니다. 프레임을 한 칸 더 넘겨 세 파일이 모두 있는 프레임에 도달하면 정상 상태로 복귀합니다.

팁 FW 모델 출력과 HW 실측 덤프를 원본과 동시에 프레임 단위로 훑을 때 이 미러링이 가장 강력합니다. `;` 로 프레임을 넘기면서 `Ctrl+B` 의 `worst` 블록 박스가 프레임마다 어디로 이동하는지 보면, 특정 패턴에서만 터지는 보상 오류를 빠르게 잡힐 수 있습니다.

7.6. 슬라이드쇼 (자동 재생)

키	동작
<code>Shift+A</code>	슬라이드쇼 자동 재생 토글
<code>Shift+Up</code>	재생 간격 +1초
<code>Shift+Down</code>	재생 간격 -1초

`Shift+A` 를 누르면 지정된 간격마다 `;` 를 누른 것과 같은 동작이 자동으로 반복됩니다. 재생 대상 패널은 **토글한 순간의 활성 패널**로 고정되므로, 재생 중에 `Tab` 을 눌러도 재생 대상은 바뀌지 않습니다. 다시 `Shift+A` 를 누르면 정지합니다.

항목	값
기본 간격	3.0 초
조절 단위	1.0 초
범위	0.5 초 ~ 10.0 초 (경계에서 클램프)

영속화 되지 않음. av 를 다시 실행하면 3.0 초로 복귀

재생 중에는 상태바에 재생 삼각형 아이콘과 함께 2.4s / 3.0s 형태로 (남은 시간 / 설정 간격) 이 청록색으로 표시됩니다.

주의 Shift+Up / Shift+Down 은 슬라이드쇼가 켜져 있을 때만 간격을 조절합니다. 꺼져 있으면 같은 키가 원래 기능인 화면 팬(--pan-step 만큼 상하 이동) 으로 동작합니다. 간격이 안 바뀐다면 먼저 Shift+A 로 재생을 켜십시오.

팁 --pair 나 --comp 와 함께 쓰면 슬라이드쇼도 원본 디렉토리 기준으로 진행되므로, 두(또는 세) 디렉토리가 프레임 단위로 정렬된 채 자동 재생됩니다. 100프레임 시퀀스를 0.5 초 간격으로 돌리며 눈으로 훑어 “튀는 프레임”을 먼저 표시해 두고, 그 구간만 ; / a 로 정밀 검사하는 방식이 효율적입니다.

7.7. 파일 열기

방법	동작
Shift+Ctrl+O (macOS: Shift+Cmd+O)	네이티브 열기 다이얼로그 — 활성 패널 에 로드
메뉴 File > Open Image A...	패널 A 에 로드
메뉴 File > Open Image B...	패널 B 에 로드
메뉴 File > Open Image...	활성 패널에 로드 (단축키와 동일)
드래그앤드롭	A 가 비어 있으면 A, 아니면 B 에 로드

다이얼로그의 파일 필터는 png;jpg;jpeg;bmp;tga;hdr;ppm;pgm 과 All files 두 가지입니다.

Open Images 창에는 Clear other image when loading single 체크박스가 있습니다. 켜면 한쪽을 새로 열 때 반대쪽 패널을 비웁니다. 기본은 꺼짐이며, 시퀀스 탐색으로 로드할 때는 이 옵션과 무관하게 반대쪽을 건드리지 않습니다.

참고 어떤 경로로 열든 로드 성공 시에만 패널이 교체됩니다. 손상된 파일을 열어도 기존 영상과 뷰포트, 시퀀스 목록은 그대로 남고 붉은 실패 토스트만 뜹니다. 또 로드가 성공하면 그 파일의 디렉토리로 시퀀스가 다시 구성되므로, 다른 디렉토리의 파일을 열면 그 순간부터 새 디렉토리를 탐색하게 됩니다.

7.8. 저장 — Save Images 창

Shift+Ctrl+S (macOS: Shift+Cmd+S) 또는 메뉴 File > Save Images... 로 저장 창을 토글합니다. 이 단축키는 **문맥 인식**입니다.

열려 있는 창	뜨는 저장 창
히스토그램	차트 저장 창 (PNG + CSV, 6장 참조)
H-Line / V-Line Cut	차트 저장 창 (PNG + CSV, 6장 참조)
통계 창	통계 저장 창 (6장 참조)
그 외	Save Images — 이 절에서 설명

Save Images 창은 세 개의 저장 대상을 한 화면에서 다룹니다.

대상	활성 조건과 내용
Image A	패널 A 의 영상 원본 픽셀
Image B	패널 B 의 영상 원본 픽셀
Diff	A·B 가 모두 로드되고 diff 모드가 <code>none</code> 이 아닐 때만 활성. 현재 diff 모드와 amplify 를 그대로 반영한 결과 영상

각 행은 체크박스, 경로 입력칸, `Save` 버튼으로 구성됩니다. 하단의 `Save All Checked` 를 누르면 체크된 항목을 한꺼번에 저장하고, 하나라도 실패하면 그 지점에서 멈추고 붉은 오류 메시지를 표시합니다. 성공하면 초록색으로 `Saved: <파일명>` 이 뜹니다.

7.8.1. 기본 경로

대상	기본 경로
Image A	원본과 같은 디렉토리에 <code><A의 stem>_save.png</code>
Image B	원본과 같은 디렉토리에 <code><B의 stem>_save.png</code>
Diff	A 의 디렉토리에 <code><A stem>_vs_<B stem>_diff.png</code>

영상이 로드돼 있지 않으면 각각 `./image_a_save.png`, `./image_b_save.png` 가 쓰입니다. 포맷을 바꾸면 세 경로의 확장자가 자동으로 따라 바뀝니다.

7.8.2. 포맷 옵션

옵션	값
Format	PNG (기본) BMP PPM
Mode (PPM 전용)	Binary (P6) (기본) ASCII (P3)
Bits (PPM 전용)	8 (기본) 10 12 16

PPM 의 비트 심도 선택은 DDI 검증에서 특히 유용합니다. 10비트 패널 데이터를 `P6 / 10bit` 로 뽑으면 `maxval` 이 `1023` 인 PPM 이 되어, 8비트로 놀리지 않은 값을 외부 스크립트에서 그대로 읽을 수 있습니다.

주의 저장 시 알파 채널은 제외되고 RGB 3채널로 기록됩니다. HDR/float 영상은 `[0, 1]` 범위로 클램프된 뒤 정수화되므로 범위를 벗어난 값은 잘립니다. 또한 diff 모드가 `SSIM` 또는 `FLIP` 인 상태에서 `Diff` 를 저장하면 해당 히트맵이 아니라 **Pixel Absolute** 결과가 저장됩니다. SSIM/FLIP 맵을 그대로 파일로 남기려면 헤더리스 `--diff-out` 을 쓰십시오 (8장 참조).

7.8.3. 스크린샷과 저장의 차이

`Shift+Ctrl+S` 는 “화면 캡처”가 아닙니다. av 는 창을 그대로 캡처하는 별도의 스크린샷 기능을 제공하지 않으며, 저장되는 것은 오버레이·격자·박스·상태바가 제외된 **원본 픽셀 데이터**(또는 그 diff 결과)입니다. 검증 리포트에는 이쪽이 정답입니다 — 화면 축소로 리샘플링된 이미지가 아니라 실제 값이 남기 때문입니다.

worst 블록 박스나 커튼 분할 같은 **화면 그대로**의 그림이 필요하다면 OS 의 캡처 도구 (macOS `Shift+Cmd+4`, Windows `Win+Shift+S`)를 쓰거나, 아래의 클립보드 복사로 픽셀 데이터를 가져와 리포트 도구에서 합성하십시오.

7.8.4. 패널 우클릭 컨텍스트 메뉴

패널 위에서 오른쪽 버튼을 **0.5초 이상 길게** 눌렀다 떼면 (드래그 없이, `Ctrl` 을 누르지 않은 상태) 작은 컨텍스트 메뉴가 뜹니다.

항목	동작
Save As...	네이티브 저장 다이얼로그. 확장자(.png / .bmp / .ppm)로 포맷 자동 결정
Copy to Clipboard	그 패널의 영상을 PNG 로 클립보드에 복사

짧게 누르거나 드래그하면 컨텍스트 메뉴 대신 우클릭 드래그 줌 영역 선택으로 동작합니다 (2장 참조).

7.9. 클립보드 복사 (2단계 키)

버그 리포트나 메신저에 붙여 넣을 때는 파일을 거치지 않고 바로 클립보드로 복사하는 편이 빠릅니다. av 는 오조작을 막기 위해 **2단계 키**를 씁니다.

키 조합	동작
Ctrl+C 또는 Cmd+C	복사 모드 진입 (하단에 안내 배너)
→ 1	Image A 를 PNG 로 복사
→ 2	Image B 를 PNG 로 복사
→ 3	현재 diff 모드의 Diff 영상을 PNG 로 복사
Esc	복사 모드 취소
5초 경과	자동 취소 (타임아웃)

Ctrl+C 를 누르면 화면 하단에 노란 테두리 배너가 뜹니다.

Copy: 1 = A 2 = B 3 = Diff (Esc to cancel)

여기서 1 / 2 / 3 (숫자 키패드 포함)을 누르면 해당 영상이 PNG 로 인코딩되어 시스템 클립보드에 등록되고, 초록 배너로 Copied: Image A 가 **1.5초** 표시됩니다. Esc 를 누르면 조용히 빠져나옵니다. 1 / 2 / 3 도 Esc 도 아닌 다른 키를 누르면 복사 모드가 취소된 뒤 그 키의 원래 기능이 그대로 수행되므로, 모드에 갇히는 일은 없습니다.

참고

- 복사 데이터는 알파를 포함한 RGBA PNG 이며, MIME 타입은 image/png 로 등록됩니다.
- Shift+Space 로 A/B 를 스왑한 상태에서도 1 은 언제나 화면 왼쪽에 보이는 영상을 복사합니다 (표시 위치 기준).
- 3 (Diff) 은 A·B 가 모두 로드돼 있어야 하며, 저장과 마찬가지로 SSIM/FLIP 모드는 Pixel Absolute 로 대체됩니다.

주의 클립보드 데이터는 SDL3 의 콜백 방식으로 등록됩니다. 즉 av 프로세스가 살아 있는 동안 붙여 넣기 요청에 응답합니다. macOS 와 Windows 에서는 PNG 클립보드를 받는 앱에 곧바로 붙여 넣을 수 있지만, Linux(X11/Wayland)에서는 클립보드 매니저가 없으면 av 를 종료하는 순간 내용이 사라 집니다. 원격 세션이나 CI 장비에서는 클립보드 대신 Save Images 나 헤더리스 --diff-out 을 쓰는 편 이 확실합니다.

7.10. 실무 시나리오 — 100프레임 시퀀스에서 회귀 프레임 찾기

보상 IP 펌웨어를 갱신한 뒤, 100프레임 테스트 패턴 시퀀스에서 화질이 무너진 프레임을 찾아 리포트하는 전형적인 흐름입니다.

예시 — 회귀 프레임 추적

1단계 — 헤드리스로 범위를 좁힙니다. 눈으로 100장을 보기 전에 수치로 후보를 고릅니다.

```
av --pair --metrics /work/golden/frame_0001.png /work/hw_out > psnr.csv
sort -t, -k4 -g psnr.csv | head -5
```

`psnr_db` 컬럼이 가장 낮은 다섯 프레임이 나옵니다. 여기서 `frame_0057.png` 가 `52.1 dB` 로 다른 프레임 (`> 60 dB`)보다 확연히 낮다고 가정합니다.

2단계 — 해당 프레임을 GUI 로 엽니다. 프레임 정렬이 깨지지 않도록 반드시 `--pair` 로 엽니다.

```
av --pair /work/golden/frame_0057.png /work/hw_out
```

3단계 — 앞뒤 프레임을 훑습니다. `a` 로 56, 55번을, `;` 로 58, 59번을 확인합니다. 두 패널이 항상 같은 파일명으로 함께 움직이므로 프레임이 어긋날 걱정이 없습니다. 상단 토스트로 지금 몇 번인지, 상태바 `A [57/100]` 으로 위치를 확인합니다.

4단계 — 블링크로 차이를 눈에 띄게 만듭니다. `,` 를 누르면 A 와 B 가 한 패널에서 자동 교대되어 미세한 차이가 “깜빡임”으로 드러납니다. 너무 빠르면 `<`, 너무 느리면 `>` 로 `0.1` 초씩 조절합니다 (기본 `0.5` 초, 범위 `0.1 ~ 2.0` 초). 블링크 중에는 두 패널의 뷰포트가 강제로 동기화되므로 픽셀 단위로 정렬된 상태에서 비교됩니다.

5단계 — 증거를 남깁니다. `Ctrl+3` 으로 Pixel Absolute diff 를 켜고 `Ctrl+C` → `3` 으로 diff 영상을 클립보드에 복사해 이슈에 바로 붙여 넣습니다. 파일로 남겨야 하면 `Shift+Ctrl+S` 로 `Save Images` 를 열고 `Diff` 를 체크한 뒤 `Save All Checked` 를 누릅니다.

팁 결손 프레임 점검도 같은 흐름에 포함됩니다. CSV 의 `missing` 상태 행이 있으면 HW 덤프에서 빠진 프레임이며, GUI 에서 그 프레임으로 이동하면 B 패널에 `No matching image: frame_XXXX.png` 가 표시되어 즉시 확인됩니다.

8. 헤드리스 CLI와 CI 연동

av 는 GUI 뷰어이지만, 같은 실행 파일이 창·OpenGL·SDL 을 전혀 열지 않는 **헤드리스 모드** 로도 동작합니다. 이 장에서는 DDI 보상 IP 의 출력 프레임을 원본과 비교해 수치 리포트를 뽑고, 임계값을 넘으면 빌드를 떨어뜨리는 CI 게이트를 만드는 방법을 다룹니다. 모든 헤드리스 모드는 계산 후 즉시 종료하며, 표준 출력에는 기계가 파싱할 수 있는 순수 데이터만 흘러보냅니다.

8.1. 헤드리스 실행 모델

8.1.1. 창을 열지 않는 모드 전수

`main()` 은 CLI 를 파싱한 직후, SDL 을 초기화하기 **전에** 아래 순서대로 헤드리스 모드를 검사합니다. 먼저 걸린 것 하나만 실행되고 프로세스는 그 반환값으로 종료합니다. 즉 여러 헤드리스 플래그를 동시에 주면 아래 표의 위쪽이 이깁니다.

순위	플래그	하는 일
1	<code>--batch <file -></code>	매니페스트의 임의 A,B 쌍을 지표 계산
2	<code>--comp-batch</code>	3개 디렉토리의 같은 이름 프레임 전체를 블록 비교
3	<code>--comp-out <file -></code>	3영상 블록별 CSV/JSON 테이블
4	<code>--metrics</code>	A 대비 B 의 PSNR/SSIM/FLIP/MSE/MAE
5	<code>--diff-out <png></code>	diff 시각화를 PNG 로 저장
6	<code>--validate</code>	float/HDR 의 NaN/Inf/음수/>1 스캔
7	<code>--probe <X,Y></code>	픽셀 한 점의 CIE 컬러메트리
8	<code>--uniformity</code>	평판 균일도·무라 지표

이 모드들은 X11/Wayland 디스플레이도, GPU 도 필요 없습니다. SSH 세션, 도커 컨테이너, GitHub Actions 러너처럼 화면이 없는 환경에서 그대로 실행됩니다. 디코딩은 전부 CPU 경로(`stb_image`)로 처리되므로 GUI 에서 보이는 것과 같은 포맷(PNG/BMP/PPM/PGM/HDR 등, 7장 참조)을 그대로 읽습니다.

8.1.2. stdout 은 데이터, stderr 는 요약

헤드리스 출력의 가장 중요한 규약은 **스트림 분리** 입니다.

- **stdout**: 헤더 한 줄 + 데이터 행만. 주석·진행 로그·합계가 섞이지 않습니다.
- **stderr**: 사람이 읽는 한 줄 요약(프레임 수, 평균/중앙값/p95, 게이트 판정, FAIL 프레임 이름 나열)과 오류 메시지.

따라서 `> report.csv` 로 리다이렉트하면 파일에는 순수 CSV 만 남고, 요약은 터미널·CI 로그에 그대로 보입니다.

```
av --metrics ref.png ip_out.png > metrics.csv # 파일 = 순수 CSV
av --metrics ref.png ip_out.png 2> /dev/null # 요약만 버리기
av --metrics ref.png ip_out.png > /dev/null # 요약만 보기
```

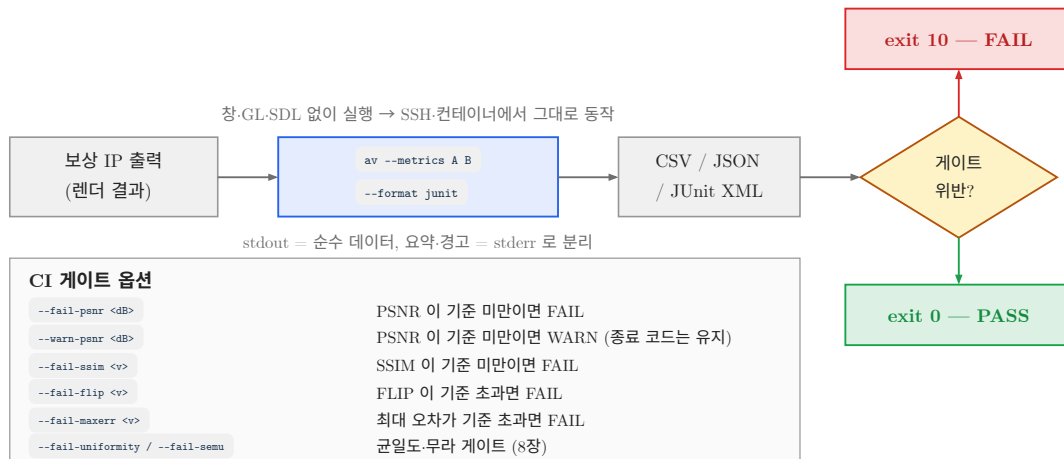


그림 8.1: 헤드리스 모드의 CI 연동 흐름. 창을 띄우지 않고 종료 코드로 합격/불합격을 알린다.

주의 `--format json` 과 `--format junit` 에서는 stderr 요약 줄이 **출력되지 않습니다**. 집계값(mean/median/p95/min/max)과 게이트 판정은 JSON 의 `summary` 객체 안에 들어가고, JUnit 은 `tests / failures / skipped` 속성으로 대체됩니다. CSV 모드에서만 stderr 한 줄 요약이 나옵니다.

8.2. 지표 추출 — `--metrics`

8.2.1. 기본 사용법

```
av --metrics ref.png ip_out.png
```

첫 줄이 헤더, 둘째 줄부터 프레임 한 장당 한 행입니다. 실제 출력 예:

```
file,width,height,psnr_db,psnr_r,psnr_g,psnr_b,psnr_y,ssim,flip,mse,mae,max_error,msigned,psnr_cb,psnr_cr
test_a.png,100,100,7.6564,7.6564,inf,7.6564,24.7091,0.973462,0.549758,7436.17,57.5,202,10.6353,11.8849,12.915
# stderr: [metrics] frames=1 missing=0 mean_psnr=7.6564dB median=7.6564
# p95=7.6564 min=7.6564 mean_ssim=0.973462 mean_flip=0.549758
```

8.2.2. 16개 열의 의미

수치 필드는 모두 `%.6g` 로 찍히며, 무한대는 `inf` / `-inf` 문자열입니다. 단위 표의 “code” 는 8비트 이미지의 코드값(0~255)을, HDR(float) 이미지는 선형광 정규화값(peak 1.0)을 뜻합니다.

열	단위/범위	의미
<code>file</code>	—	A 의 파일명(basename). <code>--batch</code> 에서 label 을 주면 그 label
<code>width</code>	px	A 의 가로 크기
<code>height</code>	px	A 의 세로 크기
<code>psnr_db</code>	dB	종합 PSNR. R/G/B 채널 PSNR 중 유한한 값만 평균 (아래 설명)
<code>psnr_r</code>	dB	R 채널 PSNR (peak 255, HDR 은 1.0)
<code>psnr_g</code>	dB	G 채널 PSNR
<code>psnr_b</code>	dB	B 채널 PSNR
<code>psnr_y</code>	dB	Rec.709 루마 Y' PSNR. 사람 눈에 가까운 단일 수치
<code>ssim</code>	0~1	구조 유사도. 1 = 완전 동일, 높을수록 좋음
<code>flip</code>	0~1	FLIP 지각 오차 평균. 0 = 완전 동일, 낮을수록 좋음
<code>mse</code>	code ²	R/G/B MSE 의 산술평균
<code>mae</code>	code	R/G/B MAE 의 산술평균

<code>max_error</code>	code	세 채널을 통틀어 가장 큰 절대오차 한 픽셀
<code>msigned</code>	code	루마 가중 평균 부호 오차 (밝기 편향). 아래 설명
<code>psnr_cb</code>	dB	Rec.709 크로마 Cb PSNR ($Cb = (B-Y)/1.8556$)
<code>psnr_cr</code>	dB	Rec.709 크로마 Cr PSNR ($Cr = (R-Y)/1.5748$)

SSIM·FLIP 의 알고리즘적 의미와 GUI 상의 히트맵 표현은 4장을 참조하세요. 여기서는 CSV 로 떨어지는 스칼라 점수만 다룹니다.

8.2.3. `psnr_db` — “채널별 PSNR 평균” 관례

av 의 종합 PSNR 은 전체 MSE 하나로 계산하지 않고, **R/G/B 각각의 PSNR 을 구한 뒤 유효한 것만 평균** 냅니다. 유효 조건은 `0 < psnr < 999` 이므로, 어떤 채널이 A 와 B 에서 완전히 동일해 PSNR 이 무한대가 되면 그 채널은 평균에서 **빠집니다**. 세 채널이 모두 동일하면 남는 값이 없으므로 `psnr_db` 자체가 `inf` 가 됩니다.

이 관례는 GUI 의 Image Info 창(`i`)에 표시되는 PSNR, `--comp` 의 전역 PSNR 과 완전히 같습니다(3장·5장 참조). 따라서 화면에서 본 값과 CI 리포트의 값이 어긋나지 않습니다.

주의 위 예시 행에서 `psnr_g` 가 `inf` 인데 `psnr_db` 는 7.6564 입니다. G 채널이 평균에서 빠져 R/B 두 채널만 평균된 결과입니다. 보상 IP 가 특정 채널만 손대는 경우(예: R 게인만 보정) 종합 PSNR 이 실제보다 **나쁘게** 보일 수 있으므로, 채널별 `psnr_r` / `psnr_g` / `psnr_b` 열을 함께 확인하세요.

8.2.4. `msigned` — 밝기 편향 읽는 법

`msigned` 는 채널별 평균 부호 오차 `mean(A[c] - B[c])` 를 Rec.709 가중치로 합친 값입니다.

```
msigned = 0.2126*mean(Ar-Br) + 0.7152*mean(Ag-Bg) + 0.0722*mean(Ab-Bb)
```

절대값 지표(`mse`, `mae`)와 달리 부호가 살아 있으므로 **어느 쪽으로 치우쳤는지** 를 알려줍니다.

- `msigned` > 0 → A(원본)가 평균적으로 더 밝음 = B(보상 출력)가 어두워짐
- `msigned` < 0 → 보상 출력이 원본보다 밝아짐
- `msigned` ≈ 0 인데 `mae` 가 크면 → 편향 없는 노이즈성 오차

팁 DDI 보상 IP 검증에서 `msigned` 는 감마·오프셋 계수 부호 실수를 잡는 가장 빠른 신호입니다. `mae` 는 그대로인데 `msigned` 만 프레임 전체에서 한쪽 부호로 몰려 있으면, 알고리즘 오차가 아니라 DC 오프셋/게인 세팅 오류일 가능성이 높습니다.

8.2.5. 시퀀스 전체 실행 — `--pair` 조합

`--pair` 를 함께 주면 A 의 디렉토리를 전부 훑어 같은 파일명을 B 디렉토리에서 찾아 짝짓고, **프레임당 한 행** 을 출력합니다. 파일 순서는 GUI 의 next/prev 와 동일한 파일명 코드포인트 정렬입니다(7장 참조).

```
av --pair --metrics golden/f0001.png build/ip_out
```

- 첫 번째 인자는 A 디렉토리 안의 **파일** 이어야 합니다(디렉토리만 주면 안 됨).
- 두 번째 인자는 B 디렉토리 자체 또는 그 안의 아무 파일이나 줄 수 있습니다.
- 두 디렉토리가 같으면 비교 의미가 없으므로 stderr 경고 후 exit 1 입니다.
- B 에 같은 이름이 없으면 그 행은 `file,,,missing,,...` 플레이스홀더가 되고 stderr 요약의 `missing=` 카운트에 잡힙니다.
- 디코드 실패는 `decode_error`, 크기/포맷 불일치는 `mismatch` 로 표시됩니다.

예시 — 시퀀스 회귀 리포트

```
av --pair --metrics golden/f0001.png build/ip_out > run.csv
# stdout(run.csv):
#   file,width,height,psnr_db,...
#   f0001.png,1080,2392,41.2831,...
#   f0002.png,1080,2392,40.9773,...
# stderr:
#   [metrics] frames=2 missing=0 mean_psnr=41.1302dB median=41.1302
#           p95=41.2529 min=40.9773 mean_ssim=0.9971 mean_flip=0.0184
```

8.2.6. 출력 포맷 — `--format csv|json|junit`

기본값은 `csv` 입니다. 값은 `csv`, `json`, `junit` 셋 중 하나여야 하며 다른 값을 주면 즉시 오류 종료합니다.

csv (기본) — 위에서 본 16열 헤더 + 행. stderr 에 한 줄 요약.

json — 프레임 배열 + 집계 요약. 무한대는 JSON 숫자로 표현할 수 없으므로 `null` 이 됩니다. 게이트를 켜다면 프레임마다 `verdict`, 요약에 `gate` 가 붙습니다.

```
av --metrics ref.png ip_out.png --format json --fail-psnr 30
```

```
{
  "frames": [
    {
      "file": "test_a.png", "width": 100, "height": 100, "psnr_db": 7.6564,
      "psnr_r": 7.6564, "psnr_g": null, "psnr_b": 7.6564, "psnr_y": 24.7091,
      "ssim": 0.973462, "flip": 0.549758, "mse": 7436.17, "mae": 57.5,
      "max_error": 202, "msigned": 10.6353, "psnr_cb": 11.8849, "psnr_cr": 12.915,
      "verdict": "FAIL"
    }
  ],
  "summary": {
    "frames": 1, "missing": 0,
    "psnr": {
      "mean": 7.6564, "median": 7.6564, "p95": 7.6564, "min": 7.6564, "max": 7.6564
    },
    "ssim": {
      "mean": 0.973462, ...
    },
    "flip": {
      "mean": 0.549758, ...
    },
    "gate": {
      "verdict": "FAIL", "fail": 1, "warn": 0
    }
  }
}
```

행이 `missing` / `decode_error` / `mismatch` 이면 수치 필드 대신 `{"file": "...", "status": "missing"}` 형태로만 나옵니다.

junit — CI 서버가 바로 읽는 테스트 리포트 XML. 프레임 하나가 testcase 하나이고, 게이트 FAIL 이 `<failure>`, 플레이스홀더 행이 `<skipped>` 입니다.

```
av --metrics ref.png ip_out.png --format junit --fail-psnr 30 > junit.xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
<testsuite name="av-metrics" tests="1" failures="1" skipped="0">
  <testcase name="test_a.png">
    <failure message="metric gate">psnr=7.6564 ssim=0.973462
flip=0.549758 maxerr=202</failure>
  </testcase>
</testsuite>
```

참고 `--format` 은 `--metrics` 와 `--batch` 에 적용됩니다. `--comp-out` 은 `csv` 와 `json` 만 지원하고 `junit` 을 주면 `stderr` 경고 후 `CSV` 로 떨어집니다. `--probe` / `--uniformity` / `--validate` / `--comp-batch` 는 항상 `CSV` 입니다.

8.3. CI 게이트 — `--fail-*` / `--warn-psnr`

8.3.1. 다섯 개의 임계값

게이트 옵션 중 하나라도 주면 게이트가 활성화되고, 프레임마다 `PASS/WARN/FAIL` 판정이 붙습니다. 지정하지 않은 임계값은 내부적으로 `-1` 이라 검사되지 않습니다. **방향에 주의하세요** — `PSNR/SSIM` 은 “낮으면 실패”, `FLIP/MaxErr` 는 “높으면 실패” 입니다.

옵션	방향	판정
<code>--fail-psnr <dB></code>	<code>psnr_db < dB</code>	FAIL. 단 <code>psnr_db</code> 가 <code>inf</code> (완전 동일)면 검사 자체를 건너뛴
<code>--warn-psnr <dB></code>	<code>psnr_db < dB</code>	WARN. 종료 코드에는 영향 없음(0 유지)
<code>--fail-ssim <v></code>	<code>ssim < v</code>	FAIL. 동일 영상은 <code>ssim =1</code> 이라 통과
<code>--fail-flip <v></code>	<code>flip > v</code>	FAIL. FLIP 은 낮을수록 좋으므로 부등호 방향이 반대
<code>--fail-maxerr <v></code>	<code>max_error > v</code>	FAIL. 단일 픽셀 스파이크를 잡는 용도

한 프레임이라도 `FAIL` 이면 프로세스는 `exit 10` 으로 끝납니다. `WARN` 만 있으면 `exit 0` 입니다(로 그에만 표시). 게이트는 `--metrics` 와 `--batch` 에서 동작하고, `--uniformity` 는 별도의 `--fail-uniformity` / `--fail-semu` 를 씁니다.

8.3.2. 게이트 출력

`CSV` 모드에서는 `stderr` 요약 줄 끝에 `| GATE ...` 가 붙고, 그다음 줄에 문제 프레임 목록이 나열됩니다.

예시 — 게이트 FAIL 시 `stderr`

```
av --pair --metrics golden/f001.png out/ --fail-psnr 30 --warn-psnr 40
# stderr:
# [metrics] frames=2 missing=0 mean_psnr=7.6564dB median=7.6564 p95=7.6564
#           min=7.6564 mean_ssim=0.973462 mean_flip=0.549758
#           | GATE FAIL fail=2 warn=0
# [metrics] FAIL:f001.png FAIL:f002.png
# exit code: 10
```

팁 DDI 양산 회귀에서는 `--fail-psnr` 을 “절대 넘으면 안 되는 하한”으로, `--warn-psnr` 을 “품질이 서서히 떨어지는 신호”로 두 단계로 거는 것이 안전합니다. 예: `--fail-psnr 38 --warn-psnr 42` . 여기에 `--fail-maxerr 8` 을 더하면 평균은 멀쩡한데 한두 픽셀만 크게 튀는 보상 LUT 경계 아티팩트를 잡을 수 있습니다.

8.4. 픽셀 컬러메트리 — `--probe <X,Y>`

8.4.1. 무엇을 출력하나

이미지 A(그리고 선택적으로 B)의 픽셀 좌표 (X,Y) 한 점을 `CIE` 값으로 변환해 11열 `CSV` 로 출력합니다. GUI 의 `Shift+V` 컬러메트리 벌룬(3장 참조)과 같은 색 변환 코어를 씁니다.


```
av --probe 540,1196 panel_white.png
av --probe 540,1196 ref.png ip_out.png      # A / B / delta 세 행
```

열	단위	의미
which	—	행 종류: A / B / delta
X	—	CIE 1931 삼자극치 X (D65)
Y	—	CIE 1931 Y — 상대 휘도
Z	—	CIE 1931 Z
x	—	CIE 1931 색도 x
y	—	CIE 1931 색도 y
uprime	—	CIE 1976 u'
vprime	—	CIE 1976 v'
CCT	K	McCamy 근사 상관색온도
Duv	—	흑체 궤적으로부터의 편차
Lstar	0~100	CIE L* (명도)

8비트 이미지는 화소값을 sRGB 감마 인코딩된 표시값으로 보고 디코드하며, HDR(float) 이미지는 선형 광 그대로 취급합니다.

8.4.2. 검증된 기준값

순백색(255,255,255) 픽셀을 찍으면 sRGB 의 정의상 D65 백색점이 정확히 나와야 합니다. 실제 실행 결과입니다.

예시 — D65 백색점 확인

```
av --probe 32,32 white.png
```

```
which,X,Y,Z,x,y,uprime,vprime,CCT,Duv,Lstar
A,0.950456,1,1.08906,0.3127,0.329,0.19783,0.46832,6505.08,0.00320742,100
```

x=0.3127, y=0.3290, CCT ≈6504K, Lstar=100 — 교과서상의 D65 값과 일치합니다. 색 파이프라인이 정상인지 CI 에서 확인하는 스모크 테스트로 그대로 쓸 수 있습니다.

8.4.3. B 를 함께 준 경우의 delta 행

두 번째 이미지를 주면 A, B 행에 이어 delta 행이 나옵니다. 이 행은 앞의 XYZ/xy/u'v' 칸을 비워 두고 뒤의 세 칸만 채웁니다.

- CCT 칸 → ΔCCT (B - A, 단위 K)
- Duv 칸 → $\Delta u'v'$ (두 점 사이의 색도 거리)
- Lstar 칸 → ΔE_{76} (CIE 1976 색차)

```
which,X,Y,Z,x,y,uprime,vprime,CCT,Duv,Lstar
A,...,...
B,...,...
delta,,,,,,,,-118.4,0.00214,1.83
```

팁 보상 전/후 패넌의 같은 좌표를 찍어 $\Delta u'v'$ 를 보면 보상 IP 가 색을 얼마나 틀어놓았는지 한 줄로 확인됩니다. 일반적으로 $\Delta u'v'$ 0.004 정도가 육안 식별 한계 근처이므로, 이를 CI 임계로 쓰기 좋습니다(수치는 스크립트에서 직접 비교 — `--probe` 에는 게이트 옵션이 없습니다).

좌표가 이미지 범위를 벗어나면 stderr 에 범위 안내를 출력하고 exit 3 입니다.

8.5. 평판 균일도와 무라 — `--uniformity`

8.5.1. 무엇을 재나

플 화이트/그레이 같은 **플랫 필드** 영상 한 장을 받아 휘도 균일도, 비균일도, 색 편차, 무라 지수를 한 번에 계산합니다. 패넌 점등 검사 이미지나 보상 전후 플랫 필드를 CI 로 추적할 때 씁니다.

```
av --uniformity flat_white.png
av --uniformity flat_before.png flat_after.png
av --pair --uniformity flat/f001.png flat_after/
```

8.5.2. 11개 열

```
file,role,width,height,Lmean,uni9_pct,uni13_pct,uni25_pct,nonuni_cv_pct,duv_max,semu
```

열	의미
file	이미지 파일명. Δ 행은 <code>delta</code>
role	A / B / B-A
width , height	픽셀 크기
Lmean	화면 전체 평균 CIE Y (상대 휘도). 8비트는 sRGB 디코드, HDR 은 선형광
uni9_pct	ICDM 9점 휘도 균일도 % = $100 \cdot L_{min} / L_{max}$
uni13_pct	ICDM 13점 균일도 %
uni25_pct	ICDM 25점 균일도 %
nonuni_cv_pct	전 픽셀 변동계수 % = $100 \cdot \sigma / \mu$. 낮을수록 좋음
duv_max	25개 측정점 $u'v'$ 중 화면 평균 $u'v'$ 로부터의 최대 거리
semu	SEMU 무라 지수 프록시. 평탄하면 0, 클수록 무라가 눈에 뵈

8.5.3. 균일도 % 의 정의와 측정점 배치

uni9 / uni13 / uni25 는 모두 $100 \cdot L_{min} / L_{max}$ 입니다. **100% 가 완벽** 이고 값이 클수록 좋습니다(CV 와 방향이 반대이니 혼동에 주의하세요). 각 측정점은 한 픽셀이 아니라 반지름 `max(1, min(W,H)/40)` 인 정사각 박스 평균을 씁니다 — 센서 노이즈나 디터 패턴에 흔들리지 않게 하기 위한 처리입니다.

측정점 격자는 ICDM 관례를 따릅니다.

- **9점**: 가로/세로 각각 1/6, 3/6, 5/6 위치의 3×3
- **13점**: 위 9점 + 1/3, 2/3 위치의 안쪽 2×2 네 점
- **25점**: 가로/세로 각각 1/10, 3/10, 5/10, 7/10, 9/10 의 5×5

nonuni_cv_pct 는 격자 점이 아니라 **전 픽셀** 의 표준편차/평균이므로, 격자 사이에 숨은 얼룩까지 반영합니다. 균일도 %는 좋은데 CV 가 크다면 측정점을 비껴간 국부 결함이 있다는 뜻입니다.

8.5.4. SEMU 무라 지수 — 계산 방식과 한계

semu 는 다음 순서로 계산됩니다.

1. 휘도 필드에 2차 다항식 배경면 $\{1, x, y, x*x, x*y, y*y\}$ 을 최소제곱으로 피팅해 뺍니다. 부드러운 셰이딩·기울기·비네팅은 이 단계에서 깨끗이 제거되므로, 순수 그라디언트 영상은 잔차가 거의 0 이 됩니다 (그건 무라가 아니라 셰이딩이니깐요).
2. 잔차의 Weber 대비 맵에서 최대값 C_{max} 를 구합니다.
3. $|c| \geq 0.5 * C_{max}$ 인 픽셀 면적 S 를 프레임 대비 % 로 구합니다.
4. $semu = 100 * C_{max} / (1.97 / S^{0.33} + 0.72)$ 로 가시성 정규화를 적용합니다.

주의 `semu` 는 **프로시** 입니다. 표준 SEMU 정의는 결함의 물리적 크기와 시청 거리를 요구하지만, av 는 이미지 한 장만 보므로 면적을 “프레임 대비 %” 로 가정합니다. 실제 픽셀 피치·시청 거리는 모델링 되지 않으므로 **절대 합격/불합격 판정에 쓰지 말고**, 같은 해상도·같은 촬영 조건에서의 상대 추세(보상 전 대비 후, 리비전 간 비교)로만 사용하세요. 실행할 때마다 stderr 에도 같은 경고가 찍힙니다.

8.5.5. A/B/Δ 세 행

이미지를 두 장 주거나 `--pair` 를 쓰면 A 행, B 행에 이어 `delta,B-A` 행이 따라옵니다. Δ 행은 단순 차(B - A)이므로 부호 해석이 중요합니다.

- `uni*_pct` 가 **양수** → 보상 후 균일도가 좋아짐
- `nonuni_cv_pct` 가 **음수** → 비균일도가 줄어듦 (개선)
- `semu` 가 **음수** → 무라가 줄어듦 (개선)

예시 — 보상 전후 균일도 비교

```
av --uniformity flat.png flat_mura.png
```

```
file,role,width,height,Lmean,uni9_pct,uni13_pct,uni25_pct,nonuni_cv_pct,duv_max,semu
flat.png,A,256,256,0.57758,100,100,100,0.000123628,1.73743e-14,0
flat_mura.png,B,256,256,0.571199,74.2218,74.2218,74.2218,5.2796,1.38294e-14,11.8958
delta,B-A,256,256,-0.00638171,-25.7782,-25.7782,-25.7782,5.27948,-3.54484e-15,11.8958
```

완전 평탄한 A 는 균일도 100%, CV ≈ 0 , SEMU 0 입니다. 중앙에 원형 얼룩을 넣은 B 는 균일도 74.2%, CV 5.28%, SEMU 11.9 로 떨어지고 Δ 행이 정확히 그 악화폭을 보여줍니다.

8.5.6. 균일도 게이트

옵션	판정
<code>--fail-uniformity <pct></code>	<code>uni25_pct < pct</code> 이면 FAIL. 25점 균일도 기준
<code>--fail-semu <v></code>	<code>semu > v</code> 이면 FAIL

```
av --uniformity flat_after.png --fail-uniformity 88 --fail-semu 1.5
```

하나라도 FAIL 이면 `exit 10`, 아니면 0 입니다. A 행과 B 행 모두 각각 판정되며 (Δ 행은 판정 대상이 아님), stderr 에 `| GATE FAIL fail=N` 이 붙습니다. 디코드에 실패하면 `exit 4` 입니다.

8.6. 매니페스트 배치 — `--batch <file|->`

8.6.1. 왜 필요한가

`--pair` 는 “두 디렉토리에 같은 파일명” 이라는 제약이 있습니다. `--batch` 는 그 제약을 없애고, **임의의 A,B 쌍 목록** 을 파일 하나로 넘겨 한 번에 처리합니다. 파일명이 서로 다른 골든 이미지와 IP 출력, 여러 테스트 케이스를 섞은 회귀 스위트에 적합합니다.

8.6.2. 매니페스트 형식

- 한 줄에 한 쌍: `A<TAB>B` 또는 `A<TAB>B<TAB>label`
- 구분자는 반드시 **TAB** (공백 아님). 각 필드의 앞뒤 공백/탭은 제거됩니다.
- `#` 로 시작하는 줄은 주석, 빈 줄(공백/탭만 있는 줄 포함)은 무시
- CRLF 줄바꿈도 그대로 처리됩니다
- TAB 이 없는 줄은 stderr 경고 후 건너뛰니다
- `label` 을 주면 CSV `file` 열에 그 label 이 들어갑니다(없으면 A 의 basename)

예시 — 매니페스트 파일 예 (pairs.tsv)

```
# DDI 보상 IP 킥 스윙트
# A(원본)<TAB>B(보상 충격)<TAB>label
golden/gradient.png out/gradient_fw.png gradient-fw
golden/gradient.png out/gradient_hw.png gradient-hw
golden/skin_tone.png out/skin_tone_hw.png skin

# 코너 케이스 (파일명이 서로 달라도 됨)
ref/checker_1080.png out/checker_result.png checker
ref/checker_1080.png ref/checker_1080.png self-identity
```

8.6.3. 실행

```
av --batch pairs.tsv # 파일에서 읽기
cat pairs.tsv | av --batch - # stdin (파이프)
av --batch pairs.tsv --format junit > junit.xml
av --batch pairs.tsv --fail-psnr 38 --fail-maxerr 8
```

출력은 `--metrics` 와 완전히 동일한 16열이고, `--format` 과 모든 `--fail-*` / `--warn-psnr` 게이트가 그대로 적용됩니다.

예시 — 배치 실행 결과

```
av --batch pairs.tsv
```

```
file,width,height,psnr_db,psnr_r,psnr_g,psnr_b,psnr_y,ssim,flip,mse,mae,max_error,msigned,psnr_cb,psnr_cr
frame01,100,100,7.6564,7.6564,inf,7.6564,24.7091,0.973462,0.549758,7436.17,57.5,202,10.6353,11.8849,12.915
identical,100,100,inf,inf,inf,inf,inf,1,0,0,0,0,inf,inf
# stderr: [batch] frames=2 missing=0 mean_psnr=7.6564dB median=7.6564
# p95=7.6564 min=7.6564 mean_ssim=0.986731 mean_flip=0.274879
```

`identical` 행처럼 완전히 같은 두 파일은 모든 PSNR 이 `inf`, `ssim` =1, `flip` =0 이 됩니다. `inf` 행은 평균 PSNR 집계에서 제외되므로 `mean_psnr` 이 7.6564 로 남아 있는 점에 주의하세요(SSIM/FLIP 평균에는 포함됩니다).

파일을 못 열면 exit 3, 유효한 쌍이 하나도 없어도 exit 3 입니다.

8.7. diff 이미지 내보내기 — `--diff-out <png>`

8.7.1. 기본 사용법

GUI 를 띄우지 않고 4장의 diff 시각화를 PNG 파일로 바로 저장합니다. CI 가 FAIL 한 프레임의 “증거 이미지”를 아티팩트로 남길 때 씁니다.

```
av --diff-out diff.png ref.png ip_out.png
av --diff-out diff.png --diff-mode signed --amplify 8 ref.png ip_out.png
av --diff-out diff.png --diff-mode flip --sbs ref.png ip_out.png
```

- `--diff-mode` 를 생략하면(기본 `none`) 절대차(abs)로 저장합니다.
- `--amplify <val>` 은 차이 증폭 배율입니다. 기본 `1.0`, 유효 범위 `0.1~100`. 미세한 보상 오차를 눈으로 보이게 하려면 `8~30` 정도를 씁니다.
- 저장 후 stderr 에 `[diff-out] wrote <경로> (WxH)` 를 찍고 `exit 0` 입니다.

8.7.2. `--sbs` 합성

`--sbs` 를 붙이면 `A | Δ | B` 를 가로로 이어 붙인 폭 3배 이미지를 저장합니다. 리뷰어가 원본과 결과를 나란히 놓고 볼 수 있어 PR 코멘트에 첨부하기 좋습니다.

예시 — 증거 이미지 만들기

```
av --diff-out build/diff_f0007.png --diff-mode signed --amplify 8 --sbs \
golden/f0007.png build/ip_out/f0007.png
# stderr: [diff-out] wrote build/diff_f0007.png (3240x2392)
```

입력이 `1080×2392` 였다면 `--sbs` 로 `3240×2392` 가 됩니다. `signed` 모드라 보상 출력이 원본보다 어두운 곳은 붉게, 밝은 곳은 푸르게 나타나 편향 방향이 한눈에 보입니다.

8.7.3. 주의점

주의 헤드리스 `diff` 는 CPU 경로로 재현되므로 GUI 셰이더와 완전히 같은 모드 집합을 갖지 않습니다. `abs · rel · falsecolor · signed` 는 그대로 재현되고, `flip` 은 FLIP 엔진의 magma 히트맵으로 저장됩니다. 그러나 `ssim` 은 절대차로 대체되고, `alphablend · enhance · highlight` 도 절대차로 떨어집니다. 정확한 SSIM 맵이 필요하다면 GUI 에서 저장하세요(4장 참조).

참고 `--diff-out` 은 **단일 쌍 전용** 입니다. `--pair` 를 같이 줘도 시퀀스를 순회하지 않고 A/B 한 쌍만 처리합니다. 시퀀스 전체의 `diff` 가 필요하다면 아래 CI 스크립트 예처럼 셀 루프로 돌리세요. 두 이미지의 크기가 다르면 `exit 5`, FLIP 계산 실패는 `exit 6`, PNG 쓰기 실패는 `exit 7` 입니다.

8.8. 결함 스캔 — `--validate`

8.8.1. 무엇을 잡나

`float/HDR` 이미지 한 장을 훑어 표현 불가능하거나 범위를 벗어난 픽셀을 셉니다. R/G/B 세 채널만 검사하며 알파는 보지 않습니다.

```
av --validate ip_out.hdr
```

```
class,count
nan,0
inf,0
negative,0
superwhite,1440
# first offenders: class,x,y,channel
# superwhite,17,0,R
# superwhite,17,0,G
# superwhite,17,0,B
```

class	조건	의미
nan	isnan(v)	NaN. 0 나눗셈·미초기화 버퍼의 전형적 증상
inf	isinf(v)	무한대. 오버플로/발산
negative	v < 0	음수 광량. 보상 계수가 과보정한 결과
superwhite	v > 1	1.0 초과. HDR 이면 정상일 수 있음

처음 발견된 최대 20개는 # 주석 줄로 class,x,y,channel 위치까지 알려줍니다. CSV 파서는 이 줄을 주석으로 건너뛰면 됩니다.

8.8.2. 종료 코드와 8비트 입력

- nan 또는 inf 가 하나라도 있으면 exit 8. negative / superwhite 만 있으면 exit 0 입니다(HDR 에서 는 정상 범주일 수 있으므로).
- 8비트 이미지를 주면 구조상 이런 값이 불가능하므로 카운트를 전부 0 으로 찍고 stderr 에 8-bit image (no float data) - nothing to validate 를 남긴 뒤 exit 0 으로 끝냅니다.

팁 보상 IP 의 float 중간 출력(.hdr)을 CI 초입에서 --validate 로 한 번 걸러두면, NaN 이 섞인 프레임으로 PSNR 을 계산하다가 원인 불명의 이상값을 쫓는 시간을 아낄 수 있습니다. GUI 에서 같은 검사를 오버레이로 보려면 / 키를 쓰세요(3장 참조).

8.9. 블록 비교 헤드리스 — --comp-out / --comp-batch

5장의 3영상 블록 비교(--comp)를 CI 에서 수직로 뽑는 두 가지 모드입니다. 둘 다 --comp 모드를 요구하며, 위치 인자 세 개(원본, img2, img3)가 모두 있어야 합니다(세 번째 인자를 주면 --comp 는 자동 활성화 됩니다).

8.9.1. 블록별 테이블 — --comp-out

```
av --comp orig.png fw.png hw.png --comp-out blocks.csv --blk 16x16
av --comp orig.png fw.png hw.png --comp-out - # stdout 으로
av --comp orig.png fw.png hw.png --comp-out - --format json
```

인자가 - 이면 stdout, 그 외에는 그 경로의 파일에 씁니다(열 수 없으면 exit 3). 행은 **전체 블록**이며 row-major(by 바깥 루프, bx 안쪽 루프) 순서입니다.

열	의미
bx , by	블록 그리드 인덱스 (0-based)
x , y	블록 좌상단 픽셀 좌표
w , h	실제 블록 크기. 오른쪽/아래 가장자리는 잘려서 작아짐
mse_img2	img2 블록 MSE (R/G/B 평균)
psnr_img2	img2 블록 PSNR dB. MSE 가 0 이면 inf
mse_img3	img3 블록 MSE
psnr_img3	img3 블록 PSNR dB
dpsnr	psnr_img2 - psnr_img3 . 양수 = img2 우세

dpsnr 의 특수값: img2 만 완전 일치면 inf , img3 만 완전 일치면 -inf , 둘 다 완전 일치면 0 입니다.

stderr 에는 승패 요약 한 줄이 나옵니다. 승패 기준은 블록 MSE 비의 $d = 10 \cdot \log_{10}(\text{mse_img3} / \text{mse_img2})$ 로, $d > 1$ 이면 img2 승, $d < -1$ 이면 img3 승, 그 사이는 무승부입니다(즉 1 dB 이내는 동률로 봅니다).

예시 — 블록 테이블 뽑기

```
av --comp orig.png fw.png hw.png --comp-out - --blk 32x32
```

```
bx,by,x,y,w,h,mse_img2,psnr_img2,mse_img3,psnr_img3,dpsnr
0,0,0,0,32,32,448.583,21.6124,0,inf,-inf
1,0,32,0,32,32,3698.15,12.4510,0,inf,-inf
2,0,64,0,32,32,15823.6,6.1378,0,inf,-inf
3,0,96,0,4,32,26141.7,3.9575,0,inf,-inf
...
# stderr:
# [comp] blocks=16 nonzero=16 img2_psnr=7.6564 img3_psnr=999
#       | wins img2=0 img3=16 tie=0 (|d|>1dB)
```

마지막 열의 `w=4` 는 폭 100px 이미지를 32px 블록으로 나눌 때 남는 자투리입니다. `stderr` 의 `img3_psnr=999` 는 전역 PSNR 이 무한대(완전 일치)라는 내부 표현입니다.

주의 헬프 문구와 달리, `--comp-out` 이 실제로 반영하는 것은 `--blk` 뿐입니다. CSV 는 **모든** 블록을 출력하므로 `--num_blk` 는 행 수를 바꾸지 않고(그 옵션은 GUI 의 worst 박스 개수 전용), `--blk-metric` 도 열 값에 영향을 주지 않습니다 (`mse_*` / `psnr_*` 열은 항상 RGB 기준). 랭킹 기준을 바꿔 보고 싶다면 `--comp-batch` 의 `worst2` / `worst3` 열이나 GUI 를 쓰세요.

`--format json` 을 주면 같은 필드를 객체 배열로 냅니다. `inf` 표현을 위해 `psnr_img2` / `psnr_img3` / `dpsnr` 은 **문자열** 로 인코딩됩니다.

```
[
  {
    "bx":0,"by":0,"x":0,"y":0,"w":32,"h":32,
    "mse_img2":448.583,"psnr_img2":"21.6124",
    "mse_img3":0,"psnr_img3":"inf","dpsnr":"-inf"},
    ...
  ]
```

8.9.2. 프레임 요약 — `--comp-batch`

세 이미지 각각의 **디렉토리** 에서 같은 이름을 가진 프레임을 전부 훑어, 프레임당 한 행으로 요약합니다. 알고리즘 리비전 두 개(예: FW 레퍼런스 vs HW RTL) 의 추세를 CI 에서 추적하는 데 씁니다.

```
av --comp orig/f001.png fw/f001.png hw/f001.png --comp-batch --blk 16x16
```

열	의미
<code>file</code>	프레임 파일명
<code>psnr_img2</code>	img2 의 원본 대비 전역 PSNR dB (완전 일치는 <code>inf</code>)
<code>psnr_img3</code>	img3 의 전역 PSNR dB
<code>dpsnr</code>	<code>psnr_img2 - psnr_img3</code> . 어느 한쪽이 <code>inf</code> 면 <code>0</code>
<code>win2</code>	img2 가 1 dB 이상 이긴 블록 수
<code>win3</code>	img3 가 1 dB 이상 이긴 블록 수
<code>tie</code>	1 dB 이내 무승부 블록 수
<code>worst2</code>	img2 의 #1 worst 블록. <code>bx:by@psnr</code> 형식, 없으면 <code>-</code>

worst3	img3 의 #1 worst 블록
status	ok / missing (짝 없음·디코드 실패) / mismatch (크기 불일치)

`--blk-metric rgbly|chroma` 는 `worst2` / `worst3` 의 랭킹 기준에 실제로 반영됩니다(기본 `rgb`). `--blk` 는 블록 크기를, 따라서 승패 카운트를 바꿉니다.

예시 — 리비전 추세 요약

```
av --comp orig/f001.png fw/f001.png hw/f001.png --comp-batch --blk 16x16
```

```
file,psnr_img2,psnr_img3,dpsnr,win2,win3,tie,worst2,worst3,status
f001.png,7.6564,inf,0,0,49,0,6:6@3.96,-,ok
f002.png,7.6564,7.6564,0.0000,0,0,49,6:6@3.96,6:6@3.96,ok
# stderr:
# [comp-batch] frames=2 missing=0 mean_psnr img2=7.6564 img3=7.6564
# | total wins img2=0 img3=49 tie=49 (|d|>1dB)
```

`f001.png` 는 `img3` 가 원본과 완전히 같아 `psnr_img3=inf`, `worst3=-` 이고 49개 블록 전부 `img3` 승입니다. `worst2=6:6@3.96` 은 그리드 (6,6) 블록의 PSNR 이 3.96 dB 로 가장 나쁘다는 뜻입니다 — 이 좌표를 GUI 에 그대로 입력해 현장 확인할 수 있습니다(5장 참조).

유효 프레임이 하나도 없으면 `exit 3` 입니다. 두 `comp` 모드 모두 `--fail-*` 게이트를 지원하지 않으므로, 임계 판정은 CSV 를 받아 스크립트에서 하세요.

8.10. 종료 코드

CI 스크립트는 `av` 의 종료 코드만 보고도 무슨 일이 있었는지 구분할 수 있습니다.

코드	의미
0	성공. WARN 만 발생한 경우도 0 입니다
1	CLI 파싱·검증 실패. 알 수 없는 옵션, 잘못된 <code>--blk</code> / <code>--format</code> /hex, 값 누락, <code>--comp</code> 에 이미지 3장 미달, <code>--comp-out</code> / <code>--comp-batch</code> 를 <code>--comp</code> 없이 사용, <code>--pair</code> 의 두 디렉토리가 동일
2	SDL/윈도우/ImGui 초기화 실패. GUI 경로에서만 발생
3	헤드리스 인자 오류. 이미지 인자 부족, <code>--probe</code> 좌표 형식/범위 오류, 매니페스트 열기 실패, 유효 쌍 0개, <code>--comp-out</code> 파일 열기 실패, <code>--comp-batch</code> 유효 프레임 0개
4	이미지 디코드 실패 (파일 없음·지원하지 않는 포맷·손상)
5	크기/포맷 불일치. 단일 쌍 <code>--metrics</code> , <code>--diff-out</code> , <code>--comp-out</code> 에서 발생
6	<code>--diff-out</code> 의 FLIP 계산 실패
7	<code>--diff-out</code> 의 PNG 쓰기 실패 (경로 없음·권한·디스크)
8	<code>--validate</code> 에서 NaN 또는 Inf 발견
10	CI 게이트 FAIL. <code>--fail-psnr</code> / <code>--fail-ssim</code> / <code>--fail-flip</code> / <code>--fail-maxerr</code> 또는 <code>--fail-uniformity</code> / <code>--fail-semu</code> 위반

참고 `--pair --metrics` 처럼 여러 프레임을 도는 경우, 개별 프레임의 `mismatch` 는 해당 행만 플레이스홀더로 남기고 전체 종료 코드를 5 로 만들지 않습니다. `exit 5` 는 **단일 쌍** 실행에서만 나옵니다. 시퀀스 실행에서는 게이트 결과(0 또는 10)가 종료 코드를 결정합니다.

8.11. 실전 CI 스크립트

8.11.1. (a) bash 회귀 게이트 루프

시퀀스 전체를 게이트에 통과시키고, FAIL 한 프레임만 골라 증거 diff 이미지를 만드는 스크립트입니다.

`set -e` 를 쓰지 않는 것이 핵심입니다 — `exit 10` 은 “정상적으로 판정된 실패” 이므로 우리가 직접 처리해야 합니다.

```
#!/usr/bin/env bash
# scripts/av_regression.sh - DDI 보상 IP 회귀 게이트
set -uo pipefail

REF=golden          # 원본(보상 전) 프레임 디렉토리
OUT=build/ip_out     # 보상 IP 출력 프레임 디렉토리
REPORT=build/metrics.csv
MIN_PSNR=38
mkdir -p build

first=$(ls "$REF"/*.png | head -n 1)
if [ -z "$first" ]; then echo "no reference frames in $REF"; exit 1; fi

# 1) 시퀀스 전체 지표 + 2단계 게이트 (stdout=CSV, stderr=요약은 CI 로그로)
av --pair --metrics "$first" "$OUT" \
  --fail-psnr "$MIN_PSNR" --warn-psnr 42 \
  --fail-ssim 0.985 --fail-flip 0.08 --fail-maxerr 8 \
  > "$REPORT"
rc=$?

case "$rc" in
  0) echo "== GATE PASS ==" ;;
  10) echo "== GATE FAIL == 아래 프레임이 인계를 넘었습니다:" ;;
  *) echo "av 실행 실패 (exit $rc)"; exit "$rc" ;;
esac

# 2) FAIL 프레임 추출 - psnr_db(4영)가 숫자이면서 인계 미달인 행만
# ('inf'/'missing'/'mismatch' 같은 문자열 행은 정규식으로 걸러낸다)
fails=$(awk -F, -v t="$MIN_PSNR" \
  'NR>1 && $4 ~ /^[0-9.]+$/ && $4+0 < t { print $1 }' "$REPORT")

# 3) FAIL 프레임의 증거 diff PNG 생성 (아티팩트 첨부용)
for f in $fails; do
  echo "  - $f"
  av --diff-out "build/diff_$f" --diff-mode signed --amplify 8 --sbs \
    "$REF/$f" "$OUT/$f"
done

# 4) 플랫폼 픽드 균일도도 함께 확인 (별도 게이트, 별도 exit)
if [ -f "$OUT/flat_white.png" ]; then
  av --uniformity "$REF/flat_white.png" "$OUT/flat_white.png" \
    --fail-uniformity 88 --fail-semu 1.5 > build/uniformity.csv
  urc=$?
```

```
[ "$src" -eq 10 ] && { echo "== UNIFORMITY GATE FAIL =="; rc=10; }
fi

exit "$rc"
```

팁 awk 필터에서 `$4 ~ /^[0-9.]+$` 를 반드시 넣으세요. `psnr_db` 열에는 완전 일치 프레임의 `inf` 나 플레이스홀더의 `mismatch / missing` 문자열이 들어올 수 있는데, awk 구현에 따라 이들이 0 으로 형변환되어 **멀쩡한 프레임을 FAIL 로 오인** 하게 됩니다.

8.11.2. (b) GitHub Actions — JUnit 리포트 업로드

`--format junit` 으로 표준 테스트 리포트를 만들고, 게이트가 실패해도 리포트를 아티팩트로 남기는 워크플로입니다. `if: always()` 가 그 역할을 합니다.

```
# .github/workflows/image-quality.yml
name: image-quality

on: [push, pull_request]

jobs:
  metrics:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build & install av
        run: |
          cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
          cmake --build build -j"${nproc}"
          cmake --install build --prefix "$HOME/.local"
          echo "$HOME/.local/bin" >> "$GITHUB_PATH"

      - name: Generate IP output frames
        run: ./scripts/run_ip.sh golden build/ip_out

      # 헤더리스 - 디스플레이된 GPU 도 펍은 없습니다
      - name: Metrics gate (JUnit)
        run: |
          mkdir -p reports
          av --pair --metrics golden/f0001.png build/ip_out \
            --format junit \
            --fail-psnr 38 --fail-ssim 0.985 --fail-flip 0.08 \
            > reports/av-metrics.xml

      - name: Metrics table (CSV, 추세 보관용)
        if: always()
        run: |
          av --pair --metrics golden/f0001.png build/ip_out \
            > reports/av-metrics.csv
```

```

- name: Uniformity gate
  if: always()
  run: |
    av --uniformity golden/flat_white.png build/ip_out/flat_white.png \
      --fail-uniformity 88 --fail-semu 1.5 \
      > reports/av-uniformity.csv

- name: Upload reports
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: av-reports
    path: |
      reports/av-metrics.xml
      reports/av-metrics.csv
      reports/av-uniformity.csv

- name: Publish JUnit result
  if: always()
  uses: mikepenz/action-junit-report@v4
  with:
    report_paths: reports/av-metrics.xml

```

동작 방식:

1. `Metrics gate` 스텝이 `exit 10` 을 내면 그 스텝이 실패하고 잡이 빨간불이 됩니다.
2. 뒤따르는 스텝은 모두 `if: always()` 라 계속 실행되어 CSV/균일도 리포트와 아티팩트 업로드가 끝까지 진행됩니다.
3. `action-junit-report` 가 `<failure>` 가 붙은 testcase 를 PR 체크에 프레임 이름 그대로 표시하므로, 어느 프레임이 회귀했는지 PR 화면에서 바로 보입니다.

팁 야간 회귀에는 `--comp-batch` 를 한 스텝 더 붙여 FW 레퍼런스와 HW RTL 의 프레임별 `dpsnr` · `win2` / `win3` 추세를 CSV 로 축적해 두면, 특정 커밋에서 어느 블록부터 벌어지기 시작했는지 되짚기 쉽습니다. `worst2` / `worst3` 의 `bx:by@psnr` 좌표를 그대로 GUI 에 넣어 현장 확인하는 흐름은 5장을 참조하세요.

9. 색 관리와 룩(LUT · CDL)

DDI 보상 IP 를 검증할 때 화면에서 보이는 색과 파일에 들어 있는 숫자는 반드시 구분해야 합니다. “보상 후 영상이 좀 누렇게 보인다”는 말은 파일의 픽셀 값이 바뀌었다는 뜻일 수도 있고, 보고 있는 뷰어가 색을 변환해 보여 준다는 뜻일 수도 있습니다. 이 장에서는 av 가 파일의 값을 화면에 올리기가까지 거치는 단계, 그 중 어디에 ICC 프로파일과 룩(LUT/CDL)이 끼어드는지, 그리고 **룩이 지표 계산에는 전혀 영향을 주지 않는다**는 av 의 핵심 설계 원칙을 소스 수준에서 확인합니다. 마지막으로 CIE 컬러메트리 계산을 담당하는 공용 색 코어를 다룹니다.

9.1. 표시 파이프라인 — 어느 단계에서 무엇이 적용되는가

av 의 화면 표시는 다음 순서로 진행됩니다. 파일 디코드 → 색 관리 → 룩(LUT/CDL) → 채널 선택 → 화면 표시. 이 순서는 `src/ui/main_window.cpp` 의 텍스트 업로드, `src/gl_texture.cpp` 의 내부 포맷, `src/shader_sources.h` 의 `IMAGE_FRAG_SRC` 프래그먼트 셰이더에서 그대로 확인할 수 있습니다.

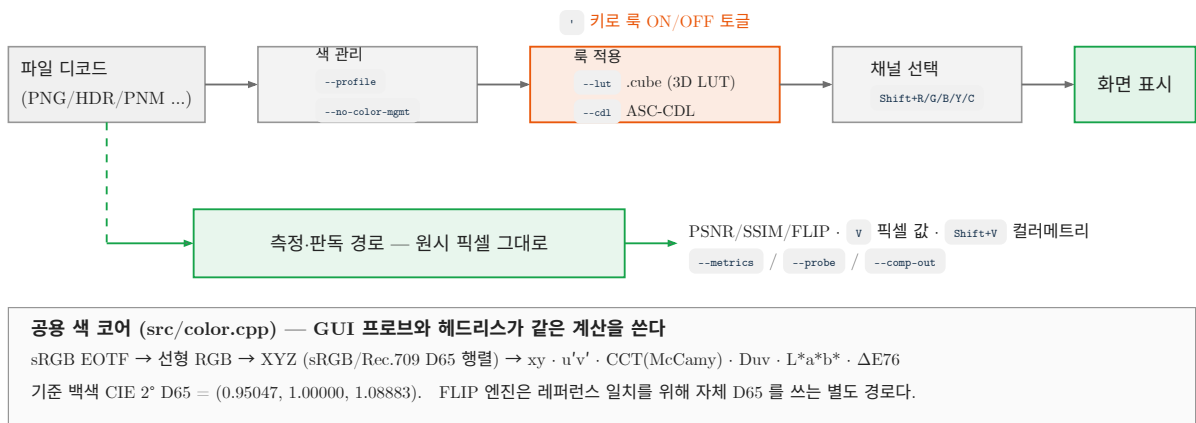


그림 9.1: 표시 파이프라인과 측정 경로의 분리. 룩(LUT/CDL)은 화면에만 적용되며 지표·픽셀 값 판독에는 영향을 주지 않는다.

단계	담당 코드	하는 일
디코드	<code>src/image_loader.cpp</code>	PNG/JPG/BMP/TGA/PGM/PPM/HDR 을 CPU 버퍼로. LDR 은 8bit RGBA, HDR 은 RGBA32F, PPM 은 원본 16bit 값을 별도 보관
텍스처 업로드	<code>src/gl_texture.cpp</code>	LDR 은 <code>GL_RGBA8</code> / <code>GL_RGB8</code> / <code>GL_R8</code> , HDR 은 <code>GL_RGBA32F</code> . sRGB 내부 포맷을 쓰지 않으므로 업로드 단계에서 값이 변형되지 않습니다
색 관리	<code>--profile</code> / <code>--no-color-mgmt</code>	옵션으로 의도를 기록. 현재 릴리스에서는 변환이 적용되지 않습니다 (아래 절 참조)
룩(LUT/CDL)	<code>src/lut.cpp</code> + 셰이더 <code>u_lut</code>	<code>--lut</code> / <code>--cdl</code> 로 로드한 격자를 3D 텍스처로 샘플링. 키로 ON/OFF
채널 선택	셰이더 <code>u_channel</code>	<code>Shift+R/G/B</code> 단일 채널, <code>Shift+Y</code> Luma, <code>Shift+C</code> 복귀 (3장 참조)
픽셀 그리드	셰이더	좀 16배 이상에서 격자선을 0.4 비율로 섞어 표시

셰이더 본문의 실제 순서는 다음과 같습니다. 룩이 채널 선택보다 **먼저** 적용된다는 점이 중요합니다. 즉 `Shift+R` 로 R 채널만 보고 있을 때 화면에 뜨는 밝기는 “LUT 를 통과한 뒤의 R” 입니다.

```

out_color = texture(u_tex, uv); // 1. 원본 샘플
if (u_lut_enabled == 1) // 2. 룩
    out_color.rgb = texture(u_lut, clamp(out_color.rgb, 0.0, 1.0)).rgb;
if (u_channel == 1) out_color = vec4(vec3(out_color.r), out_color.a); // 3. 채널
// ... u_channel 4 = Luma = dot(rgb, vec3(0.2126, 0.7152, 0.0722))
if (u_zoom >= 16.0) { ... } // 4. 픽셀 그라드

```

주의 `--software` 로 실행한 소프트웨어 렌더러는 CPU 경로(`cpu_render_image()` 뒤에 `lut_apply()`)를 쓰기 때문에 **채널 선택이 룩보다 먼저** 적용됩니다. 채널 필터와 룩을 동시에 켜 상태에서 GPU 모드와 `--software` 모드의 화면이 다르게 보일 수 있습니다. 룩을 정량적으로 비교할 때는 한쪽 렌더 경로로 통일하십시오.

참고 보간 방식도 두 경로가 다릅니다. GPU는 `GL_TEXTURE_3D`의 `GL_LINEAR` 하드웨어 삼선형 보간, 소프트웨어 렌더러는 `lut_apply()`의 CPU 삼선형 보간을 씁니다. 결과는 대체로 같지만 격자 사이 값에서 1~2 LSB 차이가 날 수 있습니다.

9.2. 룩은 지표와 픽셀 판독에 영향을 주지 않는다

이 절이 이 장에서 가장 중요합니다. av의 룩은 **표시 전용(display transform)**입니다. LUT나 CDL을 걸어 둔 채로 PSNR을 재도, 픽셀 값을 읽어도, 컬러메트리를 프로브해도, 파일에 저장해도 결과는 룩을 걸지 않았을 때와 완전히 동일합니다.

경로	룩 적용	근거
패널 A / B 화면	적용	<code>render_single()</code> → 이미지 셰이더 <code>u_lut_enabled</code>
<code>--comp</code> 세 패널	적용	세 패널 모두 <code>render_single()</code> 을 통과 (5장 참조)
Diff 패널	미적용	Diff는 <code>DIFF_FRAG_SRC</code> 를 쓰며 LUT 유니폼 자체가 없음 (4장 참조)
SSIM / FLIP 히트맵	미적용	<code>lut_on</code> 계산에서 두 모드를 명시적으로 제외
<code>V</code> 픽셀 값 풍선	미적용	<code>ImageEntry</code> 의 원본 픽셀 배열을 직접 읽음
<code>Shift+V</code> 컬러메트리	미적용	같은 원본 배열에서 <code>color::srgb_to_xyz()</code> 호출
히스토그램 · 라인컷 · 통계	미적용	모두 CPU 픽셀 배열 기반 (6장 참조)
<code>Ctrl+S</code> 저장 / 클립보드	미적용	<code>src/image_save.cpp</code> , <code>src/clipboard_image.cpp</code> 에 LUT 참조 없음
<code>--metrics</code> <code>--probe</code> <code>--uniformity</code> <code>--comp-out</code> <code>--diff-out</code> <code>--validate</code> <code>--batch</code>	미적용	<code>src/metrics_cli.cpp</code> 전체에 LUT 참조가 없음

헤드리스에서는 아예 파일조차 읽지 않습니다. `src/main.cpp` 는 헤드리스 모드를 먼저 디스패치하고 나서(그리고 대부분 그 자리에서 종료) `AppState` 를 만들며, `.cube` / `.cdl` 파싱은 그보다 뒤인 `apply_cli_options()` 안에서 일어나기 때문입니다.

예시 — 룩을 걸어도 PSNR 은 그대로

같은 두 영상에 대해 룩 유무만 바꿔 지표를 뽑아 보면 CSV 가 바이트 단위로 동일합니다. CI 게이트(8장 참조)를 쓰는 팀은 이 성질 덕분에 “검토용 룩”을 스크립트에 남겨 뒀도 안전합니다.

```
av --metrics orig.png comp.png > a.csv
av --metrics orig.png comp.png --cdl shadow_lift.cdl > b.csv
diff a.csv b.csv          # 차이 없음 (--cdl 은 헤드리스에서 파싱조차 되지 않는다)
```

팁 반대로 말하면 “룩을 적용한 결과물”을 지표로 평가할 수는 없습니다. 보상 IP 의 출력에 감마 룩을 씌운 상태를 정량 평가해야 한다면, 룩을 미리 구워 넣은 PNG 를 따로 만들어 그 파일을 `--metrics` 에 넣어야 합니다.

9.3. ICC 프로파일과 색 관리 스위치

9.3.1. `--profile <file>`

ICC 프로파일 파일 경로를 지정합니다. 기본값은 없음(빈 문자열)이며 `.av.ini` 에 저장되지 않으므로 실행 때마다 지정해야 합니다. 값을 생략하면 `Expected value after --profile` 를 출력하고 종료 코드 1 로 끝납니다.

```
av --profile /usr/share/color/icc/sRGB.icc orig.png comp.png
```

9.3.2. `--no-color-mgmt`

색 관리를 끄고 파일의 값을 그대로 화면에 올리라는 스위치입니다. 기본값은 꺼짐 (`no_color_mgmt = false`) 이고 역시 영속화되지 않습니다.

```
av --no-color-mgmt orig.png comp.png
```

9.3.3. 현재 지원 범위와 한계

정직하게 밝혀야 할 사실이 있습니다. 두 옵션은 `CliOptions::icc_profile` 과 `CliOptions::no_color_mgmt` 에 값을 기록만 하며, 현재 릴리스의 렌더 파이프라인은 이 값을 참조하지 않습니다. 빌드 시스템은 `lcms2` 를 선택적으로 찾아 `AV_HAS_LCMS2` 를 정의하지만(`CMakeLists.txt`), 소스 어디에서도 이 매크로를 쓰지 않습니다. 즉 두 옵션의 유무와 관계없이 `av` 의 표시는 항상 `--no-color-mgmt` 상태 입니다.

항목	현재 동작
<code>--profile <file></code>	경로만 저장. 프로파일 변환 미적용. 존재하지 않는 파일을 줘도 오류 없음
<code>--no-color-mgmt</code>	플래그만 저장. 이미 기본 동작이므로 화면 변화 없음
텍스처 내부 포맷	<code>GL_RGBA8</code> 등 선형 포맷 — GPU 의 자동 sRGB 감마 변환 없음
모니터 프로파일	OS 에 위임. <code>av</code> 는 개입하지 않음
영속화	둘 다 <code>.av.ini</code> 에 저장되지 않음

참고 이것은 검증 도구로서는 오히려 안전한 기본값입니다. 픽셀 값 검증이 목적이라면 뷰어가 색을 손대지 않는 편이 낫습니다. `av` 로 코드값 128 인 회색을 열고 `v` 로 읽으면 항상 128 이 나오며, 어

떤 프로파일 환경에서 실행하든 같은 값이 나옵니다. DDI 보상 IP 의 출력 LUT 를 코드값 단위로 대조할 때는 이 성질에 의존하십시오.

주의 따라서 av 의 화면 색을 “절대적인 색” 으로 신뢰해서는 안 됩니다. 광색역 모니터에 연결하면 sRGB 이미지가 과포화되어 보입니다. 이는 av 가 아니라 OS/모니터 프로파일 문제이며, av 는 그 위에서 값을 그대로 흘려보낼 뿐입니다. 색을 “보는” 판단이 필요할 때는 sRGB 로 캘리브레이션된 모니터를 쓰고, 수치 판단은 반드시 `Shift+V` 프로브나 `--probe` 로 하십시오.

9.4. 3D/1D LUT — `--lut <file.cube>`

9.4.1. 무엇을, 왜

`.cube` 파일에 담긴 색 변환을 **양쪽 패널의 화면에** 적용합니다. 보상 IP 가 전제하는 감마 커브를 흉내 내 보거나, 어두운 영역의 차이를 눈으로 보기 위해 색다우를 들어 올릴 때 씁니다. 로드 성공하면 즉시 커진 상태로 시작합니다 (`lut_enabled = true`).

```
av --lut looks/panel_gamma22.cube orig.png comp.png
```

로드에 실패하면 `stderr` 에 다음을 출력하고 **특 없이 정상 실행을 계속**합니다.

```
[lut] failed to load .cube: looks/panel_gamma22.cube
```

9.4.2. `.cube` 파싱 규칙

파서는 `src/lut.cpp` 의 `lut_load_cube()` 입니다. 규칙은 아래 표가 전부입니다.

요소	처리
<code>LUT_3D_SIZE N</code>	3D 격자. 데이터 개수가 정확히 <code>N*N*N*3</code> 이어야 하며 아니면 로드 실패
<code>LUT_1D_SIZE N</code>	1D 커브. 데이터 개수가 정확히 <code>N*3</code> 이어야 함. 채널별 커브를 $N \times N \times N$ 격자로 구워 넣음
<code>TITLE</code>	무시
<code>DOMAIN_MIN / DOMAIN_MAX</code>	무시 — 입력 도메인은 언제나 <code>[0,1]</code> 로 가정
<code>\#</code> 로 시작하는 줄	주석으로 건너뛸
빈 줄 / 공백 줄	건너뛸
데이터 줄	공백으로 구분된 float 3개(R G B). 3개 미만이면 그 줄만 조용히 무시
데이터 순서	R 이 가장 빠르게 변하는 표준 순서. 내부 인덱스는 $((b*N + g)*N + r)*3$
격자 크기 상한	코드상 없음. GPU 3D 텍스처 한계(<code>GL_MAX_3D_TEXTURE_SIZE</code> , 최소 보장 256)와 메모리가 실질 한계
실무 권장	17 / 33 / 65 — 업계 표준 크기

주의 `DOMAIN_MIN / DOMAIN_MAX` 를 무시한다는 점을 잊지 마십시오. 10bit 코드값 도메인 (`DOMAIN_MAX 1023 1023 1023`)으로 작성된 `.cube` 를 그대로 넣으면 값이 전혀 다르게 해석됩니다. av 에 넣기 전에 도메인을 `0.0 ~ 1.0` 으로 정규화해 다시 뽑으십시오.

9.4.3. 보간과 적용 범위

입력 RGB 는 `[0,1]` 로 클램프된 뒤 삼선형(trilinear) 보간으로 샘플링됩니다. GPU 모드에서는 `GL_RGB32F` 3D 텍스처 + `GL_LINEAR` + `GL_CLAMP_TO_EDGE` , 소프트 웨어 모드에서는 `lut_apply()` 가 8개 이웃 격자점을

직접 섞습니다. 텍스처 업로드는 첫 프레임에 지연 수행되므로(`main_window.cpp`), 큰 LUT 를 걸면 첫 프레임만 살짝 느릴 수 있습니다.

9.4.4. 1D LUT 의 함정 — 메모리

`LUT_1D_SIZE N` 은 채널별 1D 커브지만, av 는 이를 **$N \times N \times N$ 3D 격자로 구워 넣습니다.** 즉 1D 점 개수가 그대로 3D 한 번의 길이가 됩니다. 메모리는 `$N * N * N * 3 * 4$` 바이트로 폭증합니다.

N	메모리	판정
33	약 0.4 MB	권장
65	약 3.1 MB	권장
256	약 192 MB	비권장
1024	약 12 GB	사실상 불가 — 할당 실패/스왑

주의 DaVinci Resolve 등에서 뽑은 1024점짜리 1D `.cube` 를 그대로 `--lut` 에 주면 12 GB 를 할당하려 시도합니다. 1D 커브는 64점 이하로 리샘플하거나, 아예 33점 3D `.cube` 로 변환해서 쓰십시오.

9.4.5. 룩 ON/OFF 토글 — `[L]` 키

`[L]` (아포스트로피) 를 누르면 룩 적용이 토글됩니다. 조건은 두 가지입니다. `Ctrl` / `Cmd` 가 눌러 있지 않아야 하고, `--lut` 또는 `--cdl` 로 로드된 격자가 실제로 있어야 합니다. 아무 룩도 로드하지 않은 상태에서 누르면 아무 일도 일어나지 않습니다.

예시 — 보상 결과의 감마 편차를 눈으로 잡기

보상 IP 출력이 원본보다 어둡게 나오는 것 같은데 PSNR 은 45 dB 로 나쁘지 않은 상황입니다.

```
av --cdl looks/shadow_lift.cdl orig.png comp.png
```

1. 실행하면 두 패널 모두 색도우가 들린 상태로 뜹니다. 저계조 차이가 눈에 띕니다.
2. `[L]` 를 눌러 룩을 끕니다 → 원래 화면으로 즉시 돌아갑니다.
3. `[L]` 를 반복해 누르면서 “룩을 걸었을 때만 보이는 밴딩” 이 어느 패널에 있는지 확인합니다.
4. 위치를 찾았으면 `[V]` 로 실제 코드값을 읽습니다. 이 값은 **룩과 무관한 원본 값**이므로 그대로 리포트에 쓸 수 있습니다.

주의 룩이 켜져 있는지 알려 주는 상태바 표시나 오버레이는 없습니다. 화면이 이상해 보이는데 원인을 모르겠다면 먼저 `[L]` 를 눌러 보십시오. 스크린샷을 리포트에 붙일 때도 룩이 켜진 채 캡처하지 않았는지 반드시 확인해야 합니다.

9.5. ASC-CDL — `--cdl <file.cdl>`

9.5.1. 수식과 적용 순서

ASC-CDL 은 색보정 업계 표준인 SOP(Slope/Offset/Power) + SAT(Saturation) 파라미터 집합입니다. av 는 이를 읽어 $33 \times 33 \times 33$ 격자로 구운 뒤 LUT 와 완전히 같은 경로로 적용합니다. 계산 순서는 다음과 같습니다.

```
# 1) SOP - 채널별 (c = R, G, B)
v  = in[c] * slope[c] + offset[c]
v  = max(v, 0.0)                                # 음수는 0 으로 클램프 (pow 정의 보호)
out[c] = pow(v, power[c])
```



```
# 2) SAT - Rec.709 루터 기준
luma = 0.2126*out[R] + 0.7152*out[G] + 0.0722*out[B]
out[c] = clamp(luma + sat * (out[c] - luma), 0.0, 1.0)
```

순서가 핵심입니다. **채널별 SOP 가 먼저, 채도(SAT)가 나중**입니다. 채도를 먼저 적용하는 다른 툴과는 결과가 다르므로, 그레이딩 팀에서 받은 값을 그대로 넣었는데 색이 안 맞으면 이 순서를 먼저 의심하십시오.

9.5.2. 파싱 범위

파서는 `lut_load_cdl()` 이며 XML 을 완전히 파싱하지 않고 태그 이름으로 첫 번째 블록을 찾아 읽는 방식입니다.

요소	처리
확장자	<code>.cdl</code> / <code>.ccc</code> 모두 동일하게 처리 (내용만 봄)
<code><Slope></code>	필수. 없으면 로드 실패. float 3개
<code><Offset></code>	선택. 기본 <code>0 0 0</code>
<code><Power></code>	선택. 기본 <code>1 1 1</code>
<code><Saturation></code>	선택. 기본 <code>1.0</code>
노드 위치	<code><SOPNode></code> / <code><SATNode></code> 안팎 무관 — 파일 전체에서 태그를 찾음
여러 개의 ColorCorrection	첫 번째 것만 사용. <code>.ccc</code> 라이브러리의 두 번째 이후 항목은 무시
네임스페이스 접두사	미지원. <code><cdl:Slope></code> 형태는 인식하지 못함
격자 크기	33 고정 (CLI 로 변경 불가)

`--lut` 과 `--cdl` 을 동시에 주면 `--lut` 이 우선하며 `--cdl` 은 조용히 무시 됩니다. 두 룩을 겹쳐 적용할 수는 없습니다.

예시 — 보상 IP 의 채널 게인을 CDL 로 재현

드라이버 레지스터에 R 게인 `+5%`, B 게인 `-5%`, B 감마 `1.10`, 채도 `1.10` 이 설정돼 있다고 합시다.

이를 `ip_v3.cdl` 로 적으면 다음과 같습니다.

```
<ColorCorrection id="ip_v3">
  <SOPNode>
    <Slope>1.05 1.00 0.95</Slope>
    <Offset>0.00 0.00 0.01</Offset>
    <Power>1.00 1.00 1.10</Power>
  </SOPNode>
  <SATNode><Saturation>1.10</Saturation></SATNode>
</ColorCorrection>
```

```
av --cdl ip_v3.cdl orig.png comp.png
```

이 설정에서 입력 중간 회색 `(0.25, 0.25, 0.25)` 는 `(0.2637, 0.2500, 0.2118)`, 8bit 로는 `(67, 64, 54)` 로 변환됩니다. 흰색 `(1,1,1)` 은 `(255, 255, 242)` 가 되어 화이트 포인트가 노란 쪽으로 이동한 것을 바로 볼 수 있습니다.

9.5.3. 33×33×33 베이킹의 정밀도 한계

CDL 은 격자에 구워지므로 격자점 사이 값은 선형 보간됩니다. 커브가 강한 비선형이면 **첫 격자 구간(코드값 0 ~ 8)**에서 오차가 커집니다. 예를 들어 `<Power>0.45 0.45 0.45</Power>` (감마 리프트) 를 적용했을 때 실제 값은 다음과 같습니다.

입력 8bit	수식대로면	av 표시	설명
1	21	7	첫 격자 구간 안 — 오차 14 LSB
2	29	13	첫 격자 구간 안 — 오차 16 LSB
4	39	27	첫 격자 구간 안 — 오차 12 LSB
8	54	54	$8/255$ 가 첫 격자점 $1/32$ 과 거의 일치 — 정확
16	73	73	정확
32	100	100	정확
128	187	187	정확

즉 격자점($k/32$) 위에서는 정확하고, 0 과 첫 격자점 사이에서만 커브가 직선으로 잘려 나갑니다. 블랙 디테일을 이 룯으로 판정하면 안 되는 이유입니다.

주의 룯은 “눈으로 보기 위한 도구” 이지 정밀 감마 검증 도구가 아닙니다. 감마 특성을 숫자로 검증해야 한다면 룯을 쓰지 말고 `--probe` 의 `Lstar` 열이나 히스토그램, 라인컷(6장 참조)을 쓰십시오. 그래도 룯의 저계조 정확도가 필요하다면, CDL 대신 65점 이상의 `.cube` 를 직접 구워 `--lut` 으로 넣으면 격자가 촘촘해집니다.

9.6. 색 코어 — 컬러메트리 계산의 단일 소스

9.6.1. 변환 체인

`src/color.cpp` 는 GL/SDL/ImGui 에 전혀 의존하지 않는 순수 계산 모듈이며, 모든 계산이 double 정밀도입니다. 변환 체인은 다음과 같습니다.

함수	내용
<code>srgb_to_linear()</code>	sRGB EOTF (IEC 61966-2-1). $c \leq 0.04045$ 이면 $c/12.92$, 아니면 $((c+0.055)/1.055)^{2.4}$
<code>linear_to_srgb()</code>	역변환. $c \leq 0.0031308$ 이면 $c*12.92$, 아니면 $1.055*c^{(1/2.4)} - 0.055$
<code>srgb_to_xyz()</code>	감마 인코딩된 sRGB 값 → 선형화 → XYZ (LDR 이미지용)
<code>lin_rgb_to_xyz()</code>	이미 선형인 값 → XYZ (HDR float 이미지용)
<code>xyz_to_xy()</code>	CIE 1931 색도 $x = X/(X+Y+Z)$, $y = Y/(X+Y+Z)$
<code>xyz_to_upvp()</code>	CIE 1976 UCS $u' = 4X/(X+15Y+3Z)$, $v' = 9Y/(X+15Y+3Z)$
<code>cct_mccamy()</code>	McCamy 1992 근사. $n = (x-0.3320)/(y-0.1858)$, $CCT = -449*n^3 + 3525*n^2 - 6823.3*n + 5520.33$ (단위 K)
<code>duv_from_upvp()</code>	$u'v' \rightarrow 1960$ UCS $(u, v = 2v'/3) \rightarrow$ Ohno/Krystek 6차 다항식. 부호는 흑체궤적 위(+)/아래(-)
<code>xyz_to_lab()</code>	CIE Lab. 기준 백색 CIE 2° D65 = $(0.95047, 1.00000, 1.08883)$
<code>delta_e76()</code>	CIE76 색차 $\sqrt{dL^2 + da^2 + db^2}$ ($L*a*b*$ 각 성분 차의 제곱합 제곱근)

sRGB → XYZ 행렬은 Bradford 적응된 표준 행렬이며, Y 행이 곧 Rec.709 휘도 가중치입니다.

```

X = 0.4123907993 R + 0.3575843394 G + 0.1804807884 B
Y = 0.2126390059 R + 0.7151686788 G + 0.0721923154 B
Z = 0.0193308187 R + 0.1191947798 G + 0.9505321522 B

```

참고 av 안에는 서로 다른 두 개의 “휘도” 개념이 공존하며 혼동하면 안 됩니다. **CIE Y** (광도적 휘도)는 위 행렬의 Y 행을 **선형** RGB 에 적용한 값으로, 컬러메트리 프로브와 `--uniformity` 가 씁니다. 반면 **Rec.709 Y'** (비디오 루마)는 `0.2126R' + 0.7152G' + 0.0722B'` 를 **감마 인코딩된** 값에 그대로 적용한 것으로, diff/SSIM/luma-PSNR 과 `Shift+Y` Luma 채널 뷰가 씁니다.

9.6.2. GUI 프로브와 헤드리스가 같은 코어를 쓴다

`Shift+V` 컬러메트리 풍선말(3장 참조), ROI 통계 창의 ROI 평균 색도(6장 참조), 헤드리스 `--probe`, `--uniformity` 는 모두 같은 `color::` 함수를 호출합니다. 따라서 GUI 에서 눈으로 읽은 값과 CI 스크립트가 뽑은 CSV 값이 어긋날 수 없습니다.

전입점	표시	소스
<code>Shift+V</code>	A xy 0.3127,0.3290 6505K 행, B 행, 그리고 $\Delta u'v' \cdot \Delta CCT$ 행	<code>main_window.cpp</code>
ROI 통계 창	ROI 평균의 $xy \cdot u'v' \cdot CCT \cdot Duv$, 두 장이면 Δ	<code>chart_windows.cpp</code>
<code>--probe <X,Y></code>	<code>which,X,Y,Z,x,y,uprime,vprime,CCT,Duv,Lstar</code> CSV	<code>metrics_cli.cpp</code>
<code>--uniformity</code>	ICDM 균일도 + 색편차 <code>duv_max</code>	<code>metrics_cli.cpp</code>

입력 값 해석 규칙도 공통입니다. PPM 원본 값이 있으면 그 `maxval` 로 정규화한 뒤 sRGB 로, HDR float 이면 선형광으로, 그 외 LDR 8bit 이면 `/255` 후 sRGB 로 봅니다.

예시 — 흰색 픽셀의 컬러메트리 검증

코드값 `(255,255,255)` 인 8bit PNG 와 `(128,128,128)` 인 PNG 를 프로브하면 다음이 나옵니다.

```
av --probe 4,4 white.png gray.png
```

```

which,X,Y,Z,x,y,uprime,vprime,CCT,Duv,Lstar
A,0.950456,1,1.08906,0.3127,0.329,0.19783,0.46832,6505.08,0.00320742,100
B,0.205166,0.215861,0.235085,0.3127,0.329,0.19783,0.46832,6505.08,0.00320742,53.585
delta,,,,,,,,,0,2.77556e-17,46.415

```

흰색의 XYZ 가 D65 백색 `(0.95047, 1.0, 1.08883)` 과 일치하고, `x=0.3127`, `y=0.3290`, `CCT ≈ 6505 K` 로 교과서 값이 그대로 나옵니다. 코드값 128 은 무채색이므로 색도가 동일하고 `Lstar` 만 `53.585` 로 떨어집니다. 이 값은 “코드값 50% = 밝기 50%” 가 아니라는 것을 보여 주는 좋은 근거 자료입니다.

주의 `Duv` 가 정확히 0 이 아니라 `0.0032` 로 나오는 것은 오류가 아닙니다. D65 자체가 흑체궤적에서 살짝 위에 있는 광원이기 때문입니다.

또한 검은 픽셀 `(0,0,0)` 처럼 `X+Y+Z = 0` 인 경우 `xy` 가 `(0,0)` 으로 폴백되며, 그 값을 McCamy 식에 넣으면 `CCT = 2021 K`, `Duv = 0.23` 같은 **무의미한 숫자**가 나옵니다. 매우 어두운 영역이나 클리핑된 영역의 CCT/Duv 는 판독하지 마십시오. McCamy 근사 자체도 흑체궤적 근방에서만 유효합니다.

9.6.3. FLIP 은 별도의 D65 경로다

`src/flip_engine.cpp` 는 자체 D65 상수 `(0.950428545, 1.0, 1.088900371)` 을 가지고 있으며, 위의 색 코어를 쓰지 않습니다. 값이 미세하게 다른 것은 실수가 아니라 **의도된 분리**입니다. FLIP 은 NVIDIA 레퍼런스 구현과 수치가 일치해야 하는 검증된 코드이므로, 컬러메트리를 정리하면서 FLIP 을 건드려 회귀를 만들지 않도록 일부러 손대지 않았습니다.

참고 정리하면 av 에는 두 개의 D65 가 있습니다. 컬러메트리(프로브 · ROI · 균일도)는 CIE 표준 D65 `(0.95047, 1.0, 1.08883)` 을 써서 발표된 레퍼런스와 값이 맞고, FLIP 지표는 NVIDIA 레퍼런스와 값이 맞습니다. 두 값을 통일하려는 시도는 어느 한쪽의 검증을 깨뜨립니다.

9.7. 실무 활용 — DDI 검증 시나리오

9.7.1. 패널 측정 데이터와 대조하기

CA-410 이나 CS-2000 같은 색채휘도계로 패널을 실측하고, 그 데이터를 av 의 프로브 값과 대조하는 흐름입니다. 먼저 성격 차이를 이해해야 합니다.

측정기 값	패널이 실제로 방출한 빛의 색도. 백라이트 · 컬러필터 · 셀 특성이 모두 반영됨
av 프로브 값	파일의 코드값을 이상적인 sRGB 디스플레이가 표시한다고 가정했을 때의 색도. 패널 특성은 반영되지 않음

따라서 두 값을 직접 빼서 비교하면 안 됩니다. 실무에서는 다음 순서로 씁니다.

1. 테스트 패턴(예: R/G/B/W 풀필드, 그레이스케일 스텝)의 **입력 파일**을 `--probe` 로 찍어 “의도한 색도” 를 확보합니다.
2. 같은 패턴을 패널에 띄워 측정기로 “실제 색도” 를 찍습니다.
3. 두 색도의 차이($\Delta u'v'$, ΔCCT)가 패널+IP 가 만들어 낸 오차입니다.
4. 보상 IP 를 태운 출력 파일을 다시 `--probe` 로 찍어, IP 가 의도한 방향으로 보정했는지 확인합니다.

```
# 원본과 보상 결과의 같은 차이를 한 번에 비교 (delta 행이 자동 축적)
av --probe 960,540 pattern/W100.png comp/W100.png
```

`delta` 행의 `CCT` 열은 ΔCCT , `Duv` 열은 $\Delta u'v'$ 거리, `Lstar` 열은 $\Delta E76$ 입니다 (8장 참조). 풀필드 패턴이라면 한 점 대신 `--uniformity` 로 9/13/25점 균일도와 `duv_max` 를 한 번에 뽑는 편이 낫습니다.

팁 측정 데이터로부터 “이 패널에서 이렇게 보인다” 를 **미리 보고** 싶다면, 측정 결과로 3D LUT 을 만들어 `--lut` 으로 걸면 됩니다. 이렇게 하면 av 화면이 패널 시뮬레이터가 됩니다. 다만 이 상태의 화면은 어디까지나 예측이며, 지표와 프로브 값은 여전히 원본 파일 기준이라는 점을 잊지 마십시오.

9.7.2. LUT 으로 감마 보정 후 비교하기

보상 IP 의 결함은 대부분 저계조(블랙 디테일, 밴딩, 디터 패턴)에서 먼저 드러나는데, 감마 2.2 로 인코딩된 이미지를 그대로 보면 이 영역이 너무 어두워 눈에 띄지 않습니다. 이때 새도우를 들어 올리는 룩을 임시로 걸어 두고 비교하면 검출률이 크게 올라갑니다.

```
# shadow_lift.cdl: power 0.45 (감마 리프트), slope/offset 은 상등
av --cdl shadow_lift.cdl orig.png comp.png
```

권장 작업 순서입니다.

1. `--cdl` 또는 `--lut` 으로 룩을 걸고 실행합니다.
2. `'` 로 룩을 껐다 켜며 “룩을 켜었을 때만 보이는 차이” 를 찾습니다.

3. 의심 영역을 찾으면 Diff 모드(4장 참조)로 전환합니다. Diff 패널에는 룩이 적용되지 않으므로 원본 차이가 그대로 보입니다.
4. **V** 와 **Shift+V** 로 원본 코드값과 색도를 읽어 기록합니다.
5. 최종 판정은 `--metrics` / `--comp-out` 수치로 합니다. 룩과 무관합니다.

예시 — 세 영상 비교에 룩을 걸들이기

원본 · FW 보상 · HW 보상 세 장을 블록 비교하면서 저계조를 들여다보는 조합입니다.

```
av --comp orig.png fw.png hw.png --blk 16x16 --num_blk 24 --cdl shadow_lift.cdl
```

세 패널 모두에 같은 룩이 걸리므로 비교 조건이 동일합니다. 위스트 블록 랭킹과 PSNR 은 룩과 무관하게 원본 픽셀로 계산되므로, “눈으로 본 인상” 과 “숫자” 를 같은 화면에서 동시에 확인할 수 있습니다 (5장 참조).

9.8. 요약과 주의점

옵션 / 키	기본값	요점
<code>--profile <file></code>	없음	경로만 저장. 현재 변환 미적용. 영속화 안 됨
<code>--no-color-mgmt</code>	꺼짐	플래그만 저장. av 는 원래 원시 값을 그대로 표시
<code>--lut <file.cube></code>	없음	3D/1D <code>.cube</code> . 로드 성공 시 즉시 ON. 도메인은 <code>[0,1]</code> 가정
<code>--cdl <file.cdl></code>	없음	ASC-CDL SOP+SAT 를 33 격자로 베이킹. <code>--lut</code> 이 있으면 무시됨
<code>'</code>	—	룩 ON/OFF. 로드된 룩이 없으면 무동작. 세션 한정, <code>.av.ini</code> 미저장
<code>Shift+V</code>	꺼짐	컬러메트리 풍선말. <code>--probe</code> 와 같은 코어 (3장 참조)

주의 이 장에서 반드시 기억할 세 가지입니다.

1. **룩은 화면에만 적용됩니다.** 지표(PSNR/SSIM/FLIP), 픽셀 값 판독, 컬러메트리, 파일 저장, 클립보드 복사, 모든 헤드리스 출력은 룩의 영향을 받지 않습니다.
2. **룩이 켜졌다는 표시가 화면에 없습니다.** 스크린샷을 리포트에 넣기 전에 `'` 로 상태를 확인하십시오.
3. `--profile` **은 아직 색을 바꾸지 않습니다.** av 의 화면 색을 절대 기준으로 삼지 말고, 수치 판정은 `Shift+V` 와 `--probe` 로 하십시오.

10. 설정 영속화와 문제 해결

av 는 종료할 때 화면 상태의 일부를 홈 디렉토리의 텍스트 파일에 적어 두고, 다음 실행 때 그대로 복원합니다. 어떤 항목이 저장되고 어떤 항목은 매번 초기화되는지 정확히 알아 두면 “어제와 화면이 다르다”는 혼란을 없앨 수 있습니다. 이 장에서는 설정 파일의 위치와 저장되는 키 전수, 설정을 되돌리는 방법, 모드에 따라 의미가 바뀌는 키의 대조표, 그리고 현장에서 자주 부딪히는 증상별 해결책을 다룹니다.

10.1. 설정 파일 `.av.ini`

10.1.1. 위치와 이름

설정 파일 이름은 점으로 시작하는 `.av.ini` 입니다 (`av.ini` 가 아닙니다). 경로는 환경변수 `HOME` 하나만으로 결정되며, ICC 프로파일이나 XDG 규격 디렉토리는 사용하지 않습니다.

환경	실제 경로
macOS	<code>/Users/<사용자>/ .av.ini</code>
Linux	<code>/home/<사용자>/ .av.ini</code>
Windows — MSYS2 / Git Bash / WSL	<code>\$HOME/.av.ini</code> (셸이 <code>HOME</code> 을 설정함)
Windows — cmd.exe / PowerShell	<code>HOME</code> 이 없으면 현재 작업 디렉토리 의 <code>.av.ini</code>

주의 Windows 의 `cmd.exe` 나 PowerShell 에는 보통 `HOME` 이 정의돼 있지 않습니다. 이때 av 는 경로를 `.` 로 대체하므로, 설정이 **av 를 실행한 폴더**에 남습니다. 폴더를 바꿔 실행하면 설정이 초기화된 것처럼 보입니다. 항상 같은 설정을 쓰려면 실행 전에 `setx HOME %USERPROFILE%` 로 `HOME` 을 한 번 정의해 두십시오.

10.1.2. 저장 시점과 읽는 시점

시점	동작
프로그램 시작	<code>.av.ini</code> 를 읽어 상태에 반영 (파일이 없으면 조용히 무시)
곧이어 CLI 적용	CLI 옵션이 같은 항목을 덮어씀 — CLI 가 항상 이깁니다
실행 중	저장하지 않음. 키를 눌러 토글해도 파일은 그대로
정상 종료	<code>Q</code> 또는 창 닫기 → 이벤트 루프 종료 후 파일 전체를 새로 씀
비정상 종료	<code>kill -9</code> , 크래시, 정전 → 저장되지 않음

즉 저장은 “변경 즉시”가 아니라 “정상 종료 시 1회”입니다. 실행 중에 손으로 파일을 편집해도 종료할 때 전부 덮어써지므로, 파일을 직접 고칠 때는 반드시 av 를 먼저 종료하십시오.

참고 av 를 여러 개 띄우고 각각 다른 상태로 종료하면 **마지막에 종료한 인스턴스**의 상태만 남습니다. 한쪽에서는 히스토그램을 열어 두고 다른 쪽에서는 닫은 채 끄면, 나중에 끈 쪽이 이깁니다.

10.1.3. 파일 형식

`key=value` 한 줄에 한 항목, `#` 으로 시작하는 줄은 주석, `=` 가 없는 줄은 무시하는 아주 단순한 형식입니다. 모르는 키도 그냥 건너뛴다. 실제로 저장된 파일은 다음과 같은 모습입니다.

```
# av configuration
show_borders=1
pixel_format=0
```

```

magnifier_active=1
show_crosshair=0
sync_viewports=1
show_ui=0
show_info=0
show_pixel_info=0
show_histogram=0
show_hline_cut=0
show_vline_cut=0
show_stats=0
show_hotkey_help=0
show_scatter_plot=0
histogram_compare=0
channel_mode=0
windowed_mode=0
overlay_active=0
zoom=0

```

불리언 값은 “0 이면 거짓, 그 외 문자열은 모두 참”으로 해석합니다. `show_ui=true` 라고 써도 참으로 동작합니다.

10.1.4. 저장되는 키 전수

파일에 기록되는 키는 정확히 19개이며, 위 예시의 순서 그대로 매번 새로 씁니다.

키	값	단축키	의미 (기본값)
<code>show_borders</code>	0 / 1	B	패널 테두리 표시 (기본 1)
<code>show_ui</code>	0 / 1	U	메뉴바 + 상태바 오버레이 (기본 0)
<code>windowed_mode</code>	0 / 1	W	타이틀바 있는 창 모드 (기본 0)
<code>show_crosshair</code>	0 / 1	M	십자선 오버레이 (기본 0)
<code>magnifier_active</code>	0 / 1	Ctrl+M	돋보기 (diff 아닌 모드, 기본 1)
<code>overlay_active</code>	0 / 1	O	Overlay/Blend 비교 모드 (기본 0)
<code>sync_viewports</code>	0 / 1	S	A/B 뷰포트 동기 (기본 1)

키	값	단축키	의미 (기본값)
<code>show_info</code>	0 / 1	I	Image Info 창 (기본 0)
<code>show_pixel_info</code>	0 / 1	V	커서 픽셀 값 풍선 (기본 0)
<code>show_histogram</code>	0 / 1	Ctrl+H	히스토그램 창 (기본 0)
<code>show_hline_cut</code>	0 / 1	Ctrl+L	수평 라인 컷 창 (기본 0)
<code>show_vline_cut</code>	0 / 1	Ctrl+Y	수직 라인 컷 창 (기본 0)
<code>show_stats</code>	0 / 1	Ctrl+S	Image Statistics 창 (기본 0)
<code>show_scatter_plot</code>	0 / 1	Ctrl+T	Scatter Plot 창 (기본 0)
<code>show_hotkey_help</code>	0 / 1	Ctrl+Shift+H	핫키 레퍼런스 창 (기본 0)
<code>histogram_compare</code>	0 / 1	— (창 안 체크박스)	히스토그램 A/B 겹쳐 보기 (기본 0)

키	값	단축키	의미 (기본값)
---	---	-----	----------

<code>channel_mode</code>	0 ~ 4	<code>Shift+C</code> 등	채널 표시: 0 = RGB(<code>Shift+C</code>), 1 = R(<code>Shift+R</code>), 2 = G(<code>Shift+G</code>), 3 = B(<code>Shift+B</code>), 4 = Luma Y'(<code>Shift+Y</code>) (기본 0)
<code>pixel_format</code>	0 ~ 2	<code>Ctrl+X</code>	픽셀 값 표기: 0 = 10진, 1 = 0x80, 2 = 80h (기본 0)
<code>zoom</code>	0 또는 배율	<code>--zoom</code>	초기 줌: 0 = fit, >0 = 고정 배율 (기본 0). 적용 시 0.125 ~ 256 으로 클램프

주의 `zoom` 은 “지금 화면의 배율”이 아니라 **초기 배율 설정**입니다. `Z` 나 휠로 확대해도 이 값은 바뀌지 않습니다. 값이 바뀌는 경로는 `--zoom` 을 명시했을 때뿐이며, 한 번 `av --zoom 4 img.png` 로 띄우면 `zoom=4` 가 저장되어 그 다음부터는 인자 없이 실행해도 400% 로 시작합니다. 되돌리려면 `av --zoom fit img.png` 로 한 번 띄웠다 끄거나 `.av.ini` 의 `zoom=` 줄을 지우십시오.

10.1.5. 저장은 되지만 다음 실행에 반영되지 않는 두 키

`sync_viewports` 와 `windowed_mode` 는 파일에 기록되지만, 시작 시 CLI 옵션 적용 단계에서 **무조건** CLI 기본값으로 덮어씁니다. 따라서 실질적으로는 매번 초기화됩니다.

키	시작 시 실제 값	원하는 상태로 시작하는 방법
<code>sync_viewports</code>	<code>--sync</code> 기본값 = 켜짐	끄고 시작하려면 <code>--no-sync</code> 를 붙입니다
<code>windowed_mode</code>	<code>--windowed</code> 기본값 = 꺼짐	창 모드로 시작하려면 <code>--windowed</code> 를 붙입니다

팁 자주 쓰는 조합은 셸 alias 로 굳혀 두는 편이 확실합니다.

```
alias avw='av --windowed --geometry 1600x900'
alias avn='av --no-sync'
```

10.1.6. 영속화되지 않는 상태

아래 항목은 파일에 기록되지 않으므로 실행할 때마다 기본값에서 출발합니다. 검증 조건을 재현하려면 CLI 옵션으로 지정하거나 스크립트로 감싸야 합니다.

항목	시작값 / 지정 방법
Diff 모드	항상 <code>none</code> . <code>--diff-mode</code> 또는 <code>-d</code> 로 지정 (4장 참조)
Diff amplify / threshold	1.0 / 0 . <code>--amplify</code> 로 지정
<code>--comp</code> 관련 전부	<code>--comp</code> <code>--blk</code> <code>--num_blk</code> <code>--blk-metric</code> <code>--grid</code> 로 매번 지정 (5장 참조)
worst 박스, 그리드, 히트맵, 승패맵	<code>show_worst</code> 만 켜짐으로 시작, 나머지는 꺼짐
LUT / CDL 적용	<code>--lut</code> / <code>--cdl</code> 로 로드해야 켜짐 (9장 참조)
불량 픽셀 오버레이 <code>/</code>	항상 꺼짐
컬러메트리 표시 <code>Shift+V</code>	항상 꺼짐
ROI 선택 <code>Ctrl+E</code>	항상 꺼짐
블링크 간격 / 슬라이드쇼 간격	0.5 초 / 3.0 초 고정
pan step	32 px. <code>-p</code> 또는 <code>--pan-step</code> 으로 지정
패널 테두리 색	magenta/yellow/cyan. <code>-bc</code> 로 지정
Pathfinder 미니맵	항상 이미지 미니맵 모드(1)로 시작
활성 패널	항상 A

10.1.7. 창 배치 파일 `.av_imgui.ini`

히스토그램·통계·핫키 레퍼런스 같은 도구 창의 **위치와 크기**는 `.av.ini` 가 아니라 ImGui 가 관리하는 별도 파일 `.av_imgui.ini` 에 저장됩니다. 이 파일 이름은 상대 경로이므로 **av** 를 실행한 작업 디렉토리에 생깁니다. 창 배치가 화면 밖으로 밀려나 보이지 않는다면 이 파일을 지우면 됩니다.

```
rm .av_imgui.ini # 창 배치창 초기화 (토글 상태는 유지)
```

10.2. 설정 초기화와 부분 되돌리기

10.2.1. 전체 초기화

가장 확실한 방법은 파일을 지우는 것입니다. av 를 종료한 상태에서 실행하십시오.

```
rm ~/.av.ini
rm .av_imgui.ini
```

다음 실행에서 av 는 소스에 하드코딩된 기본값(위 표의 “기본값” 열)으로 시작하고, 종료할 때 파일을 다시 만듭니다.

10.2.2. 특정 설정만 되돌리기

로더는 **파일에 있는 키만** 상태에 반영합니다. 따라서 되돌리고 싶은 항목의 줄만 지우면 그 항목만 기본값으로 돌아옵니다.

예시 — 히스토그램 창만 안 뜨게 되돌리기

1. av 를 종료합니다.
2. `~/.av.ini` 에서 `show_histogram=1` 줄을 삭제하거나 `show_histogram=0` 으로 고칩니다.
3. av 를 다시 띄우면 히스토그램 창이 닫힌 상태로 시작합니다.

같은 작업을 셸 한 줄로 하려면:

```
sed -i.bak '/^show_histogram=/d' ~/.av.ini
```

주의 숫자 키(`pixel_format`, `channel_mode`, `zoom`)에 숫자가 아닌 값이나 빈 값을 넣으면 파싱 예외가 발생해 av 가 시작 즉시 죽습니다. `zoom=` 처럼 값을 비워 두지 말고, 지울 때는 줄 전체를 지우십시오. 불리언 키는 어떤 문자열이 들어와도 안전합니다.

10.2.3. 설정을 고정해서 잠그는 방법

여러 사람이 쓰는 검증 장비나 CI 이미지에서는 “누가 뭘 눌러도 다음 사람에게 같은 화면”이 필요합니다. 저장 루틴은 파일을 열지 못하면 조용히 넘어가므로, 원하는 상태로 한 번 저장한 뒤 파일을 읽기 전용으로 만들면 그대로 굳습니다.

```
av golden.png hw.png # 원하는 상태로 댄춘 뒤 Q 로 종료
chmod 444 ~/.av.ini # 이후 종료 시 저장 시도는 무시됨
```

10.3. 모드별 키 재정의 대조표

가장 헷갈리는 부분입니다. `--comp` (3영상 블록 비교) 모드에서는 같은 키가 다른 일을 합니다. 아래 표의 키들은 **모드에 따라 의미가 완전히 바뀌므로** 반드시 구분해서 기억하십시오.

키	일반 모드 (A/B, diff)	<code>--comp</code> 모드 (3영상)
<code>S</code>	뷰포트 동기 on/off 토글	img2 ↔ img3 패널 위치 교환 (통계 재계산)

G	뷰를 이미지 중앙으로 리셋	블록 그리드 경계선 표시 토글
Ctrl+G	동작 없음	블록 PSNR 히트맵 토글 (노랑→빨강)
Tab	활성 패널 전환 (A ↔ B)	마우스가 올라간 패널의 worst 블록 순회
Shift+Tab	활성 패널 전환	worst 블록 역순 순회
[/]	amplify ± 0.5 (AlphaBlend 에서는 alpha $\pm 1\%$)	img2 의 worst 블록만 이전/다음
Shift+[/ Shift+]	threshold ± 1 (AlphaBlend 에서는 alpha $\pm 10\%$)	img3 의 worst 블록만 이전/다음
Ctrl+D	diff 픽셀 리스팅 창	worst 블록 리스트 창 (행 클릭 → 3패널 점프)
Ctrl+B	동작 없음	worst 블록 사각형 표시 토글
Ctrl+W	동작 없음	승패 맵 토글 (파랑 = img2 우세, 주황 = img3 우세)
,	A/B 블링크 비교기	3-way 블링크 (orig → img2 → img3)
;/ a	활성 패널의 시퀀스만 이동	원본 기준 이동 + img2/img3 가 파일명으로 따름

모드와 무관하게 같은 동작을 하는 키도 함께 정리합니다. **B** 는 두 모드 모두 패널 테두리 토글이고, **Ctrl+S** 는 두 모드 모두 통계 창, **Shift+Ctrl+S** 는 저장 창입니다. 즉 **S** 계열에서 헛갈리는 것은 **수식 키 없는 S** 하나뿐입니다.

주의 `--comp` 모드에서는 **S** 가 패널 교환에 지정되므로 **키보드로 sync 를 토글할 수 없습니다**. 필요하면 **U** 로 UI 오버레이를 켜고 View 메뉴의 “Sync Viewports” 체크박스를 켜십시오.

10.3.1. `--comp` 가 매 프레임 강제로 끄는 상태

`--comp` 는 3영상 레이아웃을 지키기 위해 매 프레임 아래 상태를 초기화합니다. 키를 눌러 커더라도 다음 프레임에 곧바로 꺼지므로 “눌렀는데 아무 일도 안 일어난다”로 보입니다. 정상 동작입니다.

강제 해제되는 것	관련 키
Diff 모드 전부	Ctrl+0 ~ Ctrl+7 , Ctrl+9
A/B 스왑	Shift+Space
A/B 블링크	, — 대신 3-way 블링크로 재정의됨
Overlay/Blend	0

10.4. 문제 해결 Q&A

10.4.1. Q1. OpenGL 초기화에 실패하거나 원격 화면에서 창이 안 뜹니다

av 는 OpenGL 3.3 Core 를 먼저 시도하고, 실패하면 스스로 SDL 소프트웨어 렌더러로 내려갑니다. 폴백은 세 단계 모두에서 일어나며 각각 표준 오류에 한 줄씩 남습니다.

```
[av] GL window creation failed: ... - falling back to software renderer
[av] GL context creation failed: ... - falling back to software renderer
[av] gladLoadGL failed - falling back to software renderer
```

성공하면 `OpenGL 4.6 ... / Renderer: ...` 가, 소프트웨어 모드면 `Using SDL software renderer (no OpenGL)` 가 출력됩니다. 폴백조차 실패하면 종료 코드 **2** 로 끝납니다.

처음부터 소프트웨어 렌더러를 쓰려면 `--software` 를 붙입니다.

```
av --software golden.png hw.png
```

환경변수 `SSH_CONNECTION` 이 설정돼 있으면 (즉 SSH X11 forwarding 이면) `--software` 없이도 자동으로 소프트웨어 모드로 전환하고 다음을 출력합니다.

```
[av] X11 forwarding detected (SSH) - using software renderer
```

참고 소프트웨어 모드에서는 Linux X11 오류 핸들러도 함께 설치되어 MIT-SHM/GLX 오류를 무시합니다. 원격 화면에서 자주 보이던 X 오류 종료가 이 경로로 방지됩니다.

10.4.2. Q2. SSH 로 접속한 서버라 GUI 를 못 씁니다. 수치만 뽑고 싶습니다

창을 아예 열지 않는 헤드리스 모드를 쓰십시오. `--metrics` 는 PSNR/SSIM/FLIP/MSE/MAE 를 CSV 로 출력하고 종료합니다.

```
av --metrics golden.png hw.png
av --metrics --pair --format json /work/golden/f0001.png /work/hw_out
```

블록 단위 비교, diff PNG 내보내기, 균일도, CI 게이트까지 모두 헤드리스로 가능합니다. 자세한 사용법과 종료 코드 규약은 8장을 참조하십시오.

10.4.3. Q3. 두 영상 크기가 달라 비교가 안 됩니다 (종료 코드 5)

헤드리스 비교는 크기.포맷이 다르면 계산을 포기하고 종료 코드 5 를 돌려줍니다.

```
av --metrics 1920x1080.png 1280x720.png ; echo $?
# [metrics] size/format mismatch
# 5
```

`--diff-out` 은 `[diff-out] size mismatch: 1920x1080 vs 1280x720 ,` `--comp-out` 은 `[comp] size/format mismatch vs orig (img2:BAD img3:ok)` 를 남기고 역시 5 입니다. “포맷 불일치”에는 한쪽만 HDR/float 인 경우도 포함됩니다.

GUI 에서는 다르게 동작합니다. Image Info 창의 PSNR 은 `N/A (size/format mismatch)` 로 정직하게 표시하지만, diff 캔버스와 통계 창은 두 영상을 **선형 인덱스 기준으로 겹쳐** 계산하므로 숫자가 나오긴 합니다. 그 숫자는 의미가 없습니다.

주의 크기가 다르면 먼저 크롭/리사이즈로 해상도를 맞추십시오. DDI 검증에서는 보상 IP 가 패널 해상도 그대로 출력했는지부터 확인하는 편이 빠릅니다. `--pair` 로 시퀀스를 돌릴 때 특정 프레임만 크기가 다르면 그 프레임의 행만 `mismatch` 상태로 표시되고 나머지는 정상 집계됩니다.

10.4.4. Q4. HDR 이미지에 NaN 이나 Inf 가 섞인 것 같습니다

GUI 에서는 `/` 를 눌러 불량 픽셀 오버레이를 켵니다. NaN/Inf/음수/1.0 초과 픽셀이 표시됩니다 (float 이미지에만 의미가 있습니다).

배치로 확인하려면 `--validate` 를 씁니다. 클래스별 개수를 CSV 로 내고, 처음 발견한 20개의 좌표를 주석으로 덧붙입니다.

```
av --validate hdr_out.hdr
# class,count
# nan,3
```

```
# inf,0
# negative,128
# superwhite,4096
# # first offenders: class,x,y,channel
# # nan,512,300,R
```

NaN 또는 Inf 가 하나라도 있으면 종료 코드 8 로 끝나므로 CI 게이트로 바로 쓸 수 있습니다. 음수나 1.0 초과는 정상 HDR 에서도 나타날 수 있어 종료 코드를 바꾸지 않습니다. 8비트 이미지를 넣으면 구조적으로 불가능하므로 전부 0 을 출력하고 8-bit image (no float data) - nothing to validate 를 알린 뒤 0 으로 끝냅니다.

10.4.5. Q5. 16비트 PPM 의 픽셀 값이 이상하게 보입니다

av 는 PPM(P2/P3/P5/P6)을 읽을 때 원본 정수값을 `pixels_orig` 에 따로 보관하고, 화면 표시용 8비트 버퍼와 정밀 연산용 float 버퍼를 추가로 만듭니다. 픽셀 값 풍선과 diff 리스팅은 **원본 정수값**을 우선 사용하므로, 10비트 PPM 이면 0 ~ 1023 범위의 값이 그대로 나옵니다. 우선순위 규칙은 3장에서 자세히 설명합니다.

값이 0 ~ 255 로 눌러 보인다면 대개 다음 중 하나입니다.

- 파일에 `maxval` 이 255 로 적혀 있다 — 헤더가 8비트로 저장된 것입니다.
- 소프트웨어 렌더러로 실행 중이다 — **화면**은 8비트 텍스처로 그려집니다. 값 풍선은 여전히 원본 정수값을 보여 주므로 숫자를 믿으십시오.
- 확장자가 `.ppm` / `.pgm` 이 아니라 다른 형식이다 — PPM 경로를 타지 않으면 `pixels_orig` 가 만들어지지 않습니다.

주의 `--no-color-mgmt` 와 `--profile` 은 CLI 파서가 받아들이기는 하지만, 현재 빌드의 표시 파이프라인에서 이 값을 소비하는 코드가 없습니다. 즉 두 옵션을 붙여도 화면 결과는 바뀌지 않습니다. 밝기, 색이 예상과 다르다면 LUT/CDL 적용 여부()와 채널 모드 (Shift+C 로 RGB 복귀)를 먼저 확인하십시오. 9장을 함께 참조하십시오.

10.4.6. Q6. --comp 인데 diff 키가 안 먹습니다

정상입니다. `--comp` 는 인자 파싱 단계에서 diff 모드를 `none` 으로 되돌리고, 실행 중 매 프레임 다시 `none` 으로 강제합니다. 3영상 레이아웃(`orig | img2 | img3`)에는 diff 패널이 들어갈 자리가 없기 때문입니다. 두 영상 diff 가 필요하다면 `--comp` 없이 두 장만 여십시오.

```
av --comp golden.png fw.png hw.png # diff 불가, 블록 비교 전용
av -d golden.png hw.png # 두 장 diff
```

주의 세 번째 영상 로드 실패하면 3패널 레이아웃 조건이 깨져 화면은 2패널로 떨어지지만 `--comp` 상태는 그대로라 diff 는 여전히 강제 해제됩니다. “2패널인데 diff 도 안 되는” 상태가 보이면 시작 로 그에서 `Failed to load image C:` 를 확인하십시오.

10.4.7. Q7. --comp 인데 블록 박스가 안 보입니다

다음 순서로 확인하십시오.

확인	내용
토글 상태	<code>Ctrl+B</code> 로 worst 박스를 꺾을 수 있습니다. 기본값은 켜짐입니다

패널	worst 박스와 상세 풍선은 비교 패널(img2/img3) 전용입니다. 왼쪽 원본 패널에는 박스가 그려지지 않습니다
개수	<code>--num_blk</code> 가 작으면 그만큼만 그림니다 (기본 <code>16</code>). <code>--num_blk 64</code> 로 늘려 보십시오
줌	fit 줌에서 <code>8x8</code> 블록은 화면에서 몇 픽셀에 불과해 2px 두께 사각형끼리 겹쳐 뭉개져 보입니다. <code>1</code> ~ <code>8</code> 로 확대하거나, <code>Tab</code> 으로 순회하면 뷰포트가 해당 블록으로 자동 이동합니다
그리드	<code>G</code> 그리드 선은 셀이 화면에서 4px 미만 이면 아예 그리지 않습니다. fit 줌에서 <code>--grid 16x16</code> 이 안 보이는 것은 이 규칙 때문입니다
통계	영상을 바꾸거나 <code>s</code> 로 교환한 직후에는 블록 통계가 무효화되고 다음 프레임에 다시 계산됩니다

블록 번호 라벨(`#1`, `#3P` 등)은 박스가 화면에서 가로 22px, 세로 14px 을 넘을 때만 표시됩니다. 마젠타 색과 `P` 접미사는 피크 픽셀 오차로 뽑힌 블록입니다 (5장 참조).

10.4.8. Q8. 키를 눌렀는데 아무 반응이 없습니다

원인은 대개 셋 중 하나입니다.

원인	확인과 대처
모드별 재정의	이 장의 대조표를 확인하십시오. <code>s</code> <code>G</code> <code>Tab</code> <code>[</code> <code>]</code> <code>Ctrl+D</code> 는 <code>--comp</code> 에서 뜻이 다릅니다
ImGui 입력 포커스	히스토그램·저장 창 입력란에 커서가 있으면 수식키 없는 평문 키가 그 창으로 먹힙니다. 패널을 한 번 클릭하거나 <code>Esc</code> 로 창을 닫으십시오. <code>Ctrl</code> / <code>Cmd</code> 조합은 항상 통과합니다
활성 패널	<code>;</code> <code>a</code> 같은 시퀀스 키는 활성 패널 에만 적용됩니다. <code>Tab</code> 으로 바꾸고, 테두리가 붉은 쪽이 활성입니다 (7장 참조)

또 하나 알아 둘 것은 복사 모드입니다. `Ctrl+C` 를 누르면 av 는 “어느 이미지를 복사할지” 기다리는 상태로 들어가고, 이어서 누른 키가 `1` / `2` / `3` 이면 각각 A/B/Diff 를 PNG 로 클립보드에 복사합니다. 그 외의 키를 누르면 대기 상태만 풀리고 그 키는 원래 동작을 그대로 수행하므로 키를 잃어버리지는 않습니다. `Esc` 만은 아무 동작 없이 대기 상태를 취소합니다.

10.4.9. Q9. `Ctrl+D` 를 눌러도 diff 픽셀 리스팅 창이 뜨지 않습니다

리스팅 목록은 **diff 렌더 경로에서만** 계산됩니다. diff 모드가 꺼져 있으면 목록이 계산되지 않았으므로 창은 열리자마자 스스로 닫힙니다. 두 영상이 완전히 동일할 때도 같은 이유로 닫힙니다.

```
av -d golden.png hw.png # diff 모드를 켜둔 뒤 Ctrl+D
```

채널 모드가 R/G/B 중 하나면 “그 채널만 동일”해도 창이 닫힙니다. `Shift+C` 로 RGB 로 되돌린 뒤 다시 시도하십시오.

10.4.10. Q10. 이 파일 형식은 왜 안 열립니까

시퀀스 스캔과 파일 인식에 쓰이는 확장자 목록은 다음 8종이며 대소문자는 무시합니다.

```
.png .jpg .jpeg .bmp .tga .pgm .ppm .hdr
```

`.tif` / `.tiff`, `.exr`, `.webp`, `.dng` 등은 지원하지 않습니다. PPM 계열은 P2/P3 (ASCII) 와 P5/P6 (binary) 를 자체 파서로 읽으며 `maxval` 은 `65535` 까지 허용합니다. `.hdr` 은 Radiance RGBE 로 읽어 float 버퍼를 만듭니다.

로드에 실패하면 표준 오류에 이유가 남고, **기존 이미지는 그대로 유지**됩니다.

```
[image_loader] failed to load: shot.tiff - unknown image type
```

팁 DDI 검증 덤프가 TIFF 나 16비트 PNG 로 나온다면, 전처리로 PPM(P6, `maxval 1023`)이나 PNG 로 변환해 두는 편이 av 의 원본 정수값 표시와 잘 맞습니다.

10.4.11. Q11. 한글 파일명이 깨지고 글자가 작게 나옵니다

av 의 UI 폰트는 ImGui 기본 폰트(ProggyClean 13px)와 Roboto-Medium(26px / 18px) 두 가지뿐이며 **CJK 글리프를 포함하지 않습니다**. 따라서 파일명 토스트나 경로에 한글이 들어가면 해당 글자는 표시되지 않습니다. 영문·숫자 파일명을 쓰는 것이 유일한 해결책입니다 (파일을 읽고 비교하는 동작 자체는 정상입니다).

Roboto 폰트 경로는 **빌드 시점의 절대 경로**로 박힙니다. 빌드 트리를 지우거나 다른 장비로 실행 파일만 복사하면 폰트 로드가 실패하고 기본 폰트로 대체되어, Image Info 창과 픽셀 값 풍선의 글자가 갑자기 작아집니다. 기능에는 영향이 없습니다. 큰 글자가 필요하면 빌드 트리를 유지한 채 설치하십시오.

10.4.12. Q12. 설정이 저장되지 않습니다

증상	원인
강제 종료 후 초기화	<code>kill -9</code> , 크래시, 로그아웃 시 저장 루틴이 실행되지 않습니다. <code>Q</code> 나 창 닫기로 종료하십시오
폴더를 바꾸면 초기화	<code>HOME</code> 미설정 (주로 Windows cmd/PowerShell). Q1 위쪽의 경로 표를 참조하십시오
아무리 해도 안 바뀜	<code>.av.ini</code> 가 읽기 전용이거나 홈 디렉토리에 쓰기 권한이 없습니다. 저장 실패는 오류 메시지 없이 조용히 무시됩니다
sync/창 모드만 안 남음	“저장은 되지만 반영되지 않는 두 키” 절 참조 — 설계상 매번 CLI 기본값으로 시작합니다
여러 창을 띄웠을 때	마지막에 종료한 인스턴스가 파일을 덮어씁니다

10.5. 성능 팁

10.5.1. 큰 이미지와 시퀀스

av 는 디코드한 이미지를 LRU 캐시에 최대 **8장**까지 보관하고, 파일의 수정 시각과 크기로 유효성을 검사합니다. 덕분에 `;` / `a` 로 앞뒤를 오갈 때 최근 8장은 디스크 재디코드 없이 즉시 뜹니다. 반대로 말하면 메모리도 8장분을 차지합니다.

입력	이미지 1장당 CPU 버퍼	설명
8비트 PNG/JPG	약 4 bytes/px	RGBA8 한 벌
10~16비트 PPM	약 26 bytes/px	RGBA8 + 원본 uint16 RGB + RGBA32F
<code>.hdr</code> (float)	약 16 bytes/px	RGBA32F (소프트웨어 모드에서는 RGBA8 도 추가)

4K(3840×2160) 10비트 PPM 한 장이 대략 200MB 이므로, 캐시가 가득 차면 1.5GB 이상을 쓸 수 있습니다. GPU 텍스처도 캐시 항목마다 하나씩 잡힙니다. 메모리가 빠듯한 장비에서 대용량 시퀀스를 돌릴 때는 GUI 대신 헤드리스 `--metrics --pair` 나 `--comp-batch` 를 쓰는 편이 훨씬 가볍습니다 (8장 참조).

팁 프레임을 넘기면서 같은 영역을 확대해 보려면 `--zoom` 으로 배율을 고정하십시오. 시퀀스 로드마다 `fit` 으로 되돌아가지 않고 지정 배율 + 중앙 정렬을 유지합니다.

10.5.2. 무거운 계산이 언제 일어나는가

작업	시점과 특성
SSIM / FLIP 히트맵	모드 진입이나 영상 변경 시 워커 스레드 에서 계산. 상태바에 <code>SSIM: computing..</code> 이 뜨는 동안에도 화면은 계속 반응합니다

<code>--comp</code> 블록 스캔	영상 로드/교환 후 첫 프레임에 1회. 이후 캐시되어 토글은 즉시 반응합니다
diff 픽셀 리스팅	diff 렌더 중 지연 계산. 영상 크기와 콘텐츠 버전이 같으면 재사용
창 드래그	창을 옮기는 동안 렌더를 건너뛵니다. 이동이 멈추고 100ms 뒤 자동 재개

10.5.3. 소프트웨어 렌더러 사용 시 제약

`--software` 또는 SSH 자동 감지로 소프트웨어 모드가 되면 다음이 달라집니다.

항목	소프트웨어 모드에서의 동작
리샘플링	매 프레임 CPU 로 패널 크기만큼 다시 그립니다. 창이 클수록 느려집니다
LUT / CDL	출력 픽셀마다 CPU 로 적용. GPU 3D 텍스처 경로를 쓰지 않습니다
HDR 표시	float 값을 0 ~ 1 로 클램프한 8비트 텍스처로 표시. 값 읽기는 원본 유지
16비트 PPM 표시	8비트 텍스처로 표시. 값 읽기는 원본 정수값 유지
diff / overlay / 채널	CPU 경로로 모두 지원 — 기능이 빠지지 않습니다

팁 원격 화면에서 반응이 느리면 전체화면 대신 창을 작게 띄우십시오. 기본은 테두리 없는 최대화 창이라 화면 전체를 CPU 로 그립니다.

```
av --software --windowed --geometry 1024x640 golden.png hw.png
```

11. 부록 — 전체 레퍼런스

이 장은 설명이 아니라 **찾아보기** 를 위한 장입니다. 앱 안의 핫키 창 (`Ctrl+Shift+H`)과 `av --help` 에 실려 있는 항목을 하나도 빠뜨리지 않고 표로 옮겼고, 헤더리스 출력의 열 이름, 종료 코드, 지원 포맷, 용어 정의를 덧붙였습니다. 각 항목의 배경과 실전 활용은 표의 “관련 장” 열이 가리키는 장을 보십시오.

11.1. 키보드 단축키 전수표

아래 표는 `src/ui/main_window.cpp` 의 `render_hotkey_help_window()` 가 앱 안에서 보여 주는 목록과 **1:1 로 대응** 합니다. 카테고리 이름도 앱 창의 `Category` 열과 같습니다. 비교 열에는 소스(`src/app.cpp` 의 스캔코드 처리)에서 확인한 모드 제약과 주의점을 적었습니다.

참고 표기 규칙 두 가지입니다. 첫째, `Cmd` 는 SDL 의 GUI 수식키로, macOS 에서는 `Command`, Windows/Linux 에서는 `Super(Win)` 키입니다. 둘째, 텍스트 입력 위젯에 포커스가 있을 때는 문자 키가 그 위젯으로 먼저 들어갑니다 (복사 모드 진입은 `WantTextInput` 을 확인해 이 경우에만 막습니다).

11.1.1. Navigation — 이동

키	동작	비고
<code>H</code> / <code>Left</code>	왼쪽으로 1픽셀 팬	이미지 좌표 기준. 줌 배율과 무관하게 항상 1픽셀 (2장)
<code>L</code> / <code>Right</code>	오른쪽으로 1픽셀 팬	<code>sync</code> 가 켜져 있으면 A/B 가 함께 움직입니다
<code>K</code> / <code>Up</code>	위로 1픽셀 팬	—
<code>J</code> / <code>Down</code>	아래로 1픽셀 팬	—
<code>Shift</code> + <code>H</code> <code>L</code> <code>K</code> <code>J</code> / 방향키	빠른 팬 (<code>pan_step</code> 픽셀)	기본 32픽셀. <code>-p</code> / <code>--pan-step</code> 으로 변경. <code>Shift+Up</code> / <code>Shift+Down</code> 은 슬라이드쇼가 켜져 있으면 간격 조절로 가로칩니다
<code>Cmd+Shift</code> + <code>H</code> <code>L</code> <code>K</code> <code>J</code>	해당 방향 끝단으로 점프	<code>fit</code> 이 자동으로 해제됩니다. Windows/Linux 에서는 <code>Super(Win)</code> 키
<code>G</code>	이미지 중심으로	<code>--comp</code> 모드에서는 <code>G</code> 가 블록 그리드 토글 로 바뀝니다 (5장)
마우스 좌드래그	이미지 팬	ROI 모드(<code>Ctrl+E</code>) 와 Curtain 모드에서는 드래그가 그쪽으로 소비됩니다

11.1.2. Zoom — 줌

키	동작	비고
<code>+</code> / <code>=</code> / <code>Numpad+</code>	줌 인 (2배)	줌 단계 목록에서 한 칸 위 (2장)
<code>-</code> / <code>Numpad-</code>	줌 아웃 (1/2배)	—
<code>Z</code>	줌 인	왼손 조작용 대체 키
<code>Shift+Z</code>	줌 아웃	<code>X</code> 도 같은 동작을 합니다 (핫키 창 목록에는 없음)
<code>O</code>	핫키 창 표기: Fit to window	실제 동작은 $2^{-0} = 1$, 즉 1:1(100%) + 중앙 정렬 입니다. <code>fit</code> 이 필요하면 <code>F</code> 를 쓰십시오 (2장)
<code>1</code> - <code>8</code>	2^{-n} 배율 (1=2배 ... 8=256배)	항상 중앙 정렬이 함께 적용됩니다. <code>Ctrl</code> 을 같이 누르면 diff 모드 전환이 됩니다
<code>F</code>	fit 토글	끄면 직전 배율로 복귀

<code>Space</code>	1:1 줌 + 중앙 정렬	<code>Shift+Space</code> 는 A/B 스왑
마우스 휠	줌 인/아웃	<code>Alt</code> 를 같이 누르면 시퀀스 이동으로 바뀝니다
마우스 우드래그	선택 영역으로 줌	드래그한 사각형이 패널을 채우도록 배율·팬을 계산합니다

11.1.3. Display — 표시

키	동작	비고
<code>U</code>	UI 오버레이(메뉴바/상태바) 토글	기본값 off. <code>~/av.ini</code> 에 저장됩니다
<code>I</code>	Image Info 창 토글	<code>--comp</code> 에서는 img2·img3 의 원본 대비 PSNR 이 함께 표시됩니다
<code>V</code>	커서 픽셀값 풍선말 토글	3장
<code>S</code>	뷰포트 sync 토글 (A ↔ B)	<code>--comp</code> 모드에서는 img2 ↔ img3 패널 교환 으로 바뀝니다
<code>W</code>	윈도우 모드 토글 (타이틀바)	끄면 borderless + maximized. <code>--windowed</code> 로 시작 상태 지정
<code>Tab</code>	활성 패널 전환 (A ↔ B)	<code>--comp</code> 에서는 worst 블록 순회로 바뀝니다 (5장)
<code>R</code>	이미지 시계방향 90도 회전	두 패널 모두 회전하고 뷰포트는 fit + 중앙으로 초기화
<code>Ctrl+R</code>	이미지 반시계방향 90도 회전	<code>Shift+R</code> 은 Red 채널이므로 혼동 주의
<code>Shift+Space</code>	A/B 영상 스왑	두 영상이 모두 로드된 경우에만 동작
<code>M</code>	크로스헤어 오버레이 토글	<code>~/av.ini</code> 에 저장
<code>Ctrl+M</code>	돋보기 토글	diff 모드가 아닐 때만. Enhance 모드에는 자체 돋보기가 있습니다 (4장)
<code>P</code>	Pathfinder: 이미지 미니맵	fit 상태에서는 표시되지 않습니다
<code>Ctrl+P</code>	Pathfinder: 개략도(schematic) 모드	같은 키를 다시 누르면 숨김
<code>B</code>	패널 테두리 토글	<code>-nb</code> 로 숨긴 채 시작 가능. <code>--comp</code> 에서 <code>Ctrl+B</code> 는 worst 박스
<code>/</code>	불량 픽셀 오버레이 토글	float/HDR 영상의 NaN·Inf·음수 <code>>1</code> 표시. 헤드리스 대응은 <code>--validate</code>
<code>Ctrl+X</code>	픽셀값 표기 형식 순환	Decimal → <code>0x80</code> → <code>80h</code> . <code>X</code> 단독은 줌 아웃 (3장)

11.1.4. Channel — 채널

키	동작	비고
<code>Shift+R</code>	Red 채널만 표시	채널 모드는 diff 픽셀 리스트의 필터로도 쓰입니다 (4장)
<code>Shift+G</code>	Green 채널만 표시	—
<code>Shift+B</code>	Blue 채널만 표시	—
<code>Shift+Y</code>	Rec.709 루마(Y') 그레이스케일 토글	다시 누르면 RGB 로 복귀 (토글형)
<code>Shift+C</code>	RGB 표시 (기본값)	채널 모드는 <code>~/av.ini</code> 에 저장됩니다

11.1.5. Analysis — 분석

키	동작	비고
<code>Ctrl+H</code>	히스토그램 창 토글	열면 라인컷·통계 창은 자동으로 닫힙니다 (배타적) — 6장
<code>Ctrl+L</code>	H-Line Cut 토글	커서가 놓인 행의 수평 프로파일
<code>Ctrl+Y</code>	V-Line Cut 토글	커서가 놓인 열의 수직 프로파일

Ctrl+S	통계 창 토글	Shift+Ctrl+S 는 저장 다이얼로그이므로 구분 주의
Ctrl+E	ROI 선택 모드 토글 (좌드래그로 선택)	모드를 끄면 선택 영역도 함께 초기화됩니다
Shift+V	컬러메트리 프로브 (xy / u'v' / CCT) 를 풍선말에 추가	켜면 픽셀 풍선말(V)도 자동으로 켜집니다 (3장, 9장)
Ctrl+T	산점도(Scatter Plot) 토글	A 픽셀값 대 B 픽셀값의 상관 산점도

11.1.6. Overlay — 오버레이

키	동작	비고
0	Overlay/Blend 비교 모드 토글	두 영상을 한 캔버스에 겹칩니다. <code>~/av.ini</code> 에 저장
Curtain 모드	좌드래그로 분할선 이동	모드 선택은 메뉴 View → Overlay → Blend mode / Curtain mode . 분할선 기본 위치는 0.5

11.1.7. Sequence — 시퀀스

키	동작	비고
;	디렉토리 시퀀스 다음 영상 (활성 패널)	<code>--pair</code> / <code>--comp</code> 에서는 상대 패널도 같은 파일명으로 따릅니다 (7장)
A	디렉토리 시퀀스 이전 영상 (활성 패널)	Ctrl · Cmd 조합에서는 동작하지 않습니다
N / Shift+N	다음 / 이전 영상 (구형 핫키)	; / A 와 완전히 동일한 동작
Tab	활성 패널 전환 (A ↔ B)	Display 카테고리과 같은 키. <code>--comp</code> 에서는 블록 순회
Alt + 마우스 휠	커서 아래 패널의 영상 이동	활성 패널이 아니라 커서가 놓인 패널이 대상입니다
Shift+A	슬라이드쇼 자동 재생 토글	시작할 때의 활성 패널이 대상 패널로 고정됩니다
Shift+Up	슬라이드쇼 간격 +1초	최대 10초. 슬라이드쇼가 꺼져 있으면 팬으로 동작
Shift+Down	슬라이드쇼 간격 -1초	최소 0.5초

11.1.8. Blink — 블링크 비교기

키	동작	비고
,	블링크 비교기 토글 (A/B 자동 교대)	두 영상이 모두 로드된 경우에만. 켜는 순간 두 뷰포트가 활성 패널 값으로 정렬되고 sync 가 강제로 켜집니다 (끄면 원래 sync 로 복원)
< / > (Shift+, / Shift+.)	블링크 간격 -/+0.1초	범위 0.1 – 2.0초. 블링크(또는 <code>--comp</code> 3-way 블링크)가 켜져 있을 때만 반응

11.1.9. Diff — 차이 비교

키	동작	비고
Ctrl+D	diff 픽셀 리스트 창 토글	<code>--comp</code> 모드에서는 worst 블록 리스트 창 으로 바뀝니다
Ctrl+2	Diff: Alpha Blend 토글	B 가 없으면 경고 토스트가 뜹니다
Ctrl+3	Diff: Pixel Absolute 토글	<code>-d</code> / <code>--diff</code> 로 시작 시 이 모드
Ctrl+4	Diff: Pixel Relative 토글	—

Ctrl+5	Diff: Enhance 토글 (remap + 돋보기)	—
Ctrl+6	Diff: False Color 토글	—
Ctrl+7	Diff: SSIM 토글	—
Ctrl+0	Diff: FLIP 토글 (지각 오차맵, magma)	—
Ctrl+1	Diff: Signed 토글 (파랑=A<B, 흰색=동일, 빨강=A>B)	—
Ctrl+9	Diff: Highlight 토글 (빨강)	—
[/]	주 diff 파라미터 -/+	일반 모드: amplify ±0.5 (범위 0.5 – 100). AlphaBlend: alpha ±1%
Shift+[/ Shift+]	보조 파라미터 -/+	일반 모드: threshold ±1 (범위 0 – 255). AlphaBlend: alpha ±10%
\	주 diff 파라미터 리셋	amplify = 1.0, AlphaBlend는 alpha = 50%
Ctrl+8	허용오차(tolerance) 기반 diff 토글	threshold 를 켜고 끄는 스위치
Ctrl+\	diff threshold 리셋 (0)	—

주의 `--comp` 모드에서는 diff 엔진이 통째로 비활성화됩니다. 위 Diff 키 중 **Ctrl+D** · **[/]** · **Shift+[/ Shift+]** 는 모두 comp 전용 동작으로 재배치됩니다. 4장과 5장을 함께 보십시오.

11.1.10. Comp — 3영상 블록 비교 (`--comp`)

키 / 옵션	동작	비고
<code>av --comp <orig> <img2> <img3></code>	3영상 비교: orig img2 img3 (diff 비활성)	핫키 창에도 CLI 형태로 실행 중인 항목입니다
<code>--blk <WxH></code>	worst-PSNR 탐색 블록 크기	기본 8x8 . 범위 2 – 1024. 8 처럼 한 값만 주면 8x8 로 해석
<code>--num_blk <N></code>	윤곽선을 그릴 worst 블록 개수	기본 16. 최소 1 로 클램프. <code>--num_blk</code> 도 같은 옵션
Ctrl+B	worst 블록 박스 토글	빨강=worst → 노랑 순. 마젠타 P 는 픽 픽셀 스파이크 픽
G	전체 블록 그리드 경계선 토글	셀 크기는 <code>--grid</code> 값 (기본 16x16), <code>--blk</code> 와 독립
Ctrl+W	승패(win/loss) 맵	파랑=img2 우세, 주황=img3 우세. 판정 기준 d > 1dB
Ctrl+G	블록 PSNR 히트맵 (img2/img3)	노랑 → 빨강 = 나쁨. 50dB 이상은 투명
Ctrl+D	worst 블록 리스트 창	행을 클릭하면 세 패널이 동시에 그 블록으로 점프
S	img2 ↔ img3 패널 교환	통계도 함께 따라갑니다 (슬롯 교환 후 재계산)

Tab / Shift+Tab	마우스가 놓인 패널의 worst 블록 순회	패널 밖이면 img2+img3 병합 심각도 순
[/]	img2(가운데) 의 worst 블록만 이전/다음	마우스 위치와 무관하게 대상 고정
Shift+[/ Shift+]	img3(오른쪽) 의 worst 블록만 이전/다음	—
,	3-way 블링크: 한 패널에서 orig → img2 → img3	간격 조절은 < / >
; / A	orig 디렉토리의 다음/이전 영상	img2·img3 는 같은 파일명으로 미러링
블록 박스 위에 마우스 올리기	상세 풍선말	블록 PSNR/MSE, 상대 영상의 같은 블록 PSNR, 채널별 MSE, 최악 픽셀의 orig/현재/차이. 반대편 비교 패널에는 그 패널 자신의 통계 로 같은 형식의 풍선말이 뜨고, orig 패널에는 박스만 표시
I	Info 창: img2 와 img3 의 원본 대비 PSNR	—

11.1.11. File / Copy / Help

키	동작	비고
Shift+Ctrl+O	이미지 파일 열기	활성 패널에 로드. macOS 는 Shift+Cmd+O 도 동작
Shift+Ctrl+S	저장 다이얼로그	열려 있는 분석 창에 따라 이미지/차트/통계 저장 창이 선택됩니다 (7장)
Q	종료	확인 창 없이 즉시. 종료 직전에 <code>~/av.ini</code> 가 기록됩니다
Ctrl/Cmd+C → 1	Image A 를 PNG 로 클립보드 복사	복사 모드는 5초 후 자동 취소. Esc 로도 취소
Ctrl/Cmd+C → 2	Image B 를 PNG 로 클립보드 복사	숫자 키패드 1 / 2 / 3 도 인식
Ctrl/Cmd+C → 3	Diff 영상을 PNG 로 클립보드 복사	현재 diff 모드의 결과 이미지
Ctrl+Shift+H	이 핫키 레퍼런스 창 토글	열림 상태가 <code>~/av.ini</code> 에 저장되므로 다음 실행에도 유지됩니다

11.1.12. 핫키 창 목록에는 없지만 실제로 동작하는 키

소스의 스캔코드 처리에는 있으나 핫키 창 `entries[]` 에는 실려 있지 않은 키입니다.

키	동작	비고
Esc	열려 있는 것을 하나씩 닫기	블링크 → diff 리스트 → 핫키 창 → 히스토폴그램/라인컷/통계 → ROI 통계/산점도 → Info/풍선말 → 저장창/크로스헤어 → ROI 모드/Overlay 순 (1장)
X	줌 아웃	Z 와 짝. Ctrl+X 는 픽셀값 형식 순환
'	3D LUT / CDL 룩 표시 토글	<code>--lut</code> 또는 <code>--cdl</code> 로 파일이 로드된 경우에만 반응. <code>av --help</code> 의 <code>--lut</code> 설명에만 적혀 있습니다 (9장)
Numpad 1 Numpad 2 Numpad 3	복사 모드에서 A / B / Diff 선택	Ctrl/Cmd+C 로 복사 모드에 들어간 뒤 숫자열 키와 동일하게 동작

11.2. CLI 옵션 전수표

기본 문법입니다. 옵션과 이미지 경로의 순서는 자유이며, 위치 인자는 등장 순서대로 imageA → imageB → imageC 로 지정됩니다 (최대 3개, 초과 시 오류).

```
av [options] [imageA] [imageB]
av --comp [options] <orig> <img2> <img3>
```

11.2.1. GUI 옵션 — 창과 표시

옵션	인자	기본값	설명	장
<code>--fullscreen</code>	—	off	전체 화면으로 시작	1장
<code>--windowed</code>	—	off	타이틀바 + 크기 조절 가능한 창으로 시작	1장
<code>--geometry</code>	<WxH>	1280x720	초기 창 크기	1장
<code>--software</code>	—	off	SDL 소프트웨어 렌더러 강제 (OpenGL 미사용)	10장
<code>-nb</code>	—	off	패널 테두리를 숨긴 채 시작	1장
<code>-bc</code>	<A> <D>	ff00ff ffff00 00ffff	A/B/Diff 패널 테두리 색 (6자리 hex, 알파 230 고정)	1장
<code>--zoom</code>	fit 0 1 2 ...	.av.ini 저장값 (없으면 fit)	초기 배율. 0 / fit = 창 맞춤, 1 = 1:1. 지정 시 저장값을 덮어쓰고 이후 실행에도 유지	2장
<code>--sync</code>	—	on	A/B 뷰포트 동기화 켜기	2장
<code>--no-sync</code>	—	—	A/B 뷰포트 동기화 끄기	2장
<code>-p</code> , <code>--pan-step</code>	<N>	32	<code>Shift</code> + hjkl 의 점프 크기 (이미지 픽셀)	2장

11.2.2. GUI 옵션 — 비교와 색

옵션	인자	기본값	설명	장
<code>--diff-mode</code>	none alphablend abs rel highlight falsecolor ssim flip signed enhance	none	시작 시 diff 모드. 목록에 없는 값은 오류	4장
<code>-d</code> , <code>--diff</code>	—	—	<code>--diff-mode abs</code> 의 단	4장

				추 형	
<code>--amplify</code>	<code><val></code>		<code>1.0</code>	diff 증 폭. 도 움 말 표 기 범 위 0.1 - 100 (GUI 의 <code>[</code> / <code>]</code> 조 절 범 위 는 0.5 - 100)	4장
<code>--pair</code>	—		off	image 디 렉 토 리 에 서 같 은 파 일 명 을 짜 지 어 시 퀀 스 를 연 동. 두 디렉토리	3장

					가 서 로 달 라 야 합 니 다
--comp	—		off	3	5장 영 상 블 록 비 교. 이 미 지 3 장 필 수, diff 는 강 제 로
				none	세 번 째 위 치 인 자 를 주 면 자 동 활 성 화
--blk	<WxH> 또는 <N>		8x8	comp	5장 블 록 크 기. 각 변 2 — 1024

<code>--num_blk</code>	<code><N></code>	<code>16</code>	운 곽 선 을 그 릴 worst 블 록 개 수 (최 소 1). <code>-- num- blk</code> 별 칭	5장
<code>--blk-metric</code>	<code>rgb y chroma</code>	<code>rgb</code>	worst 5장 랭 킹 기 준. 리 스 트 는 지 표 MSE 75% + 피 크 픽 셀 25%	
<code>--grid</code>	<code>[WxH N]</code> (선택)	off / 크기 <code>16x16</code>	시 작 시 블 록 그 리 드 표 시. 값 은 2 - 1024 범	5장

			위 의 숫 자 형 태 일 때 만 소 비 (파 일 명 오 인 방 지)
--profile	<file>	—	ICC 9장 색 프로 파일 경로
--no-color-mgmt	—	off	색 9장 관 리 비 활 성 화
--lut	<file.cube>	—	3D/1D 9장 .cube LUT 를 두 패 널 표 시 에 적 용 ( 토 글)
--cdl	<file.cdl>	—	ASC- 9장 CDL( .cdl slope/ offset/

				power + sat 록. -- lut 가 있 으 면 -- lut 우 선	
--version	—			버 전 과 갱 신 날 짜 를 출 력 하 고 종 료 (exit 0)	1장
-h , --help	—			도 움 말 을 출 력 하 고 종 료 (exit 0)	1장

11.2.3. 헤드리스 옵션

아래 옵션이 하나라도 있으면 창·OpenGL·SDL 을 열지 않고 계산 후 즉시 종료합니다. 여러 개를 동시에 주면 --batch → --comp-batch → --comp-out → --metrics → --diff-out → --validate → --probe → --uniformity 순으로 먼저 걸린 하나만 실행됩니다.

옵션	인자	기본값	설명	장
--metrics	—	off	A 대 B 의 PSNR/SSIM/FLIP/MSE/MAE 를 16열 CSV 로 출력. --pair 와 함께 쓰면 시퀀스 전체를 프레임당 한 줄로	8장

<code>--batch</code>	<code><file -></code>	—	매니페스트의 임의 A,B 쌍을 지표 계산. TAB 구분 <code>A<TAB>B[<TAB>label]</code> , # 주석, - = stdin. 같은 파일명 제약 없음	8장
<code>--format</code>	<code>csv json junit</code>	<code>csv</code>	<code>--metrics</code> / <code>--batch</code> 출력 형식. <code>--comp-out</code> 은 <code>csv json</code> 만 (<code>junit</code> 지정 시 <code>csv</code> 로 대체)	8장
<code>--diff-out</code>	<code><path></code>	—	<code>--diff-mode</code> (또는 FLIP) 결과를 PNG 로 저장	8장
<code>--sbs</code>	—	<code>off</code>	<code>--diff-out</code> 을 A diff B 가로 3연결 합성으로 저장	8장
<code>--validate</code>	—	<code>off</code>	float/HDR 영상의 NaN/Inf/음수/ <code>>1</code> 픽셀 스캔 (CSV). GUI 대응은 <code>/</code>	8장
<code>--probe</code>	<code><X,Y></code>	—	픽셀 X,Y 의 컬러메트리(XYZ/xy/u'v'/CCT/Duv/L*) 를 CSV 로. B 가 있으면 B 행과 delta 행 추가	9장
<code>--uniformity</code>	—	<code>off</code>	평판 균일도: ICDM 9/13/25점 %, CV, 색 $\Delta u'v'$, SEMU 무라 프록시. A[,B] $\rightarrow$ A/B/ Δ 행	8장
<code>--comp-out</code>	<code><file -></code>	—	comp 블록별 표를 파일 또는 stdout(-) 으로. <code>--comp</code> 필수	5장, 8장
<code>--comp-batch</code>	—	<code>off</code>	세 디렉토리의 같은 이름 프레임 전체를 프레임당 한 줄로 요약. <code>--comp</code> 필수	5장, 8장

11.2.4. CI 게이트 옵션

옵션	인자	기본값	FAIL 조건
<code>--fail-psnr</code>	<code><dB></code>	미설정(-1)	<code>psnr_db</code> < 값
<code>--fail-ssim</code>	<code><v></code>	미설정	<code>ssim</code> < 값
<code>--fail-flip</code>	<code><v></code>	미설정	<code>flip</code> > 값
<code>--fail-maxerr</code>	<code><v></code>	미설정	<code>max_error</code> > 값
<code>--warn-psnr</code>	<code><dB></code>	미설정	<code>psnr_db</code> < 값 $\rightarrow$ WARN 만. 종료 코드는 0 유지
<code>--fail-uniformity</code>	<code><pct></code>	미설정	<code>uni25_pct</code> < 값
<code>--fail-semu</code>	<code><v></code>	미설정	<code>semu</code> > 값

게이트가 하나라도 설정되어 있고 FAIL 이 한 건이라도 있으면 프로세스는 **exit 10** 으로 끝납니다. 자세한 CI 연동 스크립트는 8장을 참조하십시오.

```
av --metrics --pair --fail-psnr 40 --warn-psnr 45 \
/data/ddi/orig/f0001.png /data/ddi/hw > metrics.csv
```

11.2.5. 인자 검증 규칙

규칙	내용
위치 인자	최대 3개. 4번째부터는 <code>Too many image arguments (max 3)</code> 로 exit 1
<code>--comp</code>	imageA/B/C 가 모두 있어야 합니다. 하나라도 없으면 exit 1
세 번째 위치 인자	<code>--comp</code> 를 쓰지 않아도 세 번째 경로가 주어지면 comp 모드가 자동으로 켜집니다
<code>--comp-out</code> / <code>--comp-batch</code>	<code>--comp</code> (이미지 3장) 없이 쓰면 exit 1
<code>--pair</code>	경로 2개 필요. 두 디렉토리가 같으면 “Nothing to compare” 로 exit 1
알 수 없는 -- 옵션	Unknown option: 출력 후 exit 1
값 누락	Expected value after <옵션> 출력 후 exit 1

11.3. 종료 코드

코드	의미	발생 조건
0	성공	정상 종료. 게이트가 WARN 만 낸 경우, <code>--validate</code> 가 <code>negative</code> / <code>superwhite</code> 만 찾은 경우, 8비트 영상에 <code>--validate</code> 를 건 경우 포함
1	CLI 파싱.검증 실패	알 수 없는 옵션, 잘못된 <code>--blk</code> / <code>--format</code> / <code>--blk-metric</code> / <code>--diff-mode</code> / hex 색, 값 누락, 위치 인자 4개 이상, <code>--comp</code> 이미지 부족, <code>--comp-out</code> / <code>--comp-batch</code> 를 <code>--comp</code> 없이 사용, <code>--pair</code> 두 디렉토리 동일
2	초기화 실패	<code>SDL_Init</code> / <code>SDL_CreateWindow</code> / <code>SDL_CreateRenderer</code> / ImGui 백엔드 / <code>MainWindow::init()</code> 실패. GUI 경로에서만 발생
3	헤드리스 인자 오류	이미지 인자 부족, <code>--probe</code> 좌표 형식 오류 또는 범위 밖, 디렉토리에 영상 없음, 매니페스트 열기 실패, 유효 쌍 0개, <code>--comp-out</code> 출력 파일 열기 실패, <code>--comp-batch</code> 유효 프레임 0개
4	이미지 디코드 실패	파일 없음.지원하지 않는 포맷.손상. <code>--metrics</code> / <code>--diff-out</code> / <code>--validate</code> / <code>--probe</code> / <code>--uniformity</code> / <code>--comp-out</code> 공통
5	크기/포맷 불일치	단일 쌍 <code>--metrics</code> , <code>--diff-out</code> , <code>--comp-out</code> 에서 A 와 B(또는 orig 와 img2/img3)의 해상도.자료형이 다를 때. 시퀀스 실행에서는 해당 행만 <code>mismatch</code> 로 남고 종료 코드가 되지 않습니다
6	FLIP 계산 실패	<code>--diff-out</code> <code>--diff-mode flip</code> 에서 FLIP 엔진이 실패
7	PNG 쓰기 실패	<code>--diff-out</code> 의 출력 경로가 없거나 권한.디스크 문제
8	불량 픽셀 발견	<code>--validate</code> 에서 NaN 또는 Inf 가 한 개 이상. <code>negative</code> / <code>superwhite</code> 만으로는 8 이 되지 않습니다
10	CI 게이트 FAIL	<code>--fail-psnr</code> / <code>--fail-ssim</code> / <code>--fail-flip</code> / <code>--fail-maxerr</code> / <code>--fail-uniformity</code> / <code>--fail-semu</code> 중 하나 이상 위반

팁 `set -e` 를 쓰는 CI 스크립트에서는 `exit 10` 이 스크립트를 즉시 죽여 리포트 수집 단계가 실행되지 않습니다. 게이트 판정 명령만 `|| rc=$?` 로 감싸십시오 (8장).

11.4. CSV / JSON 스키마 요약

모든 헤드리스 CSV 는 첫 줄이 헤더이고, stdout 에는 데이터만 나갑니다. 사람이 읽는 요약 줄과 오류 메시지는 stderr 로 분리됩니다. 무한대(완전 동일)는 `inf` , 수치 형식은 `%.6g` 입니다.

11.4.1. `--metrics` / `--batch` — 16열

#	열 이름	의미
1	<code>file</code>	A 의 파일명(basename). <code>--batch</code> 에서 label 을 주면 그 label
2	<code>width</code>	영상 폭(픽셀)
3	<code>height</code>	영상 높이(픽셀)
4	<code>psnr_db</code>	종합 PSNR(dB). R/G/B 중 유한한 채널들의 평균, 모두 동일하면 <code>inf</code>
5	<code>psnr_r</code>	Red 채널 PSNR
6	<code>psnr_g</code>	Green 채널 PSNR
7	<code>psnr_b</code>	Blue 채널 PSNR
8	<code>psnr_y</code>	Rec.709 루마 PSNR
9	<code>ssim</code>	SSIM 점수 (0 – 1)
10	<code>flip</code>	FLIP 평균 오차 (0 – 1, 작을수록 좋음)

11	mse	RGB 3채널 평균 MSE
12	mae	RGB 3채널 평균 MAE
13	max_error	채널을 통틀어 가장 큰 절대 오차
14	msigned	루마 가중 평균 부호 오차 (A - B). 부호가 밝기 편향 방향
15	psnr_cb	Rec.709 Cb 크로마 PSNR
16	psnr_cr	Rec.709 Cr 크로마 PSNR

행 상태가 정상이 아니면 `<name>,,,<tag>` 뒤에 빈 필드가 채워진 플레이스홀더 행이 나옵니다. `<tag>` 는 `missing` (짜 파일 없음), `decode_error` (디코드 실패), `mismatch` (크기/포맷 불일치) 중 하나입니다.

예시 — 헤드리스 지표 한 쌍 확인

```
av --metrics /data/ddi/orig/f0001.png /data/ddi/hw/f0001.png
#
file,width,height,psnr_db,psnr_r,psnr_g,psnr_b,psnr_y,ssim,flip,mse,mae,max_error,msigned,psnr_cb,psnr_cr
#
f0001.png,1920,1080,42.1337,41.88,42.55,41.96,42.61,0.99312,0.0184,3.98,1.21,37,-0.164,48.3,47.1
```

`msigned` 가 음수이므로 보상 결과(B) 가 원본(A) 보다 전체적으로 밝습니다.

`--format json` 은 같은 값을 `frames[]` 배열과 `summary` 객체(프레임 수, `psnr` / `ssim` / `flip` 의 mean/median/p95/min/max, 게이트 판정)로 내보내고, `--format junit` 은 `<testsuite>` 아래 프레임당 `<testcase>` 를 만들며 FAIL 은 `<failure>`, 상태 행은 `<skipped>` 로 표시합니다.

11.4.2. `--probe` — 11열

#	열 이름	의미
1	which	A / B / delta 중 하나
2	x	CIE XYZ 의 X
3	y	CIE XYZ 의 Y (휘도)
4	z	CIE XYZ 의 Z
5	x	CIE 1931 색도 x
6	y	CIE 1931 색도 y
7	uprime	CIE 1976 u'
8	vprime	CIE 1976 v'
9	CCT	상관 색온도 (K). delta 행에서는 ΔCCT
10	Duv	Planckian locus 로부터의 거리. delta 행에서는 Δu'v'
11	Lstar	CIE L*. delta 행에서는 ΔE76

참고 `delta` 행은 2 - 8 열이 비어 있고 9/10/11 열만 채워집니다. 즉 같은 열 번호라도 A/B 행과 delta 행의 의미가 다릅니다. 파싱 스크립트에서 반드시 `which` 열로 분기하십시오.

11.4.3. `--uniformity` — 11열

#	열 이름	의미
1	file	파일명. Δ 행에서는 delta
2	role	A / B / B-A

3	<code>width</code>	영상 폭
4	<code>height</code>	영상 높이
5	<code>Lmean</code>	화면 전체 평균 휘도(CIE Y)
6	<code>uni9_pct</code>	ICDM 9점 휘도 균일도 % — 격자점 휘도의 $100 * Lmin / Lmax$
7	<code>uni13_pct</code>	ICDM 13점 균일도 % (3×3 + 내부 4점)
8	<code>uni25_pct</code>	ICDM 25점 균일도 %. <code>--fail-uniformity</code> 가 보는 값
9	<code>nonuni_cv_pct</code>	전 픽셀 휘도의 변동계수 CV %
10	<code>duv_max</code>	25점 각각의 u'v' 와 화면 평균 u'v' 사이 최대 $\Delta u'v'$
11	<code>semu</code>	SEMU 무라 프록시. <code>--fail-semu</code> 가 보는 값

계산 실패 행은 3열에 `error` 가 들어간 플레이스홀더가 됩니다. B-A 행은 각 항목의 단순 차이(B - A)이므로 `uni*_pct` 는 + 가, `cv` · `semu` 는 - 가 “보상으로 좋아졌다” 는 뜻입니다.

11.4.4. `--comp-out` — 11열 (블록 단위)

#	열 이름	의미
1	<code>bx</code>	블록 열 인덱스 (0 시작)
2	<code>by</code>	블록 행 인덱스 (0 시작)
3	<code>x</code>	블록 좌상단 픽셀 X. <code>bx * blk_w</code>
4	<code>y</code>	블록 좌상단 픽셀 Y. <code>by * blk_h</code>
5	<code>w</code>	블록 실제 폭 (오른쪽 끝은 잘릴 수 있음)
6	<code>h</code>	블록 실제 높이
7	<code>mse_img2</code>	orig 대비 img2 의 블록 MSE
8	<code>psnr_img2</code>	orig 대비 img2 의 블록 PSNR(dB). 완전 동일이면 <code>inf</code>
9	<code>mse_img3</code>	orig 대비 img3 의 블록 MSE
10	<code>psnr_img3</code>	orig 대비 img3 의 블록 PSNR(dB)
11	<code>dpsnr</code>	<code>psnr_img2 - psnr_img3</code> . + 면 img2 우세, 한쪽만 <code>inf</code> 면 <code>inf / -inf</code>

행은 worst 목록이 아니라 **영상의 모든 블록** 입니다 (`nbx * nby` 줄). stderr 로 블록 수, 0 이 아닌 블록 수, 전역 PSNR, `|d| > 1dB` 기준 승패 집계가 한 줄 나옵니다.

```
av --comp --comp-out - --blk 16x16 --num_blk 24 --blk-metric y \
orig/f0001.png fw/f0001.png hw/f0001.png | head -3
```

11.4.5. `--comp-batch` — 10열 (프레임 단위)

#	열 이름	의미
1	<code>file</code>	프레임 파일명 (orig 디렉토리 기준)
2	<code>psnr_img2</code>	프레임 전역 PSNR (orig 대비 img2)
3	<code>psnr_img3</code>	프레임 전역 PSNR (orig 대비 img3)
4	<code>dpsnr</code>	<code>psnr_img2 - psnr_img3</code>
5	<code>win2</code>	img2 가 이긴 블록 수 (<code>d > 1dB</code>)
6	<code>win3</code>	img3 가 이긴 블록 수 (<code>d < -1dB</code>)
7	<code>tie</code>	무승부 블록 수 (<code>abs(d) <= 1dB</code>)
8	<code>worst2</code>	img2 의 #1 worst 블록 <code>bx:by@psnr</code>

9	worst3	img3 의 #1 worst 블록	bx:by@psnr
10	status	ok / missing (짝 프레임 없음) / mismatch (크기 불일치)	

11.4.6. --validate — 2열

열 이름	의미
class	nan / inf / negative / superwhite 네 행이 항상 이 순서로 나옵니다
count	해당 분류에 속한 채널값 개수 (RGB 3채널을 각각 셈)

값이 하나라도 있으면 그 아래에 `# first offenders: class,x,y,channel` 주석과 최대 20개의 위치가 `#` 주석으로 따라붙습니다. 주석 줄이므로 CSV 파서에서 `#` 로 시작하는 줄만 건너뛰면 됩니다.

11.5. 지원 파일 포맷

디렉토리 시퀀스 스캔과 파일 열기 대화상자가 인식하는 확장자는 다음 8종입니다 (`SUPPORTED_IMG_EXTS` , 대소문자 무시).

```
.png .jpg .jpeg .bmp .tga .pgm .ppm .hdr
```

포맷	확장자	내부 표현	비고
PNG	.png	RGBA8	8/16비트, 알파 지원. 저장·클립보드 복사의 기본 포맷
JPEG	.jpg .jpeg	RGBA8	손실 압축. 압축 아티팩트가 지표에 그대로 잡히므로 회귀 기준 영상으로는 권장하지 않습니다
BMP	.bmp	RGBA8	무손실. 저장 포맷으로도 선택 가능
TGA	.tga	RGBA8	알파 채널 포함
PGM	.pgm	원본 정수 + RGBA8	P2(ASCII) / P5(binary) 그레이스케일. <code>maxval</code> 보존
PPM	.ppm	원본 정수 + RGBA8	P3(ASCII) / P6(binary) 컬러. <code>maxval</code> 이 255 가 아닌 10/12비트 파일도 원본값 그대로 유지 (최대 65535)
HDR	.hdr	RGBA32F	Radiance RGBE. float 로 유지되어 <code>--validate</code> 와 <code>/</code> 오버레이의 대상이 됩니다

참고 PNM 계열은 av 자체 파서가 먼저 시도하고(P2/P3/P5/P6 매직 넘버 확인), 실패하면 `stb_image` 로 넘어갑니다. 그래서 10비트 PPM 의 원본값이 8비트로 눌리지 않고 픽셀 풍선말·통계·라인컷에 그대로 나타납니다 (3장). 저장 시에도 `P6 / 10bit` 를 고르면 `maxval 1023` 인 PPM 이 나옵니다 (7장).

주의 헤드리스 경로(`src/metrics_cli.cpp`)는 GUI 와 달리 `stb_image` 만 사용합니다. 즉 `--metrics` 등에서 PPM 은 stb 의 PNM 디코더를 타므로 8비트로 정규화된 값이 쓰입니다. 10비트 원본값 자체를 다뤄야 하는 검증은 GUI 경로나 PNG 16비트를 쓰십시오.

11.6. 용어집

11.6.1. 지표 용어

용어	정의
PSNR	Peak Signal-to-Noise Ratio. $20 \log_{10}(\text{peak}) - 10 \log_{10}(\text{MSE})$ (dB). <code>peak</code> 는 8비트 영상 255, float 영상 1.0. 값이 클수록 원본에 가깝고, 완전히 같으면 무한대(<code>inf</code>)입니다. av 의 종합 PSNR 은 R/G/B 채널 PSNR 중 유한하고 999 미만인 값들의 평균입니다

SSIM	Structural Similarity Index. 국소 창에서 밝기·대비·구조 세 요소를 비교해 0 – 1 로 냅니다. 1 이 완전 일치. 블러·구조 왜곡처럼 PSNR 이 잘 못 잡는 결함에 민감합니다
FLIP	NVIDIA 의 지각 오차 지표. 사람이 두 영상을 번갈아 볼 때 느끼는 차이를 0(동일) – 1 로 모델링합니다. av 는 magma 색맵으로 오차맵을 그리고 평균값을 <code>flip</code> 열에 씁니다. 값이 작을수록 좋으므로 게이트도 “초과 시 FAIL” 입니다
MSE	Mean Squared Error. 픽셀 차이의 제곱 평균. av 의 <code>mse</code> 열은 R/G/B 세 채널 MSE 의 평균입니다. 큰 오차에 가중이 크게 실립니다
MAE	Mean Absolute Error. 픽셀 차이 절대값의 평균. MSE 보다 이상치에 둔감해서 “전반적으로 얼마나 어긋나 있는가” 를 봅니다
msigned	루마 가중 평균 부호 오차. 채널별 평균 부호 오차(A – B)에 Rec.709 루마 계수 <code>0.2126 / 0.7152 / 0.0722</code> 을 곱해 더한 값입니다. 0 에 가까우면 편향 없음, 음수면 B(보상 결과)가 전체적으로 밝고 양수면 어둡습니다. PSNR/MSE 가 버리는 방향 정보 를 담습니다

11.6.2. 색 용어

용어	정의
$\Delta u'v'$	CIE 1976 UCS 색도 평면에서 두 색 사이의 유클리드 거리. 지각적으로 균등한 척도라서 색 편차 판정에 씁니다. 통상 0.004 이하면 나란히 놓고도 구분하기 어렵다고 봅니다
CCT	Correlated Color Temperature. 상관 색온도(K). av 는 McCamy 1992 근사식을 CIE 1931 xy 에 적용합니다. 백색점이 “따뜻한가/차가운가” 를 한 숫자로 요약합니다
Duv	Planckian locus(흑체 궤적)로부터의 부호 있는 거리. CIE 1960 uv 로 변환한 뒤 6차 다항 근사로 계산합니다. + 면 궤적 위쪽(초록 기미), - 면 아래쪽(자홍 기미)입니다. CCT 만으로는 알 수 없는 백색점의 색 틀어짐을 잡습니다
SEMU	무라의 가시성 을 정규화한 지표. av 는 배경면을 뺀 잔차의 최악 Weber 대비 <code>Cmax</code> 를 결함 면적 <code>s</code> (화면 대비 %)의 가시성 계수로 나눕니다 — <code>semu = 100 * Cmax / (1.97 / S^0.33 + 0.72)</code> . 평탄한 화면은 0, 국소 얼룩은 대비·면적에 따라 커집니다. 화소 피치와 시청 거리를 모델링하지 않은 프록시 이므로 절대 합격선보다 같은 조건에서의 상대 비교 에 쓰십시오
ICDM	Information Display Measurements Standard. 디스플레이 측정 규격으로, 평판 균일도를 화면상의 9점·13점·25점 격자에서 측정하도록 정의합니다. av 의 <code>uni9_pct / uni13_pct / uni25_pct</code> 가 이 격자를 따릅니다
LUT	Look-Up Table. 입력 색을 표에서 찾아 출력 색으로 바꾸는 변환입니다. av 는 <code>.cube</code> 형식의 3D/1D LUT 를 <code>--lut</code> 로 받아 표시에만 적용합니다 (토큰). 지표 계산은 룩을 적용하기 전 원시 픽셀로 합니다
CDL	ASC Color Decision List. slope / offset / power 세 파라미터에 채도(sat) 를 더한 표준 색 보정 기술입니다. av 는 <code>.cdl / .ccc</code> 파일을 <code>--cdl</code> 로 읽어 LUT 와 같은 표시 전용 경로에 적용합니다

11.6.3. `--comp` 와 실무 용어

용어	정의
spike 픽	<code>--comp</code> 의 worst 블록 목록 중 블록 내 최대 픽셀 오차 기준으로 뽑힌 항목입니다. 목록의 위쪽 3/4 은 지표 MSE 순, 나머지 1/4 이 이 방식입니다. 화면에서는 마젠타 색 박스와 <code>P</code> 표시, 리스트 창에서는 순번 옆 <code>P</code> 로 구분합니다. 63픽셀이 정확하고 1픽셀만 크게 튀는 점 결함처럼 MSE 에 묻히는 결함을 잡기 위한 장치입니다 (5장)
win/loss	블록별로 <code>dpsnr = psnr_img2 - psnr_img3</code> 를 계산해 <code>> 1dB</code> 면 img2 승, <code>< -1dB</code> 면 img3 승, 그 사이는 무승부로 집계한 것입니다. 두 알고리즘(예: FW 보상 대 HW 보상) 중 어느 쪽이 화면의 어느 영역에서 유리한지 보는 지도이며, <code>Ctrl+W</code> 맵과 <code>--comp-batch</code> 의 <code>win2 / win3 / tie</code> 열로 나타냅니다
DDI	Display Driver IC. 패널을 구동하는 드라이버 칩입니다. 이 매뉴얼의 주 사용 맥락은 DDI 안의 보상 IP(디무라·감마·색보정 등)가 만든 출력 프레임을 원본과 비교해 화질 회귀를 검증하는 작업입니다

무라 (mura)	패널 표면에 나타나는 얼룩·국소 휘도 또는 색 불균일을 가리키는 용어입니다. 전체적인 밝기 기울기(shading)와 달리 국소적이며, 사람 눈에 “얼룩”으로 인지됩니다. av의 <code>--uniformity</code> 는 2차 다항 배경면을 최소자승으로 제거해 shading을 걷어낸 뒤 남은 잔차로 SEMU를 계산하므로, 순수한 기울기는 0에 가깝게, 국소 얼룩은 값이 살아남게 설계돼 있습니다
ROI	Region of Interest. 관심 영역입니다. <code>Ctrl+E</code> 로 모드를 켜고 좌드래그로 사각형을 지정하면 그 영역만의 통계와 컬러메트리를 볼 수 있습니다 (6장)